

Hướng dẫn sử dụng bảng điều khiển 3X-UI

العربية · English · Español · فارسی · Bahasa Indonesia · 日本語 · Português · Русский · Türkçe · Українська · Tiếng Việt · 简体中文 · 繁體中文

Phiên bản 3X-UI: 3.4.0. Hướng dẫn được soạn theo phiên bản này và phù hợp với nó. Tóm tắt các thay đổi của 3.4.0 so với 3.3.1 – trong mục «[Có gì mới trong 3.4.0](#)».

Hướng dẫn chi tiết bằng tiếng Việt về bảng điều khiển web **3X-UI** (quản lý Xray-core): các tính năng, cấu hình và vận hành, với phân tích từng trường và nút bật tắt trong giao diện.

Tên và nhãn tương ứng với giao diện bảng điều khiển; các thuật ngữ tiếng Việt được trình bày đúng như hiển thị trong giao diện. Các từ *inbound* / *outbound* không được dịch.

Mục lục

- Có gì mới trong 3.4.0
- 1. Giới thiệu, yêu cầu và cài đặt
 - 1.1. 3X-UI là gì
 - 1.2. Hệ điều hành và kiến trúc được hỗ trợ
 - 1.3. Phương thức cài đặt
 - 1.4. Lần khởi động đầu tiên và thông tin đăng nhập mặc định
 - 1.5. Vị trí tệp
 - 1.6. Lệnh quản lý `x-ui` (menu script)
 - 1.7. Lệnh con `x-ui` (không có menu tương tác)
 - 1.8. Di chuyển SQLite → PostgreSQL
- 2. Đăng nhập bảng điều khiển và bảo mật truy cập
 - 2.1. Biểu mẫu đăng nhập
 - 2.2. Xác thực hai yếu tố (2FA / TOTP)
 - 2.3. Giới hạn số lần đăng nhập (login limiter / bảo vệ chống dò mật khẩu)
 - 2.4. Đổi tên đăng nhập và mật khẩu quản trị viên
 - 2.5. Đường dẫn bí mật (URI-path / webBasePath) và cổng bảng điều khiển
 - 2.6. Thời gian tồn tại phiên (timeout)
 - 2.7. LDAP (đồng bộ và xác thực)
- 3. Tổng quan / Dashboard
 - 3.1. Nguyên tắc thu thập dữ liệu chung
 - 3.2. CPU
 - 3.3. Bộ nhớ (RAM)
 - 3.4. Bộ nhớ hoán đổi (Swap)
 - 3.5. Ổ đĩa (Storage)
 - 3.6. Thời gian hoạt động (Uptime)
 - 3.7. Tải hệ thống (Load average)
 - 3.8. Mạng: tốc độ và tổng lưu lượng
 - 3.9. Địa chỉ IP của máy chủ
 - 3.10. Kết nối TCP/UDP
 - 3.11. Trạng thái Xray và quản lý tiến trình
 - 3.12. Cập nhật bảng điều khiển (3X-UI)
 - 3.13. Cập nhật tệp địa lý (GeoIP / GeoSite)
 - 3.14. Sao lưu và khôi phục cơ sở dữ liệu
 - 3.15. Các yếu tố giao diện bổ sung
- 4. Inbounds: tạo và các tham số chung
 - 4.1. Các trường chung của biểu mẫu
 - 4.2. Sniffing
 - 4.3. Allocate (chiến lược phân bổ cổng)
 - 4.4. External Proxy (Proxy ngoài)
 - 4.5. Fallbacks
 - 4.6. Đặt lại lưu lượng định kỳ
 - 4.7. JSON của inbound (nâng cao)
 - 4.8. Thao tác với inbound: QR / Edit / Reset / Delete và thống kê

- 5. Giao thức
 - 5.1. Danh sách các giao thức được hỗ trợ
 - 5.2. Giao thức nào hỗ trợ TLS / REALITY / transport
 - 5.3. VLESS
 - 5.4. VMess
 - 5.5. Trojan
 - 5.6. Shadowsocks
 - 5.7. Dokodemo-door / Tunnel (bộ chuyển tiếp trong suốt)
 - 5.8. SOCKS / HTTP (giao thức `mixed`)
 - 5.9. WireGuard (inbound)
 - 5.10. Hysteria (mặc định v2)
 - 5.11. MTPROTO (proxy cho Telegram)
 - 5.12. Bảng tóm tắt nhanh về lựa chọn giao thức
- 6. Transport (Stream Settings)
 - 6.1. Lựa chọn mạng truyền tải
 - 6.2. RAW / TCP (`tcpSettings`)
 - 6.3. mKCP (`kcpSettings`)
 - 6.4. WebSocket (`wsSettings`)
 - 6.5. gRPC (`grpcSettings`)
 - 6.6. HTTPUpgrade (`httpupgradeSettings`)
 - 6.7. XHTTP / SplitHTTP (`xhttpSettings`)
 - 6.8. Transport Hysteria (`hysteriaSettings`)
 - 6.9. Các tham số liên quan
- 7. Bảo mật kết nối: TLS,XTLS và REALITY
 - 7.1. Sự khác biệt: TLS vs XTLS vs REALITY
 - 7.2. Chế độ «Không» (`none`)
 - 7.3. Chế độ TLS
 - 7.4. Chế độ REALITY
 - 7.5. Khuyến nghị thực tế về cấu hình
- 8. Khách hàng (Clients)
 - 8.1. Các trường của khách hàng
 - 8.2. Liên kết với inbound
 - 8.3. Thao tác với khách hàng
 - 8.4. Thao tác hàng loạt
 - 8.5. Tìm kiếm, bộ lọc và sắp xếp
 - 8.6. Biểu tượng và trạng thái
- 9. Nhóm khách hàng
 - 9.1. Nhóm khách hàng là gì và tại sao cần thiết
 - 9.2. Liên kết nhóm với khách hàng, inbound, node và giao thức
 - 9.3. Danh mục nhóm và nhóm «trống»
 - 9.4. Các trường và cột của nhóm
 - 9.5. Tạo nhóm
 - 9.6. Đổi tên nhóm
 - 9.7. Thêm khách hàng vào nhóm
 - 9.8. Xóa khách hàng khỏi nhóm (không xóa bản thân khách hàng)
 - 9.9. Đặt lại lưu lượng nhóm

- 9.10. Xóa nhóm và xóa khách hàng của nhóm
- 9.11. Liên kết với trang «Khách hàng»
- 9.12. Tóm tắt các endpoint API
- 9.13. Lưu lượng theo nhóm
- 10. Đăng ký (Subscription)
 - 10.1. subId là gì và cách hình thành liên kết
 - 10.2. Cài đặt máy chủ đăng ký
 - 10.3. Định dạng đầu ra
 - 10.4. Trang thông tin đăng ký và mã QR
 - 10.5. Mẫu trang đăng ký tùy chỉnh
- 11. Xray: định tuyến, outbounds, DNS và tiện ích mở rộng
 - 11.1. Cấu trúc trình chỉnh sửa: tab/chế độ
 - 11.2. Cài đặt chung (General)
 - 11.3. Quy tắc định tuyến (routing)
 - 11.4. Outbounds (kết nối đi)
 - 11.5. Bộ cân bằng tải (Balancers)
 - 11.6. DNS
 - 11.7. Fake DNS
 - 11.8. WireGuard / WARP / NordVPN
 - 11.9. Reverse proxy và TUN
 - 11.10. Log và thống kê (Stats, metrics)
 - 11.11. Lưu, khởi động lại và chuyển đổi tự động
 - 11.12. Outbound từ đăng ký (với tự động cập nhật)
 - 11.13. Xoay vòng IP trong WARP
- 12. Node (đĩa bảng điều khiển, master/slave)
 - 12.1. Tóm tắt ở đầu danh sách
 - 12.2. Thêm và chỉnh sửa node
 - 12.3. Kiểm tra TLS (cho node https)
 - 12.4. Thông tin hiển thị cho mỗi node
 - 12.5. Thao tác với node
 - 12.6. Lịch sử số liệu
 - 12.7. Cách đồng bộ inbounds và khách hàng
 - 12.8. Chuỗi node (node con / node trung gian)
 - 12.9. Node: điểm mới trong 3.3.0
- 13. Cài đặt bảng điều khiển
 - 13.1. Lưu và khởi động lại bảng điều khiển
 - 13.2. Cài đặt chung (tab «Bảng điều khiển» / *General*)
 - 13.3. Truy cập bảng điều khiển: IP, cổng, đường dẫn, tên miền, chứng chỉ
 - 13.4. Phiên, proxy bảng điều khiển và proxy tin cậy (tab «Proxy và máy chủ» / *Proxy and Server*)
 - 13.5. Bot Telegram (tab «Bot Telegram» / *Telegram Bot*)
 - 13.6. Ngày và giờ (tab «Ngày và giờ» / *Date and Time*)
 - 13.7. Lưu lượng bên ngoài và hành vi Xray (tab «Lưu lượng ngoài» / *External Traffic*)
 - 13.8. Khác: mẫu cấu hình Xray và URL kiểm tra
 - 13.9. Tài khoản quản trị viên và token API
 - 13.10. Thay đổi API trong 3.3.0 (quan trọng đối với tích hợp)

- 14. Bot Telegram
 - 14.1. Bật và cấu hình bot
 - 14.2. Menu chính và các nút
 - 14.3. Lệnh bot
 - 14.4. Quản lý khách hàng (chỉ quản trị viên)
 - 14.5. Thông báo và báo cáo
 - 14.6. Sao lưu và log
 - 14.7. Đặc điểm hoạt động
 - 15. Cơ sở dữ liệu địa lý (geoip / geosite và tùy chỉnh)
 - 15.1. geoip.dat và geosite.dat là gì
 - 15.2. Tập địa lý chuẩn và cập nhật chúng
 - 15.3. Tài nguyên địa lý người dùng (Custom GeoSite / GeoIP)
 - 15.4. Xác thực và giới hạn
 - 15.5. Kiểm tra tự động khi khởi động bảng điều khiển
 - 15.6. Sử dụng cơ sở dữ liệu địa lý trong quy tắc định tuyến
 - 16. Vận hành: sao lưu, log, cập nhật, CLI
 - 16.1. Sao lưu và khôi phục cơ sở dữ liệu
 - 16.2. Xem log
 - 16.3. Mức độ và cấu hình ghi log Xray
 - 16.4. Quản lý Xray: dừng và khởi động lại
 - 16.5. Khởi động lại và cập nhật bảng điều khiển
 - 16.6. Tác vụ định kỳ (cron)
 - 16.7. Menu console và CLI (`x-ui`)
 - 16.8. Gỡ cài đặt bảng điều khiển
 - 16.9. Lệnh `x-ui migrateDB`
-

Có gì mới trong 3.4.0

Mục này liệt kê ngắn gọn các thay đổi của phiên bản **3.4.0** so với 3.3.1, hiển thị với người dùng bảng điều khiển, được nhóm theo các mục của hướng dẫn. Chi tiết về từng tính năng – trong mục tương ứng bên dưới.

3. Tổng quan / Dashboard

- **Lịch sử số liệu hệ thống: thêm khoảng thời gian tổng hợp 12h/24h/48h** – Trong cửa sổ lịch sử số liệu hệ thống, các giá trị 12h, 24h và 48h đã được thêm vào các khoảng thời gian trung bình – giờ đây các biểu đồ (CPU, RAM, lưu lượng, gói tin, kết nối, đĩa, trực tuyến, tải) có thể xem trong khoảng thời gian lên đến hai ngày.

4. Inbounds: tạo và các tham số chung

- **Inbound: cảnh báo xung đột với cổng Xray API dự trữ** – Khi tạo hoặc sửa inbound, bảng điều khiển giờ không cho phép chiếm cổng nội bộ Xray API dự trữ (theo mặc định 62789 trên 127.0.0.1): TCP-inbound cục bộ trên cổng này trên loopback bị từ chối với lỗi xung đột cổng. Trên node, hạn chế không áp dụng – chúng có Xray riêng.
- **Tunnel/TPProxy: nhận streamSettings không có khóa security** – Inbound loại tunnel/TPProxy, có streamSettings không chứa khối security, giờ mở và lưu mà không có lỗi xác thực.
- **Inbounds: cột Speed (tốc độ trực tiếp) trong danh sách** – Trong danh sách inbounds xuất hiện cột Speed với tốc độ lưu lượng hiện tại (tải lên/xuống) theo từng inbound.

5. Giao thức

- **Shadowsocks-2022: tái tạo PSK máy khách khi thay đổi phương thức có kích thước khóa khác** – Đối với Shadowsocks-2022: khi thay đổi phương pháp mã hóa sang phương pháp có kích thước khóa khác (ví dụ giữa aes-256 và aes-128), PSK của máy khách giờ tự động được tái tạo cho độ dài mới khi lưu inbound. Hệ quả: các máy khách bị ảnh hưởng cần lấy lại đăng ký (các liên kết cũ sẽ ngừng hoạt động).
- **WireGuard: đã xóa trường workers** – Trường workers đã bị xóa khỏi biểu mẫu WireGuard (inbound và outbound): xray-core v26.6.22 không còn sử dụng nó nữa. Các cấu hình đã lưu trước đây hoạt động không thay đổi – trường đơn giản bị bỏ qua.
- **VLESS+XHTTP: flow xtls-rprx-vision trong liên kết và đăng ký** – Đối với VLESS trên XHTTP, flow xtls-rprx-vision giờ được đưa vào liên kết và đăng ký một cách chính xác (kể cả cho XHTTP+REALITY và định dạng Clash/Mihomo). Trước đây bảng điều khiển lưu flow, nhưng máy khách nhận cấu hình không có vision.

6. Transport (Stream Settings)

- **XHTTP: đổi tên các trường sessionId + Session ID Table / Length** – Trong transport XHTTP các trường phiên đã được đổi tên: Session ID Placement và Session ID Key (trước đây – Session Placement / Session Key). Đã thêm hai tham số. Session ID Table – tập ký tự để tạo định danh phiên: có thể chọn định sẵn (ALPHABET, Base62, hex, number, v.v.) hoặc nhập chuỗi ASCII tùy ý; trống – giá trị mặc định xray-core. Session ID Length – độ dài hoặc phạm vi (ví dụ 8-16) của các định danh được tạo; chỉ tính khi Session ID Table được đặt, giá trị tối thiểu phải lớn hơn 0. Các inbound đã lưu cũ được di chuyển tự động.

- **Inbound: pres Real client IP để xác định IP thực sau CDN/relay** – Trong cài đặt socket (sockopt) của inbound xuất hiện tùy chọn Real client IP – pres để xác định IP thực của khách truy cập khi lưu lượng đến qua CDN hoặc relay (nếu không, địa chỉ của bên trung gian được ghi lại). Có ba tùy chọn: Off / direct (không xử lý), Cloudflare CDN (tin tưởng X-Forwarded-For) và L4 relay / Spectrum (PROXY) (chấp nhận header PROXY protocol). Các pres loại trừ nhau và cảnh báo nếu transport đã chọn không hỗ trợ chúng. Các trường này không bao giờ được gửi cho máy khách trong đăng ký.
- **gRPC: header Trusted X-Forwarded-For giờ được tính** – Header Trusted X-Forwarded-For giờ được tính và trên transport gRPC (trước đây – chỉ WebSocket, HTTPUpgrade và XHTTP). Đối với inbound gRPC, bảng điều khiển không còn hiển thị cảnh báo về header không được hỗ trợ.

7. Bảo mật kết nối: TLS, XTLS và REALITY

- **TLS: các trường mới Verify Peer Cert By Name, Curve Preferences, Master Key Log, ECH Sockopt** – Verify Peer Cert By Name – tên (phân cách bằng dấu phẩy) mà máy khách kiểm tra chứng chỉ máy chủ thay vì SNI; thay thế hiện đại cho allowInsecure đã bị xóa, được truyền trong liên kết và đăng ký, không ghi vào máy chủ. Curve Preferences – giới hạn và thứ tự đường cong trao đổi khóa TLS (ví dụ X25519MLKEM768, X25519); trống – giá trị mặc định. Master Key Log – đường dẫn để ghi khóa TLS (định dạng SSLKEYLOGFILE) để gỡ lỗi trong Wireshark; để trống trong môi trường sản xuất. ECH Sockopt – tham số socket để nhận cấu hình ECH (dialerProxy, Domain Strategy, TCP Fast Open, Multipath TCP).
- **REALITY: giới hạn tốc độ fallback (Limit Fallback) và Master Key Log** – Đối với mỗi hướng (Upload và Download): After Bytes – số byte được đi qua ở tốc độ đầy trước khi bắt đầu giới hạn (0 – giới hạn từ byte đầu tiên); Bytes Per Sec – giới hạn tốc độ lưu lượng fallback để các thăm dò không sử dụng máy chủ như kênh miễn phí (0 – không giới hạn, tắt hướng); Burst Bytes Per Sec – dự trữ cho các đột biến ngắn hạn. Cũng thêm trường Master Key Log (đường dẫn đến tệp SSLKEYLOGFILE để gỡ lỗi).
- **TLS: các nút điện Pinned Peer Cert SHA-256 từ chứng chỉ inbound và theo SNI** – Bên cạnh trường Pinned Peer Cert SHA-256 giờ có hai nút tự động điền thay vì nút hash ngẫu nhiên trước đây. Nút đầu tiên điền SHA-256 từ chứng chỉ của chính inbound. Nút thứ hai lấy hash từ chứng chỉ trực tiếp của máy chủ bằng cách thực hiện kết nối TLS theo SNI được chỉ định (serverName phải được điền). Các hash thu được được thêm vào trường (phân cách bằng dấu phẩy) và đưa vào liên kết để ghim chứng chỉ trên máy khách.
- **TLS: OCSP-stapling mặc định tắt cho chứng chỉ inbound mới** – Đối với inbound mới, OCSP stapling mặc định tắt (khoảng thời gian 0). Điều này loại bỏ các lỗi trong log xray đối với chứng chỉ không có OCSP responder (ví dụ Let's Encrypt). Trường vẫn còn – có thể bật cho các chứng chỉ hỗ trợ stapling.
- **REALITY: tương thích với trường dest (bí danh target)** – Nếu REALITY-inbound được tạo với trường dest (bởi các phiên bản bảng điều khiển cũ, qua API hoặc các công cụ bên ngoài), giờ nó tải chính xác: giá trị dest được điền vào trường Target. Trước đây Target trống, và việc lưu lại sẽ phá vỡ REALITY.

8. Khách hàng (Clients)

- **Tab «Links» trong trình chỉnh sửa khách hàng (liên kết ngoài và đăng ký)** – Trên đó, nút **Add External Link** thêm các liên kết chia sẻ bên ngoài (`vless://` , `vmess://` , `trojan://` , `ss://` , `hysteria2://` , `wireguard://`), và nút **Add External Subscription** – URL của đăng ký từ xa. Tất cả những điều này được trộn vào đầu ra đăng ký của khách hàng đó (định dạng raw, JSON và Clash): các liên kết được thêm như vậy, và đăng ký từ xa được bảng điều khiển tải xuống định kỳ và kết hợp cấu hình của chúng với cấu hình riêng.

- **Trường «Giới hạn IP» giờ bị tắt nếu không có Fail2ban** – Trường **Giới hạn IP** giờ chỉ hoạt động khi Fail2ban được cài đặt và hoạt động. Nếu Fail2ban không được cài đặt (hoặc hệ thống Windows, hoặc tính năng bị tắt trên máy chủ), trường trong trình chỉnh sửa khách hàng bị khóa, và khi di chuột sẽ hiển thị gợi ý đề xuất cài đặt Fail2ban từ menu bash `x-ui`. Khi cập nhật bảng điều khiển, giới hạn IP đã lưu của khách hàng trên máy chủ không có Fail2ban sẽ bị đặt lại về 0, vì nó không được áp dụng ở đó dù sao.
- **Xóa khách hàng không được liên kết, xuất và nhập khách hàng** – Trong menu **Thêm** trên trang **Khách hàng** đã thêm ba thao tác. **Xuất khách hàng** hiển thị danh sách JSON của tất cả khách hàng (ở định dạng `{client, inboundIds}`) với các nút sao chép và tải xuống (`clients-export.json`). **Nhập khách hàng** nhận JSON tương tự: khách hàng có liên kết được tái tạo và liên kết với inbound, khách hàng không có liên kết được khôi phục như các bản ghi riêng lẻ, và email đã tồn tại không bị ghi đè (chúng được đánh dấu là bỏ qua). **Xóa khách hàng không được liên kết** xóa tất cả khách hàng không được liên kết với bất kỳ inbound nào, cùng với lưu lượng, nhật ký IP và liên kết ngoài của họ; thao tác không thể hoàn tác.
- **Nhật ký IP của khách hàng: thời gian kết nối và tên node** – Trong nhật ký IP của khách hàng (nút xem bên cạnh trường «Giới hạn IP» và trong thẻ «Thông tin khách hàng»), mỗi bản ghi giờ bao gồm, ngoài IP, thời gian truy cập gần nhất và nhân node (`@ tên_node`) mà qua đó kết nối được ghi lại – trong câu hình đa bảng điều khiển, có thể thấy khách hàng kết nối qua node nào.
- **Đặt lại nhân nhóm của khách hàng trong trình chỉnh sửa đơn** – Giờ nếu trong trình chỉnh sửa một khách hàng xóa trường **Nhóm** và lưu, nhân nhóm được gỡ bỏ chính xác – trước đây khách hàng có thể tiếp tục hiển thị dưới nhóm cũ cho đến khi lưu lại.
- **Danh sách khách hàng tự động cập nhật (thăm dò nền)** – Danh sách khách hàng giờ tự động cập nhật: bảng điều khiển vài giây một lần kéo trang hiện tại, vì vậy khách hàng mới kết nối và thứ tự sắp xếp đã thay đổi xuất hiện mà không cần làm mới thủ công.

10. Đăng ký (Subscription)

- **Hosts được quản lý: ghi đè liên kết đăng ký theo host** – Trong phiên bản 3.4.0 đã thêm mục Hosts (mục menu bên). Đối với mỗi inbound có thể đặt một hoặc nhiều endpoint Host, được thay thế vào liên kết đăng ký thay vì địa chỉ, cổng và tham số TLS của chính inbound – điều này thuận tiện để phân phối lưu lượng qua CDN hoặc relay. Host có Remark và mô tả, liên kết với inbound, Address (trông – kế thừa địa chỉ inbound) và Port (0 – kế thừa cổng inbound), tham số Security (same/tls/none/reality), cũng như Host header, Path, Mux, Sockopt, Final Mask, loại trừ khỏi định dạng đăng ký (raw/json/clash) và tham số cho Clash/mihomo. Host được sắp xếp trong inbound và hỗ trợ thao tác hàng loạt.
- **Remark Template đã thay thế bộ tạo mô hình remark; biến {{VAR}}** – Bộ tạo tên hồ sơ cũ (chọn Inbound/Email/External Proxy và dấu phân cách) đã được thay thế bằng trường «Remark Template». Trong đó bạn viết định dạng tên tùy ý, chèn biến bằng nút: nhận dạng khách hàng (`{{EMAIL}}`), (`{{INBOUND}}`), (`{{HOST}}`), (`{{ID}}`), (`{{SUB_ID}}`), (`{{COMMENT}}`), (`{{TELEGRAM_ID}}`), lưu lượng (`{{TRAFFIC_USED}}`), (`{{TRAFFIC_LEFT}}`), (`{{TRAFFIC_TOTAL}}`), (`{{UP}}`), (`{{DOWN}}`), (`{{USAGE_PERCENTAGE}}`) và thời hạn/trạng thái (`{{DAYS_LEFT}}`), (`{{TIME_LEFT}}`), (`{{EXPIRE_DATE}}`), (`{{JALALI_EXPIRE_DATE}}`), (`{{STATUS}}`), (`{{STATUS_EMOJI}}`). Biến được thay thế riêng lẻ cho mỗi khách hàng khi tạo đăng ký, có xem trước. Các đoạn phân cách bằng ký tự «» với giá trị không giới hạn được ẩn tự động, và thông tin về sử dụng và thời hạn chỉ hiển thị trên liên kết đầu tiên của khách hàng. Nếu để trống trường, mô hình remark cũ sẽ được sử dụng.
- **Liên kết ngoài và đăng ký từ xa theo từng khách hàng (tab Links)** – Đây bạn có thể đính kèm liên kết chia sẻ bên ngoài (Add External Link) và địa chỉ đăng ký bên ngoài (Add External Subscription) cho từng khách hàng – chúng sẽ được đưa vào đăng ký riêng của khách hàng đó (định dạng RAW, JSON và Clash). Đăng ký

ngoài được bảng điều khiển tải xuống và kết hợp với cấu hình khách hàng. Điều này thuận tiện để cung cấp cho khách hàng các máy chủ bổ sung trên inbound của bạn.

- **Happ: ẩn cài đặt máy chủ trong máy khách (Hide server settings)** – Trên tab Happ của cài đặt đăng ký đã thêm nút bật tắt «Hide server settings». Khi bật, trong máy khách Happ, khả năng xem và thay đổi cài đặt máy chủ bị ẩn. Tùy chọn chỉ áp dụng cho máy khách Happ.
- **Tên node không còn được thêm vào tên hồ sơ trong đăng ký** – Tên node không còn được thêm vào tên hồ sơ trong đăng ký. Trong ứng dụng máy khách chỉ hiển thị remark của inbound do quản trị viên đặt, không có hậu tố nội bộ dạng «@tên-node».
- **Nhãn mô hình remark được đổi tên Other → External Proxy (sau đó thay thế bằng mẫu)** – Không cần ghi chép riêng: việc đổi tên mục mô hình remark «Other» thành «External Proxy» được hấp thụ bởi quá trình chuyển sang trường Remark Template mới, nơi bộ tạo mô hình từ UI bị loại bỏ.
- **Độ chính xác của liên kết đăng ký: SS2022, Shadowrocket, SIP002 obfs, XHTTP trong Clash** – Đã cải thiện tính tương thích của các liên kết đăng ký được tạo ra: đã sửa mã hóa SS2022, deep-link cho Shadowrocket, xuất Shadowsocks+obfs ở định dạng SIP002 (plugin obfs-local) và bộ trường đầy đủ XHTTP trong đăng ký Clash/Mihomo. Không cần cài đặt riêng – liên kết đơn giản được máy khách nhận dạng chính xác hơn.
- **Mô hình remark đăng ký: mục «Other» được đổi tên thành «External Proxy»** – Trong cài đặt đăng ký trong mô hình remark, mục «Other» được đổi tên thành «External Proxy» (nguồn – remark của proxy ngoài). Hành vi không thay đổi, cài đặt hiện tại vẫn tương thích.
- **Đăng ký: chọn biên remark bằng cách nhấp vào chip (Remark variable picker)** – Bên cạnh trường Remark Template có sẵn bộ biên-chip được nhóm: nhấp vào biên {{VAR}} chèn nó vào mẫu, khi di chuột hiển thị mô tả. Trong các trường remark và tên host cũng cho phép ký hiệu đơn giản hóa trong ngoặc đơn – {DATA_LEFT}, {EXPIRE_DATE}, {PROTOCOL}, {TRANSPORT}, v.v.; bảng điều khiển tự động chuyển đổi sang định dạng nội bộ {{...}}.

11. Xray: định tuyến, outbounds, DNS và tiện ích mở rộng

- **Định tuyến và Outbounds được đưa vào các mục riêng trong menu bên** – Bắt đầu từ phiên bản này «Outbounds» và «Định tuyến» (Routing) được đưa vào các mục riêng trong menu bên (ngay bên dưới «Hosts»), mỗi mục có địa chỉ riêng – /outbound và /routing . Trước đây định tuyến mở bên trong submenu «Cấu hình Xray», còn outbounds – như tab trang Xray. Trong submenu «Cấu hình Xray» giờ chỉ còn: Cơ bản, Bộ cân bằng tải, DNS và Mẫu nâng cao. Liên kết trực tiếp đến /outbound và /routing và tải lại trang hoạt động như mong đợi.
- **Quy tắc định tuyến có thể bật và tắt bằng nút bật tắt** – Quy tắc định tuyến riêng lẻ giờ có thể **tắt** tạm thời bằng nút bật tắt mà không cần xóa. Trong bảng quy tắc có cột «Bật» với nút bật tắt, trong biểu mẫu quy tắc trường «Bật» – cũng là nút bật tắt. Quy tắc bị tắt không đưa vào cấu hình Xray cuối cùng. Quy tắc thống kê dịch vụ (api) không thể tắt – nút bật tắt của nó bị khóa.
- **Xuất quy tắc định tuyến và outbounds mở trong cửa sổ xem trước modal** – Nút «Xuất» quy tắc định tuyến và outbounds giờ không tải tệp ngay lập tức, mà mở cửa sổ modal với xem trước JSON và các nút «Sao chép» và «Tải xuống». Trên tab «Định tuyến» «Nhập» và «Xuất» được tập hợp vào menu thả xuống «thêm» (như trên tab Outbounds).
- **Kiểm tra tất cả outbounds giờ kiểm tra cả outbounds từ đăng ký; direct/dns không còn được kiểm tra** – Nút «Kiểm tra tất cả» trên trang «Outbounds» giờ kiểm tra cả outbounds nhận từ đăng ký (bảng «từ đăng ký») – các hàng của chúng cũng được tô màu theo kết quả. Đồng thời outbounds `freedom` («direct») và

dns không còn được kiểm tra trong bất kỳ chế độ nào (đây không phải proxy): nút kiểm tra ở chúng không khả dụng, và «Kiểm tra tất cả» bỏ qua chúng.

- **FinalMask: mảng Lengths/Delays theo từng đoạn thay vì Length/Delay đơn lẻ** – Trong mask fragment (FinalMask), các trường đơn lẻ Length và Delay được thay thế bằng danh sách Lengths và Delays: cho mỗi đoạn có thể đặt phạm vi độ dài riêng (ví dụ 100-200) và độ trễ tính bằng ms (ví dụ 10-20 hoặc 0). Các hàng có thể thêm và xóa; các giá trị đã lưu trước đây được chuyển tự động.
- **Loopback outbound: thêm khối Sniffing** – Ở outbound loại loopback đã xuất hiện khối Sniffing với các tham số tương tự như ở inbound: bật, destOverride, Metadata Only, Route Only và danh sách tên miền bị loại trừ.
- **Hysteria2 / Salamander: chế độ Gecko (packetSize) và TLS cho mask Realm** – Trong mask UDP (FinalMask) cho Hysteria2, các khả năng được mở rộng. Ở mask Salamander đã thêm bộ chọn Mode: chế độ Gecko thêm phần đệm ngẫu nhiên cho gói tin với các trường kích thước Min/Max (từ 1 đến 2048, mặc định 512-1200) để bảo vệ chống phân tích độ dài gói tin. Ở mask Realm đã xuất hiện khối TLS Config không bắt buộc: Server Name (SNI), ALPN (h3/h2/http/1.1), Fingerprint và Allow Insecure.
- **Nhập liên kết chia sẻ vào outbound lưu cài đặt xmux** – Khi nhập outbound từ liên kết chia sẻ, giờ lưu cài đặt bộ ghép kênh **xmux** (XHTTP): trước đây chúng bị mất lặng lẽ. Sau khi nhập, các giá trị được điền vào biểu mẫu con XMUX.
- **Tag outbounds từ đăng ký giữ nguyên ký tự không phải ASCII (chữ Cyrillic)** – Tag outbounds nhận từ đăng ký giờ giữ nguyên ký tự không phải ASCII (ví dụ chữ Cyrillic) và vẫn đọc được, thay vì chỉ còn là chữ số.

12. Node (đa bảng điều khiển, master/slave)

- **Node: chế độ kiểm tra TLS mới – Mutual TLS (chứng chỉ máy khách)** – Trong biểu mẫu node, chế độ kiểm tra TLS giờ có bốn tùy chọn: Verify (CA hệ thống), Pin (ghim chứng chỉ theo SHA-256), Skip (không kiểm tra) và Mutual TLS mới (chứng chỉ máy khách). Trong chế độ Mutual TLS, bảng điều khiển xác nhận bổ sung bản thân với node bằng chứng chỉ máy khách do CA riêng của nó phát hành; API-token cho node như vậy trở thành tùy chọn. Để bật mTLS: trên node đặt chế độ Mutual TLS, sao chép CA của bảng điều khiển quản lý từ mục Node mTLS, đăng ký nó trên node như CA gốc đáng tin cậy và khởi động lại node.
- **Node: mục «Node mTLS» – sao chép CA bảng điều khiển và CA gốc đáng tin cậy** – Trên trang Node đã thêm mục Node mTLS để cấu hình xác thực TLS lẫn nhau giữa các bảng điều khiển. Nút «Sao chép CA của bảng điều khiển này» sao chép chứng chỉ gốc của bảng điều khiển vào clipboard – nó cần được chuyển đến các node được quản lý, những node sẽ kiểm tra chứng chỉ máy khách của bảng điều khiển. Trường «CA gốc đáng tin cậy» được sử dụng khi chính bảng điều khiển là node: dán CA của bảng điều khiển quản lý vào đây để yêu cầu chứng chỉ máy khách của nó, và khởi động lại bảng điều khiển. Mutual TLS được bật tùy ý; nếu các trường trống, node hoạt động theo sơ đồ cũ – chỉ với API-token.
- **Định tuyến kết nối đi của bảng điều khiển đến node (Connection outbound)** – Trong biểu mẫu node xuất hiện trường «Connection outbound» (kết nối đi). Nếu chọn tag Xray-outbound trong đó, lưu lượng của bảng điều khiển đến API node này sẽ đi qua outbound được chỉ định (bảng điều khiển tự thêm bridge-inbound trên loopback vào cấu hình hoạt động và áp dụng mà không cần khởi động lại). Giá trị trống = kết nối trực tiếp. Trong danh sách các tag được nhóm thành «Outbounds» và «Bộ cân bằng tải», outbounds blackhole bị ẩn.
- **Node: định tuyến lưu lượng bảng điều khiển → node qua outbound đã chọn («Connection outbound»)** – Trong biểu mẫu node xuất hiện trường «Connection outbound»: nó cho phép hướng lưu lượng yêu cầu của bảng điều khiển đến node qua outbound Xray đã chọn (có thể dùng outbounds thông thường và bộ

cân bằng tải). Bảng điều khiển tự động thêm loopback-bridge inbound vào cấu hình hoạt động và áp dụng thay đổi mà không cần khởi động lại. Để trống để kết nối trực tiếp.

- **Node: xóa node bị chặn khi vẫn còn inbounds được liên kết** – Chỉ có thể xóa node sau khi tất cả inbounds đã được gỡ liên kết khỏi nó. Nếu ít nhất một inbound vẫn được liên kết với node, bảng điều khiển sẽ từ chối xóa với lỗi – trước tiên hãy gỡ liên kết hoặc xóa các inbounds đó, sau đó xóa node.
- **Node: trên trang node hiển thị tốc độ trực tiếp của các inbound được đặt trên node** – Trên trang Node cho các inbound được đặt trên node, giờ hiển thị máy khách trực tuyến, bộ đếm và tốc độ truyền hiện tại. Chip «đã kết thúc» (ended) chỉ đếm máy khách hết hạn và hết lưu lượng (máy khách bị ngắt kết nối không còn nằm trong đó).

14. Bot Telegram

- **Thông báo: bus sự kiện mới với các kênh Telegram và Email (SMTP)** – Đã thêm hệ thống thông báo sự kiện với hai kênh phân phối – Telegram và Email. Trên tab thông báo, các sự kiện được nhóm theo thẻ: Outbound (ngã xuống/phục hồi), Xray Core (kết thúc bất thường), Nodes (node không khả dụng/phục hồi), System (tải CPU và bộ nhớ cao với ngưỡng % có thể cấu hình), Security (nỗ lực đăng nhập). Mỗi nhóm có nút bật tắt chính và bộ đếm sự kiện được chọn. Bộ sự kiện được bật cấu hình riêng cho Telegram và Email; mặc định bật «nỗ lực đăng nhập» và «tải CPU cao».
- **Thông báo: kênh Email/SMTP mới và cài đặt máy chủ SMTP** – Đã thêm kênh thông báo mới qua email thông qua SMTP. Trên tab cài đặt SMTP: bật thông báo email, máy chủ và cổng SMTP (mặc định 587), tên người dùng, mật khẩu (lưu ẩn), danh sách người nhận (phân cách bằng dấu phẩy) và loại mã hóa – none, STARTTLS (mặc định) hoặc TLS. Nút «Gửi email kiểm tra» kiểm tra kết nối và hiển thị giai đoạn nào (kết nối, xác thực, gửi) xảy ra lỗi. Trên tab thứ hai chọn các sự kiện sẽ nhận email.
- **Thông báo: cảnh báo tải bộ nhớ cao (ngưỡng RAM)** – Đèn cảnh báo tải CPU cao đã thêm cảnh báo tải bộ nhớ cao. Trong nhóm sự kiện «System» đã xuất hiện «Memory high (%)» với trường ngưỡng riêng (mặc định 80%); bảng điều khiển mỗi phút kiểm tra tải RAM và khi vượt ngưỡng gửi thông báo đến các kênh đã chọn.

15. Cơ sở dữ liệu địa lý (geoip / geosite và tùy chỉnh)

- **Cập nhật cơ sở dữ liệu địa lý: trạng thái theo từng tệp và bỏ qua khởi động lại nếu không có thay đổi** – Cập nhật cơ sở dữ liệu địa lý (geoip/geosite, bao gồm bộ IR và RU) giờ hiển thị trạng thái theo từng tệp: đã cập nhật, đã cập nhật hoặc lỗi tải xuống. Khởi động lại Xray (và do đó gián đoạn kết nối đang hoạt động) chỉ xảy ra nếu ít nhất một tệp thực sự được cập nhật; khi không có thay đổi, bảng điều khiển không khởi động lại. Hành vi tương tự với lệnh x-ui update-all-geofiles.

16. Vận hành: sao lưu, log, cập nhật, CLI

- **Giới hạn IP của khách hàng chỉ hoạt động khi cài đặt fail2ban; trường bị khóa nếu không** – Giới hạn số lượng IP của khách hàng giờ chỉ áp dụng nếu fail2ban được cài đặt trên máy chủ. Nếu không, trường «IP Limit» trong biểu mẫu khách hàng và khi thêm hàng loạt trở nên không khả dụng với gợi ý giải thích (trên Windows – thông báo riêng), và các giới hạn đã đặt trước đây trên các máy chủ như vậy bị tự động đặt lại về 0, vì chúng không được áp dụng dù sao. Chặn fail2ban giờ áp dụng cho cả TCP và UDP.
- **Tự động cài đặt fail2ban khi cài đặt và cập nhật bảng điều khiển** – Khi cài đặt và cập nhật bảng điều khiển trên máy chủ thông thường, fail2ban giờ được cài đặt và cấu hình tự động (trước đây điều này chỉ

xảy ra trong Docker), để tính năng giới hạn IP hoạt động ngay từ đầu. Hành vi được kiểm soát bởi biến môi trường XUI_ENABLE_FAIL2BAN: cài đặt được thực hiện nếu biến không được đặt hoặc bằng true. Chạy thử công có thể bằng lệnh x-ui setup-fail2ban; lỗi fail2ban không ngắt cài đặt hoặc cập nhật.

- **Ghi đề cổng bảng điều khiển qua biến XUI_PORT** – Đã thêm biến môi trường XUI_PORT, đặt cổng bảng điều khiển web chỉ trong thời gian chạy tiến trình hiện tại, không thay đổi giá trị webPort đã lưu trong cơ sở dữ liệu. Cho phép giá trị từ 1 đến 65535; giá trị trống, không chính xác hoặc ngoài phạm vi bị bỏ qua (sử dụng webPort) với cảnh báo trong log. Khi sử dụng Docker với mạng bridge, cổng được xuất bản của container phải trùng với XUI_PORT, ví dụ XUI_PORT=8080 và ports: «8080:8080».
- **CLI: cờ -webCert/-webCertKey giờ áp dụng trong lệnh con setting** – Trong CLI các cờ -webCert và -webCertKey giờ hoạt động trong lệnh con x-ui setting (trước đây chúng bị bỏ qua lặng lẽ và bảng điều khiển vẫn không có HTTPS). Khi chỉ định chúng, có thể ngay lập tức đặt đường dẫn đến chứng chỉ và khóa của bảng điều khiển web, mà không cần gọi lệnh con cert riêng lẻ.
- **Tên tệp sao lưu cơ sở dữ liệu được đặt theo địa chỉ máy chủ** – Tệp sao lưu cơ sở dữ liệu giờ được đặt tên theo địa chỉ máy chủ, không phải x-ui.db / x-ui.dump cố định. Khi tải xuống từ trình duyệt, tên lấy từ địa chỉ bảng điều khiển trong thanh địa chỉ, nếu không – từ tên miền web đã cấu hình, và khi không có – từ IP công cộng (trước tiên IPv4, sau đó IPv6). Như vậy các bản sao lưu từ các máy chủ khác nhau dễ phân biệt. Phần mở rộng vẫn là .db cho SQLite và .dump cho PostgreSQL.
- **Hỗ trợ cài đặt và cập nhật trên máy chủ chỉ có IPv6** – Script cài đặt và cập nhật giờ hoạt động chính xác trên các máy chủ chỉ có IPv6: tải xuống bản phát hành và các tệp phụ không còn bắt buộc sử dụng IPv4, vì vậy bảng điều khiển có thể được cài đặt và cập nhật trên máy chủ không có địa chỉ IPv4.

1. Giới thiệu, yêu cầu và cài đặt

1.1. 3X-UI là gì

3X-UI là bảng quản lý web mã nguồn mở dành cho máy chủ **Xray-core**. Bảng điều khiển cung cấp giao diện web đa ngôn ngữ thống nhất để triển khai, cấu hình và giám sát nhiều loại giao thức proxy và VPN: từ một VPS đơn lẻ đến các cấu hình phân tán gồm nhiều nút.

3X-UI là bản fork nâng cao của dự án X-UI gốc. So với bản gốc, 3X-UI bổ sung hỗ trợ nhiều giao thức hơn, tăng độ ổn định, thống kê lưu lượng theo từng client và nhiều tính năng tiện dụng khác.

Tính năng chính:

- **Inbound đa giao thức** – VLESS, VMess, Trojan, Shadowsocks, WireGuard, Hysteria2, HTTP, SOCKS (Mixed), Dokodemo-door / Tunnel, TUN và **MTPROTO** (proxy Telegram, thêm vào từ phiên bản 3.3.0).
- **Transport và mã hóa hiện đại** – TCP (Raw), mKCP, WebSocket, gRPC, HTTPUpgrade và XHTTP, được bảo vệ bằng TLS, XTLS và REALITY.
- **Fallback** – phục vụ nhiều giao thức trên cùng một cổng (ví dụ: VLESS và Trojan trên cổng 443) thông qua cơ chế fallback trong Xray.
- **Quản lý theo từng client** – hạn ngạch lưu lượng, ngày hết hạn, giới hạn IP, hiển thị trạng thái "trực tuyến", liên kết mời một chạm, mã QR và đăng ký.
- **Thống kê lưu lượng** – theo từng inbound, client và outbound, có thể đặt lại.
- **Hỗ trợ nhiều nút** – quản lý và mở rộng quy mô trên nhiều máy chủ từ một bảng điều khiển duy nhất.
- **Outbound và định tuyến** – WARP, NordVPN, quy tắc định tuyến tùy chỉnh, bộ cân bằng tải, chuỗi proxy.
- **Máy chủ đăng ký tích hợp** với nhiều định dạng đầu ra.
- **Telegram bot** để giám sát và quản lý từ xa.
- **REST API** với tài liệu Swagger tích hợp sẵn.
- **Lưu trữ linh hoạt** – SQLite (mặc định) hoặc PostgreSQL.
- **13 ngôn ngữ giao diện**, chủ đề tối và sáng.
- **Tích hợp Fail2ban** để áp dụng giới hạn IP theo từng client.

Lưu ý quan trọng: dự án chỉ dành cho mục đích sử dụng cá nhân. Không khuyến khích sử dụng cho các mục đích bất hợp pháp hoặc trong môi trường sản xuất.

1.2. Hệ điều hành và kiến trúc được hỗ trợ

Hệ điều hành

Script cài đặt xác định bản phân phối theo trường `ID` trong `/etc/os-release` (hoặc `/usr/lib/os-release`). Các hệ điều hành được hỗ trợ chính thức:

Ubuntu, Debian, Armbian, Fedora, CentOS, RHEL, AlmaLinux, Rocky Linux, Oracle Linux, Amazon Linux, Virtuozzo, Arch, Manjaro, Parch, openSUSE (Tumbleweed / Leap), Alpine và Windows.

Trên các hệ thống thuộc họ Alpine, dịch vụ OpenRC được sử dụng (`rc-service` / `rc-update`), còn lại dùng `systemd`. Với CentOS 7, các gói được cài qua `yum`, với các phiên bản mới hơn – qua `dnf`. Nếu bản phân phối không được nhận dạng, script mặc định sẽ thử dùng trình quản lý gói `apt-get`.

Kiến trúc bộ xử lý

Kiến trúc được xác định theo kết quả của `uname -m` và được quy về một trong các giá trị được hỗ trợ:

Giá trị <code>uname -m</code>	Kiến trúc 3X-UI
<code>x86_64</code> , <code>x64</code> , <code>amd64</code>	<code>amd64</code>
<code>i*86</code> , <code>x86</code>	<code>386</code>
<code>armv8*</code> , <code>arm64</code> , <code>aarch64</code>	<code>arm64</code>
<code>armv7*</code> , <code>arm</code>	<code>armv7</code>
<code>armv6*</code>	<code>armv6</code>
<code>armv5*</code>	<code>armv5</code>
<code>s390x</code>	<code>s390x</code>

Nếu kiến trúc không có trong danh sách này, script sẽ hiển thị thông báo «Unsupported CPU architecture!» và dừng cài đặt.

Các phụ thuộc cơ bản

Trước khi cài đặt bảng điều khiển, script tự động cài đặt một bộ gói cơ bản (tên gói khác nhau tùy bản phân phối): `cron` / `cronie` / `dcron`, `curl`, `tar`, `tzdata` / `timezone`, `socat`, `ca-certificates`, `openssl`.

1.3. Phương thức cài đặt

Phương thức 1. Script cài đặt (khuyến nghị)

Cài đặt được thực hiện bằng một lệnh duy nhất với quyền root:

```
bash <(curl -Ls https://raw.githubusercontent.com/mhsanaei/3x-ui/master/install.sh)
```

Script bắt buộc yêu cầu quyền root: nếu chạy không phải với quyền root, sẽ hiển thị «Please run this script with root privilege» và kết thúc với lỗi.

Các bước mà trình cài đặt thực hiện:

1. Xác định hệ điều hành và kiến trúc.
2. Cài đặt các phụ thuộc cơ bản.
3. Tải xuống archive bản phát hành `x-ui-linux-<arch>.tar.gz` và giải nén vào thư mục `/usr/local/x-ui`.
4. Tải xuống script quản lý `x-ui.sh` và cài đặt nó như lệnh `/usr/bin/x-ui`.

5. Tạo thư mục log `/var/log/x-ui`.
6. Chạy cấu hình ban đầu: chọn cơ sở dữ liệu, tạo thông tin đăng nhập, chọn cổng, tùy chọn cấu hình SSL.
7. Cài đặt và khởi động dịch vụ tự khởi động (systemd unit `x-ui.service` hoặc script init OpenRC cho Alpine).

Chọn cơ sở dữ liệu khi cài đặt. Trình cài đặt đề xuất:

- 1) SQLite (mặc định, khuyến nghị khi số client < 500) — một file `/etc/x-ui/x-ui.db`, không cần cấu hình.
- 2) PostgreSQL (khuyến nghị khi có nhiều client hoặc nhiều nút). PostgreSQL có thể được cài đặt cục bộ (tạo người dùng và cơ sở dữ liệu chuyên dụng với tên `xui`) hoặc chỉ định DSN tới máy chủ đã có sẵn. Các tham số kết nối được ghi vào file môi trường dịch vụ (`/etc/default/x-ui`, `/etc/conf.d/x-ui` hoặc `/etc/sysconfig/x-ui` tùy bản phân phối) dưới dạng biến `XUI_DB_TYPE` và `XUI_DB_DSN`.

Ví dụ: ghi tham số PostgreSQL vào file môi trường dịch vụ. Sau khi chọn PostgreSQL và nhập DSN, trình cài đặt sẽ thêm vào file môi trường các dòng tương tự như sau:

```
XUI_DB_TYPE=postgres
XUI_DB_DSN=postgres://xui:S3cretPass@127.0.0.1:5432/xui?sslmode=disable
```

Ở đây `xui` là tên người dùng và cơ sở dữ liệu, `127.0.0.1:5432` là địa chỉ và cổng máy chủ, `sslmode=disable` phù hợp cho kết nối cục bộ (với máy chủ từ xa thường dùng `require`).

Cài đặt phiên bản cụ thể (cũ hơn). Có thể chỉ định rõ tag phiên bản — trình cài đặt sẽ tải xuống bản phát hành tương ứng:

```
bash <(curl -Ls https://raw.githubusercontent.com/mhsanaei/3x-ui/v2.4.0/install.sh)
v2.4.0
```

Phiên bản tối thiểu cho phép cài đặt theo cách này là `v2.3.5`; nếu chỉ định phiên bản cũ hơn sẽ hiển thị «Please use a newer version (at least v2.3.5)».

Phương thức 2. Docker

Chạy với cơ sở dữ liệu SQLite mặc định:

```
docker compose up -d
```

Để chạy với dịch vụ PostgreSQL tích hợp, cần bỏ comment các dòng `XUI_DB_*` trong `docker-compose.yml` và khởi động với profile:

```
docker compose --profile postgres up -d
```

Image bao gồm Fail2ban (hoạt động mặc định) để áp dụng giới hạn IP theo client. Fail2ban chặn các vi phạm thông qua `iptables`, điều này yêu cầu khả năng `NET_ADMIN`. Trong `docker-compose.yml` khả năng này đã được cung cấp qua `cap_add`. Khi chạy thủ công qua `docker run`, các khả năng cần được thêm thủ công, nếu không các lệnh chặn sẽ chỉ được ghi log chứ không được áp dụng:

Ví dụ: lệnh docker run đầy đủ. Phương án tối thiểu với ánh xạ cổng bảng điều khiển, khả năng mạng và volume lưu trữ cơ sở dữ liệu:

```
docker run -d \
  --name 3x-ui \
  --restart unless-stopped \
  --cap-add=NET_ADMIN --cap-add=NET_RAW \
  -v $PWD/db:/etc/x-ui \
  -v $PWD/cert:/root/cert \
  -p 2053:2053 \
  ghcr.io/mhsanaei/3x-ui:latest
```

Volume `/etc/x-ui` lưu trữ file `x-ui.db` giữa các lần khởi động lại container, nếu không cài đặt và tài khoản sẽ bị mất.

```
docker run -d --cap-add=NET_ADMIN --cap-add=NET_RAW ... ghcr.io/mhsanaei/3x-ui
```

Trong Docker, bảng điều khiển là tiến trình chính của container: tự khởi động được kiểm soát bởi chính sách khởi động lại container (ví dụ: `restart: unless-stopped`), chứ không phải dịch vụ bên trong container.

1.4. Lần chạy đầu tiên và thông tin đăng nhập mặc định

Khi cài đặt lần đầu (khi vẫn đang dùng thông tin đăng nhập mặc định), trình cài đặt **tạo ngẫu nhiên** tên người dùng, mật khẩu và đường dẫn web, cũng như cổng:

Tham số	Cách tạo khi cài đặt	Ghi chú
Tên người dùng (Username)	chuỗi ngẫu nhiên 10 ký tự	được tạo tự động
Mật khẩu (Password)	chuỗi ngẫu nhiên 10 ký tự	được tạo tự động
Đường dẫn web bảng điều khiển (WebBasePath)	chuỗi ngẫu nhiên 18 ký tự	bảo vệ bảng điều khiển khỏi bị phát hiện qua URL gốc
Cổng bảng điều khiển (Port)	mặc định là cổng ngẫu nhiên trong khoảng 1024–62000; có thể đặt thủ công nếu muốn	giá trị "nhà máy" của <code>webPort</code> là <code>2053</code> , nhưng trình cài đặt sẽ ghi đè lên

Cuối quá trình cài đặt, script hiển thị tóm tắt: tên người dùng, mật khẩu, cổng, đường dẫn web, token API và liên kết truy cập (Access URL) dạng:

```
https://<tên-miền-hoặc-IP>:<cổng>/<đường-dẫn-web>
```

Nếu chứng chỉ SSL chưa được cấu hình, liên kết sẽ dùng `http://` và script sẽ hiển thị cảnh báo về việc cần cấu hình SSL (mục menu 19).

Bắt buộc phải thay đổi thông tin đăng nhập. Vì thông tin đăng nhập và mật khẩu được tạo ngẫu nhiên, cần **lưu lại ngay sau khi cài đặt**. Có thể thay đổi bất kỳ lúc nào qua mục menu «Reset Username & Password» (xem bên dưới) hoặc từ giao diện web trong cài đặt bảng điều khiển. Sau khi đặt lại, script

nhắc nhở: «Please use the new login username and password to access the X-UI panel. Also remember them!».

Sau khi cài đặt, lệnh `x-ui` được dùng để mở menu quản lý (xem mục 1.6).

1.5. Vị trí các file

Đường dẫn	Mục đích
<code>/usr/local/x-ui/</code>	thư mục cài đặt bảng điều khiển (nhị phân <code>x-ui</code> , script <code>x-ui.sh</code>)
<code>/usr/local/x-ui/bin/xray-linux-<arch></code>	nhị phân Xray-core (trên armv5/armv6/armv7 được đổi tên thành <code>xray-linux-arm</code>)
<code>/usr/bin/x-ui</code>	script quản lý (lệnh <code>x-ui</code>)
<code>/etc/x-ui/x-ui.db</code>	file cơ sở dữ liệu SQLite (mặc định)
<code>/var/log/x-ui/</code>	thư mục log bảng điều khiển
<code>/etc/systemd/system/x-ui.service</code>	systemd unit dịch vụ (không dùng cho Alpine)
<code>/etc/init.d/x-ui</code>	script init OpenRC (chỉ dành cho Alpine)
<code>/etc/default/x-ui</code> · <code>/etc/conf.d/x-ui</code> · <code>/etc/sysconfig/x-ui</code>	file biên môi trường dịch vụ (đường dẫn phụ thuộc vào bản phân phối); nơi ghi <code>XUI_DB_TYPE</code> / <code>XUI_DB_DSN</code>

Thư mục cơ sở dữ liệu có thể được ghi đè bằng biên môi trường `XUI_DB_FOLDER` (mặc định `/etc/x-ui`), còn thư mục nhị phân Xray – bằng biên `XUI_BIN_FOLDER` (mặc định `bin` tương đối so với thư mục bảng điều khiển). Tên file cơ sở dữ liệu là `x-ui.db`.

Ví dụ: chuyển cơ sở dữ liệu sang ổ đĩa riêng. Để lưu `x-ui.db` không phải trong `/etc/x-ui` mà trên ổ đĩa được mount, ví dụ `/data`, hãy đặt biên trong file môi trường dịch vụ và khởi động lại bảng điều khiển:

```
echo 'XUI_DB_FOLDER=/data/x-ui' >> /etc/default/x-ui
mkdir -p /data/x-ui
systemctl restart x-ui
```

Đường dẫn đầy đủ tới cơ sở dữ liệu sẽ là `/data/x-ui/x-ui.db`.

Các biên môi trường chính

Biên	Mục đích	Mặc định
<code>XUI_DB_TYPE</code>	backend cơ sở dữ liệu: <code>sqlite</code> hoặc <code>postgres</code>	<code>sqlite</code>
<code>XUI_DB_DSN</code>	chuỗi kết nối PostgreSQL (khi <code>XUI_DB_TYPE=postgres</code>)	–
<code>XUI_DB_FOLDER</code>	thư mục file cơ sở dữ liệu SQLite	<code>/etc/x-ui</code>
<code>XUI_INIT_WEB_BASE_PATH</code>	URI đường dẫn ban đầu của bảng điều khiển web (chỉ khi khởi tạo lần đầu)	<code>/</code>

Biến	Mục đích	Mặc định
XUI_DB_MAX_OPEN_CONNS	số kết nối tối đa (pool PostgreSQL)	—
XUI_DB_MAX_IDLE_CONNS	số kết nối nhàn rỗi tối đa (pool PostgreSQL)	—
XUI_ENABLE_FAIL2BAN	bật áp dụng giới hạn IP qua Fail2ban	true
XUI_LOG_LEVEL	mức độ ghi log (debug , info , warning , error)	info
XUI_DEBUG	chế độ gỡ lỗi	false

Ví dụ: tạm thời bật ghi log chi tiết. Để chẩn đoán sự cố, nâng mức log lên `debug` và khởi động lại dịch vụ:

```
echo 'XUI_LOG_LEVEL=debug' >> /etc/default/x-ui
systemctl restart x-ui
x-ui log      # xem log gỡ lỗi
```

Sau khi chẩn đoán xong, hãy trả lại giá trị `info` để log không phình to.

Đường dẫn ban đầu của bảng điều khiển web qua môi trường. Biến `XUI_INIT_WEB_BASE_PATH` đặt URI đường dẫn bảng điều khiển web (`webBasePath`) khi khởi tạo cài đặt lần đầu. Điều này tiện lợi khi triển khai trong Docker hoặc qua systemd để cố định ngay đường dẫn truy cập bảng điều khiển. Giá trị được chuẩn hóa tự động — dấu gạch chéo đầu và cuối được thêm vào nếu cần, còn giá trị rỗng hoặc chỉ gồm khoảng trắng sẽ bị bỏ qua (khi đó áp dụng đường dẫn mặc định `/`). Biến này chỉ ảnh hưởng **đến khởi tạo lần đầu**: nếu cài đặt đã được tạo, đường dẫn thay đổi trong giao diện web hoặc qua mục menu «Reset Web Base Path».

1.6. Lệnh quản lý `x-ui` (menu script)

Sau khi cài đặt, lệnh `x-ui` (chạy với quyền root) mở menu tương tác «3X-UI Panel Management Script». Chọn mục bằng cách nhập số tương ứng (phạm vi 0–27). Nhiều mục cũng có thể dùng dưới dạng lệnh con cho script (xem mục 1.7).

Menu được chia thành các khối theo chủ đề.

Cài đặt và cập nhật

- **1. Install** — cài đặt bảng điều khiển (chạy `install.sh`). Trước khi cài đặt, script kiểm tra xem bảng điều khiển đã được cài chưa.
- **2. Update** — cập nhật tất cả các thành phần x-ui lên phiên bản mới nhất. Dữ liệu không bị mất; sau khi cập nhật, bảng điều khiển tự động khởi động lại. Yêu cầu xác nhận.
- **3. Update Menu** — cập nhật chỉ script quản lý (`x-ui.sh` / lệnh `x-ui`) lên phiên bản hiện tại mà không cài đặt lại bảng điều khiển.
- **4. Legacy Version** — cài đặt phiên bản cụ thể (cũ hơn) của bảng điều khiển. Script yêu cầu nhập số phiên bản (ví dụ: `2.4.0`) và tải xuống bản phát hành tương ứng.
- **5. Uninstall** — gỡ cài đặt hoàn toàn bảng điều khiển **cùng với Xray**. Dịch vụ được dừng và vô hiệu hóa, các thư mục `/etc/x-ui/` và `/usr/local/x-ui/` , file môi trường dịch vụ và script quản lý bị xóa. Yêu cầu xác nhận (mặc định «không»).

Thông tin đăng nhập và cài đặt

- **6. Reset Username & Password** – đặt lại tên người dùng và mật khẩu bảng điều khiển. Có thể nhập giá trị tùy chỉnh hoặc để trống để tạo ngẫu nhiên (tên ngẫu nhiên – 10 ký tự, mật khẩu ngẫu nhiên – 18 ký tự). Ngoài ra đề xuất tắt xác thực hai yếu tố (2FA) nếu đã được cấu hình. Sau khi đặt lại, bảng điều khiển khởi động lại.
- **7. Reset Web Base Path** – đặt lại đường dẫn web bảng điều khiển: tạo đường dẫn ngẫu nhiên mới (18 ký tự), bảng điều khiển khởi động lại. Dùng khi đường dẫn cũ bị lộ hoặc bị quên.
- **8. Reset Settings** – đặt lại tất cả cài đặt bảng điều khiển về giá trị mặc định. **Thông tin đăng nhập (tên người dùng và mật khẩu) và dữ liệu tài khoản không bị mất.** Yêu cầu xác nhận; sau khi đặt lại, bảng điều khiển khởi động lại.
- **9. Change Port** – thay đổi cổng bảng điều khiển web. Yêu cầu nhập số cổng (1–65535); sau khi đặt cần khởi động lại để cổng có hiệu lực.
- **10. View Current Settings** – xem cài đặt hiện tại (`x-ui setting -show`). Hiển thị backend cơ sở dữ liệu đang dùng (SQLite hoặc PostgreSQL với mật khẩu được che trong DSN) và liên kết truy cập (Access URL). Nếu SSL chưa được cấu hình, đề xuất cấp chứng chỉ Let's Encrypt cho địa chỉ IP.

Quản lý dịch vụ

- **11. Start** – khởi động dịch vụ bảng điều khiển. Nếu bảng điều khiển đang chạy, hiển thị thông báo rằng không cần khởi động lại.
- **12. Stop** – dừng dịch vụ bảng điều khiển.
- **13. Restart** – khởi động lại dịch vụ bảng điều khiển.
- **14. Restart Xray** – khởi động lại chỉ nhân Xray-core mà không khởi động lại bảng điều khiển (qua `systemctl reload x-ui`, trong Docker – bằng tín hiệu `USR1` tới tiến trình bảng điều khiển).
- **15. Check Status** – kiểm tra trạng thái dịch vụ (`systemctl status x-ui` hoặc `rc-service x-ui status`).
- **16. Logs Management** – quản lý log: xem log gỡ lỗi (Debug Log, qua `journalctl`) và, ngoại trừ Alpine, xóa tất cả log (Clear All logs).

Tự khởi động

- **17. Enable Autostart** – bật tự khởi động bảng điều khiển khi hệ điều hành khởi động (`systemctl enable x-ui` hoặc `rc-update add`).
- **18. Disable Autostart** – tắt tự khởi động khi hệ điều hành khởi động.

Trong Docker, tự khởi động được kiểm soát bởi chính sách khởi động lại container, vì vậy các mục này chỉ hiển thị gợi ý tương ứng.

Bảo mật và mạng

- **19. SSL Certificate Management** – quản lý chứng chỉ SSL qua acme.sh: cấp chứng chỉ cho tên miền, thu hồi, gia hạn bắt buộc, xem các tên miền hiện có, chỉ định đường dẫn chứng chỉ cho bảng điều khiển, cũng như cấp chứng chỉ ngắn hạn (~6 ngày, tự động gia hạn) cho địa chỉ IP.
- **20. Cloudflare SSL Certificate** – cấp chứng chỉ SSL qua xác thực DNS Cloudflare.
- **21. IP Limit Management** – quản lý giới hạn số IP theo client (dựa trên Fail2ban): xem và gỡ chặn, v.v.

- **22. Firewall Management** – quản lý tường lửa (mở/đóng cổng và xem các quy tắc).
- **23. SSH Port Forwarding Management** – cấu hình chuyển tiếp cổng SSH để truy cập bảng điều khiển từ máy cục bộ qua tunnel SSH.

Hiệu suất và bảo trì

- **24. Enable BBR** – bật/tắt thuật toán kiểm soát tắc nghẽn TCP BBR (submenu với các mục Enable BBR / Disable BBR).
- **25. Update Geo Files** – cập nhật cơ sở dữ liệu geo (file `.dat`) với lựa chọn nguồn: Loyalsoldier (`geoip.dat`, `geosite.dat`), chocolate4u (`geoip_IR.dat`, `geosite_IR.dat`), runetfreedom (`geoip_RU.dat`, `geosite_RU.dat`) hoặc All (tất cả cùng lúc). Sau khi cập nhật, bảng điều khiển khởi động lại.
- **26. Speedtest by Ookla** – chạy kiểm tra tốc độ mạng qua Speedtest by Ookla.
- **27. PostgreSQL Management** – quản lý phiên bản PostgreSQL tích hợp/liên kết (kích hoạt và các thao tác liên quan).
- **0. Exit Script** – thoát khỏi menu.

1.7. Lệnh con `x-ui` (không có menu tương tác)

Để sử dụng trong script, lệnh `x-ui` hỗ trợ các lệnh con trực tiếp (chạy `x-ui` không có đối số sẽ mở menu):

Lệnh	Hành động
<code>x-ui</code>	mở menu quản lý
<code>x-ui start</code>	khởi động bảng điều khiển
<code>x-ui stop</code>	dừng bảng điều khiển
<code>x-ui restart</code>	khởi động lại bảng điều khiển
<code>x-ui restart-xray</code>	khởi động lại Xray
<code>x-ui status</code>	trạng thái dịch vụ hiện tại
<code>x-ui settings</code>	cài đặt hiện tại
<code>x-ui enable</code>	bật tự khởi động khi hệ điều hành khởi động
<code>x-ui disable</code>	tắt tự khởi động
<code>x-ui log</code>	xem log
<code>x-ui banlog</code>	xem log chặn Fail2ban
<code>x-ui update</code>	cập nhật bảng điều khiển
<code>x-ui update-all-geofiles</code>	cập nhật tất cả file geo
<code>x-ui migrateDB [file]</code>	chuyển đổi <code>.db</code> <code>?</code> <code>.dump</code> (SQLite)
<code>x-ui legacy</code>	cài đặt phiên bản cũ
<code>x-ui install</code>	cài đặt bảng điều khiển

Lệnh	Hành động
<code>x-ui uninstall</code>	gỡ cài đặt bảng điều khiển

1.8. Di chuyển SQLite → PostgreSQL

Có thể chuyển cài đặt hiện tại từ SQLite sang PostgreSQL:

```
x-ui migrate-db --dsn "postgres://xui:password@127.0.0.1:5432/xui?sslmode=disable"
# sau đó đặt XUI_DB_TYPE và XUI_DB_DSN trong /etc/default/x-ui và khởi động lại:
systemctl restart x-ui
```

File SQLite gốc vẫn được giữ nguyên – chỉ xóa thủ công sau khi đã kiểm tra backend mới hoạt động ổn định.

Ví dụ: kiểm tra việc chuyển sang PostgreSQL. Sau khi di chuyển, hãy đảm bảo bảng điều khiển thực sự đang chạy trên backend mới bằng lệnh xem cài đặt – trong kết quả đầu ra phải thấy PostgreSQL (mật khẩu trong DSN được che):

```
x-ui settings | grep -i -E 'db|dsn'
```

Nếu bảng điều khiển mở được và các tài khoản còn đó, có thể xóa file `x-ui.db` gốc.

2. Đăng nhập vào bảng điều khiển và bảo mật truy cập

Phần này mô tả tất cả những gì liên quan đến xác thực quản trị viên của bảng điều khiển 3X-UI: biểu mẫu đăng nhập, xác thực hai yếu tố (TOTP), bảo vệ chống dò mật khẩu, thay đổi thông tin đăng nhập, thay đổi đường dẫn bí mật và cổng bảng điều khiển, thời gian tồn tại phiên, cũng như đồng bộ hóa/xác thực qua LDAP.

2.1. Biểu mẫu đăng nhập

Trang đăng nhập được phục vụ tại gốc của đường dẫn bí mật (`webBasePath`). Nếu người dùng đã đăng nhập, họ sẽ tự động được chuyển hướng đến `.../panel/` . Trang có công tắc chủ đề, lựa chọn ngôn ngữ giao diện và chính biểu mẫu đăng nhập.

Các trường biểu mẫu:

Trường	Gợi ý/tiêu đề	Bắt buộc	Mô tả
Tên đăng nhập	«Имя пользователя»	Có	Tên đăng nhập của quản trị viên. Giá trị trống bị từ chối ngay ở phía máy khách, còn trên máy chủ – bằng thông báo «Введите имя пользователя».
Mật khẩu	«Пароль»	Có	Mật khẩu quản trị viên. Giá trị trống bị từ chối bằng thông báo «Введите пароль».
Mã 2FA	«Код 2FA»	Chỉ khi bật 2FA	Trường này xuất hiện chỉ khi bảng điều khiển đã bật xác thực hai yếu tố. Mã 6 chữ số từ ứng dụng xác thực.

Nút «**Войти**» gửi biểu mẫu đến `POST /login` .

Hành vi và thông báo:

- Khi đăng nhập thành công, hiển thị «Вход выполнен успешно» và chuyển hướng đến `.../panel/` .
- Với bất kỳ lỗi thông tin đăng nhập nào hoặc mã 2FA sai, máy chủ trả về thông báo **duy nhất**: «Неверные данные учетной записи.» (tiếng Anh: *Invalid username or password or two-factor code.*). Điều này được thực hiện có chủ ý – bảng điều khiển không gợi ý cụ thể điều gì sai (tên đăng nhập, mật khẩu hay mã), để không tạo điều kiện cho việc dò mật khẩu.
- Trường «Код 2FA» được bảng điều khiển hiện hoặc ẩn dựa trên yêu cầu `POST /getTwoFactorEnable` , trả về trạng thái 2FA hiện tại ngay cả trước khi xác thực.
- Nếu phiên máy chủ hết hạn, ở yêu cầu tiếp theo sẽ hiển thị «Сессия истекла. Войдите в систему снова» và người dùng được chuyển hướng đến trang đăng nhập.

Ghi chú về CSRF: trước khi gửi biểu mẫu, máy khách lấy CSRF token (`GET /csrf-token`); các yêu cầu `/login` và `/logout` được bảo vệ bằng kiểm tra CSRF.

Ví dụ: đăng nhập qua API. Khi 2FA tắt, chỉ cần tên đăng nhập và mật khẩu; khi 2FA bật, thêm trường `twoFactorCode` :

```
# Без 2FA
curl -i -X POST https://panel.example.com:2053/мой-секрет/login \
-H 'Content-Type: application/x-www-form-urlencoded' \
--data 'username=admin&password=ВашПароль'

# С включённой 2FA — добавляется 6-значный код
curl -i -X POST https://panel.example.com:2053/мой-секрет/login \
-H 'Content-Type: application/x-www-form-urlencoded' \
--data 'username=admin&password=ВашПароль&twoFactorCode=123456'
```

Khi thành công, máy chủ sẽ trả về `Set-Cookie` với cookie phiên — đây là cookie cần truyền trong các yêu cầu tiếp theo đến `/panel/api/...`.

2.2. Xác thực hai yếu tố (2FA / TOTP)

2FA trong 3X-UI được triển khai theo chuẩn **TOTP** và tương thích với mọi ứng dụng xác thực (Google Authenticator, Aegis, FreeOTP, v.v.). Các tham số được cố định: thuật toán **SHA1**, 6 chữ số, chu kỳ **30** giây, issuer `3x-ui`, nhãn `Administrator`.

Ví dụ: otpauth URI được mã hóa trong mã QR. Nếu ứng dụng xác thực không quét được camera, có thể thêm token thủ công qua liên kết sau (thay `JBSWY3DPEHPK3PXP` bằng Base32 secret của bạn):

```
otpauth://totp/3x-ui:Administrator?secret=JBSWY3DPEHPK3PXP&issuer=3x-
ui&algorithm=SHA1&digits=6&period=30
```

Các tham số `algorithm=SHA1`, `digits=6`, `period=30` tương ứng với các giá trị cố định của bảng điều khiển — không cần thay đổi chúng.

Cài đặt nằm trong phần **Настройки** → **Учетная запись**, tab «**Двухфакторная аутентификация**».

Phần tử	Văn bản	Mô tả
Công tắc	«Включить 2FA»	Bật/tắt xác thực hai yếu tố.
Mô tả	«Добавляет дополнительный уровень аутентификации для повышения безопасности.»	Gợi ý bên dưới công tắc.

Cách bật 2FA

Khi bật công tắc, bảng điều khiển **tạo cục bộ một secret mới** — chuỗi ngẫu nhiên mã hóa Base32 (bảng chữ cái `A–Z` và `2–7`). Một cửa sổ «Включить двухфакторную аутентификацию» mở ra với hướng dẫn từng bước:

1. «Отсканируйте этот QR-код в приложении для аутентификации или скопируйте токен рядом с QR-кодом и вставьте его в приложение». Bên dưới mã QR hiển thị bản thân secret dạng văn bản — khi nhập vào mã QR, secret được sao chép vào clipboard (hiển thị «Скопировано»).

2. **«Введите код из приложения»** — cần nhập mã 6 chữ số do ứng dụng tạo ra. Mã được kiểm tra **phía trình duyệt**: bảng điều khiển tự tính TOTP hiện tại theo secret vừa tạo và so sánh với mã đã nhập. Nếu mã sai — «Неверный код»; trường chỉ chấp nhận đúng 6 chữ số.

Chỉ sau khi xác nhận thành công, secret và cờ bật mới được lưu lại. Khi lưu, hiển thị «Двухфакторная аутентификация была успешно установлена».

Lưu ý: các thay đổi trong phần cài đặt được áp dụng bằng nút chung **«Сохранить»**, sau đó thường cần khởi động lại bảng điều khiển («Сохраните изменения и перезапустите панель для их применения»). Khi lần đầu bật 2FA, máy chủ còn **vô hiệu hóa tất cả các phiên đang hoạt động** (tăng «login epoch»), vì vậy sau khi áp dụng cài đặt cần đăng nhập lại — lần này đã có mã 2FA.

Cách tắt 2FA

Nhập lại công tắc sẽ mở cửa sổ «Отключить двухфакторную аутентификацию» với gợi ý «Введите код из приложения, чтобы отключить двухфакторную аутентификацию.». Sau khi nhập mã đúng, cờ và secret được xóa, hiển thị «Двухфакторная аутентификация была успешно удалена».

Kiểm tra mã khi đăng nhập

Khi đăng nhập, máy chủ lấy secret đã lưu và so sánh TOTP hiện tại với mã 2FA được gửi. Không khớp được coi là đăng nhập thất bại, nhưng người dùng vẫn thấy thông báo chung «Неверные данные учетной записи.».

Khôi phục quyền truy cập (recovery)

3X-UI **không có** cơ chế «mã khôi phục» riêng biệt. Nếu mất quyền truy cập vào ứng dụng xác thực, không thể khôi phục đăng nhập qua giao diện bảng điều khiển. Cách duy nhất là tắt 2FA trực tiếp trong cơ sở dữ liệu trên máy chủ: đặt lại khóa `twoFactorEnable` thành `false` (và nếu cần xóa `twoFactorToken`) trong bảng cài đặt, sau đó khởi động lại bảng điều khiển. Vì vậy, khi bật 2FA nên lưu secret (Base32 token) ở nơi an toàn.

Ví dụ: tắt khẩn cấp 2FA trên máy chủ. Sau khi truy cập máy chủ qua SSH, dùng bảng điều khiển, đặt lại các khóa trong bảng cài đặt và khởi động lại bảng điều khiển:

```
x-ui stop
sqlite3 /etc/x-ui/x-ui.db "UPDATE settings SET value='false' WHERE
key='twoFactorEnable';"
sqlite3 /etc/x-ui/x-ui.db "UPDATE settings SET value='' WHERE key='twoFactorToken';"
x-ui start
```

Sau đó, đăng nhập chỉ bằng tên đăng nhập và mật khẩu, còn 2FA có thể được thiết lập lại nếu muốn.

Liên quan đến thay đổi thông tin đăng nhập: khi thay đổi tên đăng nhập/mật khẩu (xem 2.4), 2FA **tự động tắt** trên máy chủ để secret cũ không chặn quyền truy cập với thông tin đăng nhập mới.

2.3. Giới hạn số lần đăng nhập (login limiter / bảo vệ chống dò mật khẩu)

Bảng điều khiển có bộ giới hạn đăng nhập thất bại tích hợp sẵn (tương tự fail2ban ở cấp ứng dụng). Các tham số được cố định trong mã nguồn và **không thể cấu hình** qua giao diện:

Tham số	Giá trị	Mục đích
Số lần thất bại tối đa	5	Số lần thử thất bại cho phép trong cửa sổ thời gian.
Cửa sổ tính toán	5 phút	Cửa sổ trượt để tích lũy các lần thất bại (các lần cũ hơn bị loại bỏ).
Khóa (cooldown)	15 phút	Thời gian khóa khóa sau khi vượt ngưỡng.

Cách hoạt động:

- Khóa chặn được xây dựng từ **cặp «IP + tên đăng nhập»** (tên đăng nhập được chuyển thành chữ thường, cắt khoảng trắng). Tức là khóa chặn áp dụng cho cặp cụ thể «địa chỉ + tên người dùng», không phải toàn bộ bảng điều khiển.
- Mỗi lần thử thất bại (tên đăng nhập/mật khẩu sai hoặc mã 2FA sai), bộ đếm tăng lên. Sau **5 lần thất bại trong 5 phút**, khóa bị chặn trong **15 phút**. Trong thời gian bị chặn, mọi thử nghiệm của cặp đó ngay lập tức bị từ chối bằng thông báo «Неверные данные учетной записи.», dù thông tin có đúng.
- **Đăng nhập thành công ngay lập tức đặt lại** bộ đếm và gỡ chặn cặp đó.
- Địa chỉ IP của máy khách được xác định có tính đến proxy tin cậy (xem `trustedProxyCIDRs`): header `X-Real-IP` và `X-Forwarded-For` chỉ được chấp nhận nếu yêu cầu đến từ địa chỉ tin cậy. Nếu không, sử dụng địa chỉ kết nối thực tế, còn nếu không trích xuất được – chuỗi `unknown`.

Tất cả các lần thử đều được ghi nhật ký. Với các lần thất bại, cảnh báo được ghi vào nhật ký máy chủ kèm tên người dùng, IP, lý do và, khi bị chặn, thời gian `blocked_until`. Nếu bật thông báo đăng nhập qua bot Telegram (`tgNotifyLogin` – «Уведомление о входе»), quản trị viên còn nhận được tên người dùng, IP và thời gian của cả các lần thử thành công, thất bại và bị chặn.

Ví dụ: thông báo đăng nhập trong Telegram. Khi `tgNotifyLogin` được bật, sau mỗi lần thử, quản trị viên nhận được tin nhắn dạng như sau:

```
Уведомление о входе
Пользователь: admin
IP: 203.0.113.45
Время: 2026-06-10 14:32:07
Статус: успешно
```

Với cặp «IP + tên đăng nhập» bị chặn, trạng thái sẽ chỉ ra rằng lần thử bị từ chối bởi bộ giới hạn.

2.4. Thay đổi tên đăng nhập và mật khẩu quản trị viên

Phần **Настройки** → **Учетная запись**, tab «**Учетные данные администратора**». Các trường:

Trường	Văn bản	Mô tả
Tên đăng nhập hiện tại	«Текущий логин»	Tên người dùng hiện tại. Phải khớp với tên đăng nhập hiện tại, nếu không thay đổi bị từ chối.

Trường	Văn bản	Mô tả
Mật khẩu hiện tại	«Текущий пароль»	Mật khẩu hiện tại để xác nhận danh tính.
Tên đăng nhập mới	«Новый логин»	Tên người dùng mới. Không được để trống.
Mật khẩu mới	«Новый пароль»	Mật khẩu mới. Không được để trống.

Thay đổi được áp dụng bằng nút «Подтвердить» và gửi đến `POST /panel/setting/updateUser`.

Logic và thông báo từ máy chủ:

- Nếu «Текущий логин» không khớp với thực tế hoặc «Текущий пароль» sai – «Произошла ошибка при изменении учетных данных администратора.» với giải thích «Неверное имя пользователя или пароль».
- Nếu tên đăng nhập mới hoặc mật khẩu mới trống – giải thích «Новое имя пользователя и новый пароль должны быть заполнены».
- Khi thành công – «Вы успешно изменили учетные данные администратора.». Mật khẩu được lưu dưới dạng bcrypt hash.

Ví dụ: thay đổi thông tin đăng nhập qua API. Yêu cầu cần có cookie phiên hợp lệ (lấy khi đăng nhập) và xác nhận tên đăng nhập/mật khẩu hiện tại:

```
curl -X POST https://panel.example.com:2053/мой-секрет/panel/setting/updateUser \
  -b 'session=БАША_СЕССИОННАЯ_КООКИЕ' \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  --data 'oldUsername=admin&oldPassword=СтарыйПароль&newUsername=root&newPassword=НовыйСложныйПароль'
```

Sau khi thành công, phiên hiện tại bị hủy – cần đăng nhập lại với thông tin đăng nhập mới.

Các hiệu ứng quan trọng khi thay đổi thông tin đăng nhập:

- **Tất cả các phiên hiện có đều bị hủy** (tăng bộ đếm `login_epoch` của người dùng), do đó sau khi thay đổi, bảng điều khiển tự động đăng xuất và chuyển hướng đến trang đăng nhập – cần đăng nhập lại.
- Nếu tại thời điểm thay đổi **2FA đang được bật, nó tự động tắt** (cờ và secret được đặt lại). Xác thực hai yếu tố sau khi thay đổi tên đăng nhập/mật khẩu cần được thiết lập lại.

Nếu 2FA đang bật, trước khi gửi biểu mẫu sẽ mở cửa sổ «Изменить учетные данные» với gợi ý «Введите код из приложения, чтобы изменить учетные данные администратора.» – chỉ có thể thay đổi thông tin đăng nhập sau khi xác nhận mã 2FA hiện tại.

2.5. Đường dẫn bí mật (URI path / webBasePath) và cổng bảng điều khiển

Các tham số này nằm trong phần **Настройки** → **Панель** và ảnh hưởng trực tiếp đến khả năng «ẩn» và tính khả dụng của bảng điều khiển. Được áp dụng sau khi lưu và **khởi động lại bảng điều khiển**.

Trường	Văn bản	Giá trị mặc định	Mô tả
Cổng bảng điều khiển	«Порт панели» (<code>panelPort</code>), gọi ý «Порт, на котором работает панель»	2053	Cổng TCP của giao diện web.
URI path	«URI-путь» (<code>panelUrlPath</code>), gọi ý «Должен начинаться с '/' и заканчиваться '/'»	/	Đường dẫn cơ sở bí mật (<code>webBasePath</code>). Bảng điều khiển chỉ có thể truy cập qua đường dẫn này (ví dụ: <code>/мой-секрет/</code>).
Địa chỉ IP để quản lý bảng điều khiển	«IP-адрес для управления панелью» (<code>panelListeningIP</code>), gọi ý «Оставьте пустым для подключения с любого IP»	trống	Địa chỉ mà bảng điều khiển lắng nghe. Trống = tất cả giao diện.
Tên miền bảng điều khiển	«Домен панели» (<code>panelListeningDomain</code>), gọi ý «Оставьте пустым для подключения с любых доменов и IP.»	trống	Giới hạn truy cập theo tên miền (Host).
Đường dẫn đến khóa công khai chứng chỉ bảng điều khiển	<code>publicKeyPath</code> , gọi ý «Введите полный путь, начинающийся с '/'»	trống	Chứng chỉ TLS để truy cập HTTPS vào bảng điều khiển.
Đường dẫn đến khóa riêng tư chứng chỉ bảng điều khiển	<code>privateKeyPath</code> , gọi ý tương tự	trống	Khóa riêng tư TLS.

Hành vi của đường dẫn cơ sở (`webBasePath`):

- Giá trị được chuẩn hóa tự động: nếu không bắt đầu bằng / , ký tự được thêm vào đầu; nếu không kết thúc bằng / , được thêm vào cuối. Tức là đường dẫn thực tế luôn có dạng `/.../` .
- Đường dẫn cơ sở áp dụng cho bản thân bảng điều khiển, cho các asset và cho cookie phiên (cookie chỉ được cấp cho đường dẫn này).

Khuyến nghị bảo mật (phần «Предупреждения безопасности»): bảng điều khiển tự hiển thị cảnh báo nếu cấu hình «quá công khai»:

- «Панель работает по обычному HTTP — настройте TLS для продакшна.»
- «Стандартный порт 2053 широко известен — измените его на случайный.»
- «Базовый путь по умолчанию "/" широко известен — измените его на случайный.»

Nói cách khác, đối với máy chủ thực tế, cần đặt **cổng không chuẩn**, **URI path không tầm thường** và **chứng chỉ TLS**.

Ví dụ: cấu hình bảng điều khiển «ẩn» cho môi trường sản xuất. Trong phần **Настройки** → **Панель**, đặt các giá trị xấp xỉ như sau:

Trường	Giá trị
Cổng bảng điều khiển	34571 (ngẫu nhiên, thay vì 2053)
URI path	/aXf9Qm2/ (không tầm thường, bắt đầu và kết thúc bằng /)
Đường dẫn đến khóa công khai chứng chỉ bảng điều khiển	/etc/letsencrypt/live/panel.example.com/fullchain.pem
Đường dẫn đến khóa riêng tư chứng chỉ bảng điều khiển	/etc/letsencrypt/live/panel.example.com/privkey.pem

Sau khi lưu và khởi động lại, bảng điều khiển chỉ có thể truy cập qua `https://panel.example.com:34571/aXf9Qm2/`, và các cảnh báo bảo mật sẽ biến mất.

2.6. Thời gian tồn tại phiên (timeout)

Trường «Продолжительность сессии» (`sessionMaxAge`) nằm trong phần cài đặt bảng điều khiển/khoảng thời gian.

Trường	Văn bản	Giá trị mặc định	Đơn vị	Mô tả
Thời gian tồn tại phiên	«Продолжительность сессии», gợi ý «Продолжительность сессии в системе (значение: минута)»	360	phút	Thời gian tồn tại của cookie phiên quản trị viên.

Hành vi:

- Giá trị được đặt bằng **phút** (mặc định 360 phút = 6 giờ) và khi cấu hình cookie được chuyển đổi sang giây.
- Nếu giá trị **lớn hơn 0**, cookie phiên được đặt `MaxAge` tương ứng. Sau khi hết thời hạn này, cookie ngừng hoạt động và ở yêu cầu tiếp theo, người dùng nhận được «Сессия истекла. Войдите в систему снова».
- Phiên cũng trở nên không hợp lệ sớm hơn khi thay đổi thông tin đăng nhập hoặc lần đầu bật 2FA (thông qua cơ chế `login_epoch`, xem 2.4 và 2.2) và khi đăng xuất tường minh (`POST /logout`).
- Cookie phiên được đánh dấu `HttpOnly`, với chính sách `SameSite=Lax`; cờ `Secure` được đặt khi truy cập HTTPS trực tiếp vào bảng điều khiển.

Ngoài chính timeout còn có thông báo liên quan: «Задержка уведомления об истечении сессии» (`expireTimeDiff`, gợi ý «Получение уведомления об истечении срока действия сессии до достижения порогового значения (значение: день)», mặc định `0`) – cho phép nhận cảnh báo trước.

2.7. LDAP (đồng bộ hóa và xác thực)

Phần LDAP cung cấp hai khả năng: (1) xác thực đăng nhập quản trị viên qua LDAP nếu mật khẩu cục bộ không khớp, và (2) định kỳ đồng bộ hóa trạng thái của các client (bật/tắt cờ VLESS) từ thư mục.

Cách sử dụng khi đăng nhập: máy chủ trước tiên kiểm tra bcrypt hash mật khẩu cục bộ. Nếu **không khớp** và LDAP được bật, bảng điều khiển cố gắng xác thực người dùng trong thư mục: với `Bind DN` được đặt, thực

hiện bind dịch vụ, sau đó theo bộ lọc và thuộc tính tìm kiếm bản ghi người dùng, và thử bind dưới DN tìm thấy với mật khẩu đã nhập. Thành công có nghĩa là đăng nhập. (Sau khi xác thực LDAP thành công, nếu 2FA được bật, mã TOTP vẫn được kiểm tra.)

Các trường trong phần:

Trường	Văn bản	Giá trị mặc định	Mô tả
Bật đồng bộ hóa LDAP	«Включить LDAP-синхронизацию» (<code>enable</code>)	<code>false</code>	Công tắc chính của tích hợp LDAP.
LDAP host	«LDAP-хост» (<code>host</code>)	trống	Địa chỉ máy chủ LDAP.
Cổng LDAP	«Порт LDAP» (<code>port</code>)	<code>389</code>	Cổng. Đối với LDAPS thường là 636.
Sử dụng TLS (LDAPS)	«Использовать TLS (LDAPS)» (<code>useTls</code>)	<code>false</code>	Khi bật, sử dụng scheme <code>ldaps://</code> với kiểm tra chứng chỉ máy chủ (không bỏ qua kiểm tra).
Bind DN	«Bind DN» (<code>bindDn</code>)	trống	DN của tài khoản dịch vụ để bind/tìm kiếm ban đầu. Nếu trống – không thực hiện bind (tìm kiếm ẩn danh).
Mật khẩu bind	gợi ý: «Настроено; оставьте пустым, чтобы сохранить текущий пароль.» / «Не настроено.» / «Настроено – введите новое значение для замены»	trống	Mật khẩu cho <code>Bind DN</code> . Được lưu riêng; để giữ nguyên mật khẩu cũ, để trống trường này.
Base DN	«Base DN» (<code>baseDn</code>)	trống	Gốc của cây con nơi thực hiện tìm kiếm (tìm kiếm đệ quy, toàn bộ cây con).
Bộ lọc người dùng	«Фильтр пользователя» (<code>userFilter</code>)	(<code>objectClass=person</code>)	Bộ lọc LDAP để chọn tài khoản. Khi xác thực, tên đăng nhập được thay vào bộ lọc có escaping.
Thuộc tính người dùng (username/email)	«Атрибут пользователя (username/email)» (<code>userAttr</code>)	<code>mail</code>	Thuộc tính được so khớp với tên đăng nhập/định danh client (ví dụ: <code>mail</code> hoặc <code>uid</code>).
Thuộc tính cờ VLESS	«Атрибут VLESS-флага» (<code>vlessField</code>)	<code>vless_enabled</code>	Thuộc tính xác định xem quyền truy cập VLESS của client có nên được bật hay không.
Thuộc tính cờ chung (tùy chọn)	«Общий атрибут флага (опц.)» (<code>flagField</code>), gợi ý «Если задано, переопределяет флаг VLESS – напр. <code>shadowInactive</code> .»	trống	Nếu được đặt, dùng thay cho <code>vless_enabled</code> .
Giá trị truthy	«Truthy-значения» (<code>truthyValues</code>), gợi ý «Через запятую; по умолчанию: <code>true,1,yes,on</code> »	<code>true,1,yes,on</code>	Danh sách giá trị của thuộc tính cờ được coi là «bật».

Trường	Văn bản	Giá trị mặc định	Mô tả
Đảo ngược cờ	«Инвертировать флаг» (<code>invertFlag</code>), gợi ý «Включите, когда атрибут означает «отключено» (напр. <code>shadowInactive</code>).»	<code>false</code>	Đảo ngược ý nghĩa của cờ.
Lịch đồng bộ hóa	«Расписание синхронизации» (<code>syncSchedule</code>), gợi ý «Строка типа cron, напр. <code>@every 1m</code> »	<code>@every 1m</code>	Tần suất đồng bộ hóa theo định dạng cron.
Tag các inbound	«Теги входящих» (<code>inboundTags</code>), gợi ý «Входящие, на которых LDAP-синхронизация может авто-создавать или авто-удалять клиентов.»	trống	Giới hạn các inbound nào được phép thực hiện thao tác tự động. Nếu không có inbound: «Входящие не найдены. Сначала создайте входящий.»
Tự động tạo client	«Авто-создание клиентов» (<code>autoCreate</code>)	<code>false</code>	Tạo client trong các inbound đã chỉ định nếu client xuất hiện trong thư mục.
Tự động xóa client	«Авто-удаление клиентов» (<code>autoDelete</code>)	<code>false</code>	Xóa client nếu client biến mất khỏi thư mục.
Dung lượng mặc định (GB)	«Объём по умолчанию (ГБ)» (<code>defaultTotalGb</code>)	<code>0</code>	Giới hạn lưu lượng cho các client được tạo tự động (0 = không giới hạn).
Thời hạn mặc định (ngày)	«Срок по умолчанию (дни)» (<code>defaultExpiryDays</code>)	<code>0</code>	Thời hạn cho các client được tạo tự động (0 = vĩnh viễn).
Giới hạn IP mặc định	«Лимит IP по умолчанию» (<code>defaultIpLimit</code>)	<code>0</code>	Giới hạn số lượng IP đồng thời (0 = không giới hạn).

Đặc điểm logic của cờ đồng bộ hóa: khi đọc thuộc tính cờ (`flagField` , mặc định `vless_enabled`), giá trị được coi là «bật» nếu nó nằm trong danh sách giá trị `truthy`; khi bật đảo ngược, kết quả bị đảo ngược. Thuộc tính người dùng (`userAttr`) được dùng làm khóa so khớp (email/tên) – các bản ghi không có giá trị của thuộc tính này bị bỏ qua.

Bảo mật: khuyến nghị bật **TLS (LDAPS)** để mật khẩu bind và mật khẩu được kiểm tra không bị truyền dưới dạng văn bản rõ, còn với Bind DN nên sử dụng tài khoản có quyền đọc tối thiểu cần thiết.

Ví dụ: cấu hình LDAP điển hình để đồng bộ hóa (Active Directory). Điền các trường trong phần cho một thư mục nơi trạng thái truy cập được lưu trong thuộc tính tương tự cờ `userAccountControl` , và so khớp theo email:

Trường	Giá trị
LDAP host	<code>ldap.example.com</code>
Cổng LDAP	<code>636</code>
Sử dụng TLS (LDAPS)	bật

Trường	Giá trị
Bind DN	CN=svc-3xui,OU=Service,DC=example,DC=com
Base DN	OU=Users,DC=example,DC=com
Bộ lọc người dùng	(objectClass=person)
Thuộc tính người dùng (username/email)	mail
Thuộc tính cờ VLESS	vless_enabled
Giá trị truthy	true,1,yes,on
Lịch đồng bộ hóa	@every 5m

Với cấu hình này, cứ mỗi 5 phút, bảng điều khiển sẽ duyệt qua cây con `OU=Users`, so khớp client theo `mail` và bật/tắt quyền truy cập VLESS theo giá trị `vless_enabled`.

3. Tổng quan / Bảng điều khiển

Bảng điều khiển ("Дашборд", trong giao diện tiếng Anh – *Overview*) là trang khởi đầu của panel. Trang này hiển thị trạng thái máy chủ và tiến trình Xray theo thời gian thực. Tất cả các chỉ số đều đến từ phía máy chủ. Bộ lập lịch nền tái tạo ảnh chụp trạng thái **mỗi 2 giây** và phát đến tất cả các tab đang mở qua WebSocket; mỗi phút một lần, các hàng dữ liệu tích lũy được ghi xuống đĩa. HTTP endpoint `GET /status` trả về ảnh chụp được lưu trong bộ nhớ đệm gần nhất.

Dưới đây là phân tích từng chỉ số và từng thành phần điều khiển trên trang.

3.1. Nguyên tắc chung thu thập dữ liệu

- Ảnh chụp được thu thập bằng thư viện `gopsutil`. Nếu một phép đo cụ thể thất bại, trường đó được để bằng không và một cảnh báo được ghi vào log (`get cpu percent failed`, `get uptime failed`, v.v.) – điều này không làm sập toàn bộ bảng điều khiển, chỉ ô tương ứng sẽ hiển thị 0/N-A.
- Tốc độ "tức thời" (CPU %, mạng, disk I/O) được tính như hiệu giữa ảnh chụp hiện tại và ảnh chụp trước, chia cho khoảng thời gian tính bằng giây. Vì vậy khi lần đầu tải trang, các giá trị tốc độ có thể bằng không cho đến khi phép đo thứ hai tích lũy được.
- Lịch sử có thể xem trong mục "История системы" (*System History*) – các biểu đồ được xây dựng từ cùng các hàng dữ liệu được mô tả bên dưới (xem mục 3.12).

3.2. CPU

Ô "ЦП" (CPU) hiển thị mức sử dụng bộ xử lý hiện tại theo phần trăm, cùng các thông số của bộ xử lý.

Chỉ số	Mô tả
Mức sử dụng CPU, %	Tỉ lệ thời gian xử lý đã sử dụng trong khoảng thời gian vừa qua. Được làm mịn bằng trung bình mũ (EMA, hệ số $\alpha = 0.3$) để các đột biến không làm rung chỉ số. Giá trị luôn nằm trong phạm vi 0–100 %. Ở lần đo đầu tiên trả về 0 (khởi tạo điểm cơ sở).
Bộ xử lý logic	Số lõi logic – tức là kể cả Hyper-Threading.
Lõi vật lý	Số lõi vật lý.
Tần số	Tần số cơ sở của bộ xử lý tính bằng MHz. Được truy vấn lười và lưu vào bộ nhớ đệm: phép đo thành công đầu tiên được lưu lại, lần thử lại không thường xuyên hơn 5 phút một lần, và bản thân yêu cầu bị giới hạn bởi timeout 1,5 giây (trên một số hệ thống, việc truy vấn tần số phản hồi chậm).

Mức sử dụng CPU được tính theo thuật toán như sau: nếu có triển khai nền tảng native, nó sẽ được sử dụng, nếu không – tính theo delta của các bộ đếm thời gian xử lý (busy / total). Thời gian Guest và GuestNice bị loại trừ để không tính hai lần.

3.3. Bộ nhớ (RAM)

Ô "Память" (RAM) hiển thị đã dùng và tổng cộng. Được hiển thị dưới dạng "đã dùng / tổng" và/hoặc phần trăm đã lấp đầy. Phần trăm được ghi vào lịch sử.

3.4. Bộ nhớ đệm Swap

Ô "Подкачка" (*Swap*) hiển thị đã dùng và tổng cộng. Nếu file/phần vùng swap chưa được cấu hình (tổng = 0), chỉ số bằng không; khi không có swap, giá trị 0 được ghi vào hàng lịch sử.

3.5. Đĩa (Storage)

Ô "Диск" (*Storage*) hiển thị đã dùng và tổng cộng, trong đó **chỉ tính phần vùng gốc** /. Phần trăm đã lấp đầy được ghi vào lịch sử "Использование диска" (*Disk Usage*). Riêng disk I/O (đọc / ghi, byte/s) được thu thập như delta của các bộ đếm theo khoảng thời gian – nó được hiển thị trên tab "Диск I/O" của lịch sử.

3.6. Thời gian hoạt động hệ thống (Uptime)

Chỉ số "Время работы системы" (*Uptime*). Đây là thời gian tính từ khi khởi động **toàn bộ máy chủ** (tính bằng giây), không phải thời gian hoạt động của panel hay Xray. Thời gian hoạt động của tiến trình Xray được lưu riêng (xem mục 3.9), cũng như số luồng của panel ("Потоки" / *Threads*).

3.7. Tải hệ thống (Load average)

Khối "Нагрузка на систему" (*System Load*) – mảng ba số [Load1, Load5, Load15]. Chú thích gợi ý: "Средняя загрузка системы за последние 1, 5 и 15 минут" (*System load average for the past 1, 5, and 15 minutes*). Biểu đồ lịch sử có tên "Средняя нагрузка системы (1 / 5 / 15 мин)". Các giá trị được ghi riêng vào hàng lịch sử: load1, load5, load15.

Đây là chỉ số Unix tiêu chuẩn: số trung bình các tiến trình đang chờ trong hàng đợi thực thi. Mốc tham chiếu – so sánh với số lỗi: tải trọng liên tục vượt quá số lỗi vật lý cho thấy hệ thống đang bị quá tải.

3.8. Mạng: tốc độ và tổng lưu lượng

Chỉ **các giao diện vật lý** mới được tính. Các giao diện ảo và tunnel bị loại trừ: đó là lo / lo0, cũng như tất cả những gì bắt đầu bằng loopback, docker, br-, veth, virbr, tun, tap, wg, tailscale, zt. Các giá trị được tổng hợp trên tất cả các giao diện còn lại.

Tốc độ tổng thể (*Overall Speed*) – tốc độ tức thời, delta của các bộ đếm theo khoảng thời gian:

Chỉ số	Mô tả
Tải lên / gửi đi (nhãn "Загрузка" / <i>Upload</i>)	Tốc độ đi ra, byte/s.
Tải xuống / nhận vào (nhãn "Скачать" / <i>Download</i>)	Tốc độ đi vào, byte/s.

Tổng lưu lượng dữ liệu (*Total Data*) – các bộ đếm tích lũy từ khi khởi động hệ thống:

Chỉ số	Mô tả
Đã gửi (nhãn "Отправлено" / <i>Sent</i>)	Tổng số byte đã gửi.
Đã nhận (nhãn "Получено" / <i>Received</i>)	Tổng số byte đã nhận.

Ngoài ra, tốc độ gói tin (gói/s) và tổng bộ đếm gói tin cũng được thu thập – chúng hiển thị trên tab "Сетевые пакеты" (*Network Packets*) của lịch sử. Hàng lịch sử mạng: netUp , netDown , pktUp , pktDown .

3.9. Địa chỉ IP máy chủ

Khối "IP-адреса сервера" (*IP Addresses*) hiển thị IPv4 và IPv6 . Địa chỉ bên ngoài được xác định qua các dịch vụ bên thứ ba (api4.ipify.org , ipv4.icanhazip.com , v4.api.ipinfo.io/ip , ipv4.myexternalip.com/raw , 4.ident.me , check-host.net/ip cho IPv4 và tương tự cho IPv6). Danh sách được duyệt tuần tự cho đến khi có phản hồi thành công; timeout cho mỗi yêu cầu – 3 giây.

Đặc điểm:

- Kết quả được **lưu vào bộ nhớ đệm** trong suốt vòng đời tiến trình: địa chỉ đã xác định thành công sẽ không bị yêu cầu lại.
- Nếu không có dịch vụ nào phản hồi, trường đó hiển thị N/A . Đối với IPv6, khi lần đầu nhận được N/A , các yêu cầu IPv6 sẽ bị tắt hoàn toàn để không lãng phí thời gian trên các mạng không có IPv6.
- Bên cạnh có nút "mắt" để ẩn/hiện địa chỉ – gợi ý "Скрыть или показать IP-адреса сервера" (*Toggle visibility of the IP*). Đây chỉ là ẩn trực quan trong giao diện (ví dụ cho ảnh chụp màn hình), không ảnh hưởng đến bản thân các địa chỉ.

3.10. Kết nối TCP/UDP

Khối "Количество соединений" (*Connection Stats*) hiển thị tổng số kết nối TCP và UDP đang hoạt động trên máy chủ (toàn hệ thống, không chỉ Xray). Biểu đồ lịch sử – "Активные соединения (TCP / UDP)" (*Active Connections*), hàng tcpCount , udpCount .

3.11. Trạng thái Xray và quản lý tiến trình

Thẻ "Xray" hiển thị trạng thái tiến trình Xray-core và cho phép quản lý nó.

Trạng thái

Giá trị	Nhãn	Dịch	Khi nào được đặt
running	"Запущен"	Running	Tiến trình Xray đang chạy.
stop	"Остановлен"	Stopped	Tiến trình không chạy và không có lỗi khởi động nào được ghi nhận.
error	"Ошибка"	Error	Tiến trình không chạy và có lỗi khởi động được ghi nhận. Nội dung lỗi hiển thị trong cửa sổ bật lên với tiêu đề "Ошибка при запуске Xray" (<i>An error occurred while running Xray</i>).
–	"Неизвестно"	Unknown	Hiển thị khi trạng thái chưa nhận được.

Bên cạnh trạng thái hiển thị **phiên bản Xray**.

Nút điều khiển

- **Dừng (Stop).** Gọi `POST /stopXrayService`. Khi thành công, panel phát trạng thái `stop` qua WebSocket và thông báo "Xray успешно остановлен" (*Xray service has been stopped*), khi lỗi – trạng thái `error` với nội dung lỗi. Lưu ý quan trọng: nếu panel được truy cập *qua* chính Xray, việc dừng Xray có thể ngắt kết nối với panel – khi kết nối trực tiếp với panel thì không có vấn đề này.
- **Khởi động lại (Restart).** Gọi `POST /restartXrayService`. Trước khi thực hiện hiển thị xác nhận "Перезапустить xray?" với giải thích "Перезагружает сервис xray с сохранённой конфигурацией". Khi thành công – trạng thái `running` và thông báo "Xray успешно перезапущен" (*Xray service has been restarted successfully*). Khởi động lại áp dụng cấu hình đã lưu hiện tại – sử dụng sau khi thay đổi cài đặt.

Lưu ý. Trong fork này, trang bảng điều khiển đã được bổ sung quản lý đầy đủ Start / Stop / Restart cho tất cả các loại xác thực; trong UI gốc 3x-ui không có nút "start" riêng – việc khởi động được thực hiện bằng cách khởi động lại.

Nút xem log Xray

Trên thẻ Xray có nút xem log Xray (*Logs*). Nút này chỉ xuất hiện khi access-log được cấu hình trong cấu hình Xray: trình xem tích hợp đọc chính xác file này, vì vậy nếu không có access-log, nút sẽ bị ẩn. Khả năng hiển thị của nút được gắn với thuộc tính riêng `accessLogEnable` và không còn phụ thuộc vào giới hạn IP – danh sách online và giới hạn địa chỉ IP tiếp tục hoạt động ngay cả khi không có access-log (xem mục 8).

Chọn phiên bản Xray

Mục "Выбор версии" (*Version*) cho phép chuyển Xray-core sang bản phát hành khác. Danh sách phiên bản được tải qua `GET /getXrayVersion`:

- Nguồn – GitHub API của repository `XTLS/Xray-core` (`/releases`). Các yêu cầu được lưu trong bộ nhớ đệm **15 phút**; khi GitHub gặp sự cố, danh sách thành công cuối cùng được trả về để picker không bị trống.
- Chỉ các bản phát hành dạng `X.Y.Z` và **không cũ hơn 26.4.25** mới được đưa vào danh sách.

Gợi ý: "Выберите нужную версию" (*Choose the version you want to switch to.*) và cảnh báo "Важно: старые версии могут не поддерживать текущие настройки" (*Choose carefully, as older versions may not be compatible with current configurations.*).

Chuyển đổi: `POST /installXray/:version`. Kịch bản:

Ví dụ. Chuyển sang phiên bản cụ thể của Xray-core (cookie phiên phải đã được lấy qua xác thực):

```
curl -X POST 'https://panel.example.com:2053/xpanel/installXray/v25.6.8' \
  -b cookie.txt
```

Ở đây `v25.6.8` là tag từ danh sách mà `GET /getXrayVersion` trả về. Phiên bản phải có mặt trong danh sách này, nếu không panel sẽ từ chối.

1. Phiên bản được chọn được kiểm tra xem có trong danh sách bản phát hành hiện tại không (nếu không – từ chối).
2. Xray dừng lại.
3. Archive `Xray-<os>-<arch>.zip` cho OS và kiến trúc hiện tại được tải từ GitHub (hỗ trợ amd64/64, arm64-v8a, arm32-v7a/v6/v5, 386/32, s390x; cho Windows – `xray.exe`). Kích thước archive và binary bị giới hạn 200 MB.
4. Binary được thay thế nguyên tử (qua file tạm + đổi tên) và được đánh dấu có thể thực thi.
5. Xray khởi động lại.

Trước khi chuyển đổi hiển thị hộp thoại "Переключить версию Xray" (*Do you really want to change the Xray version?*) với mô tả "Это изменит версию Xray на #version#". Khi thành công – thông báo "Xray успешно обновлён" (*Xray updated successfully*).

3.12. Cập nhật panel (3X-UI)

Khởi kiểm tra cập nhật panel. Dữ liệu đến qua `GET /getPanelUpdateInfo` :

Trường	Mô tả
Phiên bản panel hiện tại	Phiên bản panel đang được cài đặt.
Phiên bản panel mới nhất	Bản phát hành mới nhất của 3x-ui lấy từ GitHub.
Có bản cập nhật	Dấu hiệu cho thấy phiên bản mới nhất mới hơn phiên bản hiện tại. Nếu không cần cập nhật – hiển thị "Панель обновлена" / "Обновлено".

Nút "**Обновить панель**" (*Update Panel*) chạy `POST /updatePanel`. Gợi ý: "Это обновит 3X-UI до последнего релиза и перезапустит сервис панели". Trước khi chạy – xác nhận "Вы действительно хотите обновить панель?" với nội dung "Это обновит 3X-UI до версии #version# и перезапустит сервис панели".

Đặc điểm và hạn chế:

- Tự cập nhật chỉ được hỗ trợ **trên Linux** (trên các OS khác sẽ trả về lỗi).
- Script cập nhật được tải từ repository chính thức (`raw.githubusercontent.com/MHSanaei/3x-ui/main/update.sh`, giới hạn 2 MB) và được chạy qua `bash`, nếu có thể thì được cô lập qua `systemd-run`.
- Khi khởi chạy thành công, hiển thị "Обновление панели началось" (*Panel update started*); nếu kiểm tra cập nhật thất bại – "Проверка обновления панели не удалась". Trong quá trình cài đặt hiển thị cảnh báo "Установка в процессе. Не обновляйте страницу".

3.13. Cập nhật file địa lý (GeoIP / GeoSite)

Nút/hộp thoại cập nhật cơ sở dữ liệu địa lý gọi `POST /updateGeofile` (tất cả các file) hoặc `POST /updateGeofile/:fileName` (một file). Cập nhật hoạt động theo danh sách trắng nghiêm ngặt về tên và nguồn:

File	Nguồn
geoiP.dat , geosite.dat	Loyalsoldier/v2ray-rules-dat (latest)
geoiP_IR.dat , geosite_IR.dat	chocolate4u/Iran-v2ray-rules (latest)
geoiP_RU.dat , geosite_RU.dat	runetfreedom/russia-v2ray-rules-dat (latest)

Hành vi:

- Tên file được xác thực: cấm `..` , dấu gạch chéo, đường dẫn tuyệt đối; chỉ cho phép `[a-zA-Z0-9._-]` + `.dat` . Các file ngoài danh sách trắng sẽ không được tải.
- Sử dụng yêu cầu có điều kiện `If-Modified-Since` : nếu file tại máy chủ nguồn không thay đổi (HTTP 304), nó sẽ không được tải lại, chỉ dấu thời gian được cập nhật.
- Sau khi tải, Xray **khởi động lại** (để nhận các cơ sở dữ liệu mới).

Ví dụ. Cập nhật chỉ các file địa lý Nga mà không ảnh hưởng đến các file khác:

```
curl -X POST 'https://panel.example.com:2053/xpanel/updateGeofile/geoiP_RU.dat' -b cookie.txt
curl -X POST 'https://panel.example.com:2053/xpanel/updateGeofile/geosite_RU.dat' -b cookie.txt
```

Để cập nhật tất cả các file trong danh sách trắng cùng một lúc – gọi `POST /updateGeofile` không có tên file.

- Hộp thoại: "Вы действительно хотите обновить геофайл?" với "Это обновит файл #filename#" cho một file và "Это обновит все геофайлы" cho nút "Обновить все". Thành công – "Геофайлы успешно обновлены".

3.14. Sao lưu và khôi phục cơ sở dữ liệu

Khối "Бэкап и восстановление" (*Backup & Restore*). Hành vi phụ thuộc vào DBMS đang sử dụng (SQLite mặc định hoặc PostgreSQL).

Xuất cơ sở dữ liệu (Sao lưu)

Nút "Экспорт базы данных" / "Резервная копия" (*Back Up*) gọi `GET /getDb` . File được trả về dưới dạng đính kèm:

- **SQLite**: trước tiên thực hiện checkpoint (xả WAL), sau đó tải xuống file `x-ui.db` . Gợi ý: "Нажмите, чтобы скачать файл .db, содержащий резервную копию вашей текущей базы данных...".
- **PostgreSQL**: tải xuống dump `x-ui.dump` ở định dạng tùy chỉnh (`pg_dump --format=custom --no-owner --no-privileges`). Các công cụ client PostgreSQL phải được cài đặt trên máy chủ; nếu không – lỗi về việc thiếu `pg_dump` .

Nhập cơ sở dữ liệu (Khôi phục)

Nút "Импорт базы данных" / "Восстановление" (*Restore*) tải file lên qua `POST /importDB` (trường form `db`).
Gợi ý: "Нажмите, чтобы выбрать и загрузить файл .db... для восстановления базы данных из резервной копии".

Kịch bản cho **SQLite** an toàn, có rollback:

1. File được kiểm tra định dạng SQLite và lưu vào file tạm, sau đó kiểm tra tính toàn vẹn.
2. Xray dừng, DB hiện tại được đóng và đổi tên thành `*.backup` (fallback).
3. File mới được đặt vào vị trí DB làm việc, thực hiện khởi tạo và migration. Nếu có sự cố – fallback được khôi phục.
4. Xray khởi động lại.

Đối với **PostgreSQL** tải lên `.dump` (kiểm tra chữ ký PGDMP) và áp dụng qua `pg_restore --clean --if-exists --single-transaction ...`. Gợi ý trực tiếp cảnh báo: "Это заменит все текущие данные".

Thông báo: "База данных успешно импортирована", "Произошла ошибка при импорте базы данных", "...при чтении базы данных", "...при получении базы данных".

File migration (giữa SQLite và PostgreSQL)

Nút "Скачать файл миграции" (*Download Migration*) gọi `GET /getMigration` và tạo bản xuất di động để chạy panel trên DBMS khác:

- Trên **SQLite** tải xuống `x-ui.dump` (dump SQL dạng văn bản).
- Trên **PostgreSQL** tải xuống `x-ui.db` – cơ sở dữ liệu SQLite sẵn sàng, được xây dựng từ dữ liệu PostgreSQL.

3.15. Các thành phần giao diện bổ sung

- **Chỉ số client trực tuyến.** Bảng điều khiển duy trì hàng `online` (*Online Clients* / "Клиенты онлайн") – số lượng client có kết nối đang hoạt động. Được tính khi Xray đang chạy (nếu không thì 0) và được ghi vào lịch sử theo cùng chu kỳ 2 giây. Biểu đồ – tab "Онлайн".
- **Lịch sử hệ thống (biểu đồ).** Nút/mục "Графики" → "История системы" với các tab: "Пропускная способность", "Пакеты", "Диск I/O", "Онлайн", "Нагрузка", "Соединения", "Использование диска". Dữ liệu được kéo qua `GET /history/:metric/:bucket`; các khoảng tổng hợp hợp lệ (bucket, giây): **2, 30, 60, 120, 180, 300, 720, 1440, 2880** (ba khoảng cuối tương ứng với preset **12h, 24h** và **48h** trong bộ chọn khoảng thời gian), mỗi tab nhận tối đa 60 điểm. Bộ đệm vòng của metric lưu dữ liệu trong thời gian tối đa **48 giờ**, vì vậy biểu đồ (CPU, RAM, lưu lượng, gói tin, kết nối, đĩa, trực tuyến, tải) có thể xem trong khoảng thời gian đến hai ngày. Các metric hợp lệ: `cpu, mem, swap, netUp, netDown, pktUp, pktDown, diskRead, diskWrite, diskUsage, tcpCount, udpCount, online, load1, load5, load15`. Nhân "Последние 2 минуты" tương ứng với bucket = 2 (chế độ thời gian thực).

Ví dụ. Lây chuỗi mức sử dụng CPU trong khoảng ~2 phút gần nhất (bucket = 2 giây, tối đa 60 điểm) và cùng chuỗi đó được tổng hợp theo 5 phút (bucket = 300 giây):

```
curl 'https://panel.example.com:2053/xpanel/history/cpu/2' -b cookie.txt
curl 'https://panel.example.com:2053/xpanel/history/cpu/300' -b cookie.txt
```

Metric có thể thay bằng bất kỳ metric hợp lệ nào (`mem` , `netUp` , `tcpCount` , `load1` , v.v.). Bucket ngoài danh sách `2` , `30` , `60` , `120` , `180` , `300` , `720` , `1440` , `2880` sẽ bị từ chối.

- **Metric Xray** — khối riêng với mức tiêu thụ bộ nhớ và garbage collection của Xray (hàng `xrAlloc` , `xrSys` , `xrHeapObjects` , `xrNumGC` , `xrPauseNs`) và "Observatory" (trạng thái kết nối outbound). Chỉ hoạt động nếu block `metrics` được cấu hình trong cấu hình Xray (`listen 127.0.0.1:11111` , `tag metrics_out`); nếu không hiển thị "Конечная точка метрик Xray не настроена".

Ví dụ block bật ô metric Xray. Trong phần cài đặt Xray phải có đồng thời `metrics` (với `tag`) và inbound lắng nghe `tag` đó:

```
{
  "metrics": {
    "tag": "metrics_out"
  },
  "inbounds": [
    {
      "listen": "127.0.0.1",
      "port": 11111,
      "protocol": "dokodemo-door",
      "settings": { "address": "127.0.0.1" },
      "tag": "metrics_out"
    }
  ]
}
```

Địa chỉ `127.0.0.1:11111` cổ tình không được công khai ra ngoài — panel truy vấn nó cục bộ.

- **Bộ chuyển chủ đề tối.** Nằm trong menu chung/thanh tiêu đề, không phải trong bảng điều khiển. Các tùy chọn: "Тема" (*Theme*) với các lựa chọn "Темная" và "Очень темная" (*Ultra Dark*). Đây là cài đặt giao diện thuần thị giác, không ảnh hưởng đến hoạt động của panel.
- **Các liên kết khác** xung quanh bảng điều khiển (từ menu/thanh dưới): "Логи", "Конфигурация" — xem JSON kết quả của Xray (`GET /getConfigJson`), "Документация".

4. Inbounds: tạo mới và các tham số chung

Mục **«Входящие»** (tiếng Anh: *Inbounds*) – là danh sách tất cả các điểm đầu vào của Xray mà các client kết nối đến. Mỗi inbound lưu trữ cả các trường «bảng điều khiển» (ghi chú, giới hạn lưu lượng, lịch đặt lại) lẫn các khối JSON thô của cấu hình Xray (`settings` , `streamSettings` , `sniffing`).

Việc tạo mới thực hiện qua nút **«Создать подключение»** (*Add Inbound*), chỉnh sửa – qua **«Изменить подключение»** (*Modify Inbound*). Cả hai thao tác gửi yêu cầu đến các API endpoint `POST /add` và `POST /update/:id`.

Dưới đây trình bày tất cả các trường của form **không** liên quan đến cài đặt giao thức cụ thể (client, mã hóa, REALITY/TLS) và **không** liên quan đến transport/stream (các tab **«Поток»**, **«Безопасность»**) – đây là chủ đề của các mục riêng biệt.

4.1. Các trường chung của form

Remark (Ghi chú)

Tham số	Giá trị
Trường	<code>remark</code>
Kiểu	chuỗi
Mặc định	rỗng

Tên dễ đọc của inbound, hiển thị trong danh sách và trong tiêu đề các hộp thoại (**«Удалить подключение "{remark}"?»** v.v.). Nhân trường – **«Примечание»**. Không ảnh hưởng đến hoạt động của Xray, chỉ phục vụ quản trị; nên đặt tên duy nhất và có ý nghĩa vì chúng được dùng trong tên file xuất và trong xác nhận các thao tác hàng loạt.

Protocol (Giao thức)

Tham số	Giá trị
Trường	<code>protocol</code>
Nhãn	«Протокол»
Xác thực	<code>required,oneof=vmess vless trojan shadowsocks wireguard hysteria http mixed tunnel tun</code>

Danh sách thả xuống chọn giao thức inbound. Các giá trị hợp lệ:

Giá trị	Ghi chú
<code>vmess</code>	
<code>vless</code>	

Giá trị	Ghi chú
trojan	
shadowsocks	
wireguard	
hysteria	Hysteria v2 – là hysteria với streamSettings.version = 2 , không có giao thức riêng
http	
mixed	socks/http trên cùng một cổng
tunnel	
tun	được validator chấp nhận, không có hằng số giao thức riêng

Trường bắt buộc (required). Việc chọn giao thức xác định các trường cài đặt client và transport nào sẽ khả dụng (xem các mục theo từng giao thức cụ thể).

Lưu ý quan trọng: khi lưu, dịch vụ chuẩn hóa streamSettings . Cài đặt transport chỉ được giữ lại cho các giao thức vmess , vless , trojan , shadowsocks , hysteria ; với các giao thức còn lại (http , mixed , tunnel , wireguard , tun) trường streamSettings **bị xóa cưỡng bức**.

Đối với inbound kiểu tunnel /TPProxy, mà khối streamSettings không có khóa security (biên thể không transport), form mở và lưu mà không gặp lỗi xác thực streamSettings.security Invalid input .

Listen IP (IP lắng nghe)

Tham số	Giá trị
Trường	listen
Kiểu	chuỗi
Mặc định	rỗng → Xray lắng nghe trên 0.0.0.0 (tất cả IP)

Địa chỉ IP mà inbound nhận kết nối. Gợi ý trường:

«Оставьте пустым для прослушивания всех IP-адресов».

Khi tạo cấu hình Xray, giá trị rỗng được thay bằng 0.0.0.0 . Ngoài IP, trường cũng nhận **đường dẫn Unix socket** – gợi ý:

«Можно также указать путь Unix-сокета (например, /run/xray/in.sock) или имя абстрактного сокета с префиксом @ (например, @xray/in.sock), чтобы слушать сокет вместо TCP-порта – в этом случае задайте порт 0».

Như vậy, trường nhận hai dạng Unix socket: đường dẫn trong hệ thống file (`/run/xray/in.sock`) và tên socket trừu tượng với tiền tố `@` (`@xray/in.sock`). Trong cả hai trường hợp, hãy đặt `Port` bằng `0` .

Trường này được thay đổi khi cần giới hạn inbound chỉ trên một interface (ví dụ: `127.0.0.1` cho inbound chỉ hoạt động như đích fallback sau Nginx) hoặc khi inbound lắng nghe Unix socket.

Ví dụ. Inbound lắng nghe chỉ trên interface cục bộ (đích fallback điển hình sau Nginx) và Unix socket:

```
listen = 127.0.0.1    port = 8443
listen = /run/xray/in.sock    port = 0
```

Port (Cổng)

Tham số	Giá trị
Trường	<code>port</code>
Nhân	«Порт»
Xác thực	<code>gte=0, lte=65535</code>
Mặc định	— (do người dùng đặt)

Cổng TCP/UDP lắng nghe. Giá trị hợp lệ từ `0` đến `65535` . Giá trị `0` chỉ dùng kết hợp với lắng nghe trên Unix socket (xem trên).

Khi lưu, dịch vụ kiểm tra xung đột cổng: hai inbound không thể đồng thời chiếm `listen:port` trùng nhau cho cùng một transport (TCP/UDP). Transport được suy ra từ giao thức và `streamSettings / settings` : ví dụ, `hysteria` và `wireguard` luôn chiếm UDP, `kcp / quic` — UDP, còn phần lớn các giao thức khác — TCP. Khi có xung đột, việc lưu bị từ chối với lỗi.

Ngoài ra, bảng điều khiển không cho phép chiếm **cổng dành riêng của Xray API nội bộ** (tag `api` , mặc định `62789` trên `127.0.0.1`): inbound TCP cục bộ mà địa chỉ lắng nghe trùng với cổng này trên loopback sẽ bị từ chối với cùng lỗi xung đột cổng. Cổng API thực tế được đọc từ template cấu hình Xray (với giá trị dự phòng `62789`). Trên các node, hạn chế này không áp dụng — chúng có Xray riêng.

Tag Xray (Tag , duy nhất) được tạo tự động từ cổng và transport theo định dạng `in-<port>-<tcp|udp|tcpudp|any>` ; đối với inbound triển khai trên node, thêm tiền tố `n<nodeId>-` . Khi trùng, tag được thêm `-2` , `-3` v.v. Người dùng thường không chỉnh sửa tag.

Total traffic (Tổng lưu lượng, GB)

Tham số	Giá trị
Trường	<code>total</code> (tính bằng <code>byte</code>)
Nhân	«Общий расход»
Mặc định	<code>0</code>

Giới hạn lưu lượng tổng của inbound. Trong form giá trị nhập bằng gigabyte, trong cơ sở dữ liệu lưu bằng byte. Gợi ý trường:

«= Безлимит. (единица: ГБ)».

Tức là **0** có nghĩa là không giới hạn. Đây là giới hạn ở cấp toàn bộ inbound (không phải từng client); lưu lượng thực tế đã dùng được lưu trong các trường `up` (đã gửi) và `down` (đã nhận) và so sánh với `total`.

Expiry date / Duration (Ngày hết hạn / thời hạn)

Tham số	Giá trị
Trường	<code>expiryTime</code> (Unix timestamp)
Nhân	«Дата окончания» (tiếng Anh: <i>Duration</i>)
Mặc định	rỗng / 0

Thời hạn hiệu lực của inbound. Gợi ý:

«Оставьте пустым, чтобы было бесконечным».

Giá trị rỗng (**0**) có nghĩa là inbound không có thời hạn. Giá trị được lưu dưới dạng Unix timestamp; form cho phép đặt cả ngày cụ thể lẫn số ngày (đếm tương đối từ thời điểm hiện tại – nhân tiếng Anh *Duration*).

Enabled (Bật)

Tham số	Giá trị
Trường	<code>enable</code>
Nhân	«Включить» (tiếng Anh: <i>Enabled</i>)
Mặc định	đặt khi tạo

Cờ trạng thái hoạt động của inbound. Việc chuyển cờ này trong danh sách được xử lý bởi một endpoint «nhẹ» riêng `POST /setEnabled/:id`, không phải cập nhật đầy đủ – điều này được thực hiện có chủ đích để không phải tuần tự hóa lại toàn bộ khối `settings` (tất cả client) mỗi khi nhân công tắc trên inbound có hàng nghìn client. Khi tắt, inbound bị xóa khỏi Xray đang chạy; khi bật – được thêm lại.

Node / Deploy to (Node / Triển khai lên)

Tham số	Giá trị
Trường	<code>nodeId</code>
Nhân	«Развернуть на», «Локальная панель»

Tham số	Giá trị
Mặc định	rỗng (bảng điều khiển cục bộ)

Chọn nơi inbound hoạt động vật lý: trên bảng điều khiển cục bộ hoặc trên một trong các node đã đăng ký. Đặc điểm thực thi: `nodeId = 0` được chuẩn hóa thành `nil`, vì `0` không phải id node hợp lệ mà là tạo phẩm của binding form; `nil / 0` có nghĩa là bảng điều khiển cục bộ. Khi lưu inbound trên node offline, có thể xuất hiện thông báo «thay đổi sẽ được đồng bộ khi node kết nối lại».

Chiến lược địa chỉ cho liên kết (Share address strategy)

Tham số	Giá trị
Trường	chiên lược + (tùy chọn) địa chỉ tùy chỉnh
Nhãn	«Стратегия адреса для ссылок» (tiếng Anh: <i>Share address strategy</i>)
Mặc định	«Адрес прослушивания inbound» (<i>Inbound listen</i>)

Danh sách thả xuống xác định địa chỉ nào được chèn vào **các liên kết chia sẻ và mã QR xuất ra** của inbound này. Các giá trị:

Giá trị	Nhãn	Nội dung chèn vào
node	«Адрес узла» (<i>Node address</i>)	địa chỉ node mà inbound đang chạy trên đó
listen	«Адрес прослушивания inbound» (<i>Inbound listen</i>)	địa chỉ lắng nghe của chính inbound
custom	«Пользовательская» (<i>Custom</i>)	địa chỉ tùy chỉnh từ trường «Пользовательский адрес для ссылок» (<i>Custom share address</i>)

Khi chọn «Пользовательская», xuất hiện trường «Пользовательский адрес для ссылок»; nhập host hoặc IP **không có scheme và cổng** (giá trị được xác thực). Tùy chọn «Адрес узла» chỉ hiển thị trong danh sách nếu tồn tại node đang bật mà inbound này có thể chạy trên đó; nếu không, nó bị ẩn và giá trị được đặt về «Адрес прослушивания inbound».

Chiến lược này chỉ ảnh hưởng đến các liên kết chia sẻ trực tiếp và mã QR. Nó **không** ảnh hưởng đến đầu ra subscription – ở đó địa chỉ vẫn được xác định theo logic thông thường của bảng điều khiển.

4.2. Sniffing (Nghe lén)

Tab «Сниффинг» chỉnh sửa khối `sniffing` của cấu hình Xray, được lưu dưới dạng JSON thô. Sniffing cho phép Xray «nghe lén» tên miền/giao thức thực sự bên trong kết nối phục vụ mục đích định tuyến.

Trường con	Nhãn	Mục đích
<code>enabled</code>	(công tắc tab)	Bật/tắt sniffing cho inbound
<code>destOverride</code>	—	Danh sách giao thức mà địa chỉ đích bị chặn: <code>http</code> , <code>tls</code> , <code>quic</code> , <code>fakedns</code>

Trường con	Nhân	Mục đích
<code>metadataOnly</code>	«Только метаданные»	Chỉ sử dụng metadata kết nối, không đọc payload
<code>routeOnly</code>	«Только маршрутизация»	Áp dụng kết quả sniffing chỉ cho định tuyến, không ghi đè địa chỉ đích
<code>domainsExcluded</code>	«Исключённые домены»	Các domain bị loại trừ khỏi sniffing
(IP bị loại trừ)	«Исключённые IP»	Các địa chỉ IP bị loại trừ khỏi sniffing

- **`destOverride`** – tập hợp các sniffer: `http` (xác định domain từ HTTP header Host), `tls` (từ SNI), `quic` (từ QUIC ClientHello), `fakedns` (so khớp với pool FakeDNS). Thông thường để xác định domain, bật `http` và `tls`.

Ví dụ khối `sniffing` (xác định domain theo HTTP và TLS, chỉ dùng kết quả cho định tuyến, không động đến mạng cục bộ):

```
{
  "enabled": true,
  "destOverride": ["http", "tls"],
  "routeOnly": true,
  "domainsExcluded": ["courier.push.apple.com"]
}
```

- **`metadataOnly`** – khi bật, Xray không đọc nội dung gói đầu tiên và chỉ dựa vào metadata; hữu ích để không phá vỡ các giao thức mà dữ liệu không thể «nghe lén».
- **`routeOnly`** – kết quả sniffing chỉ được dùng bởi các quy tắc định tuyến; địa chỉ kết nối trong outbound không bị ghi đè bằng domain đã nhận dạng.

Lưu ý: bảng điều khiển lưu `sniffing` như một khối JSON không trong suốt và khi lưu không thêm gì vào đó – tất cả giá trị mặc định cho các ô checkbox này được tạo phía ứng dụng client. Ở dạng thô, khối có thể chỉnh sửa qua mục «JSON входящего» (xem bên dưới).

4.3. Allocate (chiến lược phân bổ cổng)

Khối `allocate` trong `streamSettings` kiểm soát cách Xray phân bổ các cổng lắng nghe. Đây là một phần cấu hình Xray; bảng điều khiển lưu và truyền nó như một phần của `streamSettings` /JSON inbound. Các tham số (theo thuật ngữ Xray-core):

Trường con	Mục đích	Giá trị / mặc định
<code>strategy</code>	Chiến lược phân bổ cổng	<code>always</code> – luôn lắng nghe trên cổng đã chỉ định (mặc định); <code>random</code> – định kỳ thay đổi các cổng lắng nghe trong phạm vi
<code>refresh</code>	Khoảng thời gian thay đổi cổng (phút) khi <code>random</code>	số nguyên phút (khuyến nghị 5; tối thiểu – 2)

Trường con	Mục đích	Giá trị / mặc định
concurrency	Số cổng giữ mở đồng thời khi random	số nguyên (mặc định 3; không quá một phần ba độ rộng dải cổng)

strategy: always giữ inbound trên một cổng (chế độ tiêu chuẩn). strategy: random dùng cho các tình huống chống chặn khi inbound định kỳ «nhảy» trong phạm vi cổng; trong trường hợp này refresh và concurrency có ý nghĩa. Chỉ thay đổi các giá trị này khi cố ý sử dụng chế độ cổng ngẫu nhiên.

Ví dụ khối allocate trong streamSettings (chế độ cổng ngẫu nhiên: giữ 3 cổng mở, thay đổi mỗi 5 phút):

```
{
  "allocate": {
    "strategy": "random",
    "refresh": 5,
    "concurrency": 3
  }
}
```

Để điều này hoạt động, port của inbound được đặt theo dải (ví dụ: 20000–20100).

4.4. External Proxy (Proxy bên ngoài)

Trường **«External Proxy»** thuộc cài đặt tạo liên kết mời và được lưu trong streamSettings của inbound. Nó xác định danh sách các địa chỉ bên ngoài thay thế (host/port, nếu cần với TLS bắt buộc – **«Принудительный TLS»**), được chèn vào liên kết client thay vì listen:port thực tế của inbound.

Được sử dụng khi client cần kết nối không trực tiếp đến server mà thông qua proxy/reverse/CDN bên ngoài: khi đó trong các liên kết chung, địa chỉ công khai của frontend đó được chỉ định. Điều này không ảnh hưởng đến quá trình nhận kết nối của Xray – đây là «trang điểm» của các liên kết được tạo ra. Các trường form liên quan: **«Принудительный TLS»**, **«Fingerprint»**, nhãn của mỗi bản ghi.

4.5. Fallbacks (Các fallback)

Mục **«Fallback'и»** xác định các quy tắc chuyển hướng kết nối không khớp với bất kỳ client nào của inbound. Khả dụng cho master inbound trên TLS transport (VLESS/Trojan TCP-TLS). Được quản lý qua các endpoint GET /:id/fallbacks / POST /:id/fallbacks .

Gợi ý mục:

«Когда соединение на этом инбаунде не совпадает ни с одним клиентом, оно перенаправляется в другое место. Выберите дочерний инбаунд ниже, чтобы поля маршрутизации (SNI / ALPN / Path / xver) заполнились автоматически из его транспорта, либо оставьте выбор пустым и задайте Dest напрямую (например, 8080 или 127.0.0.1:8080), чтобы перенаправить на внешний сервер, такой как Nginx. Каждый дочерний инбаунд должен слушать на 127.0.0.1 с security=none».

Mục fallback chỉ hiển thị cho inbound VLESS/Trojan qua RAW (TCP) với bảo mật TLS hoặc REALITY. Inbound mới khởi động với security=none , do đó mục này ban đầu có thể trông như vắng mặt. Ở trạng thái này

(VLESS/Trojan, RAW/TCP, bảo mật chưa được cấu hình), thay vì mục đó hiển thị gợi ý tích hợp: các fallback sẽ khả dụng sau khi chọn TLS hoặc Reality trong tab «**Безопасность**».

Các trường của dòng fallback

Trường	Mặc định	Mô tả
(inbound con)	—	Chọn inbound con (nhân « Выберите инбаунд »). Nếu được chọn, các trường Name/Alpn/Path/Dest có thể tự điền từ transport của nó
Name	rỗng (= bất kỳ)	Điều kiện khớp theo tên (SNI/tên). Nhân «bất kỳ» — « любой »
Alpn	rỗng	Điều kiện khớp theo ALPN
Path	rỗng	Điều kiện khớp theo đường dẫn (cho transport WS/HTTP của inbound con)
Dest	tự động	Chuyển hướng đến đâu. Placeholder « авто (listen:порт дочернего) ». Có thể chỉ định cổng (8080) hoặc host:port (127.0.0.1:8080)
Xver	0	Phiên bản PROXY protocol (« Xver »): 0 — tắt, 1 hoặc 2 — phiên bản PROXY protocol tương ứng
(thứ tự)	theo vị trí	Thứ tự áp dụng quy tắc; đặt bằng nút « Вверх »/« Вниз »

Logic lưu: toàn bộ danh sách fallback của master được thay thế nguyên tử. Dòng không có inbound con được chọn (`childId <= 0`) lần Dest được đặt **bị bỏ qua**. Nếu inbound con được chọn trùng với id của chính master, nó được đặt về không. Khi tạo JSON đầu ra: nếu Dest rỗng, nó được tính từ inbound con là `listen:port`, trong đó `0.0.0.0 / :: / ::0` được thay bằng `127.0.0.1`; các trường `name / alpn / path` rỗng không được đưa vào JSON đầu ra; `xver` chỉ được thêm nếu lớn hơn 0.

Ví dụ settings.fallbacks đầu ra (lưu lượng với `alpn=h2` đến đích WS theo đường dẫn `/ws`, tất cả còn lại — đến Nginx cục bộ trên cổng 8080):

```
{
  "fallbacks": [
    { "alpn": "h2", "path": "/ws", "dest": "127.0.0.1:2001", "xver": 1 },
    { "dest": 8080 }
  ]
}
```

Dòng cuối không có `name / alpn / path` — đây là quy tắc «mặc định», bắt tất cả còn lại.

Các nút và gợi ý của mục fallbacks

- «**Добавить фолбэк**» — thêm dòng; «**Фолбэков пока нет**» — trạng thái rỗng.
- «**Быстро добавить все подходящие**» / «**Добавить все**» — thêm dòng fallback cho mỗi inbound phù hợp chưa được kết nối. Kết quả: «Добавлено {n} фолбэк(ов)» hoặc «Нет новых подходящих инбаундов».
- «**Заполнить из дочернего**» — lấy lại các trường định tuyến (SNI/ALPN/Path/xver) từ transport của inbound con đã chọn; sau khi thực hiện — «Заполнено из дочернего».
- «**Изменить поля маршрутизации**» / «**Скрыть расширенные**» — hiện/ẩn các trường chi tiết của dòng.

- Nhấn **«Маршрутизирует, когда»** và **«По умолчанию — ловит всё остальное»** giải thích điều kiện kích hoạt của mỗi dòng.

Sau khi lưu các fallback, server gọi khởi động lại Xray để `settings.fallbacks` mới có hiệu lực.

4.6. Đặt lại lưu lượng định kỳ

Khối **«Сброс трафика»** cấu hình tự động đặt lại bộ đếm lưu lượng của inbound theo lịch. Mô tả:

«Автоматический сброс счетчика трафика через указанные интервалы».

Tham số	Giá trị
Trường	<code>trafficReset</code>
Xác thực	<code>omitempty,oneof=never hourly daily weekly monthly</code>
Mặc định	<code>never</code>
Trường đi kèm	<code>lastTrafficResetTime</code> — mốc thời gian lần đặt lại cuối (nhãn «Последний сброс»)

Danh sách thả xuống:

Giá trị	Nhãn
<code>never</code>	«Никогда»
<code>hourly</code>	«Ежечасно»
<code>daily</code>	«Ежедневно»
<code>weekly</code>	«Еженедельно»
<code>monthly</code>	«Ежемесячно»

Cho mỗi chu kỳ, một cron job được đăng ký chạy theo lịch tương ứng (`@hourly` , `@daily` , `@weekly` , `@monthly`). Job chọn tất cả inbound có `trafficReset` đã đặt và đối với mỗi inbound đặt lại bộ đếm của chính inbound đó (`up=0` , `down=0`) và lưu lượng của tất cả client của nó. Tức là việc đặt lại định kỳ ảnh hưởng đến cả inbound lẫn các client của nó.

Ví dụ giá trị trường. Để bộ đếm được đặt về không vào ngày đầu tiên mỗi tháng, chọn **«Ежемесячно»** trong form, được lưu dưới dạng:

```
{ "trafficReset": "monthly" }
```

Giá trị `never` (mặc định) tắt hoàn toàn tự động đặt lại.

4.7. JSON входящего (nâng cao)

Mục **«Разделы JSON входящего»** cung cấp truy cập trực tiếp vào các khối JSON thô của inbound. Mô tả:

«Полный JSON входящего и отдельные редакторы для settings, sniffing и streamSettings».

Các trình soạn thảo khả dụng:

Tab	Nhãn	Chỉnh sửa gì
Все	«Полный объект входящего со всеми полями в одном редакторе»	toàn bộ đối tượng Inbound
Настройки	«Обёртка блока settings Xray»	trường settings
Sniffing	«Обёртка блока sniffing Xray»	trường sniffing
Stream	«Обёртка блока stream Xray»	trường streamSettings

Các trường này được tuần tự hóa dưới dạng đối tượng JSON lồng nhau: các khối rỗng được trả về dưới dạng `null`, còn văn bản không phải JSON hợp lệ được bọc trong chuỗi để dữ liệu không bị mất. Lỗi phân tích khi lưu được hiển thị với tiền tố **«Расширенный JSON»**.

Cửa sổ xem «JSON входящего», cũng như cửa sổ nhập inbound, sử dụng trình soạn thảo mã đầy đủ tính năng với tô sáng cú pháp JSON (thay vì ô văn bản thông thường): xem cấu hình – ở chế độ chỉ đọc với tô sáng, còn nhập – ở chế độ chỉnh sửa được, giúp đọc và chỉnh sửa dễ dàng hơn.

4.8. Các thao tác với inbound: QR / Edit / Reset / Delete và thống kê

Trong danh sách và trong card inbound có các thao tác sau (menu **«Меню»**):

Thống kê lưu lượng

Hiển thị lưu lượng tổng hợp của inbound: **«Отправлено/получено»** (các trường `up / down`), **«Всего трафика»**, **«Всего подключений»**. Trong card còn có – **«Создано»**, **«Обновлено»**, **«Дата окончания»**.

Trong danh sách inbounds có cột **Speed** với tốc độ lưu lượng hiện tại theo mỗi inbound (gửi/nhận), được tính từ mức tăng bộ đếm giữa các lần truy vấn; tốc độ thực tế tương tự được hiển thị trong cửa sổ thống kê inbound. Khi lần truy vấn tiếp theo không có mức tăng, giá trị tốc độ được đặt lại.

Trong tóm tắt client trên trang inbounds, trạng thái được xác định theo ưu tiên «cạn kiệt/kết thúc»: các client đã hết hạn hoặc hết lưu lượng (và bị tác vụ tự động tắt `enable`) được xếp vào trạng thái **«Исчерпан/завершён»** (*Depleted/Ended*), không phải màu xám **«Отключён»** (*Disabled*), và không được tính hai lần. Phân loại trùng khớp với phân loại hiển thị trong card của chính client, và tính đúng các client được gán với nhiều inbound.

Mã QR và sao chép liên kết

- **«Подробнее»** – mở rộng các liên kết kết nối và subscription.
- Mã QR của client: gợi ý **«Нажмите на QR-код, чтобы скопировать»**.
- **«Копировать ссылку»** (tiếng Anh: *Copy URL*), **«Экспорт ссылок»**.

Edit (Chỉnh sửa)

«Изменить подключение» — mở form chỉnh sửa (`POST /update/:id`). Khi cập nhật, dịch vụ đọc lại bản ghi hiện có, chuyển các trường đã thay đổi, nếu cần tạo lại tag (nếu tag cũ được tạo tự động) và đồng bộ runtime Xray. Thành công — thông báo «Подключение успешно обновлено».

Reset Traffic (Đặt lại lưu lượng)

«Сбросить трафик» — đặt lại bộ đếm `up / down` của chính inbound này về không (`POST /:id/resetTraffic` , đặt `up=0` , `down=0`). Xác nhận:

«Сбросить трафик "{remark}"?» / «Сбрасывает счётчики отправки/получения этого подключения до 0».

Đặt lại lưu lượng inbound **không** ảnh hưởng đến bộ đếm của các client của nó (chúng có thao tác «Đặt lại lưu lượng client» riêng). Sau khi đặt lại, Xray được khởi động lại. Thành công — thông báo «Входящий трафик сброшен». Còn có biến thể hàng loạt — «Сброс трафика всех подключений» (`POST /resetAllTraffics`).

Delete (Xóa)

«Удалить подключение» (`POST /del/:id`). Xác nhận:

«Удалить подключение "{remark}"?» / «Подключение и все его клиенты будут удалены. Это действие нельзя отменить».

Việc xóa gỡ inbound khỏi Xray đang chạy (nếu cần với khởi động lại). Thành công — thông báo «Подключение успешно удалено». Xóa hàng loạt — `POST /bulkDel` , với báo cáo từng phần tử và không quá một lần khởi động lại Xray.

Các thao tác khác với client của inbound

Trong menu cũng có: «Клонировать» (bản sao inbound với cổng mới và danh sách client rỗng), «Удалить всех клиентов» (`POST /:id/deleteAllClients` — xóa tất cả client, bản thân inbound được giữ lại), «Удалить отключенных клиентов», «Привязать/Отвязать клиентов», «Импортировать»/«Экспорт подключений» (`POST /import`). Chi tiết các thao tác client thuộc mục về client.

5. Giao thức

Khi tạo inbound, bước đầu tiên là chọn **Giao thức** («Protocol»). Giao thức xác định phương thức xác thực và mã hoá lưu lượng mà Xray-core sẽ áp dụng cho inbound đó, tập hợp các trường trong `settings` cần điền, cũng như những loại transport (`network`) và bảo mật (TLS / REALITY) khả dụng.

Trường giao thức được đặt một lần khi tạo inbound và **không thể thay đổi khi chỉnh sửa** (trong biểu mẫu chỉnh sửa, danh sách thả xuống bị khoá). Để đổi giao thức, cần tạo một inbound mới.

5.1. Danh sách giao thức được hỗ trợ

Server chấp nhận các giá trị sau cho trường `Protocol` :

```
oneof=vmess vless trojan shadowsocks wireguard hysteria http mixed tunnel tun mtproto
```

Kể từ phiên bản **3.3.0**, giá trị `mtproto` (proxy Telegram) được bổ sung vào danh sách.

Giá trị trong cấu hình	Mục đích	Mô hình client
<code>vless</code>	Giao thức proxy chính (mặc định khi tạo inbound)	Client dùng UUID, hỗ trợ flow và mã hoá hậu lượng tử
<code>vmess</code>	Giao thức proxy cổ điển của Xray	Client dùng UUID và tham số <code>security</code>
<code>trojan</code>	Proxy giả dạng HTTPS thông thường	Client dùng mật khẩu
<code>shadowsocks</code>	Proxy Shadowsocks (bao gồm SIP022 / 2022-blake3)	Một hoặc nhiều người dùng (2022)
<code>wireguard</code>	Inbound WireGuard	Peer (không phải client)
<code>hysteria</code>	Inbound Hysteria (mặc định là phiên bản 2)	Client dùng token <code>auth</code>
<code>http</code>	Proxy HTTP cổ điển (forward proxy)	Tài khoản user/pass, không tính lưu lượng
<code>mixed</code>	Proxy kết hợp SOCKS + HTTP	Tài khoản user/pass
<code>tunnel</code>	Bộ chuyển tiếp trong suốt (xray <code>dokodemo-door</code>)	Không có client
<code>tun</code>	Giao diện TUN (chỉ hiển thị các inbound đã có)	Không có client
<code>mtproto</code>	Proxy Telegram (MTPROTO), bổ sung từ 3.3.0; được quản lý bởi tiến trình <code>mtg</code> riêng, không phải Xray	Không có client (truy cập bằng secret)

Ghi chú về `tun` : giá trị này được giữ trong danh sách để tương thích và **hiển thị** các inbound đã lưu trước đó, nhưng trong phiên bản hiện tại, backend không khuyến khích tạo loại này – hỗ trợ đã được đánh dấu lỗi thời. Việc tạo mới inbound kiểu này không có ý nghĩa thực tế.

Ghi chú về Hysteria 2: không có giao thức «hysteria2» riêng biệt. Đây là giao thức `hysteria` với trường `streamSettings.version = 2`. Sơ đồ liên kết `hysteria2://` khi tạo share-link được chọn tự động khi phiên bản stream bằng 2.

Không phải tất cả giao thức đều hỗ trợ triển khai theo node. Chỉ các giao thức sau có thể triển khai trên node: `vless`, `vmess`, `trojan`, `shadowsocks`, `hysteria`, `wireguard`. Các giao thức `http`, `mixed`, `tunnel`, `tun`, `mtproto` chỉ hoạt động trên panel cục bộ.

5.2. Giao thức nào hỗ trợ TLS / REALITY / transport

Khả năng bật lớp bảo mật và transport phụ thuộc vào giao thức và mạng được chọn (`streamSettings.network`):

Khả năng	Áp dụng cho giao thức	Mạng được phép (network)
TLS	<code>vmess</code> , <code>vless</code> , <code>trojan</code> , <code>shadowsocks</code> (và luôn có với <code>hysteria</code>)	<code>tcp</code> , <code>ws</code> , <code>http</code> , <code>grpc</code> , <code>httpupgrade</code> , <code>xhttp</code>
REALITY	<code>vless</code> , <code>trojan</code>	<code>tcp</code> , <code>http</code> , <code>grpc</code> , <code>xhttp</code>
flow (xtls-rprx-vision)	chỉ <code>vless</code>	chỉ <code>tcp</code> , khi <code>security = tls</code> hoặc <code>reality</code>
Stream / transport (tab «Luồng»)	<code>vmess</code> , <code>vless</code> , <code>trojan</code> , <code>shadowsocks</code> , <code>hysteria</code>	–

Đối với các giao thức `http`, `mixed`, `tunnel`, `tun`, `wireguard`, tab transport không khả dụng – chúng không có cài đặt stream của Xray.

5.3. VLESS

Mục đích: giao thức proxy hiện đại chính. Hỗ trợ XTLS-Vision (`flow`), REALITY, cũng như mã hoá hậu lượng tử ở cấp độ VLESS (các trường `decryption` / `encryption`). Được sử dụng mặc định cho các inbound mới.

Các trường của khối `settings` :

Trường	Giá trị mặc định	Mô tả
<code>clients</code>	<code>[]</code>	Danh sách client. Mỗi client có: <code>id</code> (UUID), <code>email</code> (bắt buộc), <code>flow</code> , giới hạn (<code>limitIp</code> , <code>totalGB</code> , <code>expiryTime</code>), <code>enable</code> , <code>tgId</code> , <code>subId</code> , <code>comment</code> , <code>reset</code>
<code>decryption</code>	<code>none</code>	Tham số giải mã phía server. Nhân trong UI: «Расшифрование» (tiếng Anh «Decryption»)
<code>encryption</code>	<code>none</code>	Tham số mã hoá tương ứng (đưa vào link client). Nhân: «Шифрование» (tiếng Anh «Encryption»)
<code>fallbacks</code>	<code>[]</code>	Danh sách fallback (xem phần về fallback); khả dụng khi <code>network = tcp</code> và <code>security = TLS</code> hoặc REALITY

Trường	Giá trị mặc định	Mô tả
testseed	(4 số: 900, 500, 900, 256)	«Vision testseed» – 4 số nguyên dương dùng cho XTLS-Vision padding. Chỉ áp dụng cho client có flow <code>xtls-rprx-vision</code> , ngược lại bị bỏ qua

flow (`xtls-rprx-vision`)

flow được đặt ở phía client, không phải inbound, và nhận một trong ba giá trị:

Giá trị	Ý nghĩa
`` (trống)	Không có XTLS-flow (mặc định)
<code>xtls-rprx-vision</code>	XTLS-Vision – chế độ được khuyến nghị cho VLESS qua TCP+TLS/REALITY
<code>xtls-rprx-vision-udp443</code>	Vision tương tự nhưng có xử lý UDP/443 (QUIC)

Trường flow chỉ có thể chọn khi đáp ứng tất cả điều kiện: giao thức `vless`, network = `tcp` và security = `tls` hoặc `reality`. Trường **Vision testseed** trong biểu mẫu chỉ hiển thị khi có cùng điều kiện.

Ngoại lệ với XHTTP: khi VLESS qua network = `xhttp` với xác thực hậu lượng tử VLESS được bật (encryption / decryption, `vlessenc`), flow `xtls-rprx-vision` cũng được cho phép – bất kể lớp bảo mật, kể cả với REALITY. Trong trường hợp này, panel truyền đúng `xtls-rprx-vision` vào share-link và subscription (bao gồm định dạng Clash/Mihomo), để client nhận cấu hình với Vision.

Giải mã / Mã hoá (xác thực hậu lượng tử VLESS)

Các trường `decryption` và `encryption` là xác thực ở cấp độ VLESS (tách biệt với TLS/REALITY transport). Mặc định cả hai đều là `none`. Trong biểu mẫu có ba nút:

- **Xác thực X25519** (tiếng Anh «X25519 auth») – tạo cặp `decryption` / `encryption` dựa trên X25519.
- **Xác thực ML-KEM-768** (tiếng Anh «ML-KEM-768 auth») – biên thể hậu lượng tử (Post-Quantum).
- **Xoá** – đặt lại cả hai trường về `none`.

Bên dưới các nút hiển thị dòng trạng thái «Đã chọn: {auth}», trong đó giá trị được xác định theo đoạn cuối cùng của chuỗi `encryption`: trống/ `none` → «None», khoá rất dài (>300 ký tự) → ML-KEM-768, ngắn → X25519, ngược lại «Tùy chỉnh».

Về mặt kỹ thuật, các nút gửi yêu cầu đến `GET /panel/api/server/getNewVlessEnc` (tạo khoá qua `xray vlessenc`) và điền **cả hai** trường với các giá trị cặp dạng `mlkem768x25519plus.native.<rtt>.<role>` (ví dụ: `decryption = mlkem768x25519plus.native.600s.server-x25519`, `encryption = mlkem768x25519plus.native.0rtt.client-x25519`). Tham số `decryption` ở lại server, `encryption` đi vào link client.

Quan trọng: khi tạo cấu hình inbound cho Xray, panel loại bỏ phần thừa: nếu trong `settings` còn lại `encryption` (thuộc về phía client), nó **bị loại** khỏi cấu hình server. Trên server chỉ giữ lại `decryption`.

Khi nào nên chọn VLESS: đây là lựa chọn mặc định được khuyến nghị cho inbound mới, đặc biệt kết hợp với REALITY (không cần chứng chỉ riêng) hoặc với TLS + XTLS-Vision.

Ví dụ: khối `settings` của VLESS-inbound với một client và XTLS-Vision. Trường `flow` nằm ở phía client, `decryption` ở lại server:

```
{
  "clients": [
    {
      "id": "d342d11e-d424-4583-b36e-524ab1f0afa4",
      "email": "user1",
      "flow": "xtls-rprx-vision",
      "limitIp": 2,
      "totalGB": 0,
      "expiryTime": 0,
      "enable": true
    }
  ],
  "decryption": "none"
}
```

Để kết hợp REALITY, khối `streamSettings` tương ứng (tab «Transport» → Security: REALITY) trông như sau:

```
{
  "network": "tcp",
  "security": "reality",
  "realitySettings": {
    "dest": "www.microsoft.com:443",
    "serverNames": ["www.microsoft.com"],
    "privateKey": "<приватный ключ X25519>",
    "shortIds": ["", "6ba85179e30d4fc2"]
  }
}
```

5.4. VMess

Mục đích: giao thức proxy cổ điển của Xray. Xác thực bằng UUID, phía client có thể cấu hình thêm phương thức mã hoá payload (`security`).

Các trường của khối `settings` :

Trường	Giá trị mặc định	Mô tả
<code>clients</code>	<code>[]</code>	Danh sách client

Mỗi client VMess (ngoài các trường chung `email` , giới hạn, `enable` , `tgId` , `subId` , `comment` , `reset`):

Trường client	Giá trị mặc định	Mô tả
id	–	UUID của client
security	auto	Phương thức mã hoá payload VMess. Các giá trị cho phép: aes-128-gcm , chacha20-poly1305 , auto , none , zero

Giá trị security :

- auto – Xray tự chọn mã hoá tùy nền tảng (khuyến nghị);
- aes-128-gcm , chacha20-poly1305 – mã hoá AEAD cố định;
- none – không mã hoá payload (chỉ có ý nghĩa khi dùng qua TLS);
- zero – không mã hoá và không xác thực payload.

Tương thích lịch sử: các bản ghi cũ có thể lưu security: "" – khi đọc, chuỗi rỗng được chuẩn hoá thành auto . Khi tạo cấu hình server, trường security của client VMess **bị xoá** khỏi settings vì không cần thiết cho inbound.

Khi nào nên chọn VMess: để tương thích với các client cũ hoặc cấu hình hiện có. Với triển khai mới, VLESS thường được ưu tiên hơn.

5.5. Trojan

Mục đích: proxy giả dạng lưu lượng HTTPS thông thường. Xác thực bằng mật khẩu. Giống VLESS, hỗ trợ fallback và (khi network = tcp) REALITY/TLS.

Các trường của khối settings :

Trường	Giá trị mặc định	Mô tả
clients	[]	Danh sách client
fallbacks	[]	Danh sách fallback (khả dụng khi network = tcp và TLS/REALITY)

Trường quan trọng của mỗi client Trojan:

Trường client	Giá trị mặc định	Mô tả
password	–	Mật khẩu client (bắt buộc, tối thiểu 1 ký tự)
email	–	Mã định danh duy nhất của client

Các trường còn lại của client là chung (limitIp , totalGB , expiryTime , enable , tgId , subId , comment , reset).

Khi nào nên chọn Trojan: khi cần giả dạng HTTPS trên cổng 443, bao gồm với fallback về web server (Nginx) cho các kết nối không xác thực.

Ví dụ: khối settings Trojan với fallback về web server cục bộ. Các kết nối không xác thực (không có mật khẩu hợp lệ) được chuyển tới Nginx lắng nghe trên 127.0.0.1:8080 :

```
{
  "clients": [
    { "password": "S3cret-Pass-1", "email": "user1" }
  ],
  "fallbacks": [
    { "dest": "127.0.0.1:8080" }
  ]
}
```

Để dùng fallback cần network = tcp và Security = TLS hoặc REALITY; nếu không, trường fallbacks không khả dụng.

5.6. Shadowsocks

Mục đích: proxy Shadowsocks nhẹ. Hỗ trợ cả các mã hoá AEAD cũ lẫn các phương thức SIP022 mới (2022-blake3-*). Có thể hoạt động ở chế độ một hoặc nhiều người dùng.

Các trường của khối settings :

Trường	Giá trị mặc định	Mô tả
method	2022-blake3-aes-256-gcm	Phương thức mã hoá inbound. Nhân trong UI: «Метод шифрования» (tiếng Anh «Encryption method»)
password	''	Mật khẩu inbound (với phương thức 2022 được tạo tự động theo phương thức được chọn)
network	tcp,udp	Transport. Nhân: «Сеть» (tiếng Anh «Network»). Các tùy chọn: tcp,udp (TCP, UDP), tcp , udp
clients	[]	Danh sách client
ivCheck	false (tắt)	Bộ chuyển đổi «ivCheck» – bảo vệ khỏi tái sử dụng IV

Phương thức mã hoá (method)

Tập hợp giá trị cho phép:

Phương thức	Loại
aes-256-gcm	AEAD cũ
chacha20-poly1305	AEAD cũ
chacha20-ietf-poly1305	AEAD cũ
xchacha20-ietf-poly1305	AEAD cũ
2022-blake3-aes-128-gcm	SS 2022 (khuyến nghị)

Phương thức	Loại
2022-blake3-aes-256-gcm	SS 2022 (mặc định)
2022-blake3-chacha20-poly1305	SS 2022, một người dùng

Lô-gíc của panel theo phương thức:

- **Phương thức 2022** (2022-blake3-*) được coi là «SS 2022». Phương thức 2022-blake3-chacha20-poly1305 – **một người dùng** (không hỗ trợ multi-user); các phương thức 2022 khác cho phép nhiều client. Trường mật khẩu (với nút tạo, tự điều chỉnh độ dài khoá theo phương thức) hiển thị trong biểu mẫu đặc biệt cho phương thức 2022.
- **Mã hoá cũ** (aes-* , chacha20-*) hoạt động theo sơ đồ cổ điển «một phương thức + một mật khẩu».

Chuẩn hoá trước khi chạy Xray: với mã hoá cũ, mỗi client phải mang `method` trùng với phương thức của inbound (nếu không Xray sẽ báo lỗi «unsupported cipher method:»). Với phương thức 2022, ngược lại – trường `method` của client phải **rỗng** (nếu không Xray từ chối inbound với lỗi «users must have empty method»). Panel tự chuẩn hoá dữ liệu khi chuyển đổi phương thức.

Tạo lại khoá client khi đổi kích thước khoá: với Shadowsocks-2022, khi đổi phương thức mã hoá sang phương thức có kích thước khoá khác (ví dụ giữa 2022-blake3-aes-256-gcm và 2022-blake3-aes-128-gcm), panel tự động tạo lại PSK client theo độ dài mới khi lưu inbound. Nếu không, các khoá cũ vẫn có độ dài cũ và Xray sẽ từ chối chúng. Hệ quả: client bị ảnh hưởng cần lấy lại subscription – các link cũ sẽ không kết nối được nữa.

Khi nào nên chọn Shadowsocks: cho triển khai đơn giản không cần giả dạng TLS; lựa chọn hiện đại là các phương thức 2022-blake3.

Ví dụ: khối settings Shadowsocks cho phương thức 2022-blake3 (chế độ nhiều người dùng). Inbound có mật khẩu riêng (khóa base64 có độ dài phù hợp), mỗi client có mật khẩu riêng, trường `method` của client **rỗng**:

```
{
  "method": "2022-blake3-aes-256-gcm",
  "password": "d2hhdGV2ZXItMzItYnl0ZS1iYXNlNjQta2V5LWlcmU=",
  "network": "tcp,udp",
  "clients": [
    {
      "email": "user1",
      "password": "Y2xpZW50LWtleS0zMl1ieXRlc1iYXNlNjQtaGVyZQ==",
      "method": ""
    }
  ]
}
```

Với mã hoá legacy (aes-256-gcm v.v.) – ngược lại: một mật khẩu cho inbound, và `method` của client phải trùng với phương thức của inbound.

5.7. Dokodemo-door / Tunnel (bộ chuyển tiếp trong suốt)

Mục đích: bộ chuyển tiếp trong suốt (trong panel — giao thức `tunnel`, thực hiện hành vi `dokodemo-door`). Nhận lưu lượng và chuyển tiếp đến địa chỉ/cổng được chỉ định, không cần xác thực và không có client.

Các trường của khối `settings`:

Trường	Giá trị mặc định	Mô tả
<code>rewriteAddress</code>	(không có)	«Переписать адрес» (tiếng Anh «Rewrite address») — địa chỉ đích để chuyển hướng lưu lượng
<code>rewritePort</code>	(không có)	«Переписать порт» (tiếng Anh «Rewrite port») — cổng đích (0–65535)
<code>allowedNetwork</code>	<code>tcp,udp</code>	«Разрешённая сеть» (tiếng Anh «Allowed network»). Tùy chọn: <code>tcp,udp</code> , <code>tcp</code> , <code>udp</code>
<code>portMap</code>	<code>{}</code>	«Сопоставление портов» — bảng ánh xạ cổng→cổng (Record<string,string>)
<code>followRedirect</code>	<code>false</code> (tắt)	«Следовать redirect» (tiếng Anh «Follow redirect») — sử dụng địa chỉ đích gốc từ kết nối bị chặn

Tab «Transport» với Tunnel: inbound kiểu này có tab «**Transport**», giới hạn ở cài đặt `sockopt` — đủ cho chế độ **TProxy** (proxy trong suốt/redirect qua `sockopt.tproxy`). Danh sách thả xuống chọn transport (`network`) và tab «Security» với Tunnel bị ẩn vì TLS/REALITY không được hỗ trợ bởi loại này.

Khi nào nên chọn: cho proxy trong suốt/chuyển tiếp cổng tới các dịch vụ nội bộ.

5.8. SOCKS / HTTP (giao thức `mixed`)

Trong bản dựng này không có giao thức `socks` riêng biệt — SOCKS và HTTP proxy được gộp vào giao thức `mixed` (kết hợp SOCKS + HTTP). Ngoài ra còn có giao thức `http` thuần túy riêng.

5.8.1. Mixed (SOCKS + HTTP)

Các trường của khối `settings`:

Trường	Giá trị mặc định	Mô tả
<code>auth</code>	<code>password</code>	«Auth» — chế độ xác thực. Tùy chọn: <code>password</code> (theo đăng nhập/mật khẩu) hoặc <code>noauth</code> (không xác thực)
<code>accounts</code>	(tùy chọn)	«Аккаунты» — danh sách cặp user/pass. Khi <code>auth = noauth</code> , trường không được ghi vào cấu hình
<code>udp</code>	<code>false</code> (tắt)	Bộ chuyển đổi «UDP» — hỗ trợ UDP qua SOCKS

Trường	Giá trị mặc định	Mô tả
<code>ip</code>	<code>127.0.0.1</code>	«UDP IP» – địa chỉ cục bộ cho các liên kết UDP. Trường chỉ hiển thị khi <code>udp</code> được bật

Tài khoản được thêm bằng nút «Добавить»; khi thêm, đăng nhập ngẫu nhiên (8 ký tự) và mật khẩu (12 ký tự) được tạo tự động và có thể chỉnh sửa.

5.8.2. HTTP (proxy thuần túy)

Mục đích: forward-proxy HTTP cổ điển. Ở cấp độ Xray không theo dõi client như «billing» (không có email/giới hạn) – chỉ có danh sách tài khoản.

Các trường của khối `settings` :

Trường	Giá trị mặc định	Mô tả
<code>accounts</code>	<code>[]</code>	«Аккаунты» – danh sách cặp user/pass (cả hai trường đều bắt buộc)
<code>allowTransparent</code>	<code>false</code> (tắt)	«Разрешить прозрачный» (tiếng Anh «Allow transparent») – chuyển tiếp yêu cầu với header Host gốc

Khi nào nên chọn SOCKS/HTTP: cho proxy cục bộ hoặc nội bộ không cần giả dạng phức tạp. `mixed` tiện lợi ở chỗ một cổng phục vụ cả client SOCKS lẫn HTTP.

5.9. WireGuard (inbound)

Mục đích: inbound WireGuard. Khác với các giao thức proxy, nó không vận hành «client» – thay vào đó cấu hình **peer** (các thiết bị mà server chấp nhận). Transport và TLS/REALITY không áp dụng cho nó.

Các trường của khối `settings` :

Trường	Giá trị mặc định	Mô tả
<code>secretKey</code>	–	Khoá bí mật của server (bắt buộc). Bên cạnh có nút tạo; khoá công khai hiển thị tự động (chỉ đọc)
<code>mtu</code>	(tuỳ chọn)	MTU của giao diện
<code>noKernelTun</code>	<code>false</code> (tắt)	«TUN без kernel» (tiếng Anh «No-kernel TUN») – dùng userspace-TUN thay vì TUN của kernel
<code>domainStrategy</code>	(tuỳ chọn)	«Domain Strategy» – chiến lược phân giải tên miền: <code>ForceIP</code> , <code>ForceIPv4</code> , <code>ForceIPv4v6</code> , <code>ForceIPv6</code> , <code>ForceIPv6v4</code>
<code>peers</code>	<code>[]</code>	Danh sách peer

Các trường của mỗi peer:

Trường peer	Giá trị mặc định	Mô tả
privateKey	(tùy chọn)	Khoá bí mật của client – được lưu để panel có thể hiển thị cấu hình cho người dùng (chỉ với inbound-peer)
publicKey	–	Khoá công khai của peer (bắt buộc)
preSharedKey (PSK)	(tùy chọn)	Khoá chia sẻ bổ sung
allowedIPs	[]	IP được phép. Khi thêm peer mới, panel tự động đề xuất địa chỉ rảnh tiếp theo (mặc định 10.0.0.2/32)
keepAlive	(tùy chọn)	«Keep-alive» – khoảng thời gian duy trì kết nối
comment	(tùy chọn)	«Comment» – nhãn tùy ý của peer; hiển thị bên cạnh tiêu đề «Peer N» và được đưa vào link chia sẻ và remark của file .conf

Nút «Добавить peer» tạo cặp khoá mới và điền allowedIPs tiếp theo. Mỗi peer có thể xoá (không thể xoá nếu chỉ còn một peer).

Trường «Comment» của peer giúp phân biệt các thiết bị: văn bản của nó hiển thị trong biểu mẫu bên cạnh tiêu đề «Peer N», cũng như được đưa vào link chia sẻ và remark của file .conf được tạo, giúp dễ nhận dạng thiết bị trong ứng dụng client. Đây là trường của panel – xray-core bỏ qua các trường không xác định của peer.

Domain Strategy và tab Transport

Ngoài peer, WireGuard-inbound còn có trường **Domain Strategy** (chiến lược phân giải tên miền: ForceIP , ForceIPv4 , ForceIPv4v6 , ForceIPv6 , ForceIPv6v4). Trường là tùy chọn và chỉ được ghi vào cấu hình nếu được đặt.

Trường **Workers** (workers , số luồng làm việc) đã bị xoá khỏi biểu mẫu WireGuard (cả inbound lẫn outbound): kể từ xray-core v26.6.22, engine không còn sử dụng nó và dựa vào cơ chế nội bộ của wireguard-go. Các cấu hình đã lưu trước đây hoạt động bình thường – trường chỉ bị bỏ qua khi phân tích, không cần di chuyển dữ liệu.

Với WireGuard cũng có tab «**Transport**» – nhưng ở dạng rút gọn: chỉ cấu hình sockopt và obfuscation **Finalmask**. Danh sách thả xuống chọn transport (network) bị ẩn vì WireGuard luôn lắng nghe qua UDP. Trong các bản ghi noise, Finalmask có trường riêng **Rand Range** (phạm vi byte 0–255, có xác thực), và phương thức obfuscation **Salamander** khả dụng cho WireGuard và Hysteria.

Khi nào nên chọn WireGuard: khi cần đúng đường hầm VPN WireGuard, không phải proxy giả dạng.

5.10. Hysteria (mặc định v2)

Mục đích: inbound Hysteria qua QUIC. Panel mặc định làm việc với phiên bản 2. Mỗi client xác thực bằng token `auth` thay vì UUID/mật khẩu. TLS với Hysteria luôn khả dụng (xem bảng khả năng ở 5.2).

Các trường của khối `settings` :

Trường	Giá trị mặc định	Mô tả
<code>version</code>	2	Phiên bản giao thức (tối thiểu 1; panel mặc định là 2)
<code>clients</code>	<code>[]</code>	Danh sách client

Trường quan trọng của mỗi client là `auth` (token, bắt buộc) cộng với các trường chung (`email` , giới hạn, `enable` , `tgId` , `subId` , `comment` , `reset`).

Các tham số bổ sung được đặt trong `streamSettings.hysteriaSettings` :

Trường	Giá trị / tùy chọn	Mô tả
<code>version</code>	cố định là 2 (trường bị khoá)	«Версия» (tiếng Anh «Version»)
<code>udpIdleTimeout</code>	(số nguyên ≥ 1, giây)	«UDP idle timeout (c)» – thời gian chờ nhận rồi UDP
<code>masquerade</code>	tắt mặc định	«Masquerade» – giả dạng web server thông thường khi có yêu cầu «không xác thực»

Khi bật `masquerade` , có thể chọn loại (`type`):

- `""` – default (trang 404);
- `proxy` – reverse proxy (các trường «Upstream URL», «Переписать Host», «Пропустить TLS verify»);
- `file` – phục vụ thư mục (trường «Директория», ví dụ `/var/www/html`);
- `string` – phản hồi cố định (các trường «Код статуса», «Body», «Заголовки»).

Khi nào nên chọn Hysteria: khi cần QUIC-transport và độ ổn định trên kênh không ổn định/di động; `masquerade` tăng khả năng ẩn của điểm vào.

5.11. MTProto (proxy cho Telegram)

Bổ sung trong phiên bản **3.3.0**. Giá trị giao thức – `mtproto` .

MTProto là giao thức proxy riêng của Telegram. Trong 3X-UI, inbound này **được phục vụ không phải bởi Xray mà bởi tiến trình `mtg` riêng biệt**, do panel quản lý. Panel định kỳ so sánh các MTProto-inbound đang bật với các tiến trình `mtg` đang chạy: khởi động các tiến trình còn thiếu, dừng các tiến trình thừa và thu thập số liệu lưu lượng từ metrics `mtg` . Do đó **tính lưu lượng** của inbound này hoạt động như các giao thức thông thường.

Gợi ý chính thức trong biểu mẫu:

«MTPROTO обслуживается отдельным процессом mtg, а не Xray. Настройки транспорта и клиенты здесь не применяются — поделитесь ссылкой ниже в Telegram.»

Hệ quả:

- Các tab «**Transport**» (**Stream Settings**) và «**Client**» **không áp dụng cho inbound này** — quyền truy cập được xác định bằng một secret, không phải danh sách client.
- MTPROTO-inbound chỉ chạy **trên panel chính**; không triển khai lên node con (node có `NodeID` được bỏ qua).
- Tab «**Sniffing**» với MTPROTO bị ẩn — giao thức này được phục vụ bởi tiến trình `mtg`, không phải Xray, nên sniffing không áp dụng.

Các trường. Được lưu trong `settings` của inbound:

Trường trong UI	Khoá	Mô tả
Remark	<code>remark</code>	Nhãn inbound.
Listen IP	<code>listen</code>	IP lắng nghe (trống = tất cả giao diện).
Port	<code>port</code>	Cổng proxy.
Секрет	<code>settings.secret</code>	Secret truy cập ở định dạng FakeTLS .
Домен прикрытия (FakeTLS)	<code>settings.fakeTlsDomain</code>	Tên miền mà proxy giả dạng lưu lượng HTTPS.

Định dạng secret (FakeTLS). Panel tự động đưa secret về dạng đúng: kết quả = `ee` + 32 ký tự hex + mã hex của tên miền che phủ, tức là `ee<hex32><hex(fakeTlsDomain)>`. Tiền tố `ee` bật chế độ FakeTLS, còn tên miền (ví dụ một trang nổi tiếng) dùng để ngụy trang lưu lượng thành HTTPS thông thường. Chỉ cần nhập tên miền — phần còn lại panel sẽ tự hoàn thiện.

Domain-fronting và các tùy chọn nâng cao của mtg

MTPROTO-inbound có các tham số bổ sung cho tiến trình `mtg`. Các trường **Domain fronting IP**, **Domain fronting port** và **Domain fronting PROXY protocol** xác định nơi `mtg` gửi lưu lượng không phải Telegram (ví dụ tới trang NGINX giả): nếu để IP trống, FakeTLS-domain sẽ được dùng qua DNS, cổng mặc định — `443`. Ngoài ra có **Accept PROXY protocol** (cho listener), **IP preference** (`prefer-ipv6` / `prefer-ipv4` / `only-ipv6` / `only-ipv4`) và **Debug logging**. Mỗi giá trị được ghi vào file `mtg-<id>.toml` chỉ khi được đặt.

Định tuyến lưu lượng Telegram qua Xray

Bộ chuyển đổi «**Route through Xray**» (mặc định tắt) và trường tùy chọn **Outbound** cho phép đặt egress Telegram dưới quyền router của Xray. Khi bật, panel nhúng SOCKS-bridge cục bộ với tag của inbound vào cấu hình Xray, và `mtg` gửi lưu lượng Telegram qua nó. Sau đó lưu lượng có thể được khớp bằng các quy tắc trong tab «**Routing**» hoặc buộc chuyển tới outbound hoặc balancer được chọn qua trường **Outbound** (nếu trường trống, các quy tắc routing quyết định).

Cách chia sẻ với người dùng. Với MTPROTO-inbound, panel tạo link mời:

Ví dụ: secret FakeTLS và link sẵn sàng. Nếu tên miền che phủ là `www.cloudflare.com`, secret được ghép như `ee` + 32 ký tự hex + mã hex tên miền, ví dụ:

```
secret = ee1a2b3c4d5e6f70819293a4b5c6d7e8f7777772e636c6f7564666c6172652e636f6d
```

Link mời sẵn sàng (gửi cho người dùng trong Telegram cùng mã QR):

```
tg://proxy?
server=203.0.113.10&port=443&secret=ee1a2b3c4d5e6f70819293a4b5c6d7e8f7777772e636c6f7564666c6172652e636f6d
```

```
tg://proxy?server=<адрес>&port=<порт>&secret=<секрет>
```

(tương đương – `https://t.me/proxy?server=...&port=...&secret=...`). Link này và mã QR cần gửi cho người dùng Telegram – khi mở, proxy được thêm ngay vào ứng dụng. Link cũng được cung cấp qua server subscription.

Khi nào nên dùng. Cách chuẩn để vượt chặn Telegram; giả dạng FakeTLS (tên miền che phủ) làm cho lưu lượng trông giống như truy cập thông thường vào trang đó.

5.12. Bảng tra nhanh chọn giao thức

- **VLESS** – lựa chọn mặc định; tốt nhất với REALITY hoặc TLS + XTLS-Vision, hỗ trợ xác thực hậu lượng tử.
- **Trojan** – giả dạng HTTPS với fallback về web server.
- **VMess** – tương thích với các client cũ.
- **Shadowsocks** – proxy đơn giản không cần TLS; lựa chọn hiện đại là các phương thức `2022-blake3*`.
- **Hysteria** – QUIC, ổn định trên kênh kém.
- **mixed / http** – proxy SOCKS/HTTP nội bộ.
- **WireGuard** – đường hầm VPN đầy đủ.
- **tunnel** – chuyển tiếp cổng trong suốt.
- **MTPROTO** – proxy vượt chặn Telegram (FakeTLS); tiến trình `mtg` riêng biệt.

6. Truyền tải (Stream Settings)

Truyền tải (trong giao diện panel – trường «**Truyền tải**», tiếng Anh *Transmission*) xác định cách Xray-core truyền dữ liệu bên trong inbound: giao thức mạng nào được sử dụng bên trên TLS/Reality và cách đóng khung lưu lượng. Các thông số này được lưu vào đối tượng `streamSettings` của cấu hình Xray và được thiết lập trong tab truyền tải của trình soạn thảo inbound. Mã hóa (TLS / Reality) được trình bày trong mục riêng – ở đây chỉ mô tả việc chọn mạng và các thông số của nó.

6.1. Chọn mạng truyền tải

Mạng được chọn trong danh sách thả xuống «**Truyền tải**» (`streamSettings.network`). Giá trị mặc định là `tcp` (hiển thị trong danh sách là **RAW**). Các tùy chọn có sẵn:

Giá trị trong danh sách	Trường <code>network</code>	Truyền tải
RAW	<code>tcp</code>	TCP thông thường (trong các phiên bản Xray mới được đổi tên thành RAW), tùy chọn có che giấu HTTP
mKCP	<code>kcp</code>	Truyền tải UDP đáng tin cậy mKCP
WebSocket	<code>ws</code>	WebSocket qua HTTP(S)
gRPC	<code>grpc</code>	Đường hầm gRPC (HTTP/2)
HTTPUpgrade	<code>httpupgrade</code>	HTTP Upgrade
XHTTP	<code>xhttp</code>	XHTTP / SplitHTTP – truyền tải ghép kênh hiện đại

Khi thay đổi giá trị, panel xóa khối cài đặt của mạng cũ và điền khối của mạng mới bằng các giá trị mặc định từ schema của nó, do đó mỗi trường của biểu mẫu con luôn có giá trị ban đầu có ý nghĩa.

Lưu ý. Trong bản build này của panel **truyền tải HTTP/2 (h2) không có trong danh sách** – nó đã bị loại khỏi tập hợp các mạng; để tạo đường hầm hai chiều giống HTTP/2, hãy sử dụng gRPC, còn để có truyền tải HTTP giả mạo hiện đại – XHTTP. Truyền tải **Hysteria** (`hysteria`) không được chọn qua danh sách này: nó được gán cứng với giao thức Hysteria và xuất hiện tự động khi inbound được tạo với giao thức Hysteria (xem mục 6.8).

Bên dưới từng mạng và từng trường của nó được phân tích riêng.

6.2. RAW / TCP (`tcpSettings`)

Truyền tải TCP cơ bản. Theo mặc định, lưu lượng được truyền «nguyên dạng»; tùy chọn có thể giả mạo thành trao đổi HTTP/1.1 thông thường.

Trường	Giá trị mặc định	Mô tả
Proxy Protocol (<code>acceptProxyProtocol</code>)	<code>false</code> (tắt)	Chấp nhận tiêu đề PROXY protocol từ bộ cân bằng tải/proxy phía trước
Che giấu HTTP (<code>header.type</code>)	<code>none</code> (tắt)	Bật che giấu lưu lượng dưới dạng HTTP/1.1

Proxy Protocol

Công tắc «**Proxy Protocol**» (`acceptProxyProtocol`). Khi bật, Xray chờ tiêu đề PROXY protocol trên kết nối đến và trích xuất IP thực của khách hàng từ đó. Chỉ bật khi trước panel có proxy ngược/bộ cân bằng tải (ví dụ HAProxy hoặc nginx với `send-proxy`) thêm tiêu đề này. Tắt theo mặc định.

Che giấu HTTP (camouflage)

Công tắc «**Che giấu HTTP**». Quản lý trường `header` :

- **Tắt** → `header.type = "none"` (trên đường truyền trường `header` đơn giản là vắng mặt). TCP thuần túy.
- **Bật** → `header.type = "http"` . Lưu lượng được đóng khung dưới dạng yêu cầu và phản hồi HTTP/1.1. Khi bật, panel ngay lập tức điền các đối tượng con `request` và `response` bằng các giá trị mặc định.

Khi bật che giấu HTTP, các trường cài đặt yêu cầu và phản hồi giả mạo sẽ xuất hiện.

Tiêu đề yêu cầu (`header.request`):

Trường	Khóa	Giá trị mặc định	Mô tả
Phiên bản yêu cầu	<code>request.version</code>	<code>1.1</code>	Phiên bản HTTP trong dòng bắt đầu yêu cầu
Phương thức yêu cầu	<code>request.method</code>	<code>GET</code>	Phương thức HTTP của yêu cầu giả mạo
Đường dẫn yêu cầu	<code>request.path</code>	<code>/</code>	Đường dẫn. Nhập danh sách giá trị cách nhau bằng dấu phẩy; trên đường truyền đây là mảng chuỗi. Nếu để trống, mặc định là <code>/</code>
Tiêu đề yêu cầu	<code>request.headers</code>	<code>{}</code> (trống)	Bảng «Tên/Giá trị» của các tiêu đề HTTP. Lưu dưới dạng map <code>tên → [giá trị]</code> (một tên có thể có nhiều giá trị)

Tiêu đề phản hồi (`header.response`):

Trường	Khóa	Giá trị mặc định	Mô tả
Phiên bản phản hồi	<code>response.version</code>	<code>1.1</code>	Phiên bản HTTP trong dòng bắt đầu phản hồi
Trạng thái phản hồi	<code>response.status</code>	<code>200</code>	Mã trạng thái HTTP của phản hồi giả mạo

Trường	Khóa	Giá trị mặc định	Mô tả
Lý do phản hồi	<code>response.reason</code>	OK	Mô tả văn bản của trạng thái (reason-phrase)
Tiêu đề phản hồi	<code>response.headers</code>	<code>{}</code> (trống)	Bảng «Tên/Giá trị» của các tiêu đề phản hồi (map <code>tên</code> → <code>giá trị</code>)

Các trường tiêu đề được chỉnh sửa theo từng dòng – mỗi dòng xác định tên tiêu đề (`Tên`) và giá trị của nó (`Giá trị`). Các thông số này chỉ dùng để che giấu diện mạo lưu lượng; chúng không ảnh hưởng đến mật mã học. Các giá trị mặc định (GET / HTTP/1.1 , phản hồi 200 OK) phù hợp với hầu hết các tình huống – chỉ nên thay đổi khi cần giả mạo một trang web/dịch vụ cụ thể.

Ví dụ `streamSettings` cho RAW với che giấu HTTP:

```
{
  "network": "tcp",
  "tcpSettings": {
    "acceptProxyProtocol": false,
    "header": {
      "type": "http",
      "request": {
        "version": "1.1",
        "method": "GET",
        "path": ["/"],
        "headers": {
          "Host": ["www.example.com"]
        }
      },
      "response": {
        "version": "1.1",
        "status": "200",
        "reason": "OK"
      }
    }
  }
}
```

Lưu ý: `path` trên đường truyền là mảng chuỗi, và mỗi tiêu đề là mảng giá trị (`Host` → `["www.example.com"]`).

6.3. mKCP (`kcpSettings`)

mKCP – truyền tải đáng tin cậy qua UDP. Hữu ích trên các kênh có mật gói và độ trễ cao, nhưng tạo ra lưu lượng phục vụ cao hơn. Tất cả các giá trị mặc định đều khớp với các giá trị được khuyến nghị trong xray-core.

Trường	Khóa	Mặc định	Cho phép	Mô tả
MTU	<code>mtu</code>	1350	576–1460	Kích thước gói tối đa (byte). Giảm khi có vấn đề phân mảnh

Trường	Khóa	Mặc định	Cho phép	Mô tả
TTI (ms)	<code>tti</code>	<code>20</code>	10–100	Khoảng thời gian truyền (ms). Nhỏ hơn – độ trễ thấp hơn nhưng chi phí cao hơn
Uplink (MB/s)	<code>uplinkCapacity</code>	<code>5</code>	≥ 0	Bảng thông tải lên ước tính (MB/s)
Downlink (MB/s)	<code>downlinkCapacity</code>	<code>20</code>	≥ 0	Bảng thông tải xuống ước tính (MB/s)
Hệ số CWND	<code>cwndMultiplier</code>	<code>1</code>	≥ 1	Hệ số cửa sổ tắc nghẽn (congestion window)
Cửa sổ gửi tối đa	<code>maxSendingWindow</code>	<code>2097152</code>	≥ 0	Kích thước cửa sổ gửi tối đa

Ghi chú về các trường:

- **Uplink / Downlink capacity** xác định mức độ tích cực mKCP chiếm dụng kênh. Đặt theo bảng thông thực tế của kênh: giá trị quá cao dẫn đến lưu lượng thừa, quá thấp – khai thác kênh không hiệu quả.
- **TTI** trực tiếp ảnh hưởng đến sự đánh đổi «độ trễ [?] chi phí»: giá trị nhỏ hơn giảm độ trễ nhưng tăng lượng gói phục vụ.
- **MTU** giới hạn kích thước một gói mKCP; giảm giá trị giúp trên các kênh mà các gói UDP lớn bị cắt hoặc mất.

Trong bản build này trường «seed» (mật khẩu che giấu mKCP) và danh sách thả xuống **loại tiêu đề/che giấu** (`none` , `srtcp` , `utp` , `wechat-video` , `dtls` , `wireguard`) trong biểu mẫu con mKCP **không có dưới dạng các trường riêng biệt** – che giấu tầng truyền tải đã được tách ra thành cơ chế chung «FinalMask» (bao gồm chế độ `mkcp-legacy`), được mô tả trong mục tương ứng. Thông số «congestion» dưới dạng hộp kiểm riêng biệt cũng không được hiển thị; kiểm soát tắc nghẽn được thiết lập thông qua `cwndMultiplier` và `maxSendingWindow` .

6.4. WebSocket (`wsSettings`)

Truyền tải WebSocket qua HTTP(S). Đi qua CDN và proxy ngược tốt, giả mạo thành lưu lượng web thông thường.

Trường	Khóa	Mặc định	Mô tả
Proxy Protocol	<code>acceptProxyProtocol</code>	<code>false</code>	Chấp nhận tiêu đề PROXY protocol từ proxy phía trước (xem mục 6.2)
Host	<code>host</code>	<code>""</code> (trống)	Giá trị tiêu đề HTTP <code>Host</code> . Chỉ định khi làm việc qua CDN/ domain fronting
Đường dẫn	<code>path</code>	<code>/</code>	Đường dẫn trong yêu cầu bắt tay WebSocket
Chu kỳ heartbeat	<code>heartbeatPeriod</code>	<code>0</code>	Khoảng thời gian gửi khung heartbeat (giây). <code>0</code> tắt heartbeat

Trường	Khóa	Mặc định	Mô tả
Tiêu đề	headers	{ } (trống)	Các tiêu đề HTTP bắt tay bổ sung. Map «Tên → Giá trị» (chỉ giá trị chuỗi, không có mảng)

Ghi chú:

- **Đường dẫn** phải khớp trên máy chủ (inbound) và ở phía khách hàng. Thường đường dẫn này được dùng để che giấu điểm truy cập ở phía máy chủ web.
- **Host** có ý nghĩa khi inbound đứng sau CDN hoặc sử dụng domain fronting; nếu không có thể để trống.
- **Chu kỳ heartbeat** giữ kết nối «còn sống» khi đi qua proxy/CDN cắt đứt phiên không hoạt động một cách tích cực. Theo mặc định (0) heartbeat bị tắt.
- Khác với RAW, bảng tiêu đề WebSocket sử dụng định dạng «phẳng» tên → giá trị (một dòng giá trị cho mỗi tiêu đề).

Ví dụ `streamSettings` cho WebSocket sau CDN:

```
{
  "network": "ws",
  "wsSettings": {
    "acceptProxyProtocol": false,
    "host": "cdn.example.com",
    "path": "/ray",
    "heartbeatPeriod": 0,
    "headers": {
      "User-Agent": "Mozilla/5.0"
    }
  }
}
```

Các giá trị `host` và `path` phải khớp ở phía khách hàng; khác với RAW, giá trị tiêu đề ở đây là chuỗi thông thường, không phải mảng.

6.5. gRPC (`grpcSettings`)

Truyền tải «nhẹ nhàng» nhất về số lượng thông số. Tạo đường hầm lưu lượng bên trong các lời gọi gRPC (qua HTTP/2); tương thích tốt với CDN hỗ trợ gRPC. Không có che giấu tiêu đề.

Trường	Khóa	Mặc định	Mô tả
Tên dịch vụ (<code>Service Name</code>)	<code>serviceName</code>	"" (trống)	Tên dịch vụ gRPC (thực tế là «đường dẫn» của đường hầm). Phải khớp ở máy chủ và khách hàng
Authority	<code>authority</code>	"" (trống)	Giá trị pseudo-header <code>:authority</code> (tương tự <code>Host</code> cho HTTP/2). Chỉ định khi làm việc qua CDN/tên miền
Multi Mode	<code>multiMode</code>	<code>false</code> (tắt)	Bật ghép kênh nhiều luồng gRPC song song bên trong một kết nối

Ghi chú:

- **Service Name** — định danh chính của kênh gRPC; phải giống nhau ở cả hai phía. Giá trị trống được phép, nhưng thường đặt một chuỗi không rõ ràng để che giấu.
- **Authority** ảnh hưởng đến `:authority` nào được gửi trong các khung HTTP/2; cần thiết chủ yếu khi proxy qua CDN.
- **Multi Mode** cho phép nhiều luồng logic đi qua một kết nối vật lý; bật để cải thiện hiệu năng khi cả máy chủ và khách hàng đều hỗ trợ điều này.

Ví dụ `streamSettings` cho gRPC:

```
{
  "network": "grpc",
  "grpcSettings": {
    "serviceName": "GunService",
    "authority": "grpc.example.com",
    "multiMode": false
  }
}
```

`serviceName` (ở đây `GunService`) đóng vai trò «đường dẫn» của đường hầm và phải khớp ở máy chủ và khách hàng.

6.6. HTTPUpgrade (`httpupgradeSettings`)

Truyền tải dựa trên cơ chế HTTP Upgrade (như WebSocket, nhưng không có bản thân giao thức WebSocket). Cũng đi qua proxy và CDN tốt. Tập hợp trường lặp lại WebSocket, nhưng **không có** chu kỳ heartbeat.

Trường	Khóa	Mặc định	Mô tả
Proxy Protocol	<code>acceptProxyProtocol</code>	<code>false</code>	Chấp nhận tiêu đề PROXY protocol từ proxy phía trước
Host	<code>host</code>	<code>""</code> (trống)	Giá trị tiêu đề HTTP Host
Đường dẫn	<code>path</code>	<code>/</code>	Đường dẫn của yêu cầu HTTP với tiêu đề Upgrade
Tiêu đề	<code>headers</code>	<code>{}</code> (trống)	Các tiêu đề HTTP bổ sung. Map «phẳng» tên → giá trị (như WebSocket)

Mục đích của các trường **Host**, **Đường dẫn** và **Tiêu đề** giống với WebSocket (mục 6.4). Heartbeat không được cung cấp cho HTTPUpgrade — đây là đặc điểm riêng của WebSocket.

6.7. XHTTP / SplitHTTP (`xhttpSettings`)

XHTTP (còn gọi là SplitHTTP) — truyền tải HTTP ghép kênh hiện đại của xray-core. Phân tách luồng đi lên và đi xuống thành các yêu cầu HTTP riêng biệt, phù hợp tốt cho CDN và môi trường có giới hạn về thời lượng

kết nối. Không phải tất cả các trường đều hiển thị trong trình soạn thảo cùng một lúc: một số xuất hiện tùy thuộc vào chế độ được chọn (`mode`) và các công tắc đã bật.

Các trường cơ bản (luôn hiển thị)

Trường	Khóa	Mặc định	Mô tả
Host	<code>host</code>	<code>""</code> (trống)	Giá trị tiêu đề HTTP <code>Host</code>
Đường dẫn	<code>path</code>	<code>/</code>	Đường dẫn cơ bản của các yêu cầu HTTP
Chế độ (<code>Mode</code>)	<code>mode</code>	<code>auto</code>	Chế độ truyền (xem bên dưới)
Server Max Header Bytes	<code>serverMaxHeaderBytes</code>	<code>0</code>	Giới hạn kích thước tiêu đề yêu cầu trên máy chủ (byte). <code>0</code> – giá trị mặc định của xray-core
Padding Bytes	<code>xPaddingBytes</code>	<code>100–1000</code>	Phạm vi đệm ngẫu nhiên (byte, định dạng <code>min–max</code>) để gây khó khăn cho việc phân tích kích thước
Tiêu đề	<code>headers</code>	<code>{}</code> (trống)	Các tiêu đề HTTP bổ sung. Map «phẳng» tên → giá trị
Phương thức HTTP Uplink	<code>uplinkHTTPMethod</code>	<code>""</code> (Default = POST)	Phương thức HTTP của các yêu cầu đi lên. Tùy chọn: trống (mặc định POST), <code>POST</code> , <code>PUT</code> , <code>GET</code> (cái sau chỉ khả dụng trong chế độ <code>packet–up</code>)
Padding Obfs Mode	<code>xPaddingObfsMode</code>	<code>false</code> (tắt)	Bật che giấu đệm nâng cao và mở các trường bổ sung (xem bên dưới)
No SSE Header	<code>noSSEHeader</code>	<code>false</code> (tắt)	Không gửi tiêu đề <code>Content–Type: text/event–stream</code> (SSE). Bật nếu nó cản trở việc đi qua các nút trung gian

Trường «Chế độ» (`mode`)

Danh sách thả xuống với các giá trị:

Giá trị	Mô tả
<code>auto</code>	Tự động chọn chế độ (mặc định)
<code>packet–up</code>	Luồng đi lên được chia thành các yêu cầu HTTP riêng biệt (một gói mỗi yêu cầu)
<code>stream–up</code>	Luồng đi lên được truyền trong một yêu cầu phát trực tiếp kéo dài
<code>stream–one</code>	Một yêu cầu phát trực tiếp hai chiều chung

Việc chọn chế độ xác định các trường bổ sung nào trở nên hiển thị.

Các trường chỉ hiển thị khi `mode` = `packet–up` :

Trường	Khóa	Mặc định	Mô tả
Tải lên được đệm tối đa	<code>scMaxBufferedPosts</code>	<code>30</code>	Số yêu cầu POST đi lên được đệm đồng thời tối đa
Kích thước tải lên tối đa (byte)	<code>scMaxEachPostBytes</code>	<code>1000000</code>	Kích thước tối đa của một yêu cầu POST đi lên (byte)
Uplink Data Placement	<code>uplinkDataPlacement</code>	<code>""</code> (Default = body)	Nơi đặt dữ liệu luồng đi lên: <code>body</code> , <code>header</code> , <code>cookie</code> , <code>query</code>
Uplink Data Key	<code>uplinkDataKey</code>	<code>""</code>	Tên khóa/tiêu đề cho dữ liệu uplink. Chỉ xuất hiện nếu <code>uplinkDataPlacement</code> được đặt và không bằng <code>body</code>

Trường chỉ hiển thị khi `mode = stream-up` :

Trường	Khóa	Mặc định	Mô tả
Stream-Up Server	<code>scStreamUpServerSecs</code>	<code>20-80</code>	Phạm vi thời gian giữ kết nối phát trực tiếp phía máy chủ (giây, định dạng <code>min-max</code>)

Các trường che giấu đệm (hiển thị khi `xPaddingObfsMode = bật`)

Trường	Khóa	Mặc định	Mô tả
Padding Key	<code>xPaddingKey</code>	<code>""</code> (placeholder <code>x_padding</code>)	Tên khóa cho đệm
Padding Header	<code>xPaddingHeader</code>	<code>""</code> (placeholder <code>X-Padding</code>)	Tên tiêu đề HTTP truyền đệm
Padding Placement	<code>xPaddingPlacement</code>	<code>""</code> (Default = <code>queryInHeader</code>)	Nơi đặt đệm: <code>queryInHeader</code> , <code>header</code> , <code>cookie</code> , <code>query</code>
Padding Method	<code>xPaddingMethod</code>	<code>""</code> (Default = <code>repeat-x</code>)	Phương thức tạo đệm: <code>repeat-x</code> hoặc <code>tokenish</code>

Vị trí phiên và chuỗi (luôn hiển thị)

Trường	Khóa	Mặc định	Mô tả
Session ID Placement	<code>sessionIDPlacement</code>	<code>""</code> (Default = <code>path</code>)	Nơi truyền định danh phiên: <code>path</code> , <code>header</code> , <code>cookie</code> , <code>query</code>
Session ID Key	<code>sessionIDKey</code>	<code>""</code> (placeholder <code>x_session</code>)	Tên khóa phiên. Chỉ xuất hiện nếu <code>sessionIDPlacement</code> được đặt và không bằng <code>path</code>

Trường	Khóa	Mặc định	Mô tả
Session ID Table	<code>sessionIDTable</code>	"" (placeholder Base62)	Bộ ký tự để tạo định danh phiên. Có thể chọn từ danh sách tự động hoàn thành được xác định trước (ALPHABET , Alphabet , BASE36 , Base62 , HEX , alphabet , base36 , hex , number) hoặc nhập chuỗi ASCII tùy ý. Trống – giá trị mặc định của xray-core
Session ID Length	<code>sessionIDLength</code>	"" (trống)	Độ dài hoặc phạm vi (ví dụ 8–16) của định danh được tạo. Chỉ hiển thị khi Session ID Table được đặt; giá trị tối thiểu phải lớn hơn 0
Sequence Placement	<code>seqPlacement</code>	"" (Default = path)	Nơi truyền số thứ tự gói: path , header , cookie , query
Sequence Key	<code>seqKey</code>	"" (placeholder x_seq)	Tên khóa chuỗi. Chỉ xuất hiện nếu seqPlacement được đặt và không bằng path

Các trường phiên được đổi tên theo xray-core v26.6.22: trước đây chúng được gọi là **Session Placement / Session Key** (`sessionPlacement` / `sessionKey`) – bây giờ là **Session ID Placement / Session ID Key** (`sessionIDPlacement` / `sessionIDKey`); phần lỗi không còn hiểu tên cũ nữa. Các inbound được lưu trước khi cập nhật sẽ được tự động chuyển sang các khóa mới – không cần lưu lại.

Khuyến nghị:

- Đối với hầu hết các cài đặt, chỉ cần để **Chế độ = auto** , đặt **Đường dẫn/Host** và (khi làm việc qua CDN) đồng bộ chúng với khách hàng.
- Các trường vị trí (`*Placement` / `*Key`) và che giấu đệm chỉ cần thiết để tinh chỉnh cho tình huống anti-DPI/CDN cụ thể; khi để trống, các giá trị mặc định của xray-core được ghi trong ngoặc sẽ được sử dụng.
- Các thông số liên quan đến phía khách hàng/outbound (ví dụ: khoảng thời gian POST lặp lại, kích thước chunk) không hiển thị trong biểu mẫu inbound – máy chủ lắng nghe bỏ qua chúng. Bộ ghép kênh XMUX, ngược lại, khả dụng trong biểu mẫu inbound (xem bên dưới).
- **Các giá trị mặc định phục vụ không được đặt.** Panel không còn ghi các giá trị mặc định phục vụ `scMaxEachPostBytes` và `scMinPostsIntervalMs` vào cấu hình XHTTP – các giá trị nội bộ của xray-core được áp dụng. Điều này loại bỏ chữ ký DPI cố định (ký tự `scMinPostsIntervalMs=30`) mà trước đây có thể bị chặn lưu lượng. Đối với các inbound đã lưu, các giá trị khớp với mặc định của xray-core không được xuất trong liên kết và đăng ký (không cần lưu lại inbound); các giá trị được đặt thủ công vẫn được lưu giữ.

Ví dụ `streamSettings` cho XHTTP (chế độ `auto`):

```
{
  "network": "xhttp",
  "xhttpSettings": {
    "host": "xhttp.example.com",
    "path": "/yourpath",
    "mode": "auto",
    "xPaddingBytes": "100-1000"
  }
}
```

Đối với hầu hết các cài đặt, bốn trường này là đủ; các trường vị trí phiên/chuỗi và che giấu đệm để trống – khi đó các giá trị mặc định của xray-core sẽ được sử dụng.

XMUX (ghép kênh kết nối)

Công tắc **XMUX** (`enableXmux`) bật lớp ghép kênh phân phối các yêu cầu song song qua một nhóm nhỏ kết nối vật lý. Khi bật, sáu trường có thể cấu hình sẽ được mở (lưu trong `xhttpSettings.xmux`):

Trường	Khóa	Mặc định	Mô tả
Max Concurrency	<code>maxConcurrency</code>	16–32	Số yêu cầu đồng thời tối đa trên một kết nối (phạm vi min–max)
Max Connections	<code>maxConnections</code>	0	Số kết nối vật lý tối đa (0 – không giới hạn)
Max Reuse Times	<code>cMaxReuseTimes</code>	"" (trống)	Số lần tái sử dụng kết nối
Max Request Times	<code>hMaxRequestTimes</code>	600–900	Số yêu cầu tối đa trên một kết nối (phạm vi)
Max Reusable Secs	<code>hMaxReusableSecs</code>	1800–3000	Thời gian kết nối có thể tái sử dụng (giây, phạm vi)
Keep Alive Period	<code>hKeepAlivePeriod</code>	"" (trống)	Chu kỳ keep-alive để duy trì kết nối

Lưu ý. Không thể đặt đồng thời **Max Connections** và **Max Concurrency** – xray-core sẽ từ chối cấu hình đó. Theo mặc định khi bật XMUX, panel đặt `Max Concurrency` = 16–32 ; nếu bạn đặt **Max Connections** (giá trị lớn hơn 0), panel sẽ xóa giá trị mặc định `Max Concurrency` để tránh xung đột.

6.8. Truyền tải Hysteria (`hysteriaSettings`)

Truyền tải **Hysteria** không được chọn trong danh sách «Truyền tải»: nó tự động được kích hoạt khi inbound được tạo với giao thức Hysteria, và bị ẩn đối với các giao thức khác (khi rời khỏi giao thức Hysteria, mạng bị buộc trở về `tcp`). Các thông số:

Trường	Khóa	Mặc định	Mô tả
Phiên bản	version	2 (cố định, trường bị khóa)	Phiên bản Hysteria. Chỉ hỗ trợ Hysteria 2
UDP idle timeout (s)	udpIdleTimeout	60	Thời gian chờ không hoạt động của phiên UDP (giây). Phạm vi cho phép – 2–600; xray-core từ chối các giá trị ngoài khoảng này khi khởi động
Masquerade	masquerade	tắt (văng mặt)	Bật che giấu trình lắng nghe thành máy chủ HTTP/3 khi thăm dò

Khi **Masquerade** được bật, sẽ xuất hiện lựa chọn loại (type) và các trường phụ thuộc vào nó:

- **"" – default (404 page)**: trả về trang 404 tiêu chuẩn (không cần trường bổ sung).
- **proxy (reverse proxy)**: proxy ngược đến trang web bên ngoài.
 - url (**Upstream URL**, placeholder `https://www.example.com`) – địa chỉ đích;
 - rewriteHost (**Ghi lại Host**, mặc định `false`) – thay thế tiêu đề `Host`;
 - insecure (**Bỏ qua xác minh TLS**, mặc định `false`) – không xác minh chứng chỉ TLS của upstream.
- **file (serve directory)**: phục vụ tệp từ thư mục.
 - dir (**Thư mục**, placeholder `/var/www/html`).
- **string (fixed body)**: phản hồi HTTP cố định.
 - statusCode (**Mã trạng thái**, mặc định `0`, phạm vi 0–599);
 - content (**Body**) – nội dung phản hồi;
 - headers (**Tiêu đề**) – map tên → giá trị.

Masquerade cho phép inbound dựa trên Hysteria trông như một máy chủ HTTP/3 thông thường đối với các thăm dò chủ động, giúp tăng tính ẩn dật. Theo mặc định, che giấu bị tắt.

Ví dụ hysteriaSettings với proxy ngược (masquerade → proxy):

```
{
  "version": 2,
  "udpIdleTimeout": 60,
  "masquerade": {
    "type": "proxy",
    "url": "https://www.example.com",
    "rewriteHost": true,
    "insecure": false
  }
}
```

Ở đây khi thăm dò, trình lắng nghe trả về phản hồi từ `https://www.example.com`, giả mạo thành một trang web HTTP/3 thông thường.

6.9. Các thông số đi kèm

Ngoài việc chọn mạng, trong cùng tab còn có hai khối chung không phụ thuộc vào truyền tải cụ thể (chi tiết – trong các mục tương ứng):

- **External Proxy** (`externalProxy`) – danh sách địa chỉ/cổng bên ngoài được thay thế vào liên kết đăng ký thay vì địa chỉ của panel.
- **Sockopt** (`sockopt`) – các tùy chọn socket cấp thấp (TCP Fast Open, mark, chiến lược tên miền, proxy trong suốt, v.v.).

Real client IP (xác định IP thực sau CDN/relay)

Khi inbound đứng sau trung gian (CDN như Cloudflare, đường hầm/relay L4 hoặc panel khác), Xray thấy địa chỉ của trung gian, không phải khách truy cập thực. Địa chỉ này xuất hiện trong danh sách khách hàng trực tuyến và theo đó số lượng IP trên mỗi khách hàng được đếm, khiến cả hai trở nên vô dụng sau proxy. Để khôi phục IP thực, trong phần **Sockopt** của biểu mẫu inbound có lựa chọn preset **Real client IP**, kết hợp các cài đặt `acceptProxyProtocol` và `trustedXForwardedFor` :

Preset	Tác dụng	Khi nào áp dụng
Off / direct	Xóa cả hai trường.	Inbound được khách hàng truy cập trực tiếp
Cloudflare CDN	Đặt <code>sockopt.trustedXForwardedFor = ["CF-Connecting-IP"]</code> .	WebSocket / HTTPUpgrade / XHTTP / gRPC sau CDN Cloudflare (đám mây màu cam)
L4 relay / Spectrum (PROXY)	Bật <code>acceptProxyProtocol = true</code> .	Đường hầm/relay L4 trước inbound hoặc Cloudflare Spectrum

Các preset loại trừ lẫn nhau: chọn một cái sẽ xóa trường của cái kia, do đó `trustedXForwardedFor` cũ không ghi đè IP được khôi phục qua giao thức PROXY. Bên dưới preset, công tắc «thô» **Proxy Protocol** và danh sách **Trusted X-Forwarded-For** vẫn hiển thị – preset chỉ điền chúng cho bạn, và khi cần thiết có thể chỉnh sửa thủ công. Nếu preset được chọn không được hỗ trợ bởi truyền tải hiện tại (ví dụ PROXY-protocol trên mKCP), biểu mẫu hiển thị cảnh báo. Các trường này chỉ liên quan đến phía máy chủ và **không bao giờ được gửi đến khách hàng trong đăng ký**.

Chỉ dùng một cái. `acceptProxyProtocol` đọc IP thực từ tiêu đề L4 của giao thức PROXY, còn `trustedXForwardedFor` – từ tiêu đề HTTP của yêu cầu; kết hợp chúng thủ công chỉ nên làm khi chuỗi trung gian của bạn yêu cầu điều đó.

- **FinalMask** (`finalmask`) – cơ chế che giấu tầng truyền tải chung (bao gồm che giấu legacy mKCP), thay thế các trường riêng biệt «seed»/«header type» bên trong các biểu mẫu con của mạng.

7. Bảo mật kết nối: TLS, XTLS và REALITY

Mỗi inbound hỗ trợ truyền qua luồng transport (VMess, VLESS, Trojan, Shadowsocks, Hysteria) đều có tab «**Bảo mật**» trong trình chỉnh sửa. Tại đây bạn cấu hình cách mã hóa và ngụy trang kênh truyền. Có ba chế độ, chuyển đổi bằng các nút radio:

Chế độ	Nhãn trong UI	Khi nào khả dụng
none	Không	Luôn luôn (trừ Hysteria, nơi TLS là bắt buộc)
tls	TLS	Dành cho VMess/VLESS/Trojan/Shadowsocks trên các mạng tcp, ws, http, grpc, httpupgrade, xhttp; với Hysteria – luôn luôn
reality	Reality	Chỉ dành cho VLESS/Trojan trên các mạng tcp, http, grpc, xhttp

Nút **Không** không hiển thị nếu giao thức là Hysteria (TLS là bắt buộc với nó). Nút **Reality** chỉ xuất hiện khi kết hợp giao thức và mạng hợp lệ (xem bảng trên).

Khi thay đổi chế độ, bảng điều khiển sẽ xây dựng lại hoàn toàn khối `streamSettings`: xóa `tlsSettings` và `realitySettings` của chế độ trước và thay thế bằng các giá trị mặc định cho chế độ đã chọn. Cụ thể, khi chọn **Reality**, bảng điều khiển ngay lập tức tự động: điền một cặp ngẫu nhiên `target` + `serverNames` (SNI) từ danh sách tên miền phổ biến tích hợp sẵn, tạo ngẫu nhiên `shortIds`, đồng thời gửi yêu cầu đến máy chủ để lấy cặp khóa X25519 mới (`privateKey/publicKey`).

7.1. Sự khác biệt: TLS vs XTLS vs REALITY

- **TLS** – mã hóa transport cổ điển theo giao thức TLS 1.2/1.3. Máy chủ phải có chứng chỉ hợp lệ (tên miền riêng + chuỗi chứng chỉ). Lưu lượng trông giống HTTPS thông thường, nhưng với kẻ kiểm duyệt tích cực thì TLS-handshake đến tên miền của bạn có thể nhận dạng được; khi bị chặn theo SNI hoặc thiếu chứng chỉ đáng tin cậy, kết nối sẽ bị chặn hoặc báo lỗi.
- **XTLS (Vision)** – đây không phải chế độ riêng trong danh sách «Bảo mật», mà là cơ chế *flow* trên nền TLS hoặc Reality. Được bật ở phía client của inbound thông qua trường **Flow** = `xtls-rprx-vision` (hoặc `xtls-rprx-vision-udp443`). Vision khả dụng cho VLESS trên mạng tcp khi `security = tls` hoặc `security = reality`, cũng như cho VLESS qua transport `xhttp` khi bật mã hóa VLESS (`vlessenc / ML-KEM`) – trong trường hợp này trường **Flow** cũng có thể đặt thành `xtls-rprx-vision`, và giá trị đó sẽ được đưa chính xác vào liên kết `vless:// (flow=xtls-rprx-vision)`. Vision giảm «mã hóa kép» (TLS-in-TLS), truyền tải trực tiếp sau khi bắt tay, giúp tăng tốc độ truyền và cải thiện khả năng ngụy trang. Do đó, tổ hợp **VLESS + Reality + Flow xtls-rprx-vision** được coi là cấu hình hiện đại được khuyến nghị.
- **REALITY** – cơ chế ngụy trang không cần chứng chỉ riêng. Máy chủ «mượn» TLS-handshake của một trang web bên ngoài thực sự (`target / serverNames`), vì vậy đối với người quan sát, kết nối không thể phân biệt được với việc truy cập trang web đó, và không cần chứng chỉ nào cả. Xác thực dựa trên cặp khóa X25519 và tập `shortIds`. REALITY kháng được active probing và chặn theo SNI, vì SNI trở nên một tên miền bên ngoài thực sự. Đổi lại – yêu cầu cấu hình chặt chẽ hơn (phải có `target` đúng với cổng, đồng bộ khóa với client).

Quy tắc lựa chọn ngắn gọn:

- có tên miền riêng và chứng chỉ hợp lệ, cần giao diện HTTPS đơn giản → **TLS** (nếu có thể kết hợp với Vision);
- không có tên miền/chứng chỉ hoặc cần ẩn danh tối đa trước DPI → **REALITY** (kết hợp Vision cho VLESS/TCP).

7.2. Chế độ «Không» (none)

Transport được truyền không có lớp bọc TLS: các khối `tlsSettings` và `realitySettings` bị loại khỏi `streamSettings`. Chế độ này không có trường bổ sung. Phù hợp khi:

- inbound chỉ lắng nghe trên `127.0.0.1` và đóng vai trò đích fallback (theo quy tắc bảng điều khiển, inbound con dành cho fallback phải lắng nghe trên `127.0.0.1` với `security=none`);
- mã hóa/ngụy trang được đảm bảo bởi lớp bên ngoài (ví dụ: reverse proxy Nginx trước bảng điều khiển);
- sử dụng giao thức có mã hóa riêng (Shadowsocks) trong mạng nội bộ.

Đối với inbound có thể truy cập từ bên ngoài, không nên dùng chế độ «Không».

Ví dụ: khối `streamSettings` cho TLS trên mạng `tcp` (VLESS/Trojan/VMess). Đây là kết quả sau khi chọn chế độ **TLS** và điền SNI cùng đường dẫn tới chứng chỉ:

```
"streamSettings": {
  "network": "tcp",
  "security": "tls",
  "tlsSettings": {
    "serverName": "vpn.example.com",
    "minVersion": "1.2",
    "maxVersion": "1.3",
    "alpn": ["h2", "http/1.1"],
    "settings": { "fingerprint": "chrome" },
    "certificates": [
      {
        "certificateFile": "/root/cert/vpn.example.com.crt",
        "keyFile": "/root/cert/vpn.example.com.key",
        "ocspStapling": 3600,
        "usage": "encipherment"
      }
    ]
  }
}
```

7.3. Chế độ TLS

Các trường của khối `tlsSettings`. Giá trị mặc định lấy từ schema của bảng điều khiển.

Các tham số chính

Trường (nhân)	Giá trị mặc định	Mô tả
SNI (<code>serverName</code>)	<code>""</code> (trống)	Server Name Indication – tên miền được cung cấp trong TLS-handshake. Phải khớp với tên miền của chứng chỉ. Placeholder tiếng Anh: «Server Name Indication».
Cipher Suites (<code>cipherSuites</code>)	<code>""</code> → Tự động	Danh sách bộ mã hóa được phép. Mặc định để trống – việc lựa chọn được giao cho Xray/Go (tùy chọn Tự động). Chỉ thay đổi khi cần hạn chế bộ mã hóa một cách rõ ràng.
Phiên bản Tối thiểu/Tối đa (<code>minMaxVersion</code>)	min = <code>1.2</code> , max = <code>1.3</code>	Phiên bản TLS tối thiểu và tối đa. Các giá trị khả dụng: <code>1.0</code> , <code>1.1</code> , <code>1.2</code> , <code>1.3</code> . Khuyến nghị giữ <code>1.2</code> – <code>1.3</code> ; không nên hạ xuống 1.0/1.1 (các phiên bản cũ, không an toàn).
uTLS (<code>settings.fingerprint</code>)	<code>chrome</code> (trong form – mục None = <code>""</code> có sẵn)	Dấu vân tay TLS được giả lập trong client hello (uTLS fingerprint), để handshake trông giống trình duyệt phổ biến. Xem danh sách bên dưới. Trong TLS, mục đầu tiên là None (<code>""</code>), tắt giả lập.
ALPN (<code>alpn</code>)	<code>["h2", "http/1.1"]</code>	Danh sách giao thức tầng ứng dụng được thỏa thuận trong TLS (chọn nhiều). Các giá trị được phép: <code>h3</code> , <code>h2</code> , <code>http/1.1</code> . Mặc định để xuất <code>h2</code> và <code>http/1.1</code> .

Các giá trị có thể có của **uTLS fingerprint** (giống nhau cho TLS và REALITY): `chrome` , `firefox` , `safari` , `ios` , `android` , `edge` , `360` , `qq` , `random` , `randomized` , `randomizednoalpn` , `unsafe` . Trong form TLS có thêm tùy chọn trống **None** (không áp dụng giả lập dấu vân tay).

Các giá trị khả dụng của **Cipher Suites** (TLS 1.3 và bộ ECDHE): `TLS_AES_128_GCM_SHA256` , `TLS_AES_256_GCM_SHA384` , `TLS_CHACHA20_POLY1305_SHA256` , `TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA` , `TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA` , `TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA` , `TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA` , `TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256` , `TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384` , `TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256` , `TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384` , `TLS_ECDHE_ECDSA_WITH_CHACHA20_POLY1305_SHA256` , `TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305_SHA256` .

Các công tắc TLS

Công tắc	Mặc định	Mô tả
Từ chối SNI không xác định (<code>rejectUnknownSni</code>)	tắt (<code>false</code>)	Nếu bật, máy chủ ngắt kết nối khi SNI do client cung cấp không khớp với tên trong chứng chỉ. Tăng khả năng ẩn danh (máy chủ không phản hồi các yêu cầu «lạ»), nhưng yêu cầu SNI của client phải khớp chính xác.
Tắt System Root (<code>disableSystemRoot</code>)	tắt (<code>false</code>)	Tắt việc sử dụng kho lưu trữ chứng chỉ gốc đáng tin cậy của hệ thống.
Tiếp tục phiên (<code>enableSessionResumption</code>)	tắt (<code>false</code>)	Bật tiếp tục phiên TLS (session resumption / session tickets).

Các tham số TLS bổ sung (vcn, đường cong, log khóa, ECH Sockopt)

Bên dưới các cài đặt TLS chính có thêm các trường bổ sung.

Trường (nhân)	Mặc định	Mô tả
Verify Peer Cert By Name (<code>settings.verifyPeerCertByName</code>)	""	Tên (phân cách bằng dấu phẩy) mà client dùng để xác minh chứng chỉ máy chủ thay vì SNI. Đây là sự thay thế hiện đại cho trường <code>allowInsecure</code> đã bị loại khỏi Xray sau ngày 2026-06-01. Giá trị chỉ dành cho bảng điều khiển: không được ghi vào config xray của máy chủ, nhưng được truyền vào liên kết mời và subscription (<code>vcn=...</code>) để client tự áp dụng. Placeholder: <code>example.com</code> .
Curve Preferences (<code>curvePreferences</code>)	""	Giới hạn và thứ tự ưu tiên các đường cong trao đổi khóa TLS (ví dụ <code>X25519MLKEM768</code> , <code>X25519</code>). Để trống – dùng giá trị mặc định của Xray-core.
Master Key Log (<code>masterKeyLog</code>)	""	Đường dẫn ghi TLS master keys theo định dạng <code>SSLKEYLOGFILE</code> (để giải mã lưu lượng trong Wireshark khi gỡ lỗi). Placeholder: <code>/path/to/sslkeylog.txt</code> . Trong môi trường production để trống – file này cho phép giải mã toàn bộ lưu lượng.
ECH Sockopt (<code>echSockopt</code>)	tắt	Công tắc với các tham số socket cho kết nối mà Xray dùng để truy vấn ECH config list. Khi bật có thêm: Dialer Proxy (<code>dialerProxy</code> – chuyển yêu cầu qua outbound chỉ định theo tag), Domain Strategy (<code>domainStrategy</code>), TCP Fast Open (<code>tcpFastOpen</code>), Multipath TCP (<code>tcpMptcp</code>). Nên để tắt nếu không cần thiết.

Các trường `verifyPeerCertByName` , `curvePreferences` , `masterKeyLog` và `echSockopt` nằm ở cấp cao nhất của `tlsSettings` và được giữ nguyên sau khi bảng điều khiển cắt xén các trường khi lưu cấu hình.

Chứng chỉ

Phần **SSL-chứng chỉ** (tiêu đề trong UI «SSL-chứng chỉ») được thiết lập dưới dạng danh sách: nhấn nút **+** để thêm mục chứng chỉ mới, nút **- Xóa** để xóa (nút xóa chỉ khả dụng khi có nhiều hơn một mục). Mặc định khi bật TLS sẽ tạo một mục trống.

Với mỗi mục, công tắc chế độ nhập (`useFile`):

- **Đường dẫn đến chứng chỉ** (giá trị `useFile = true` , mặc định) – chỉ định đường dẫn đến các file trên máy chủ:
 - **Khóa công khai** (`certificateFile`) – đường dẫn đến file chứng chỉ (`.crt` / `.pem`);
 - **Khóa riêng tư** (`keyFile`) – đường dẫn đến file khóa riêng tư (`.key`).
- **Nội dung chứng chỉ** (giá trị `useFile = false`) – nội dung được dán trực tiếp vào các trường (vùng văn bản nhiều dòng):
 - **Khóa công khai** (`certificate`) – nội dung PEM của chứng chỉ;
 - **Khóa riêng tư** (`key`) – nội dung PEM của khóa.

Bên dưới các trường chế độ «Đường dẫn đến chứng chỉ» có hai nút:

- **Đặt chứng chỉ bảng điều khiển** – điền vào các trường đường dẫn đến chứng chỉ SSL của chính bảng điều khiển. Với inbound trên bảng điều khiển trung tâm, lấy chứng chỉ của nó (`POST /panel/setting/all → webCertFile / webKeyFile`); với inbound được gán cho node – lấy chứng chỉ của chính node đó (`GET /panel/api/nodes/webCert/{nodeId}`), vì đường dẫn của bảng điều khiển trung tâm không tồn tại trên node. Nếu chứng chỉ chưa được cấu hình, hiển thị cảnh báo: «*Bảng điều khiển chưa được cấu hình chứng chỉ. Vui lòng cài đặt trước trong Cài đặt.*» (chứng chỉ của bảng điều khiển được thiết lập trong phần «Cài đặt → Bảo mật»).
- **Xóa** – xóa cả hai đường dẫn.

Các trường bổ sung của mỗi mục chứng chỉ:

Trường	Mặc định	Mô tả
OCSP Stapling (<code>ocspStapling</code>)	0 (tắt)	Khoảng thời gian cập nhật OCSP-stapling tính bằng giây (tối thiểu 0). Mặc định tắt (0) cho inbound mới: điều này loại bỏ lỗi trong log xray cho các chứng chỉ không có OCSP-responder (ví dụ: Let's Encrypt đã ngừng OCSP). Chỉ bật cho các chứng chỉ hỗ trợ stapling.
Tải một lần (<code>oneTimeLoading</code>)	tắt (<code>false</code>)	Nếu bật, chứng chỉ được đọc từ đĩa một lần khi khởi động và không được đọc lại khi file thay đổi.
Tùy chọn sử dụng (<code>usage</code>)	<code>encipherment</code>	Mục đích của chứng chỉ. Các giá trị được phép: <code>encipherment</code> (mã hóa – chứng chỉ máy chủ thông thường), <code>verify</code> (xác minh), <code>issue</code> (phát hành – máy chủ tự ký/phát hành chứng chỉ).
Build Chain (<code>buildChain</code>)	tắt (<code>false</code>)	Chỉ hiển thị khi <code>usage = issue</code> . Xây dựng chuỗi chứng chỉ.

Không có nút riêng để tạo chứng chỉ tự ký trong trình chỉnh sửa inbound: bảng điều khiển không tạo chứng chỉ tự ký tức thời cho inbound. Chứng chỉ phải được chỉ định bằng đường dẫn/nội dung, hoặc lấy từ cài đặt bảng điều khiển bằng nút «Đặt chứng chỉ bảng điều khiển». Việc phát hành/lấy chứng chỉ SSL cho chính bảng điều khiển (bao gồm tải file và liên kết tên miền) được thực hiện trong phần **Cài đặt → Bảo mật**; không có endpoint ACME/Let's Encrypt cho inbound ở đây.

ECH và ghim chứng chỉ (các trường TLS nâng cao)

Trường	Mặc định	Mô tả
ECH key (<code>echServerKeys</code>)	""	Khóa máy chủ Encrypted Client Hello.
ECH config (<code>settings.echConfigList</code>)	""	ECH config list (phần client, được đưa vào liên kết).

Trường	Mặc định	Mô tả
SHA-256 của chứng chỉ đồng đăng (<code>settings.pinnedPeerCertSha256</code>)	<code>[]</code>	Hash SHA-256 của chứng chỉ đồng đăng (chuỗi hex, phân cách bằng dấu phẩy). Gợi ý nguyên văn: « <i>Hash SHA-256 của chứng chỉ đồng đăng dưới dạng chuỗi thập lục phân (ví dụ e8e2d3...), phân cách bằng dấu phẩy. Chỉ dành cho bảng điều khiển – không được ghi vào config xray của máy chủ, nhưng được đưa vào liên kết mời để client có thể ghim chứng chỉ.</i> »

Các nút: Cạnh trường **SHA-256 của chứng chỉ đồng đăng** có hai nút tự động điền:

- **Fill from this inbound's certificate** (biểu tượng khiên) – điền hash SHA-256 của chứng chỉ của chính inbound này (lấy cục bộ qua endpoint `getCertHash`).
- **Fetch the hash by pinging the SNI (xray tls ping)** (biểu tượng tải xuống) – lấy hash của chứng chỉ máy chủ đang chạy bằng cách thực hiện kết nối TLS theo SNI đã chỉ định (trên máy chủ gọi `getRemoteCertHash`). Trường **SNI** (`serverName`) phải được điền – nếu không hiển thị gợi ý «*Set the SNI (serverName) first to ping the remote certificate.*»

Các hash thu được được thêm vào trường (phân cách bằng dấu phẩy) và đưa vào liên kết mời để client có thể ghim chứng chỉ.

- **Lấy chứng chỉ ECH mới** – yêu cầu máy chủ cung cấp cặp ECH mới theo SNI hiện tại (`POST /panel/api/server/getNewEchCert` , trên máy chủ thực thi `xray tls ech --serverName <SNI>`); điền vào các trường **ECH key** và **ECH config**.
- **Xóa** – đặt lại cả hai trường ECH về trống.

7.4. Chế độ REALITY

Các trường của khối `realitySettings` .REALITY không sử dụng chứng chỉ SSL: thay vào đó là TLS-handshake mượn từ tên miền bên ngoài và cặp khóa X25519.

Tham số ngụy trang

Trường (nhân)	Giá trị mặc định	Mô tả
Hiển thị (<code>show</code>)	tắt (<code>false</code>)	Xuất thông tin gỡ lỗi REALITY vào log Xray. Thường để tắt.
Xver (<code>xver</code>)	<code>0</code>	Phiên bản giao thức PROXY được truyền đến backend (<code>0</code> – tắt). Tối thiểu <code>0</code> .
uTLS (<code>settings.fingerprint</code>)	<code>chrome</code>	Dấu vân tay TLS được giả lập (cùng danh sách như trong TLS, nhưng không có tùy chọn None trống).

Trường (nhân)	Giá trị mặc định	Mô tả
Đích (target)	"" (bảng điều khiển điền ngẫu nhiên khi bật)	Trường bắt buộc. Tên miền thực mà REALITY mượn TLS-handshake. Gợi ý nguyên văn: « <i>Bắt buộc. Phải chứa cổng (ví dụ: example.com:443).</i> Không có cổng Xray-core sẽ không khởi động.» Xác thực trong bảng điều khiển kiểm tra sự hiện diện và tính đúng đắn của cổng; nếu không hiển thị lỗi «Đích REALITY là bắt buộc» / «Đích REALITY phải chứa cổng...» / «Đích REALITY có cổng không hợp lệ». Nút làm mới bên cạnh điền cặp ngẫu nhiên từ danh sách tích hợp.
SNI (serverNames)	[] (được điền cùng với đích)	Danh sách SNI được phép (nhập nhiều dưới dạng tag). Phải tương ứng với tên miền trong Đích . Nút làm mới điền SNI cùng với đích ngẫu nhiên.
Chênh lệch thời gian tối đa (ms) (maxTimediff)	0	Độ lệch tối đa cho phép giữa đồng hồ client và máy chủ tính bằng mili giây (0 – không giới hạn). Tối thiểu 0 .
Phiên bản client tối thiểu (minClientVer)	""	Phiên bản client Xray tối thiểu (placeholder 25.9.11). Để trống – không giới hạn.
Phiên bản client tối đa (maxClientVer)	""	Phiên bản client Xray tối đa. Để trống – không giới hạn.
Short IDs (shortIds)	[] (được tạo khi bật)	Danh sách ID ngắn (hex) để phân biệt các client. Nhập nhiều dưới dạng tag; nút làm mới tạo một bộ ngẫu nhiên.
SpiderX (settings.spiderX)	/	Đường dẫn «spider» (phần client của REALITY), được dùng khi giả lập truy cập trang web bên ngoài. Được đưa vào liên kết mời.

Đích (target) và **SNI** (serverNames) khi bật REALITY và khi nhấn nút làm mới sẽ được điền bằng cặp ngẫu nhiên từ danh sách tích hợp của bảng điều khiển: `www.amazon.com` , `aws.amazon.com` , `www.oracle.com` , `www.nvidia.com` , `www.amd.com` , `www.intel.com` , `www.sony.com` (mỗi cái với cổng :443). Hãy chọn một trang HTTPS bên thứ ba lớn, ổn định, không nằm trên cùng máy chủ của bạn.

Ví dụ: khởi `streamSettings` cho REALITY trên mạng `tcp` (VLESS). Không cần chứng chỉ – thay vào đó là tên miền mượn và cặp khóa X25519:

```
"streamSettings": {
  "network": "tcp",
  "security": "reality",
  "realitySettings": {
    "show": false,
    "xver": 0,
    "dest": "www.nvidia.com:443",
    "serverNames": ["www.nvidia.com"],
    "privateKey": "YOUR_X25519_PRIVATE_KEY",
    "shortIds": [""],
    "settings": {
      "publicKey": "YOUR_X25519_PUBLIC_KEY",
      "fingerprint": "chrome",
      "spiderX": "/"
    }
  }
}
```

Ở đây trường **Đích** (target) trong bảng điều khiển tương ứng với dest trong config Xray đã hoàn thiện. Nếu inbound REALITY được tạo với destination trong khóa dest (bởi các phiên bản bảng điều khiển cũ, qua API hoặc công cụ bên ngoài), bảng điều khiển khi phân tích sẽ chuẩn hóa dest → target khi target trống – do đó inbound đó được tải đúng cách, trường **Đích** không bị để trống, và việc lưu lại không làm hỏng REALITY đang hoạt động.

Khóa REALITY (X25519)

Trường	Mặc định	Mô tả
Khóa công khai (settings.publicKey)	""	Khóa công khai X25519 (client đặt vào cấu hình/liên kết của mình).
Khóa riêng tư (privateKey)	""	Khóa riêng tư X25519 (chỉ lưu trên máy chủ).

Các nút bên dưới khóa:

- **Lấy chứng chỉ mới** – yêu cầu máy chủ cung cấp cặp khóa mới (GET /panel/api/server/getNewX25519Cert ; trên máy chủ thực thi xray x25519), điền **Khóa riêng tư** và **Khóa công khai**. Cặp này cũng được tạo tự động khi bật chế độ REALITY lần đầu tiên.

Ví dụ: lấy cặp khóa X25519 qua API (ngoài form, ví dụ cho script). Yêu cầu trả về khóa riêng tư và công khai:

```
curl -s -b cookie.txt https://your-panel:2053/panel/api/server/getNewX25519Cert
# 0Tbet:
# {"success":true,"obj":{"privateKey":"...","publicKey":"..."}}
```

cookie.txt – file cookie phiên, lấy được sau khi đăng nhập qua POST /login .

- **Xóa** – đặt lại cả hai khóa về trống.

Chữ ký hậu lượng tử ML-DSA-65 (mldsa65)

Lớp xác thực hậu lượng tử REALITY bổ sung (không bắt buộc):

Trường	Mặc định	Mô tả
mldsa65 Seed (<code>mldsa65Seed</code>)	""	Seed khóa ML-DSA-65 phía máy chủ.
mldsa65 Verify (<code>settings.mldsa65Verify</code>)	""	Giá trị xác minh (phần client, được đưa vào liên kết).

Các nút:

- **Lấy Seed mới** – yêu cầu cập mới (`GET /panel/api/server/getNewmldsa65` ; trên máy chủ thực thi `xray mldsa65`), điền **mldsa65 Seed** và **mldsa65 Verify**.
- **Xóa** – đặt lại cả hai trường về trống.

Giới hạn tốc độ fallback và log khóa REALITY

Trong cài đặt REALITY có thể giới hạn tốc độ lưu lượng fallback – ngăn các active probe sử dụng máy chủ như một kênh miễn phí đến tên miền được mượn. Cài đặt được thiết lập riêng cho hai hướng – **Limit Fallback Upload** và **Limit Fallback Download** (`limitFallbackUpload` / `limitFallbackDownload`), mỗi hướng có cùng bộ trường:

Trường (nhân)	Mặc định	Mô tả
After Bytes (<code>afterBytes</code>)	0	Số byte được truyền ở tốc độ đầy đủ trước khi bắt đầu giới hạn. 0 – giới hạn từ byte đầu tiên.
Bytes Per Sec (<code>bytesPerSec</code>)	0	Giới hạn tốc độ lưu lượng fallback tính bằng byte mỗi giây sau ngưỡng. 0 – không giới hạn (tắt hướng này).
Burst Bytes Per Sec (<code>burstBytesPerSec</code>)	0	Dự phòng cho các đợt tăng tốc ngăn vượt quá tốc độ ổn định (kích thước token-bucket). Nếu nhỏ hơn Bytes Per Sec , sẽ được nâng lên bằng giá trị đó.

Tại đây cũng có trường **Master Key Log** (`masterKeyLog`) – đường dẫn ghi TLS master keys theo định dạng `SSLKEYLOGFILE` để gỡ lỗi trong Wireshark; trong môi trường production để trống.

7.5. Khuyến nghị thực tế về cấu hình

1. **VLESS + Reality (khuyến nghị)**: tạo VLESS-inbound trên mạng `tcp`, trong tab «Bảo mật» chọn **Reality** – bảng điều khiển sẽ tự động điền `target /SNI`, `shortIds` ngẫu nhiên và tạo khóa X25519. Nếu cần, nhấn «Lấy chứng chỉ mới» để lấy cặp khóa của riêng bạn. Với các client VLESS, bật **Flow** = `xtls-rprx-vision` (XTLS Vision) – điều này sẽ mang lại hiệu suất và khả năng ẩn danh tối đa.

Ví dụ: liên kết client cuối cùng VLESS + Reality + Vision. Đây là liên kết mời mà bảng điều khiển tạo ra cho inbound như vậy (giá trị khóa/ID chỉ mang tính minh họa):

```
vless://uuid-клиента@1.2.3.4:443?
type=tcp&security=reality&pbk=ПУБЛИЧНЫЙ_КЛЮЧ&fp=chrome&sni=www.nvidia.com&sid=6ba85179e3
0d4fc2&spx=%2F&flow=xtls-rprx-vision#my-reality
```

Ở đây **pbk** – khóa công khai X25519, **sni** – tên miền mượn từ **Đích**, **sid** – một trong các **Short IDs**, **flow=xtls-rprx-vision** – XTLS Vision đã được bật. 2. **TLS với tên miền riêng**: chọn **TLS**, điền **SNI** bằng tên miền, thêm chứng chỉ (bằng đường dẫn đến file hoặc nội dung), hoặc nhấn «Đặt chứng chỉ bằng điều khiển» nếu tên miền và chứng chỉ đã được cấu hình trong «Cài đặt → Bảo mật». Giữ **Phiên bản Tối thiểu/Tối đa** = **1.2 – 1.3** và **uTLS** = **chrome** để ngụy trang thành trình duyệt thông thường. 3. Không để chế độ **Không** cho các inbound mở ra bên ngoài – chỉ dùng cho các đích fallback cục bộ (**127.0.0.1**) hoặc khi TLS được đảm bảo bởi proxy bên ngoài. 4. Lời khuyên từ giao diện: với hầu hết các trường nâng cao có gợi ý «*Khuyến nghị để lại cài đặt mặc định*» – chỉ thay đổi khi hiểu rõ hậu quả.

8. Clients

Client (khách hàng) là tài khoản người dùng VPN: một bộ thông tin xác thực (UUID hoặc mật khẩu) được gắn với một hoặc nhiều inbound, có quota lưu lượng riêng, thời hạn sử dụng và giới hạn số kết nối đồng thời. Trong fork này, client là một thực thể độc lập (bảng `clients`): cùng một client có thể gắn với nhiều inbound cùng lúc, dùng chung UUID/mật khẩu và bộ đếm lưu lượng chung. Mục **Clients** hiển thị tất cả tài khoản trong panel bất kể inbound nào, kèm tìm kiếm, bộ lọc, sắp xếp và thao tác hàng loạt.

8.1. Các trường của client

Dưới đây là giải thích từng trường trong trình chỉnh sửa **Thêm client** / **Sửa client**.

Form client được chia thành hai tab: **Cơ bản** (email, gắn với inbound, giới hạn, thời hạn, nhóm, ghi chú, reverse tag) và **Thông tin xác thực** (UUID/mật khẩu/auth, Flow, VMess Security). Trong nhãn trường, quota được hiển thị là **Giới hạn lưu lượng (GB)**, còn thời hạn là **Thời lượng (ngày)** và **Tự gia hạn (ngày)**; các trường **Giới hạn lưu lượng (GB)** và **Giới hạn IP** có chú thích giải thích rằng `0` nghĩa là «không giới hạn». Khi chỉnh sửa client đã tồn tại, nút tạo email ngẫu nhiên bị ẩn, còn nút nhật ký IP được đặt ngay cạnh trường **Giới hạn IP** và hiển thị số địa chỉ đã ghi nhận.

Trường	Khóa JSON	Mặc định	Mô tả
Email	<code>email</code>	– (bắt buộc)	Định danh duy nhất của client
UUID	<code>id</code>	tự tạo	Định danh cho VMess/VLESS
Mật khẩu	<code>password</code>	tự tạo	Mật khẩu cho Trojan/Shadowsocks
Xác thực	<code>auth</code>	tự tạo	Mật khẩu cho Hysteria
Flow	<code>flow</code>	trống	Flow control (XTLS), chỉ dành cho VLESS
VMess Security	<code>security</code>	<code>auto</code>	Phương thức mã hóa VMess
Giới hạn IP	<code>limitIp</code>	<code>0</code> (không giới hạn)	Số IP đồng thời tối đa
Tổng đã gửi/nhận (GB)	<code>totalGB</code>	<code>0</code> (không giới hạn)	Quota lưu lượng
Thời hạn	<code>expiryTime</code>	<code>0</code> (vĩnh viễn)	Ngày hết hạn
Tự gia hạn	<code>reset</code>	<code>0</code> (tắt)	Chu kỳ đặt lại lưu lượng, ngày
ID người dùng Telegram	<code>tgId</code>	<code>0</code> (không có)	ID Telegram dạng số
ID đăng ký	<code>subId</code>	tự tạo	Định danh đăng ký
Nhóm	<code>group</code>	trống	Nhân nhóm logic
Ghi chú	<code>comment</code>	trống	Ghi chú tùy ý
Bật	<code>enable</code>	<code>true</code>	Tài khoản có hoạt động không

Email (định danh)

Trường **Email** là định danh chính và bắt buộc của client. Mặc dù có tên gọi như vậy, đây không nhất thiết phải là địa chỉ thư điện tử: bất kỳ nhân văn bản nào đều được (tên người dùng, số). Giá trị phải **duy nhất** trong toàn panel; việc tạo client thứ hai với email đã tồn tại sẽ bị từ chối (`email already in use`), trừ trường hợp `subId` cũng trùng (điều này được hiểu là gắn cùng một client).

Email **không được để trống** (`client email is required`) và **không được chứa khoảng trắng, ký tự `/` , `\` và ký tự điều khiển** («Email không được chứa khoảng trắng, `'` , `"` hoặc ký tự điều khiển»). Email được dùng trong thống kê lưu lượng, nhật ký IP, danh sách online và tên thao tác – không nên thay đổi sau khi đã tạo.

UUID / Mật khẩu / Xác thực (thông tin xác thực)

Trường thông tin xác thực cụ thể phụ thuộc vào giao thức của inbound mà client được gắn vào. Các giá trị được điền tự động nếu để trống:

- **UUID** (trường `id`) – dành cho giao thức **VMess** và **VLESS**. Nếu không đặt, UUID v4 ngẫu nhiên sẽ được tạo.
- **Mật khẩu** (trường `password`) – dành cho **Trojan** và **Shadowsocks**. Với Trojan, mặc định tạo UUID không có dấu gạch ngang. Với Shadowsocks, tạo khóa Base64 có độ dài phù hợp tùy theo phương thức mã hóa của inbound: 16 byte cho `2022-blake3-aes-128-gcm` , 32 byte cho `2022-blake3-aes-256-gcm` và `2022-blake3-chacha20-poly1305` ; với các phương thức khác – UUID không có dấu gạch ngang. Nếu khóa nhập thủ công không phù hợp với phương thức 2022-blake3, nó sẽ được thay bằng khóa tự tạo.
- **Xác thực** (trường `auth`) – mật khẩu cho **Hysteria**. Mặc định là UUID không có dấu gạch ngang.

Vì một client có thể gắn với nhiều inbound thuộc các giao thức khác nhau, bản ghi client có thể đồng thời có cả UUID, mật khẩu và auth – mỗi giao thức dùng trường riêng của mình.

Ví dụ: thông tin xác thực của client trông như thế nào trong `settings` của các inbound khác nhau. Cùng một client trong VLESS inbound được xác định theo `id` , trong Trojan – theo `password` , trong Shadowsocks – theo `password` (khóa Base64):

```
// фрагмент settings.clients y VLESS-inbound
{ "id": "b831381d-6324-4d53-ad4f-8cda48b30811", "email": "user-a", "flow": "xtls-rprx-vision" }

// тот же клиент в Trojan-inbound
{ "password": "b831381d63244d53ad4f8cda48b30811", "email": "user-a" }

// тот же клиент в Shadowsocks-inbound (метод 2022-blake3-aes-256-gcm)
{ "password": "GPy0aA3f7C00az53eaQ8eqMfRDjmBl0h7v1u3+Z+pHk=", "email": "user-a" }
```

Flow

Flow (trường `flow`) – điều khiển luồng XTLS. Chỉ áp dụng **cho VLESS** và chỉ khi inbound được cấu hình cho XTLS Vision: VLESS qua transport **TCP** với security **tls** hoặc **reality** . Giá trị hợp lệ là `xtls-rprx-vision` (cũng như `xtls-rprx-vision-udp443` theo lịch sử); giá trị trống nghĩa là không có flow.

Nếu inbound không hỗ trợ XTLS-flow, flow đã đặt sẽ **bị xóa âm thầm** khi lưu client: với cùng một client gắn với nhiều inbound, flow chỉ được áp dụng ở nơi cho phép. Chỉ nên thay đổi nếu bạn có chủ đích sử dụng VLESS-Vision.

VMess Security

VMess Security (trường `security`) – phương thức mã hóa payload cho VMess. Giá trị mặc định là `auto` (Xray tự chọn mật mã). Các giá trị hợp lệ – chuẩn cho VMess: `auto`, `aes-128-gcm`, `chacha20-poly1305`, `none`, `zero`. Trường này không được dùng cho các giao thức khác.

Giới hạn IP (kết nối đồng thời)

Giới hạn IP (trường `limitIp`) – số lượng **địa chỉ IP khác nhau** tối đa mà client có thể kết nối đồng thời. Giá trị mặc định là `0`, nghĩa là **không giới hạn**. Với giá trị dương, panel theo dõi các IP đang hoạt động của client và khi vượt quá giới hạn, tắt tài khoản bằng tác vụ nền. (Từ **3.3.1** trở đi, việc đếm IP được thực hiện qua online-stats API của nhân Xray và **không yêu cầu** nhật ký truy cập; trên các phiên bản nhân cũ hơn, panel quay lại đọc nhật ký truy cập, lúc đó phải bật nhật ký.) Dùng tính năng này để ngăn chia sẻ một đăng ký cho nhiều thiết bị: ví dụ, `2` – cho phép hai thiết bị.

Giới hạn IP được thực thi bằng **Fail2ban**, vì vậy trường **Giới hạn IP** chỉ hoạt động khi Fail2ban đã được cài đặt và hoạt động (panel kiểm tra trạng thái qua `GET /panel/api/server/fail2banStatus`). Nếu Fail2ban chưa được cài đặt, trường trong form chỉnh sửa client (và form thêm hàng loạt) bị vô hiệu hóa, và khi di chuột vào sẽ hiện chú thích đề xuất cài Fail2ban từ menu bash `x-ui` («Fail2ban is not installed, so the IP limit cannot be enforced. Install Fail2ban from the x-ui bash menu to enable this option.»); trên Windows, chú thích thông báo rằng Fail2ban không khả dụng ở đó («Fail2ban is not available on Windows, so the IP limit cannot be enforced.»), và nếu tính năng bị tắt trên máy chủ – «The IP limit feature is disabled on this server.». Khi cập nhật panel, đối với các client trên máy chủ không có Fail2ban, giới hạn IP đã lưu sẽ bị đặt về 0 bởi một lần migration duy nhất, vì nó không được áp dụng ở đó.

Ví dụ giá trị. `limitIp: 0` – không giới hạn; `limitIp: 1` – nghiêm ngặt một thiết bị cùng lúc; `limitIp: 3` – tới ba IP khác nhau. Khi có IP thứ tư đang hoạt động, tác vụ nền sẽ tắt client (`enable = false`) cho đến khi bạn thực hiện **Đặt lại giới hạn IP**.

Các thao tác liên quan: **Nhật ký IP** hiển thị danh sách các IP đã ghi nhận của client; mỗi bản ghi chứa ngoài bản thân IP còn có thời điểm truy cập cuối và nhân node (`@ tên_node`), qua đó ghi nhận kết nối – trong cấu hình đa panel có thể thấy client kết nối qua node nào. **Đặt lại giới hạn IP** xóa nhật ký IP đã tích lũy để client có thể kết nối lại mà không cần chờ bản ghi hết hạn tự nhiên.

Tổng đã gửi/nhận (GB) – quota lưu lượng

Tổng đã gửi/nhận (GB) (trường `totalGB`) – quota lưu lượng tổng cộng (gửi + nhận). Giá trị mặc định là `0` – nghĩa là **không giới hạn**. Khi đạt quota (`up + down >= total`) client được coi là **đã hết** (depleted) và bị ngắt kết nối. Trong giao diện thường nhập bằng gigabyte; trong cơ sở dữ liệu được lưu bằng byte.

Trong danh sách client, cột **Lưu lượng** hiển thị thanh màu thể hiện mức sử dụng: lượng lưu lượng đã dùng, nhân giới hạn (hoặc dấu ∞ khi không giới hạn) và chú thích khi di chuột với phân tách gửi/nhận và phần còn lại. Cùng chỉ báo thu gọn đó được hiển thị trong thẻ client trên điện thoại.

Thời hạn (Expiry)

Thời hạn (trường `expiryTime`) xác định thời điểm hết hạn tài khoản. Trường này có ba chế độ:

- **Vĩnh viễn** – `0`. Client không bao giờ hết hạn theo thời gian.
- **Ngày cụ thể** – Unix-timestamp dương (tính bằng mili giây). Khi đến thời điểm đó (`expiryTime <=` hiện tại), client được coi là hết hạn (expired) và bị ngắt kết nối. Trong giao diện thường đặt bằng cách chọn ngày hoặc thời lượng tính bằng ngày (**Thời lượng**, đơn vị – **Ngày**).
- **Bắt đầu sau lần sử dụng đầu tiên** – giá trị âm, mã hóa thời lượng. Khi client chưa truyền byte nào, thời hạn vẫn là âm («khởi động trễ»). Tại lần đếm lưu lượng đầu tiên, panel chuyển đổi nó thành ngày tuyệt đối: hiện tại + |thời lượng|. Điều này cho phép bán, ví dụ, «30 ngày kể từ lần kết nối đầu tiên» mà không biết trước khi nào client sẽ kích hoạt. Việc chuyển đổi được thực hiện một lần cho mỗi email để tất cả các inbound được gán nhận cùng một thời hạn.

Ví dụ mã hóa thời hạn. Ngày cố định 1 tháng 3 năm 2026, 00:00 UTC → `expiryTime: 1772323200000` (timestamp dương tính bằng mili giây). «30 ngày kể từ lần kết nối đầu tiên» → `expiryTime: -2592000000` (giá trị âm, $30 \times 24 \times 60 \times 60 \times 1000$); khi có byte lưu lượng đầu tiên, panel sẽ thay bằng hiện tại + 2592000000. Vĩnh viễn → `expiryTime: 0`.

Tự gia hạn (chu kỳ đặt lại lưu lượng của client)

Trường **Tự gia hạn** (trường `reset`) – chu kỳ gia hạn/đặt lại tự động tính bằng ngày. Chú thích: «Tự động gia hạn sau khi kết thúc. (0 = tắt) (đơn vị: ngày)».

- `0` – tự gia hạn **tắt** (giá trị mặc định). Khi hết hạn, client đơn giản chỉ trở thành đã hết.
- `> 0` – tác vụ nền khi hết hạn sẽ **đặt lại bộ đếm lưu lượng về không** (`up = down = 0`), **chuyển thời hạn về phía trước** `reset` ngày (nếu cần – nhiều chu kỳ cho đến khi thời hạn mới nằm trong tương lai) và nếu cần **bật lại** client. Điều này thực hiện đăng ký theo chu kỳ (ví dụ, hàng tháng). Tự gia hạn **không áp dụng cho inbound trên các node** (`node_id IS NOT NULL`).

Hệ quả quan trọng: client có `reset > 0` **bị loại trừ** khỏi khái niệm «đã hết» trong các thao tác xóa hàng loạt – lưu lượng/thời hạn của họ dự kiến sẽ được đặt lại bởi tự gia hạn, chứ không phải làm cho tài khoản trở thành ứng viên xóa.

ID người dùng Telegram

ID người dùng Telegram (trường `tgId`) – định danh Telegram dạng số của người dùng để liên kết với bot Telegram tích hợp trong panel (thông báo, tự xem thống kê). Chú thích: «ID người dùng Telegram dạng số (0 = không có)». Giá trị mặc định `0` – không có liên kết. Có thể lọc theo trường này (**Có / Không**).

ID đăng ký (subId)

ID đăng ký (trường `subId`) – định danh mà theo đó client được đưa vào **đăng ký** (subscription). Tất cả client có cùng `subId` được trả về theo một link đăng ký. Nếu để trống khi tạo, panel sẽ **tự động tạo** `subId` ngẫu nhiên (UUID). Giá trị phải **duy nhất** trong số các client có email khác (`subId already in use`) và tuân theo cùng các hạn chế ký tự như email («ID đăng ký không được chứa khoảng trắng, '/', " hoặc ký tự điều khiển»).

Nếu không có `subId`, link đăng ký của client không khả dụng («Client này không có subId, link chia sẻ chung không khả dụng.»).

Tab Links (liên kết ngoài và đăng ký)

Ngoài các tab **Cơ bản** và **Thông tin xác thực**, trong trình chỉnh sửa client còn có tab thứ ba **Links** (chú thích: «Add third-party share links and remote subscription URLs to include in this client's subscription.»). Trên đó, nút **Add External Link** thêm các link chia sẻ của bên thứ ba (`vless://`, `vmess://`, `trojan://`, `ss://`, `hysteria2://`, `wireguard://`), còn nút **Add External Subscription** – URL các đăng ký từ xa (ví dụ, `https://provider.example/sub/...`).

Tất cả những thứ liệt kê trên được trộn vào đầu ra đăng ký của client (các định dạng raw, JSON và Clash): các link được thêm nguyên vẹn, còn các đăng ký từ xa được panel định kỳ tải xuống (có cache và timeout ngắn) và kết hợp cấu hình của chúng với cấu hình riêng. Nhờ vậy, trong một link đăng ký của client có thể cung cấp cùng với máy chủ của mình cả các cấu hình ngoài.

Nhóm

Nhóm (trường `group`) – nhãn logic để gộp các client liên quan. Chú thích: «Nhãn logic để nhóm các client liên quan (ví dụ, đội, khách hàng, khu vực). Có thể lọc từ thanh công cụ.», placeholder – «ví dụ, customer-a». Trường tùy chọn (mặc định trống). Có thể lọc danh sách theo nhóm và thực hiện thao tác hàng loạt; để xóa nhãn của client, dùng thao tác **Bỏ nhóm**.

Có thể xóa nhóm ngay trong trình chỉnh sửa một client: nếu xóa trường **Nhóm** và lưu, nhãn được xóa đúng cách và client không còn hiển thị dưới nhóm cũ.

Ghi chú

Ghi chú (trường `comment`) – ghi chú văn bản tùy ý cho quản trị viên (mặc định trống). Nội dung được đưa vào tìm kiếm và có thể lọc (**Có** / **Không** ghi chú).

Bật

Bật (trường `enable`) – cờ hoạt động của tài khoản. Mặc định **bật** (`true`); khi tạo, dù cờ không được truyền, panel buộc đặt `true`. Client bị tắt (`enable = false`) không thể kết nối và trong tổng quan thuộc danh mục **không hoạt động** (deactive). Panel tự tắt các client đã hết quota, hết hạn hoặc vượt giới hạn IP.

Các trường chỉ đọc

Trong thẻ client còn hiển thị các trường dịch vụ: **Tạo lúc** (`created_at`) và **Cập nhật lúc** (`updated_at`) – nhãn thời gian tạo và lần thay đổi cuối, được điền tự động và không chỉnh sửa được. Trường **Reverse tag** (`reverse`) – Reverse tag tùy chọn cho reverse proxy VLESS đơn giản («Reverse tag tùy chọn»).

8.2. Gắn với inbound

Mỗi client phải được gắn với ít nhất một inbound – khi tạo cần ít nhất một (`at least one inbound is required`). Trong trình chỉnh sửa, đây là trường **Inbound được gắn** với chú thích **Chọn một hoặc nhiều inbound**.

- **Gắn** – thêm client vào các inbound đã chọn (cùng UUID/mật khẩu và lưu lượng chung). Các liên kết hiện có được giữ nguyên.
- **Bỏ gắn** – xóa client khỏi các inbound đã chọn. Bản ghi client vẫn được giữ (để xóa hoàn toàn dùng **Xóa**). Các cặp mà client chưa được gắn sẽ bị bỏ qua âm thầm.

Khi lưu client được gắn với nhiều inbound, các trường không tương thích với giao thức/transport cụ thể (ví dụ, Flow ngoài VLESS-Vision) sẽ tự động được đặt về giá trị hợp lệ cho từng inbound.

Phía trên danh sách chọn inbound (trong form client, khi thêm client hàng loạt và trong các cửa sổ gắn/bỏ gắn hàng loạt) có các nút **Chọn tất cả** và **Xóa tất cả**. Trong các danh sách này, mỗi inbound được ký hiệu bằng ghi chú (remark) nếu có, nếu không – bằng tag của inbound.

8.3. Thao tác với client

Đối với từng client riêng lẻ (qua thẻ **Thông tin client** hoặc menu ngữ cảnh **Hành động**) có sẵn:

Xem thông tin, mã QR và link

- **Thông tin client** – thẻ với tất cả các trường, lưu lượng đã dùng/còn lại (**Còn lại**), thời hạn và các inbound được gắn.

Yêu cầu client qua API (`GET /panel/api/clients/get/:email`) bên cạnh các trường `client` và `inboundIds` còn trả về thêm `usedTraffic` – lưu lượng thực tế đã dùng (gửi + nhận, tính cả dữ liệu từ các node), giúp dễ dàng so sánh mức tiêu thụ với quota `totalGB` .

- **Mã QR và Link** – link cấu hình của client để nhập vào ứng dụng client. Được tạo theo tất cả các inbound được gắn có giao thức được hỗ trợ (`GET /links/:email`). Nếu không có link phù hợp: «Không có link chia sẻ – trước tiên hãy gắn client với inbound có giao thức được hỗ trợ.».
- **Link đăng ký** – URL đăng ký theo `subId` (`GET /subLinks/:subId`). Chỉ khả dụng nếu client có `subId` và dịch vụ đăng ký được bật trong **Cài đặt panel** → **Đăng ký** (nếu không: «Dịch vụ đăng ký đã bị tắt.»). Ngoài ra còn có **URL đăng ký JSON**.

Đặt lại lưu lượng

Đặt lại lưu lượng (`POST /resetTraffic/:email`) đặt về không các bộ đếm gửi/nhận (`up` , `down`) của client cụ thể. Quota (`totalGB`) và thời hạn **không bị ảnh hưởng** – chỉ lượng đã dùng được đặt về không. Toast: «Đã đặt lại lưu lượng». Nếu client không được gắn với inbound nào: «Trước tiên hãy gắn client này với inbound.».

Nút **Đặt lại lưu lượng** cũng có trong form chỉnh sửa client – ở bảng dưới, cạnh **Hủy** / **Lưu** (trước khi đặt lại sẽ yêu cầu xác nhận). Nếu client bị tắt do hết lưu lượng, việc đặt lại (cả đơn lẻ lẫn hàng loạt) sẽ tự động **bật**

lại client đó (`enable = true`) và ngay lập tức phân phát thay đổi này đến các node – không cần bật lại client thủ công trên master và node nữa.

Đặt lại giới hạn IP

Xóa nhật ký IP tích lũy của client (`POST /clearIps/:email`) để đỡ chặn tạm thời do vượt giới hạn kết nối đồng thời. Toast: «Nhật ký đã được xóa».

Xóa

Xóa (`POST /del/:email`) – xóa hoàn toàn client. Hộp thoại xác nhận: tiêu đề «Xóa client {email}?», nội dung «Client sẽ bị xóa khỏi tất cả inbound được gán và bản ghi lưu lượng của nó sẽ bị hủy. Không thể hoàn tác hành động này». Xóa sẽ gỡ client khỏi **tất cả** inbound và hủy bản ghi lưu lượng của nó. Toast: «Đã xóa client».

8.4. Thao tác hàng loạt

Trong danh sách client có thể đánh dấu nhiều bản ghi (**Chọn tất cả**, **Bỏ chọn tất cả**); bộ đếm – «{count} đã chọn». Với các mục đã chọn có sẵn:

- **Xóa ({count})** (`POST /bulkDel`) – xóa theo nhóm. Xác nhận: «Xóa {count} client?», «Mỗi client được chọn sẽ bị xóa khỏi tất cả inbound được gán, bản ghi lưu lượng bị hủy. Không thể hoàn tác hành động này». Toast: «Đã xóa {count} client», khi có lỗi một phần – «Đã xóa: {ok}, thất bại: {failed}».
- **Sửa ({count}) / Điều chỉnh** (`POST /bulkAdjust`) – thay đổi hàng loạt thời hạn và/hoặc quota. Hộp thoại «Sửa {count} client» với chú thích «Giá trị dương thêm vào, giá trị âm trừ đi. Các client có thời hạn hoặc lưu lượng không giới hạn sẽ bị bỏ qua cho trường tương ứng». Các trường: **Thêm ngày** và **Thêm lưu lượng (GB)**. Logic:
 - **Thời hạn:** client có thời hạn vĩnh viễn (`expiryTime == 0`) bị bỏ qua («unlimited expiry»); đối với client có ngày, thời hạn được dịch chuyển thêm số ngày đã chỉ định; đối với client ở chế độ «sau lần sử dụng đầu tiên» (thời hạn âm), thời lượng chờ được điều chỉnh. Việc giảm vượt quá phần còn lại sẽ bị bỏ qua («reduction exceeds remaining time/delay window»).
 - **Lưu lượng:** client có không giới hạn (`totalGB == 0`) bị bỏ qua («unlimited traffic»); nếu không, quota được thay đổi theo lượng đã chỉ định, không giảm xuống dưới không.
 - Nếu không chỉ định ngày lẫn lưu lượng: «Hãy chỉ định ngày hoặc lưu lượng trước khi áp dụng». Toast: «Đã sửa: {count}» / «Đã sửa: {ok}, đã bỏ qua: {skipped}».

Ví dụ: gia hạn các client đã chọn thêm 30 ngày và thêm 50 GB. Trong hộp thoại **Sửa**, nhập **Thêm ngày** = `30` , **Thêm lưu lượng (GB)** = `50` . Để ngược lại, trừ đi một tuần và cắt quota 10 GB, nhập giá trị âm: **Thêm ngày** = `-7` , **Thêm lưu lượng (GB)** = `-10` (các client có thời hạn vĩnh viễn hoặc không giới hạn theo trường tương ứng sẽ bị bỏ qua).

- **Gắn ({count}) / Bỏ gắn ({count})** (`POST /bulkAttach / bulkDetach`) – gắn/bỏ gắn hàng loạt các client đã chọn vào các inbound đã chọn. Mục tiêu – chỉ các inbound nhiều người dùng. Kết quả bỏ gắn: «Đã bỏ gắn {detached}, đã bỏ qua {skipped}».
- **Link đăng ký ({count})** – bảng tổng hợp URL đăng ký và đăng ký JSON của các client đã chọn với nút **Sao chép tất cả**. Nếu không ai có subId: «Không có client nào trong số đã chọn có ID đăng ký».
- **Thêm vào nhóm và Bỏ nhóm** – gán và xóa nhãn nhóm.

Đặt lại lưu lượng và xóa theo trạng thái

- **Đặt lại lưu lượng tất cả client** (`POST /resetAllTraffics`) – đặt về không bộ đếm `up / down` của **tất cả** client trong panel. Xác nhận: «Đặt lại lưu lượng tất cả client?» và «Bộ đếm gửi/nhận của tất cả client được đặt về không. Quota và thời hạn không bị ảnh hưởng. Không thể hoàn tác hành động này.». Toast: «Đã đặt lại lưu lượng tất cả client».
- **Xóa đã hết** (`POST /delDepleted`) – xóa tất cả client đã **hết quota** (`total > 0 and up + down >= total`) **hoặc hết hạn** (`expiry_time > 0 and expiry_time <= hiện tại`), với điều kiện `reset = 0` (các client có tự gia hạn không bị đung đến). Xác nhận: «Xóa các client đã hết?», «Xóa tất cả client đã hết quota lưu lượng hoặc hết hạn. Không thể hoàn tác hành động này.». Toast: «Đã xóa {count} client đã hết».

Xuất, nhập và xóa client không gắn

Khi không có gì được chọn, trong menu **Thêm** trên trang **Clients** có ba thao tác.

Xuất client (`GET /clients/export`) mở trình xem với danh sách JSON tất cả client ở định dạng `{client, inboundIds}` với các nút sao chép và tải xuống (tệp `clients-export.json`). **Nhập client** (`POST /clients/import`) mở trình chỉnh sửa, nơi dán vào JSON tương tự và nhấn **Import**: các client có `inboundIds` được tạo và gắn với inbound, các client không có liên kết được khôi phục dưới dạng bản ghi «trần» riêng biệt, còn các email đã tồn tại **không bao giờ bị ghi đè** – chúng được đưa vào danh sách bỏ qua. Toast: «{count} clients imported», «{ok} imported, {failed} skipped».

Xóa client không gắn (`POST /clients/delOrphans`) – thao tác nguy hiểm: xóa tất cả client không được gắn với bất kỳ inbound nào, cùng với bản ghi lưu lượng, nhật ký IP và các link ngoài của chúng. Xác nhận: «Delete clients without an inbound?», «Removes every client that is not attached to any inbound, along with its traffic record. This cannot be undone.». Toast: «{count} unattached clients deleted». Hành động không thể hoàn tác.

8.5. Tìm kiếm, bộ lọc và sắp xếp

Phía trên danh sách có thanh tìm kiếm («Tìm kiếm email, ghi chú, sub ID, UUID, mật khẩu, auth...») – tìm theo email, ghi chú, subId, UUID, mật khẩu và auth. Bộ đếm kết quả: «Hiển thị {shown} trong {total}».

Danh sách client được cập nhật tự động: panel vài giây một lần lấy trang hiện tại, vì vậy các client vừa kết nối và thứ tự sắp xếp đã thay đổi xuất hiện mà không cần làm mới thủ công (chỉ báo tải khi thăm dò nền không nhấp nháy).

Bảng điều khiển **Lọc client** cho phép lọc theo trạng thái (danh mục), giao thức, inbound được gắn, khoảng thời hạn, khoảng lưu lượng đã dùng, có tự gia hạn hay không (**Có/Không**), có ID Telegram và ghi chú hay không, cũng như theo nhóm. Trên các panel có node, xuất hiện bộ chọn nhiều **Node**: có thể giới hạn danh sách theo client của các node đã chọn; mục riêng **Panel cục bộ** lọc các client inbound không gắn với node (bộ lọc chỉ hiện khi có node). Sắp xếp: **Cũ nhất/Mới nhất, Cập nhật gần đây, Online gần đây, Email A→Z / Z→A, Lưu lượng nhiều nhất, Còn lại nhiều nhất, Sắp hết hạn nhất**.

8.6. Biểu tượng và trạng thái

Thứ tự ưu tiên trạng thái: đã hết/hết hạn → không hoạt động → sắp hết hạn → đang hoạt động.

- **Online / Offline** – client có kết nối đang hoạt động (có trong danh sách online hiện tại) và **đã bật**. Danh sách online được cập nhật bằng các yêu cầu riêng (`/onlines` , `/onlinesByGuid`).
 - **Đã hết** (depleted) – quota đã dùng hết (`up + down >= totalGB`) **hoặc** thời hạn đã hết (`expiryTime <=` hiện tại). Client như vậy tự động bị ngắt kết nối và nằm trong phạm vi của **Xóa đã hết**.
 - **Sắp hết hạn / sắp hết lưu lượng** (expiring) – client đã bật còn ít hơn khoảng ngưỡng cho đến khi hết hạn **hoặc** còn lại ít hơn lượng ngưỡng trước khi hết quota (ngưỡng được đặt trong cài đặt panel). Không tính nếu client đã hết/đã tắt.
 - **Không hoạt động** (deactive) – client với `enable = false` (tắt thủ công hoặc bởi tác vụ nền).
 - **Đang hoạt động** (active) – đã bật, chưa hết, chưa hết hạn và còn xa các ngưỡng.
-

9. Nhóm khách hàng

Đây là tính năng của bản fork 3X-UI này. Trong dự án gốc 3x-ui (MHSanaei) không có khái niệm "nhóm khách hàng" – bản fork này bổ sung bảng nhóm riêng, trang **Nhóm** trong menu panel và các phương thức API tương ứng. Nếu bạn chuyển cấu hình sang 3x-ui gốc, nhân nhóm sẽ không được xử lý ở đâu cả.

9.1. Nhóm khách hàng là gì và dùng để làm gì

Nhóm là một nhân logic (label) có tên, có thể gán cho một hoặc nhiều khách hàng. Nhóm không tạo ra phương thức kết nối mới và không phải là inbound hay node – đây thuần túy là một nhân tổ chức, giúp lọc và xử lý hàng loạt khách hàng một cách thuận tiện.

Ý tưởng cốt lõi của mô hình khách hàng trong bản fork này: **khách hàng là thực thể cấp cao nhất, được nhận dạng bằng email** (trường `email` trong bảng `clients` có chỉ mục duy nhất). Cùng một khách hàng (một email với cùng thông tin xác thực) có thể đồng thời thuộc nhiều inbound và thậm chí nhiều node, kể cả với các giao thức khác nhau. Nhân nhóm được lưu **một lần trên mỗi khách hàng**, do đó nó tự động áp dụng cho tất cả các liên kết của khách hàng đó với các inbound cùng một lúc.

Nhân nhóm là nhân logic để phân nhóm:

Lớp	Nơi lưu trữ	Trường
Bản ghi khách hàng (CSDL)	bảng <code>clients</code>	<code>group_name</code> (mặc định là chuỗi rỗng <code>' '</code>)
Danh mục nhóm (CSDL)	bảng <code>client_groups</code>	<code>name</code> (chỉ mục duy nhất, không được rỗng)
Cài đặt inbound (Xray)	JSON <code>settings.clients[].group</code>	được sao chép vào từng đối tượng khách hàng của mỗi inbound mà khách hàng thuộc về

Tại sao điều này cần thiết trong thực tế:

- **Một khách hàng trên nhiều inbound/node.** Nếu khách hàng được "bán" như quyền truy cập vào nhiều inbound cùng lúc (ví dụ: các giao thức khác nhau hoặc các node khác nhau), nhóm cho phép quản lý khách hàng đó như một thể thống nhất: đặt lại lưu lượng, xóa, đổi tên nhân – một thao tác áp dụng cho tất cả inbound của khách hàng đó.
- **Thao tác hàng loạt và lọc.** Trên trang **Khách hàng**, danh sách có thể được lọc theo nhóm; trên trang **Nhóm**, có sẵn các thao tác hàng loạt đối với tất cả thành viên của nhóm.
- **Quản lý số lượng lớn khách hàng.** Các nhân như `vip`, `trial`, `team-A` giúp sắp xếp hàng nghìn khách hàng vào các nhóm logic mà không cần tạo thêm inbound riêng biệt.

9.2. Môi liên hệ của nhóm với khách hàng, inbound, node và giao thức

Đây là phần quan trọng nhất cần hiểu, vì việc đồng bộ nhân không hề đơn giản.

Nhóm mô tả khách hàng, không phải inbound. Nhân tồn tại trong bản ghi khách hàng

(`clients.group_name`). Khi một khách hàng được liên kết với nhiều inbound, bất kỳ khi nào nhóm thay đổi, panel sẽ duyệt qua **tất cả** các inbound mà khách hàng đó thuộc về và gán/xóa trường `group` bên trong cài đặt Xray (`settings.clients[]`). Về mặt kỹ thuật, điều này được thực hiện như sau: tìm tất cả inbound chứa khách hàng đó theo email, sau đó sửa đổi trường khách hàng có email đó trong cài đặt JSON của mỗi inbound như vậy. Do đó:

- Nhóm **không phụ thuộc vào giao thức**. Một email có thể là khách hàng VLESS trong inbound này và là khách hàng Hysteria trong inbound khác – nhân nhóm của khách hàng đó vẫn là một và được áp dụng cho cả hai (trong khi thông tin xác thực của mỗi giao thức là riêng và được lưu riêng biệt).
- Nhóm **bao gồm các node**. Các inbound thuộc về node khác với inbound của panel chính ở trường `nodeId` (đối với inbound của panel chính, trường này là `null / 0`). Nhân nhóm được áp dụng cho các đối tượng khách hàng trong inbound bất kể đó là inbound chính hay inbound node – miễn là khách hàng có email đó tồn tại ở đó.

Nhân nhóm ổn định khi đồng bộ từ node và khi tái cấu hình cài đặt. Hành vi này được thiết kế đặc biệt:

- Khi một node gửi snapshot lưu lượng, dữ liệu của node đó **không ghi đè** `group_name` và `comment` cục bộ của khách hàng trong CSDL panel. Nhóm và chú thích được coi là các trường "cục bộ của panel" – node không quản lý chúng.
- Khi tái cấu hình cài đặt inbound, giá trị `group` rỗng trong dữ liệu đến **không đặt lại** nhân đã lưu. Nhóm được quản lý qua trang **Nhóm** (chứ không phải qua chỉnh sửa cài đặt Xray của inbound), do đó "nhóm rỗng" khi tái cấu hình thông thường được hiểu là "không thay đổi", chứ không phải "xóa sạch".

Kết luận thực tế: các thao tác duy nhất **có ý xóa** nhân là xóa nhóm và xóa khách hàng khỏi nhóm một cách rõ ràng (xem bên dưới). Việc chỉnh sửa inbound thông thường hoặc đồng bộ nền với node sẽ không làm mất nhóm.

9.3. Danh mục nhóm và các nhóm "rỗng"

Danh sách nhóm trên trang được tạo bằng cách hợp nhất hai nguồn:

- Nhóm dẫn xuất (derived)** – tất cả các giá trị `group_name` không rỗng thực sự xuất hiện ở các khách hàng, cùng với số lượng khách hàng.
- Nhóm đã lưu (stored)** – các bản ghi trong bảng `client_groups` .

Sự hợp nhất này mang lại một hiệu ứng quan trọng: một nhóm có thể tồn tại **mà không có một khách hàng nào**. Nhóm như vậy được tạo bằng nút "Thêm nhóm" (bản ghi trong `client_groups`) và hiển thị trong danh sách với bộ đếm `0` . Các bản ghi này được gọi là **nhóm rỗng**. Danh sách luôn được sắp xếp theo tên không phân biệt chữ hoa/thường.

Các bộ đếm tổng quan trên trang:

Trường	Ý nghĩa
Tổng số nhóm	Tổng số nhóm (đã lưu và dẫn xuất gộp lại).
Khách hàng có nhóm	Số khách hàng có nhân nhóm không rỗng.

Trường	Ý nghĩa
Nhóm rỗng	Số nhóm tồn tại mà không có khách hàng (bộ đếm 0).
Khách hàng trong nhóm	Số khách hàng trong một nhóm cụ thể (cột bảng).

9.4. Các trường và cột của nhóm

Bản ghi nhóm trong bảng `client_groups` chứa:

Trường	Kiểu	Mặc định	Mô tả
<code>Id</code>	int	tự tăng	Khóa chính của bản ghi nhóm.
<code>Name</code>	string	— (bắt buộc)	Tên nhóm. Chỉ mục duy nhất, không được rỗng. Trong UI — cột Tên .
<code>CreatedAt</code>	int64 (ms)	thời gian tạo	Thời điểm tạo bản ghi nhóm.
<code>UpdatedAt</code>	int64 (ms)	thời gian sửa	Thời điểm sửa đổi lần cuối.

Bảng trên trang hiển thị ít nhất các cột **Tên** và **Khách hàng trong nhóm**, cùng với các nút hành động (xem bên dưới).

9.5. Tạo nhóm

Nút **Thêm nhóm**.

Các bước:

- Nhấn **Thêm nhóm**.
- Nhập tên nhóm.
- Xác nhận.

Hành vi backend (`POST /panel/api/clients/groups/create` , body `{"name": "..."}`):

- Tên được cắt bỏ khoảng trắng đầu/cuối. Tên rỗng bị từ chối với lỗi «group name is required».
- Nếu nhóm có tên như vậy đã tồn tại — lỗi «group already exists».
- Khi thành công, một bản ghi được tạo trong `client_groups` (ban đầu không có khách hàng — đây là nhóm rỗng).

Thông báo thành công: **«Nhóm «{name}» đã được tạo.»**

Ví dụ: tạo nhóm rỗng qua API. Chuẩn bị sẵn bộ nhân trước khi thêm khách hàng:

```
curl -s 'https://panel.example.com:2053/panel/api/clients/groups/create' \
-H 'Content-Type: application/json' \
-b cookie.txt \
-d '{"name": "vip"}'
```

Phản hồi khi thành công:

```
{ "success": true, "msg": "Группа «vip» создана.", "obj": null }
```

Gọi lại với cùng tên sẽ trả về `"success": false` và thông báo `group already exists`.

Tạo nhóm rỗng trước rất tiện khi bạn muốn chuẩn bị sẵn bộ nhân rồi sau đó thêm khách hàng vào qua «Thêm khách hàng...».

9.6. Đổi tên nhóm

Nút **Đổi tên**, tiêu đề hộp thoại – «**Đổi tên {name}**».

Hành vi (`POST /panel/api/clients/groups/rename`, body `{"oldName": "...", "newName": "..."}:`

- Cả hai tên đều được cắt bỏ khoảng trắng. Tên cũ rỗng – lỗi «old group name is required», tên mới rỗng – «new group name is required».
- Nếu tên mới trùng với tên cũ – không làm gì (0 khách hàng bị ảnh hưởng).
- Nếu khác, việc đổi tên được thực hiện nguyên tử:
 - bản ghi trong `client_groups` được đổi tên;
 - tất cả khách hàng có `group_name = oldName` được cập nhật thành `newName`;
 - trong **tất cả inbound** mà các khách hàng bị ảnh hưởng thuộc về (kể cả inbound node), giá trị `group` trong cài đặt Xray được sửa từ tên cũ sang tên mới.
- Sau khi đổi tên, panel đánh dấu Xray cần khởi động lại và gửi thông báo về việc thay đổi khách hàng.

Thông báo:

- Thành công: «**Nhóm đã được đổi tên cho {count} khách hàng.**»
- Xung đột tên trong UI: «**Nhóm có tên «{name}» đã tồn tại.**»

9.7. Thêm khách hàng vào nhóm

Nút **Thêm khách hàng...**, tiêu đề – «**Thêm khách hàng vào nhóm «{name}»**».

Gợi ý trong hộp thoại:

«Chọn khách hàng để thêm vào nhóm này. Các liên kết inbound hiện có được giữ nguyên; chỉ nhân nhóm thay đổi. Những khách hàng đã thuộc nhóm này sẽ không hiển thị.»

Nếu không có ai để thêm, hiển thị «**Không có khách hàng nào khác để thêm.**»

Hành vi (`POST /panel/api/clients/groups/bulkAdd`, body `{"emails": [...], "group": "..."}:`

- Tên nhóm là bắt buộc (nếu không có sẽ báo lỗi «group name is required»); danh sách email rỗng – thao tác không làm gì.
- Nếu nhóm như vậy chưa tồn tại trong `client_groups` hay trong các nhóm dẫn xuất – nhóm sẽ được tạo tự động.

- Đối với các email được chọn, khách hàng được gán `group_name = group`; **các liên kết của khách hàng với inbound không thay đổi** – chỉ nhân bị ảnh hưởng. Sau đó trường `group` được gán trong tất cả inbound của các khách hàng đó.
- Trả về số bản ghi khách hàng bị ảnh hưởng; Xray được đánh dấu cần khởi động lại.

Thông báo thành công: **«Đã thêm {count} khách hàng vào {name}.»**

Ví dụ: gán nhóm cho nhiều khách hàng bằng một yêu cầu. Khách hàng được chỉ định bằng email, các liên kết inbound không thay đổi:

```
curl -s 'https://panel.example.com:2053/panel/api/clients/groups/bulkAdd' \
-H 'Content-Type: application/json' \
-b cookie.txt \
-d '{"emails": ["alice@example.com", "bob@example.com"], "group": "vip"}'
```

Nếu nhóm `vip` chưa tồn tại, nó sẽ được tạo tự động. Sau yêu cầu, các khách hàng này trong bản ghi sẽ được gán `group_name = "vip"`, và đối tượng khách hàng trong cài đặt Xray của mỗi inbound của họ sẽ nhận trường `"group": "vip"`:

```
{ "id": "6f1b...", "email": "alice@example.com", "group": "vip", "enable": true }
```

9.8. Xóa khách hàng khỏi nhóm (không xóa bản thân khách hàng)

Nút **Xóa khách hàng...**, tiêu đề – **«Xóa khách hàng khỏi nhóm «{name}»»**.

Gợi ý:

«Chọn thành viên để xóa khỏi nhóm này. Bản thân các khách hàng được giữ lại (sử dụng «Xóa khách hàng của nhóm» để xóa hoàn toàn).»

Hành vi (`POST /panel/api/clients/groups/bulkRemove`, body `{"emails": [...]}`): về mặt kỹ thuật, đây tương đương với «Thêm vào nhóm» với nhóm rỗng. Đối với các khách hàng được chọn, `group_name` được xóa, và trường `group` bị loại bỏ khỏi cài đặt Xray trong các inbound của họ. Bản thân các khách hàng và các liên kết inbound của họ được giữ nguyên.

Thông báo thành công: **«Đã xóa {count} khách hàng khỏi {name}.»**

9.9. Đặt lại lưu lượng của nhóm

Nút **Đặt lại lưu lượng**.

Hộp thoại xác nhận:

- Tiêu đề: **«Đặt lại lưu lượng của nhóm {name}?»**
- Nội dung: **«Thao tác này sẽ đặt lại up/down về 0 cho tất cả {count} khách hàng trong nhóm này.»**

Hành vi: đối với tất cả email thành viên của nhóm, `up` và `down` trong bảng lưu lượng được đặt về 0 và trường `enable` được gán `true` (khách hàng được bật). Thao tác được thực hiện theo lô trong một giao dịch.

Thông báo thành công: «**Đã đặt lại lưu lượng cho {count} khách hàng.**»

9.10. Xóa nhóm và xóa khách hàng của nhóm

Trên trang có **hai thao tác xóa về bản chất khác nhau** – dễ nhầm lẫn, vì vậy sự khác biệt rất quan trọng.

9.10.1. Xóa nhóm (giữ lại khách hàng)

Nút «**Xóa nhóm (giữ lại khách hàng)**».

Hộp thoại:

- Tiêu đề: «**Xóa nhóm {name}?**»
- Nội dung: «**Thao tác này xóa nhóm và xóa nhân của nó khỏi {count} khách hàng. Bản thân các khách hàng không bị xóa.**»

Hành vi (`POST /panel/api/clients/groups/delete` , body `{"name": "..."}`): bản ghi nhóm bị xóa khỏi `client_groups` , `group_name` của tất cả khách hàng trong nhóm được xóa, và trường `group` bị loại bỏ khỏi các inbound của họ. **Các khách hàng, kết nối và lưu lượng của họ được giữ lại.** Xray được đánh dấu cần khởi động lại.

Thông báo thành công: «**Đã xóa nhân nhóm khỏi {count} khách hàng.**»

9.10.2. Xóa khách hàng của nhóm (xóa hoàn toàn)

Nút «**Xóa khách hàng của nhóm**».

Hộp thoại:

- Tiêu đề: «**Xóa tất cả khách hàng trong {name}?**»
- Nội dung: «**Thao tác này xóa vĩnh viễn {count} khách hàng cùng với bản ghi lưu lượng của họ. Nhân nhóm cũng sẽ bị xóa. Không thể hoàn tác.**»

Đây là thao tác hủy diệt: nó xóa bản thân các khách hàng (qua xóa hàng loạt theo email, endpoint `POST /panel/api/clients/bulkDel`), bao gồm bản ghi lưu lượng của họ, và do đó loại bỏ họ khỏi tất cả inbound.

Thông báo:

- Thành công: «**Đã xóa {count} khách hàng.**»
- Kết quả một phần: «**{ok} đã xóa, {failed} đã bỏ qua**»

Nếu nhóm rỗng, các thao tác đối với thành viên sẽ không khả dụng – hiển thị «**Nhóm này hiện chưa có khách hàng nào.**»

9.11. Liên kết với trang «Khách hàng»

Nhãn nhóm hiển thị và được sử dụng cả ngoài trang **Nhóm**:

- Trong bản ghi gọn của khách hàng có trường `group`, vì vậy trong danh sách khách hàng sẽ hiển thị nhóm mà khách hàng thuộc về.
- Danh sách khách hàng (`/panel/api/clients/list/paged`) nhận tham số lọc `group`: có thể truyền một tên hoặc nhiều tên cách nhau bằng dấu phẩy. Việc so khớp được thực hiện theo nguyên tắc "HOẶC" trong trường, không phân biệt chữ hoa/thường. Trường hợp đặc biệt: phần tử rỗng trong danh sách nhóm lọc có nghĩa là "khách hàng không có nhóm" (những khách hàng có `group` rỗng).
- Trong phản hồi trang khách hàng, mảng `groups` được trả về – danh sách đầy đủ tên các nhóm hiện có, để UI có thể xây dựng danh sách thả xuống cho bộ lọc.

Ví dụ: lọc khách hàng theo nhóm. Yêu cầu chỉ trả về khách hàng có nhãn `vip` hoặc `trial` (nhiều tên cách nhau bằng dấu phẩy, «HOẶC»):

```
GET /panel/api/clients/list/paged?group=vip,trial
```

Để lấy khách hàng **không có** nhóm, hãy truyền phần tử rỗng trong danh sách – ví dụ, giá trị lọc `group=` (chuỗi rỗng) hoặc `group=vip,` (nhãn `vip` cộng khách hàng không có nhóm).

9.12. Tổng hợp các endpoint API

Tất cả các route nhóm được gắn dưới `/panel/api/clients`:

Phương thức và đường dẫn	Mục đích	Body yêu cầu
GET <code>/panel/api/clients/groups</code>	Danh sách nhóm với số lượng khách hàng	–
GET <code>/panel/api/clients/groups/:name/emails</code>	Email của tất cả thành viên nhóm (sắp xếp theo email)	–
POST <code>/panel/api/clients/groups/create</code>	Tạo nhóm rỗng	<code>{"name"}</code>
POST <code>/panel/api/clients/groups/rename</code>	Đổi tên nhóm	<code>{"oldName", "newName"}</code>
POST <code>/panel/api/clients/groups/delete</code>	Xóa nhóm, giữ lại khách hàng (xóa nhân)	<code>{"name"}</code>
POST <code>/panel/api/clients/groups/bulkAdd</code>	Thêm khách hàng vào nhóm (theo email)	<code>{"emails": [...], "group"}</code>
POST <code>/panel/api/clients/groups/bulkRemove</code>	Xóa khách hàng khỏi nhóm (xóa nhân)	<code>{"emails": [...]}</code>
POST <code>/panel/api/clients/groups/bulkDel</code>	Xóa hoàn toàn khách hàng (được dùng bởi «Xóa khách hàng của nhóm»)	<code>{"emails": [...], "keepTraffic"}</code>

Ví dụ: kích bản vòng đời nhóm điển hình qua API.

```
# 1. Tạo nhãn trial
curl -s .../panel/api/clients/groups/create -d '{"name":"trial"}'

# 2. Gắn nhãn đó cho hai khách hàng
curl -s .../panel/api/clients/groups/bulkAdd -d '{"emails":
["u1@example.com","u2@example.com"],"group":"trial"}'

# 3. Đặt lại lưu lượng của tất cả thành viên (theo email từ /groups/trial/emails)
curl -s .../panel/api/clients/groups/bulkRemove -d '{"emails":["u2@example.com"]}'

# 4. Xóa nhóm, nhưng giữ lại khách hàng (chỉ xóa nhãn)
curl -s .../panel/api/clients/groups/delete -d '{"name":"trial"}'
```

Bước 4 xóa bản ghi nhóm và xóa `group_name` của các khách hàng trong nhóm, nhưng bản thân các khách hàng, kết nối và lưu lượng của họ vẫn được giữ lại. Để xóa vĩnh viễn bản thân các khách hàng, hãy sử dụng `bulkDel` thay thế.

Các thao tác thay đổi nhãn của khách hàng (`rename` , `delete` , `bulkAdd` , `bulkRemove`) đánh dấu Xray cần khởi động lại và gửi thông báo về việc thay đổi khách hàng.

9.13. Lưu lượng theo nhóm

Tính năng mới trong phiên bản 3.3.0: trong phần **Nhóm** (trang «Khách hàng», tab quản lý nhóm), bảng nhóm giờ đây hiển thị không chỉ số lượng khách hàng trong mỗi nhóm mà còn cả tổng lưu lượng đã sử dụng của nhóm. Cột được đặt tên là «**Lưu lượng đã sử dụng**».

Cột hiển thị gì

Đối với mỗi hàng nhóm, tổng lưu lượng được hiển thị theo tất cả khách hàng thuộc nhóm đó – tức là tổng `up + down` (lưu lượng gửi + nhận) của tất cả thành viên. Điều này cho phép trả lời nhanh câu hỏi «cả nhóm đã tải về/gửi đi tổng cộng bao nhiêu», mà không cần mở từng khách hàng và cộng tay.

Bên cạnh đó trong bảng nhóm còn hiển thị:

Cột	Ý nghĩa
Tên	Tên nhóm
Khách hàng	Số khách hàng được gắn nhãn nhóm này (trước đây cột có tên «Khách hàng trong nhóm»)
Đã gửi	Tổng <code>up</code> (lưu lượng gửi) của tất cả khách hàng trong nhóm
Đã nhận	Tổng <code>down</code> (lưu lượng nhận) của tất cả khách hàng trong nhóm
Lưu lượng đã sử dụng	Tổng <code>up + down</code> của tất cả khách hàng trong nhóm

Lưu lượng gửi và nhận được hiển thị trong các cột riêng biệt **Đã gửi** và **Đã nhận**, còn cột **Lưu lượng đã sử dụng** hiển thị tổng của chúng. Cột số lượng khách hàng đơn giản có tên là **Khách hàng**.

Phần tổng quan trên bảng còn hiển thị các tổng hợp cho tất cả nhóm – «**Tổng số nhóm**» và «**Khách hàng có nhóm**», còn tổng lưu lượng được chia thành hai thẻ: «**Tổng đã gửi / đã nhận**» (với mũi tên lên/xuống – lưu

lượng gửi và nhận riêng biệt của tất cả nhóm) và «**Tổng lưu lượng**» (với biểu tượng biểu đồ – tổng cộng của chúng).

Cách tính

Việc tính toán được thực hiện bằng một truy vấn SQL đến bảng khách hàng với phép kết hợp (`LEFT JOIN`) bảng theo dõi lưu lượng:

- theo trường nhân nhóm (`group_name`), khách hàng được nhóm lại, số lượng được đếm – đó chính là «Khách hàng trong nhóm»;
- lưu lượng được lấy là tổng `up + down` từ bảng `client_traffic` được kết hợp. Tức là cả byte gửi đi (`up`) lẫn byte nhận về (`down`) đều được cộng lại cho mỗi khách hàng;
- vì email là duy nhất cả trong bảng khách hàng lẫn bảng lưu lượng, phép kết hợp không nhân đôi lưu lượng của một khách hàng.

Đặc điểm của các giá trị:

- **Khách hàng không có bản ghi lưu lượng** được tính vào bộ đếm thành viên, nhưng cộng 0 vào tổng, do đó nhóm mới tạo hiển thị lưu lượng 0 .
- **Nhóm rỗng** (đã được tạo nhưng không có khách hàng) cũng xuất hiện trong danh sách với bộ đếm và lưu lượng bằng 0: ngoài các nhóm «được suy ra» từ nhân khách hàng, các nhóm được lưu rõ ràng cũng được đưa vào kết quả, sau đó danh sách được sắp xếp theo tên không phân biệt chữ hoa/thường.
- Khách hàng không có nhân nhóm (`group_name` rỗng) không được tính vào phép tính.

Các thao tác liên quan

Từ bảng nhóm, các thao tác đối với toàn bộ nhóm vẫn khả dụng, bao gồm «**Đặt lại lưu lượng**» – đặt `up / down` về 0 cho tất cả khách hàng của nhóm được chọn. Sau khi đặt lại như vậy, cột «Lưu lượng đã sử dụng» cho nhóm đó hiển thị 0 .

10. Đăng ký (Subscription)

Đăng ký (subscription) là cơ chế cho phép cấp cho người dùng một liên kết (URL) cố định duy nhất, qua đó ứng dụng VPN tự tải xuống và định kỳ cập nhật toàn bộ bộ cấu hình. Thay vì phải gửi thủ công cho người dùng từng liên kết riêng lẻ cho mỗi inbound, chỉ cần truyền cho họ một địa chỉ thống nhất dạng `https://domain:port/sub/<subId>`. Theo địa chỉ này, panel sẽ tổng hợp tức thì tất cả các cấu hình gắn với người dùng đó và trả về theo định dạng mà ứng dụng yêu cầu. Khi cài đặt trên máy chủ thay đổi (địa chỉ mới, xoay vòng khóa Reality, thêm inbound), người dùng sẽ nhận được cấu hình mới nhất trong lần tự động cập nhật tiếp theo mà không cần thực hiện bất kỳ thao tác nào.

Đăng ký được phục vụ bởi một máy chủ HTTP/HTTPS riêng biệt bên trong panel, chạy độc lập với web panel và lắng nghe trên cổng riêng. Điều này được thực hiện vì lý do bảo mật: cổng đăng ký có thể mở ra ngoài mà không cần mở cổng của chính panel.

10.1. subId là gì và liên kết được tạo ra như thế nào

Mỗi người dùng trong inbound có trường `subId` (trong giao diện – «ID đăng ký»). Chính giá trị này là khóa đăng ký: panel tìm trong tất cả các inbound những người dùng có `subId` khớp với giá trị được yêu cầu và gộp các cấu hình của họ thành một phản hồi duy nhất.

- Nếu nhiều người dùng (trong cùng một hoặc các inbound khác nhau) có cùng `subId`, các cấu hình của họ sẽ được đưa vào cùng một đăng ký. Đây là cách thông thường để cấp cho một người dùng nhiều máy chủ/giao thức qua một liên kết duy nhất.

Ví dụ: một người dùng – hai máy chủ qua một liên kết. Giả sử có hai inbound (VLESS trên máy chủ A và Trojan trên máy chủ B). Để cấp cho người dùng cả hai cấu hình qua một liên kết, hãy đặt cùng một `subId` cho cả hai người dùng của họ:

```
Inbound 1 (VLESS): email = ivan@vpn, subId = ivan2025
Inbound 2 (Trojan): email = ivan@vpn, subId = ivan2025
```

Khi đó tại địa chỉ `https://sub.example.com:2096/sub/ivan2025`, panel sẽ trả về cả hai cấu hình ngay lập tức. Nếu sau này thêm inbound thứ ba với cùng `subId` – nó sẽ xuất hiện với người dùng trong lần tự động cập nhật đăng ký tiếp theo mà không cần gửi lại liên kết mới.

- Nếu trường `subId` của người dùng để trống, liên kết để chia sẻ chung không khả dụng. Trong giao diện, điều này được chỉ ra bởi gợi ý: «Người dùng này không có subId, liên kết chia sẻ chung không khả dụng.»

Liên kết ngoài và đăng ký của người dùng (tab «Links»)

Trong form người dùng có tab «**Links**», nơi có thể gắn thêm các nguồn cấu hình bổ sung cho từng người dùng riêng lẻ, các nguồn này được trộn vào chính đăng ký của người dùng đó (các định dạng RAW, JSON và Clash):

- **Add External Link** – liên kết chia sẻ từ bên thứ ba (`vless://`, `trojan://`, `ss://` v.v.). Được thêm vào kết quả trả về nguyên dạng, còn với JSON/Clash thì được phân tích thêm thành cấu hình.
- **Add External Subscription** – địa chỉ đăng ký bên ngoài. Panel tự tải về (có cache và timeout ngắn) và đổ các cấu hình nhận được vào danh sách chung của người dùng.

Điều này tiện lợi để cấp cho người dùng thêm các máy chủ bổ sung ngoài các inbound của bạn qua cùng một liên kết thống nhất. Nếu phản hồi của đăng ký từ xa quá lớn, nó sẽ không bị cắt bớt một cách im lặng nữa: panel trả về lỗi và tiếp tục sử dụng giá trị đã được cache thành công lần cuối.

- Giá trị `subId` không thể đặt tùy ý: khi lưu, hệ thống kiểm tra xem nó có chứa khoảng trắng, ký tự `/`, `\` hay ký tự điều khiển không. Gợi ý xác thực tương ứng: «ID đăng ký không được chứa khoảng trắng, '/', '' hoặc ký tự điều khiển».

Liên kết cuối cùng được xây dựng theo dạng `<core>/<subPath>/<subId>` (xem phần về cài đặt máy chủ đăng ký và trường «URI reverse proxy»). Nếu không tìm thấy người dùng nào theo `subId` (người dùng đã bị xóa, `subId` không tồn tại), máy chủ trả về HTTP 404 không có nội dung. Khi có lỗi nội bộ – HTTP 500. Các ứng dụng VPN chỉ dựa vào mã phản hồi, vì vậy nội dung lỗi được để trống có chủ đích.

Thứ tự liên kết inbound trong đăng ký

Mỗi inbound có trường «**Thứ tự trong đăng ký**» (`subSortIndex`) – số từ 1, xác định vị trí của các liên kết inbound này trong kết quả đăng ký. Giá trị nhỏ hơn đứng trước; khi bằng nhau, thứ tự tạo ban đầu được giữ nguyên (theo id). Thứ tự được áp dụng cho tất cả các định dạng đầu ra – văn bản thuần, trang đăng ký, JSON và Clash. Trường này không ảnh hưởng đến thứ tự của các inbound trong chính panel.

Trường được chỉnh sửa trong form inbound cạnh cài đặt địa chỉ trong liên kết (share address) và được đồng bộ đến các node theo quy tắc thông thường. Nếu ít nhất một inbound có thứ tự khác 1, cột «**Thứ tự**» nhỏ gọn sẽ xuất hiện trong danh sách Inbounds.

10.2. Cài đặt máy chủ đăng ký

Tất cả các tham số đăng ký nằm trong phần cài đặt panel ở tab «**Đăng ký**». Dưới đây là mô tả từng tham số; trong ngoặc đơn là khóa cài đặt nội bộ và giá trị mặc định.

Bản thân phần này được chia thành các tab: «**Cài đặt panel**», «**Thông tin**», «**Hồ sơ**», «**Chứng chỉ**», «**Happ**» và «**Clash / Mihomo**». Các trường tiêu đề đăng ký, URL hỗ trợ, trang hồ sơ, thông báo và thư mục chủ đề nằm ở tab «**Hồ sơ**»; các quy tắc định tuyến Happ và Clash/Mihomo – ở các tab tương ứng; khoảng thời gian cập nhật đăng ký – ở tab «**Thông tin**».

Các tham số chính

Trường (UI)	Khóa	Giá trị mặc định	Mô tả
Bật đăng ký	<code>subEnable</code>	<code>true</code> (đã bật)	Khởi động máy chủ đăng ký riêng biệt. Gợi ý: «Tính năng đăng ký với cấu hình riêng biệt». Nếu tắt – máy chủ đăng ký không khởi động và không có liên kết nào hoạt động.
IP lắng nghe	<code>subListen</code>	trống	Địa chỉ IP mà máy chủ đăng ký chấp nhận kết nối. Gợi ý: «Để trống theo mặc định để lắng nghe tất cả địa chỉ IP».
Cổng đăng ký	<code>subPort</code>	<code>2096</code>	Cổng TCP của máy chủ đăng ký. Gợi ý: «Số cổng phục vụ dịch vụ đăng ký không được sử dụng trên máy chủ» – cổng phải rảnh và không xung đột với panel hoặc Xray.

Trường (UI)	Khóa	Giá trị mặc định	Mô tả
Đường dẫn URI	subPath	/sub/	Đường dẫn để trả về đăng ký thông thường. Gợi ý: «Phải bắt đầu bằng '/' và kết thúc bằng '/'».
Tên miền lắng nghe	subDomain	trống	Tên miền được phép truy cập đăng ký (xác thực Host). Gợi ý: «Để trống theo mặc định để lắng nghe tất cả tên miền và địa chỉ IP». Nếu được đặt – các yêu cầu với Host khác sẽ bị từ chối.

Lưu ý bảo mật quan trọng: đường dẫn mặc định /sub/ (và /json/ cho JSON) được biết đến rộng rãi và dễ đoán. Panel hiển thị cảnh báo: «Đường dẫn đăng ký mặc định "/sub/" được biết đến rộng rãi – hãy thay đổi nó.» và tương tự cho JSON. Nên đặt đường dẫn riêng không rõ ràng.

TLS / Chứng chỉ

Trường (UI)	Khóa	Mặc định	Mô tả
Đường dẫn file khóa công khai chứng chỉ đăng ký	subCertFile	trống	Đường dẫn đầy đủ đến file chứng chỉ (.crt / fullchain). Gợi ý: «Nhập đường dẫn đầy đủ bắt đầu bằng '/'».
Đường dẫn file khóa riêng tư chứng chỉ đăng ký	subKeyFile	trống	Đường dẫn đầy đủ đến file khóa riêng tư. Gợi ý: «Nhập đường dẫn đầy đủ bắt đầu bằng '/'».

Nếu cả hai đường dẫn được đặt và chứng chỉ được tải thành công, máy chủ đăng ký hoạt động qua **HTTPS**. Nếu các trường để trống hoặc không đọc được chứng chỉ – máy chủ sẽ dự phòng về **HTTP** (lỗi được ghi vào log). Sự hiện diện của TLS hợp lệ cũng ảnh hưởng đến việc tạo URL cơ sở: với cổng 443 kèm TLS và cổng 80 không có TLS, số cổng trong liên kết được bỏ qua.

Khoảng thời gian cập nhật

Trường (UI)	Khóa	Mặc định	Mô tả
Khoảng thời gian cập nhật đăng ký	subUpdates	12	Tần suất (theo giờ) ứng dụng khách nên tải lại đăng ký. Gợi ý: «Khoảng thời gian giữa các lần cập nhật trong ứng dụng khách (tính bằng giờ)».

Giá trị được truyền đến ứng dụng khách trong HTTP header `Profile-Update-Interval` ; các ứng dụng hiện đại sử dụng nó làm chu kỳ tự động cập nhật cấu hình.

Định dạng và thông tin trong phản hồi

Trường (UI)	Khóa	Mặc định	Mô tả
Mã hóa	subEncrypt	true	Gợi ý: «Mã hóa các câu hình được trả về trong đăng ký». Về mặt kỹ thuật đây không phải mã hóa mà là mã hóa Base64 toàn bộ nội dung đăng ký thông thường (định dạng mà hầu hết ứng dụng khách mong đợi). Khi tắt, các liên kết được trả về dưới dạng văn bản thuần, mỗi liên kết trên một dòng.
Hiển thị thông tin sử dụng	subShowInfo	true	Gợi ý: «Hiển thị lưu lượng còn lại và ngày hết hạn sau tên câu hình». Khi bật, các nhãn lưu lượng còn lại () và thời hạn (ví dụ: 5D, 3H) được thêm vào tên (remark) của mỗi câu hình; với người dùng đã hết hạn/không khả dụng hiển thị N/A .
Bao gồm Email trong tên	subEmailInRemark	true	Gợi ý: «Bao gồm email của người dùng trong tên hồ sơ đăng ký». Thêm email người dùng vào remark của hồ sơ.

Mẫu ghi chú (Remark Template)

Tên hiển thị (remark) của mỗi câu hình trong đăng ký được tạo theo **mẫu ghi chú** – trường «**Mẫu ghi chú**» (remarkTemplate) trên tab «**Thông tin**» của cài đặt đăng ký. Trình tạo mô hình ghi chú trước đây (chọn riêng các phần inbound/email/external proxy và ký tự phân cách) đã bị xóa khỏi giao diện; bây giờ bạn viết định dạng tên tùy ý và chèn các biến vào đó. Giá trị mặc định – {{INBOUND}} | {{TRAFFIC_LEFT}} | {{DAYS_LEFT}}D . Nếu để trống trường, mô hình ghi chú cũ (không thể tùy chỉnh qua giao diện) sẽ được áp dụng.

Các biến được nhóm theo phần **Client**, **Traffic** và **Time & status** và hiển thị cạnh trường dưới dạng chip có thể nhập {{VAR}} với gợi ý khi di chuột; nhấp vào sẽ chèn token vào mẫu, có bản xem trước trực tiếp. Mỗi biến được thay thế riêng lẻ cho từng người dùng cụ thể tại thời điểm tạo đăng ký. Cũng được phép dùng cú pháp viết tắt trong ngoặc đơn ({DATA_LEFT} , {EXPIRE_DATE} , {PROTOCOL} , {TRANSPORT} v.v.) – panel tự chuyển đổi sang định dạng nội bộ {{...}} .

Các biến khả dụng:

- **Nhận dạng người dùng:** {{EMAIL}} , {{INBOUND}} (remark của chính inbound), {{HOST}} (remark của host), {{ID}} (UUID), {{SHORT_ID}} (8 ký tự đầu của UUID), {{SUB_ID}} , {{COMMENT}} , {{TELEGRAM_ID}} , {{PROTOCOL}} , {{TRANSPORT}} .
- **Lưu lượng:** {{TRAFFIC_USED}} , {{TRAFFIC_LEFT}} , {{TRAFFIC_TOTAL}} (và các biến thể *_BYTES của chúng tính theo byte chính xác), {{UP}} , {{DOWN}} , {{USAGE_PERCENTAGE}} .
- **Thời hạn và trạng thái:** {{DAYS_LEFT}} , {{TIME_LEFT}} , {{EXPIRE_DATE}} (YYYY-MM-DD) , {{JALALI_EXPIRE_DATE}} (ngày theo lịch Jalali), {{EXPIRE_UNIX}} , {{CREATED_UNIX}} , {{RESET_DAYS}} , {{STATUS}} (active / expired / disabled / depleted), {{STATUS_EMOJI}} .

Mẫu có thể được chia thành các phân đoạn bằng ký tự gạch đứng | . Phân đoạn có biến cho giá trị «không giới hạn» (∞) – ví dụ {{TRAFFIC_LEFT}} hoặc {{DAYS_LEFT}} với người dùng không có giới hạn – sẽ tự

động bị ẩn. Ngoài ra, khối thể hiện lưu lượng và thời hạn chỉ hiển thị một lần, ở liên kết đầu tiên của người dùng, để không bị lặp lại trong mỗi cấu hình.

Ví dụ. Mẫu `{{EMAIL}} | {{TRAFFIC_LEFT}} | {{DAYS_LEFT}}D` sẽ cho người dùng còn 42 GB và 7 ngày tên dạng `ivan@vpn 42.00GB 7D`, còn người dùng không giới hạn – chỉ `ivan@vpn` (các phân đoạn có `∞` bị bỏ qua). | Mẫu ghi chú | `remarkTemplate | {{INBOUND}} | {{TRAFFIC_LEFT}} | {{DAYS_LEFT}}D` | Mẫu tự do cho tên hiển thị (remark) của mỗi cấu hình với thay thế biến `{{VAR}}`. Được thay thế riêng lẻ cho từng người dùng khi tạo đăng ký. Trình tạo «mô hình ghi chú» cũ (chọn inbound/email/external proxy và dấu phân cách) đã bị xóa khỏi giao diện và chỉ được dùng như phương án dự phòng nếu trường để trống. Chi tiết – xem «Mẫu ghi chú (Remark Template)» bên dưới. |

Siêu dữ liệu hồ sơ (header phản hồi)

Các chuỗi này được truyền đến ứng dụng khách trong HTTP header của phản hồi và hiển thị trong ứng dụng VPN dưới dạng siêu dữ liệu hồ sơ. Tất cả đều để trống theo mặc định.

Trường (UI)	Khóa	Header	Mô tả
Tiêu đề đăng ký	<code>subTitle</code>	<code>Profile-Title</code> (dạng Base64)	«Tên đăng ký mà người dùng thấy trong ứng dụng VPN». Với Clash cũng được dùng làm tên hồ sơ được nhập qua <code>Content-Disposition</code> .
URL hỗ trợ	<code>subSupportUrl</code>	<code>Support-Url</code>	«Liên kết hỗ trợ kỹ thuật hiển thị trong ứng dụng VPN».
URL hồ sơ	<code>subProfileUrl</code>	<code>Profile-Web-Page-Url</code>	«Liên kết đến trang web của bạn hiển thị trong ứng dụng VPN». Nếu không đặt, URL yêu cầu đăng ký thực tế sẽ được dùng thay thế.
Thông báo	<code>subAnnounce</code>	<code>Announce</code> (dạng Base64)	«Văn bản thông báo hiển thị trong ứng dụng VPN».

Ngoài ra, mỗi phản hồi đều truyền header `Subscription-Userinfo` với dữ liệu lưu lượng tổng hợp của người dùng: `upload`, `download`, `total` và `expire` (thời điểm hết hạn tính bằng giây). Dựa vào đó ứng dụng khách hiển thị lưu lượng còn lại và thời hạn.

Định tuyến (chỉ dành cho ứng dụng Happ)

Trường (UI)	Khóa	Mặc định	Mô tả
Bật định tuyến	<code>subEnableRouting</code>	<code>false</code>	«Cài đặt toàn cục để bật định tuyến trong ứng dụng VPN. (Chỉ dành cho Happ)». Được truyền trong header <code>Routing-Enable</code> .
Quy tắc định tuyến	<code>subRoutingRules</code>	trống	«Quy tắc định tuyến toàn cục cho ứng dụng VPN. (Chỉ dành cho Happ)». Được truyền trong header <code>Routing</code> .

| Ẩn cài đặt máy chủ | `subHideSettings` | `false` | «Ẩn cài đặt máy chủ trong đăng ký (chỉ dành cho Happ)». Khi bật, ứng dụng Happ sẽ ẩn khả năng xem và thay đổi các tham số máy chủ. Tùy chọn này chỉ có tác dụng với ứng dụng Happ. |

URI reverse proxy

Trường (UI)	Khóa	Mặc định	Mô tả
URI reverse proxy	subURI	trống	«Thay đổi URI cơ sở của URL đăng ký để sử dụng phía sau proxy server».

Nếu trường để trống, panel tự xây dựng địa chỉ cơ sở của liên kết từ tên miền và cổng đăng ký (có tính đến TLS). Nếu đăng ký được phân phối qua reverse proxy/CDN bên ngoài trên tên miền hoặc đường dẫn khác, URI cơ sở cuối cùng được đặt vào trường này và tất cả các liên kết sẽ được xây dựng từ đó. Các trường riêng lẻ tương tự cũng có cho JSON (subJsonURI) và Clash (subClashURI).

Nếu chỉ đặt subURI chung mà để trống các trường riêng lẻ cho JSON và Clash, các liên kết của các định dạng đó trên trang đăng ký sẽ kế thừa scheme và host từ subURI (chứ không phải cổng sub-server và http) – vì vậy chúng sẽ khớp với địa chỉ reverse proxy.

Ví dụ: đăng ký phía sau reverse proxy. Bản thân đăng ký lắng nghe trên 2096, nhưng bên ngoài có thể truy cập qua nginx/CDN tại https://cfg.example.com/u/. Để các liên kết trong phản hồi được xây dựng từ địa chỉ bên ngoài chứ không phải từ domain:2096 nội bộ, URI cơ sở cuối cùng được đặt vào trường «Reverse proxy URI»:

Reverse proxy URI: https://cfg.example.com/u

Khi đó liên kết cuối cùng sẽ có dạng https://cfg.example.com/u/ivan2025. Đối với các định dạng JSON và Clash, nếu cần thiết thì điền vào các trường riêng subJsonURI và subClashURI theo cách tương tự.

10.3. Các định dạng đầu ra

Đăng ký có thể được trả về theo ba định dạng độc lập, mỗi định dạng có endpoint riêng có thể bật/tắt độc lập.

Địa chỉ máy chủ và các node trong kết quả

Địa chỉ máy chủ trong các liên kết đăng ký được thay thế theo cùng chiến lược địa chỉ trong liên kết như các liên kết thông thường và mã QR trong panel: «listen» – địa chỉ liên kết có thể định tuyến, «custom» – địa chỉ tùy chỉnh do người dùng đặt (shareAddr), «node» (mặc định) – địa chỉ node. Đối với các inbound không có chiến lược được đặt rõ ràng, kết quả đăng ký không thay đổi. Điều này cho phép inbound của node được liên kết với IP công cộng cụ thể trả về địa chỉ có thể truy cập cho người dùng. Chiến lược được áp dụng cho các định dạng raw, JSON và Clash.

Tên node (Node) không được thêm vào tên (remark) của hồ sơ trong đăng ký: trong ứng dụng khách chỉ hiển thị remark của inbound do quản trị viên đặt, không có hậu tố nội bộ dạng @tên-node. Để phân biệt các mục cùng tên trong đăng ký đa node, hãy đặt cho chúng các remark khác nhau theo cách thủ công hoặc sử dụng host được quản lý (Hosts) với Remark riêng.

Nếu do mất đồng bộ giữa các node mà cùng một người dùng xuất hiện hai lần trong inbound JSON nội bộ, kết quả đăng ký sẽ tự động loại bỏ các bản sao theo email trong cả ba định dạng, vì vậy các hồ sơ trùng lặp không xuất hiện trong kết quả.

Host được quản lý (Hosts)

Phần **Hosts** (mục menu bên; trang tổng quan với số lượng Total/Enabled/Disabled và danh sách) đặt các ghi đề địa chỉ cho các liên kết đăng ký. Với mỗi inbound có thể thêm một hoặc nhiều **host** – endpoint được thay thế vào các liên kết đăng ký trả về cho người dùng **thay vì địa chỉ, cổng và tham số TLS của chính inbound đó**. Điều này tiện lợi để phân phối lưu lượng qua CDN hoặc relay mà không thay đổi inbound.

Mỗi host có thể cài đặt:

- **Remark** và mô tả (Description), liên kết với một **Inbound** cụ thể, nút chuyển **Enable** và phân công cho các node (**Nodes**).
- **Address** (trống – kế thừa địa chỉ inbound) và **Port** (`0` – kế thừa cổng inbound); **Tags** (chỉ được tính trong RAW-subscription).
- Tab **Security** – `same` / `tls` / `none` / `reality` với SNI, fingerprint, ALPN, pinned-cert, `allowInsecure` và ECH.
- Tab **Advanced** – Host header, Path, tuyến đường VLESS, Mux, Sockopt, Final Mask và loại trừ host khỏi các định dạng đăng ký riêng lẻ (`raw` / `json` / `clash`).
- Tab **Clash (mihomo)** – phiên bản IP, Mihomo X25519, trộn host (Shuffle host).

Các host được sắp xếp trong phạm vi inbound của chúng và hỗ trợ bật, tắt và xóa hàng loạt. Host được quản lý thay thế mảng External Proxy trước đây.

Liên kết thông thường (SUB) – Base64 / văn bản thuần

Định dạng cơ bản, endpoint `subPath` (mặc định `/sub/`). Luôn được bật (khi đăng ký nói chung được bật). Trả về danh sách các liên kết Xray (`vless://`, `vmess://`, `trojan://`, `ss://` v.v.) – mỗi liên kết trên một dòng. Khi bật tùy chọn «Mã hóa» (`subEncrypt`), toàn bộ danh sách được mã hóa sang Base64; khi tắt – trả về dưới dạng văn bản thuần. Định dạng này được hầu hết các ứng dụng khách hiểu (v2rayNG, V2RayTun, Sing-box, NekoBox, Streisand, Shadowrocket, Happ và các ứng dụng khác).

Ví dụ: nội dung phản hồi khi tắt «Mã hóa». Khi `subEncrypt = false`, endpoint `/sub/` trả về văn bản thuần – mỗi liên kết trên một dòng:

```
vless://3c8f...@a.example.com:443?security=reality&...#srvA-ivan
trojan://p4ss@b.example.com:443?security=tls&...#srvB-ivan
```

Khi `subEncrypt = true` (mặc định), cùng danh sách đó được mã hóa toàn bộ sang Base64 và trả về dưới dạng một chuỗi – đây chính xác là dạng mà hầu hết các ứng dụng khách mong đợi.

Đăng ký JSON (sing-box và tương thích)

Endpoint `subJsonPath` (mặc định `/json/`), được bật bằng checkbox riêng.

Trường (UI)	Khóa	Mặc định	Mô tả
Đăng ký JSON	<code>subJsonEnable</code>	<code>false</code>	«Bật/tắt endpoint JSON của đăng ký một cách độc lập.».

Trả về cấu hình JSON đầy đủ (định dạng mà sing-box và các ứng dụng khách dẫn xuất hiểu – Podkop, OpenWRT sing-box, Karing, NekoBox). Các tham số bổ sung cho định dạng này (tab `subFormats`) bao gồm:

- **Mux** (`subJsonMux` , mặc định trống) – cài đặt JSON cho multiplexing (Mux), được nhúng vào outbound của mỗi luồng đăng ký JSON. «Truyền nhiều luồng dữ liệu độc lập trong một kết nối.».
- **Final Mask** (`subJsonFinalMask` , mặc định trống) – «Mask finalmask xray (TCP/UDP) và cài đặt QUIC được thêm vào mỗi luồng đăng ký JSON. Yêu cầu phiên bản xray mới trên ứng dụng khách.». Được cấu hình qua các trường con: «Gói» (`packets`), «Độ dài» (`length`), «Khoảng thời gian» (`interval`), «Tách tối đa» (`maxSplit`), «Nhiều» (`noises` : «Loại»/ `type` , «Gói»/ `packet` , «Độ trễ (ms)»/ `delayMs` , «Áp dụng cho»/ `applyTo` , nút «+ Nhiều», cũng như «Song song» (`concurrency`), «Song song xudp» (`xudpConcurrency`) và «xudp UDP 443» (`xudpUdp443`).
- **Quy tắc định tuyến** (`subJsonRules` , mặc định trống) – các quy tắc toàn cục được thêm vào cấu hình JSON.

Đăng ký Clash / Mihomo (YAML)

Endpoint `subClashPath` (mặc định `/clash/`), được bật bằng checkbox riêng.

Trường (UI)	Khóa	Mặc định	Mô tả
Đăng ký Clash / Mihomo	<code>subClashEnable</code>	<code>false</code>	Bật tạo cấu hình YAML cho các ứng dụng khách Clash và Mihomo.
Bật định tuyến	<code>subClashEnableRouting</code>	<code>false</code>	«Thêm các quy tắc định tuyến Clash/Mihomo toàn cục vào các đăng ký YAML được tạo.».
Quy tắc định tuyến toàn cục	<code>subClashRules</code>	trống	«Các quy tắc Clash/Mihomo được thêm vào đầu mỗi đăng ký YAML trước MATCH,PROXY.».

Phản hồi được trả về với kiểu `application/yaml; charset=utf-8`. Nếu «Tiêu đề đăng ký» (`subTitle`) được đặt, nó cũng được truyền trong header `Content-Disposition (attachment; filename*=UTF-8''<title>)`, để ứng dụng Clash đặt tên cho hồ sơ được nhập bằng tên đó.

Định dạng của các liên kết và YAML được tạo ra được duy trì cập nhật cho các ứng dụng hiện đại: Shadowsocks-2022 (SS2022) không còn mã hóa userinfo sang Base64; các liên kết Shadowsocks với obfuscation http được trả về theo định dạng SIP002 với plugin `obfs-local`; cho đăng ký Clash/Mihomo, bộ đầy đủ các trường XHTTP được triển khai. Điều này không yêu cầu cài đặt riêng – các liên kết chỉ đơn giản được nhận diện chính xác hơn bởi các ứng dụng khách.

Lưu ý: trong bản build này được hỗ trợ đúng ba định dạng – liên kết thông thường (Base64/văn bản), JSON (tương thích sing-box) và Clash/Mihomo (YAML). Không có định dạng Outline riêng biệt trong máy chủ đăng ký.

10.4. Trang thông tin đăng ký và mã QR

Nếu mở liên kết đăng ký trong trình duyệt (hoặc thêm tham số `?html=1` hoặc `?view=html` vào URL, hoặc gửi header `Accept: text/html`), máy chủ thay vì phản hồi «thô» sẽ trả về **trang thông tin đăng ký** trực quan («Thông tin đăng ký»). Các ứng dụng VPN vẫn nhận phản hồi dạng máy, vì chúng không yêu cầu HTML.

Trang (ứng dụng một trang, được build bằng Vite) hiển thị:

- **Thông tin đăng ký** (khởi Descriptions):
 - «ID đăng ký» — giá trị `subId` ;
 - «Trạng thái» — «Đang hoạt động», «Không hoạt động» hoặc «Không giới hạn». Trạng thái «không hoạt động» được đặt khi người dùng bị tắt, hết giới hạn lưu lượng hoặc hết hạn;
 - «Đã tải xuống» và «Đã tải lên» — dung lượng lưu lượng;
 - «Tổng giới hạn» — giới hạn lưu lượng hoặc ∞ nếu không giới hạn;
 - «Thời hạn» — ngày kết thúc hoặc «Vĩnh viễn»;
 - lưu lượng còn lại và thời gian trực tuyến lần cuối.
 - Ngày được hiển thị theo lịch Gregorian hoặc Jalali tùy thuộc vào cài đặt «Calendar Type» của panel (`datepicker` , mặc định `gregorian`).
- **Liên kết đăng ký**: cho mỗi định dạng được bật — một hàng riêng với nhãn màu (xanh **SUB**, tím **JSON**, vàng **CLASH**), nút sao chép và nút **Mã QR** (cửa sổ pop-up, kích thước 240 px). Hàng với JSON và CLASH chỉ xuất hiện khi định dạng tương ứng được bật trong cài đặt.
- **Liên kết riêng lẻ** («Sao chép liên kết»): danh sách đầy đủ các câu hình riêng lẻ trong đăng ký, mỗi câu hình với nhãn giao thức, nút sao chép và mã QR (với liên kết post-quantum, mã QR không được tạo).
- **Nút «Sao chép tất cả câu hình»** (phía trên danh sách liên kết riêng lẻ): chỉ một lần nhấn sẽ sao chép tất cả các liên kết câu hình vào clipboard (mỗi liên kết trên một dòng mới), không cần sao chép từng cái một; khi hoàn thành hiển thị thông báo «Đã sao chép tất cả câu hình».
- **Các nút nhập nhanh vào ứng dụng** (menu thả xuống theo nền tảng): cho Android — `v2box`, `v2rayNG` (deep-link `v2rayng://install-config?url=...`), `Sing-box`, `V2RayTun`, `NPV Tunnel`, `Happ` (`happ://add/...`); cho iOS — `Shadowrocket` (qua tham số `flag=shadowrocket`), `v2box` (`v2box://install-sub?url=...&name=...`), `Streisand` (`streisand://import/...`), `V2RayTun`, `NPV Tunnel`, `Happ`. Các nút này hoặc mở deep-link của ứng dụng cần thiết với địa chỉ đăng ký đã điền sẵn, hoặc sao chép liên kết vào clipboard.

Trang thông tin được trả về với header cấm cache (`Cache-Control: no-cache`), để ứng dụng khách luôn thấy dữ liệu lưu lượng và thời hạn mới nhất.

10.5. Mẫu tùy chỉnh trang đăng ký

Từ phiên bản 3.3.0 có thể thay thế trang landing đăng ký tiêu chuẩn bằng mẫu HTML tùy chỉnh. Theo mặc định, địa chỉ đăng ký trả về trang tích hợp sẵn, nhưng nếu chỉ định thư mục với mẫu của bạn, panel sẽ render nó và điền vào đó dữ liệu hiện tại của người dùng (lưu lượng, thời hạn, liên kết v.v.).

Lưu ý quan trọng: panel **không cung cấp** sẵn các mẫu. Trong repository chỉ có thư mục `sub_templates/` với file hướng dẫn `sub_templates/README.md` ; bạn cần tự tạo chủ đề của mình.

Nơi bật

Thư mục chủ đề được đặt trong cài đặt panel:

Cài đặt → **Đăng ký** → phần «**Thông tin đăng ký**», trường «**Thư mục chủ đề đăng ký**» (`subThemeDir`).

Mô tả trường trong giao diện: «Đường dẫn tuyệt đối đến thư mục chứa mẫu tùy chỉnh (`index.html/sub.html`) cho trang đăng ký (ví dụ: `/etc/3x-ui/sub_templates/my-theme/`). Để trống để sử dụng trang mặc định.»

Cạnh đó trong cùng phần có các cài đặt liên quan, các giá trị của chúng khả dụng trong mẫu:

Trong mô tả trường «Thư mục chủ đề đăng ký» có liên kết «**Hướng dẫn mẫu** ↗» đến tài liệu về cách tạo mẫu trang đăng ký tùy chỉnh.

- «**Tiêu đề đăng ký**» (`subTitle`) – tên hiển thị với người dùng;
- «**URL hỗ trợ**» (`subSupportUrl`) – liên kết đến hỗ trợ kỹ thuật.

Tham số cài đặt

Tham số	Giá trị mặc định	Mục đích
<code>subThemeDir</code>	"" (trống)	Đường dẫn tuyệt đối đến thư mục chứa mẫu HTML của bạn. Trống = trang tích hợp mặc định.

Cách cài đặt mẫu tùy chỉnh

1. Tạo thư mục cho chủ đề trên máy chủ (ở bất kỳ đâu), ví dụ `/etc/3x-ui/sub_templates/my-theme/`.
2. Đặt vào đó file HTML với tên `index.html` hoặc `sub.html`.

Ví dụ: đường dẫn đến chủ đề. Bố cục cuối cùng trên máy chủ và giá trị trường trong cài đặt:

```
/etc/3x-ui/sub_templates/my-theme/
└─ index.html          (hoặc sub.html – có ưu tiên cao hơn)
```

Cài đặt → Đăng ký → «Thư mục chủ đề đăng ký»:
`/etc/3x-ui/sub_templates/my-theme/`

Đường dẫn phải là **tuyệt đối** (bắt đầu bằng `/`). Nếu trong thư mục không có `index.html` hay `sub.html`, panel sẽ trả về trang tích hợp. 3. Trong panel mở **Cài đặt** → **Đăng ký** và nhập đường dẫn **tuyệt đối** đến thư mục này vào trường «Thư mục chủ đề đăng ký». 4. Lưu cài đặt.

Hành vi chọn file và render:

- Nếu trong thư mục có `sub.html`, nó sẽ được dùng; nếu không thì lấy `index.html`. Tức là `sub.html` có ưu tiên cao hơn `index.html`.
- Mẫu được render bằng engine Go tiêu chuẩn `html/template`.

- Mẫu đã được phân tích **được cache** và chỉ đọc lại từ đĩa khi thời gian sửa đổi của file thay đổi. Do đó các thay đổi mẫu được nhận biết mà không cần khởi động lại panel, nhưng không có chi phí đọc/phân tích ở mỗi yêu cầu.
- Phản hồi được tạo đầy đủ vào bộ đệm và chỉ sau đó mới được gửi cho ứng dụng khách: nếu mẫu gặp lỗi trong quá trình thực thi, trang được tạo một phần (bị hỏng) sẽ không đến tay người dùng.

Hành vi mặc định và dự phòng (fallback)

- Trường trống → trang SPA tích hợp được trả về (dữ liệu được nhúng vào `window.__SUB_PAGE_DATA__`).
- Đường dẫn không tồn tại hoặc không phải thư mục → trang mặc định được sử dụng.
- Trong thư mục không có `index.html` hay `sub.html` → cảnh báo «subThemeDir set but no usable template found» được ghi vào log, trang mặc định được trả về.
- File mẫu tồn tại nhưng không thể phân tích → lỗi «custom template parse failed» được ghi vào log, trang mặc định được trả về.
- Lỗi khi thực thi mẫu → «custom template execution failed» được ghi vào log, trang mặc định được trả về.

Tức là bất kỳ vấn đề nào với mẫu tùy chỉnh cũng không «phá vỡ» đăng ký – panel luôn dự phòng về trang tích hợp. Tất cả các trang đăng ký (cả tùy chỉnh lẫn tiêu chuẩn) đều được trả về với header `Cache-Control: no-cache, no-store, must-revalidate`, để các ứng dụng khách luôn nhận được dữ liệu lưu lượng và thời hạn mới nhất.

Các biến mẫu khả dụng

Một tập hợp dữ liệu của người dùng đăng ký được truyền vào ngữ cảnh mẫu. Truy cập qua `{{ .tên }}`:

Biến	Kiểu	Mô tả
<code>{{ .sId }}</code>	string	ID đăng ký (UUID).
<code>{{ .enabled }}</code>	bool	Người dùng/đăng ký có được bật hay không.
<code>{{ .download }}</code>	string	Dung lượng tải xuống đã định dạng (ví dụ: «2.5 GB»).
<code>{{ .upload }}</code>	string	Dung lượng tải lên đã định dạng.
<code>{{ .total }}</code>	string	Tổng giới hạn lưu lượng đã định dạng.
<code>{{ .used }}</code>	string	Lưu lượng đã dùng đã định dạng (download + upload).
<code>{{ .remained }}</code>	string	Lưu lượng còn lại đã định dạng.
<code>{{ .expire }}</code>	int64	Thời hạn – Unix-time tính bằng giây (<code>0</code> = vĩnh viễn). Để dùng với JS <code>Date</code> hãy nhân với 1000.
<code>{{ .lastOnline }}</code>	int64	Thời gian trực tuyến lần cuối – Unix-time tính bằng mili giây (<code>0</code> = chưa từng).
<code>{{ .downloadByte }}</code>	int64	Tải xuống tính bằng byte chính xác.
<code>{{ .uploadByte }}</code>	int64	Tải lên tính bằng byte chính xác.
<code>{{ .totalByte }}</code>	int64	Tổng giới hạn tính bằng byte chính xác.

Biến	Kiểu	Mô tả
<code>{{ .subUrl }}</code>	string	URL trang đăng ký.
<code>{{ .subJsonUrl }}</code>	string	URL cấu hình JSON của đăng ký.
<code>{{ .subClashUrl }}</code>	string	URL cấu hình Clash/Mihomo.
<code>{{ .subTitle }}</code>	string	Tiêu đề đăng ký từ cài đặt (có thể trống).
<code>{{ .subSupportUrl }}</code>	string	URL hỗ trợ từ cài đặt (có thể trống).
<code>{{ .links }}</code>	[]string	Danh sách chuỗi cấu hình (VMess, VLESS, v.v.). Duyệt qua: <code>{{ range .links }} ... {{ end }}</code> .
<code>{{ .emails }}</code>	[]string	Danh sách email thuộc đăng ký.
<code>{{ .datepicker }}</code>	string	Định dạng lịch hiện tại của panel: <code>gregorian</code> hoặc <code>jalali</code> (lấy từ cài đặt «Loại lịch»; nếu trống – <code>gregorian</code>).

Ví dụ tối giản của nội dung mẫu sử dụng một số biến:

```

<h1>{{ .subTitle }}</h1>
<p>Đã dùng: {{ .used }} trong tổng số {{ .total }} (còn lại {{ .remained }})</p>
{{ range .links }}<div>{{ . }}</div>{{ end }}

**Ví dụ: ngày hết hạn từ `expire`.** Trường `{{ .expire }}` – là Unix-time tính bằng
**giây**, vì vậy để dùng với JavaScript cần nhân với 1000; giá trị `0` nghĩa là «không
có thời hạn»:

```html
<script>
 var exp = {{ .expire }};
 document.write(exp === 0
 ? 'Không có thời hạn'
 : 'Đến ' + new Date(exp * 1000).toLocaleDateString());
</script>

```

Lưu ý: `{{ .lastOnline }}` đã tính bằng **mili giây** – không cần nhân với 1000.

---

## ## 11. Xray: định tuyến, outbounds, DNS và tiện ích mở rộng

Mục **«Cài đặt Xray»** là trình soạn thảo mẫu cấu hình Xray-core, dựa trên đó bảng điều khiển tạo ra `config.json` cuối cùng để khởi chạy nhân. Gợi ý cho mục mẫu: **«Tập cấu hình Xray được tạo dựa trên mẫu này.»** Khác với inbounds (được lưu riêng trong cơ sở dữ liệu và được chèn vào mẫu khi tổng hợp cấu hình), tất cả những thứ còn lại – nhật ký, định tuyến, outbounds, DNS, chính sách, thống kê – đều được thiết lập tại đây.

> Quan trọng: giá trị mẫu được lưu trong cơ sở dữ liệu dưới khóa `xrayTemplateConfig`. Khi lưu, bảng điều khiển sẽ xử lý nó qua một loạt các phép biến đổi tự động (xem [11.10] (#1110-lưu-khởi-động-lại-và-các-biến-đổi-tự-động)). Bất kỳ JSON nào không hợp lệ về mặt cú pháp sẽ bị từ chối với lỗi **«xray template config invalid»**.

#### Vị trí trong menu: «Outbounds» và «Routing»

**«Outbounds»** và **«Routing»** – đây là các mục riêng biệt trong menu bên (ngay dưới «Hosts», trên «Cài đặt bảng điều khiển»), mỗi mục có địa chỉ riêng: `/outbound` và `/routing`. Các liên kết trực tiếp đến các trang này và tải lại trang hoạt động như mong đợi. Trong submenu **«Cấu hình Xray»** vẫn còn: Cơ bản, Bộ cân bằng tải, DNS và Mẫu nâng cao. Trong phần mô tả bên dưới, các mục [11.3] (#113-quy-tắc-định-tuyến-routing) và [11.4] (#114-outbounds-kết-nối-đi) tương ứng với các trang «Routing» và «Outbounds».

### ### 11.1. Cấu trúc trình soạn thảo: các tab/chế độ

Trình soạn thảo cung cấp một số chế độ hiển thị mẫu (bộ lọc theo phần JSON):

Chế độ	Nội dung hiển thị
---	---
<b>«Cơ bản»</b>	Các phần cơ bản (Nhật ký, định tuyến cơ bản, cài đặt chính)
<b>«Mẫu nâng cao»</b>	Toàn bộ mẫu JSON Xray
<b>«Tất cả»</b>	Tất cả các phần cùng một lúc

Các nhóm cài đặt logic bên trong trình soạn thảo:

- **«Cài đặt chính»** (gợi ý: **«Các thông số này mô tả các cài đặt chung»**).
- **«Nhật ký»** (xem [11.9] (#119-nhật-ký-và-thống-kê-stats-metrics)).
- **«Kết nối cơ bản»**: chặn và định tuyến trực tiếp.
- **«Inbounds»** (gợi ý: **«Thay đổi mẫu cấu hình để kết nối các máy khách cụ thể»**).
- **«Outbounds»** (xem [11.4] (#114-outbounds-kết-nối-đi)).
- **«Bộ cân bằng tải»** (xem [11.5] (#115-bộ-cân-bằng-tải-balancers)).
- **«Routing»** (gợi ý: **«Thứ tự ưu tiên của mỗi quy tắc rất quan trọng!»**, xem [11.3] (#113-quy-tắc-định-tuyến-routing)).
- **«DNS / Fake DNS»** (xem [11.6] (#116-dns)).

### ### 11.2. Cài đặt chính (General)

#### Freedom Protocol Strategy

Trường	Nhãn	Mô tả	Mặc định
---	---	---	---
<code>FreedomStrategy</code>	<b>«Cài đặt chiến lược giao thức Freedom»</b>	Chiến lược đầu ra mạng cho outbound trực tiếp (freedom). Gợi ý: <b>«Đặt chiến lược đầu ra mạng trong giao thức Freedom»</b> . Kiểm soát trường <code>domainStrategy</code> bên trong <code>settings</code> của outbound với giao thức <code>freedom</code> .   Trong mẫu tham chiếu, <code>domainStrategy</code> cho <code>freedom-outbound</code> <code>direct</code>	

là `**`AsIs`**` (địa chỉ không được phân giải, được truyền ở dạng nguyên bản). |

``domainStrategy`` cho freedom (các giá trị Xray-core): ``AsIs`` (không phân giải tên miền ở phía máy chủ), cũng như nhóm ``UseIP`` / ``UseIPv4`` / ``UseIPv6`` và các biến thể «bắt buộc» ``ForceIP*``, buộc máy chủ đầu ra phân giải tên miền và kết nối theo IP nhận được. Đổi thành ``UseIPv4`` nếu máy chủ đầu ra không có IPv6 hoặc cần chỉ đi qua IPv4.

#### #### Freedom Happy Eyeballs (IPv4/IPv6)

Trường	Nhãn	Mô tả
``FreedomHappyEyeballs``	`**Freedom Happy Eyeballs (IPv4/IPv6)**`	Gợi ý: `**«Kết nối dual-stack cho outbound trực tiếp (freedom) – hữu ích trên các máy chủ đầu ra có cả IPv4 lẫn IPv6.»` Bật thuật toán Happy Eyeballs (thử đồng thời cả hai họ địa chỉ) cho freedom-outbound.
try delay	(gợi ý)	`**Mili giây trước khi thử họ địa chỉ khác. 150–250 ms là điểm khởi đầu tốt.**` Độ trễ trước khi chuyển sang họ địa chỉ thay thế. Phạm vi khuyến nghị – 150–250 ms.

#### #### Overall Routing Strategy

Trường	Nhãn	Mô tả	Mặc định
``RoutingStrategy``	`**Cài đặt chiến lược định tuyến tên miền**`	Chiến lược phân giải DNS chung cho định tuyến. Gợi ý: `**«Đặt chiến lược định tuyến phân giải DNS chung»`. Kiểm soát trường ``routing.domainStrategy``.	Trong mẫu tham chiếu, ``routing.domainStrategy`` = `**`AsIs`**`.

``routing.domainStrategy`` xác định cách các quy tắc định tuyến IP được khớp với các yêu cầu tên miền: ``AsIs`` (chỉ quy tắc tên miền, không phân giải), ``IPIfNonMatch`` (nếu tên miền không khớp với quy tắc – phân giải và kiểm tra quy tắc IP), ``IPOnDemand`` (phân giải ngay khi gặp quy tắc IP). Để các quy tắc IP (ví dụ: ``geoip:*``) hoạt động với yêu cầu tên miền, thường cần ``IPIfNonMatch``.

#### #### Outbound Test URL

Trường	Nhãn	Mô tả	Mặc định
``outboundTestUrl``	`**URL kiểm tra outbound**`	URL để kiểm tra kết nối khi thử nghiệm outbound. Gợi ý: `**«URL để kiểm tra kết nối outbound»`. Được lưu riêng khỏi mẫu, dưới khóa ``xrayOutboundTestUrl``.	`**`https://www.google.com/generate_204`**`

Giá trị được làm sạch. Khi thử nghiệm outbound thực tế, nó còn được kiểm tra thêm như một URL công khai – đây là biện pháp bảo vệ chống SSRF: người dùng không thể đưa vào URL tùy ý (bao gồm cả URL nội bộ) thông qua máy khách, URL thử nghiệm luôn lấy từ cài đặt phía máy chủ. Giá trị rỗng khi lưu/thử nghiệm được thay bằng ``generate_204`` mặc định.

#### #### Block BitTorrent

Trường	Nhãn	Mô tả
``Torrent``	`**Chặn BitTorrent**`	Thêm vào ``routing.rules`` một quy tắc gửi lưu lượng có ``protocol: ["bittorrent"]`` đến outbound ``blocked``. Trong mẫu tham chiếu, quy tắc này có mặt theo mặc định.

#### #### Giới hạn kết nối (Connection Limits)

Gợi ý: `**«Chính sách cấp kết nối cho người dùng cấp 0. Để trống để sử dụng giá trị mặc`

định của Xray.»\* Các thông số này được ghi vào `policy.levels.0`.

Trường	Nhãn	Mô tả	Mặc định
`connIdle`	**Thời gian chờ nhàn rỗi** (giây)	*«Đóng kết nối sau khi nhàn rỗi trong số giây được chỉ định. Giảm giá trị này sẽ giải phóng bộ nhớ và bộ mô tả tệp nhanh hơn trên các máy chủ có tải cao (mặc định Xray: 300).»*	trống → mặc định Xray **300**
`bufferSize`	**Kích thước bộ đệm** (KB)	*«Kích thước bộ đệm nội bộ mỗi kết nối tính bằng KB. Đặt 0 để giảm thiểu sử dụng bộ nhớ trên các máy chủ có RAM thấp (giá trị mặc định Xray phụ thuộc vào nền tảng).»*	Placeholder: **«tự động»**.   trống → phụ thuộc nền tảng; `0` – giảm thiểu

\*\*Ví dụ (`policy.levels.0`).\*\* Các trường từ nhóm này được ghi vào chính sách cấp 0. Trên máy chủ có tải cao và RAM thấp, có thể tăng tốc giải phóng tài nguyên như sau:

```
```json
"policy": {
  "levels": {
    "0": {
      "connIdle": 120,
      "bufferSize": 0
    }
  }
}
```

Ở đây kết nối sẽ đóng sau 120 giây nhàn rỗi (thay vì 300 giây mặc định), còn bufferSize: 0 giảm thiểu mức tiêu thụ bộ nhớ cho bộ đệm. Trường để trống trong biểu mẫu đơn giản là không được đưa vào JSON – và Xray sẽ áp dụng giá trị mặc định của nó.

11.3. Quy tắc định tuyến (routing)

Danh sách quy tắc routing.rules. **Thứ tự rất quan trọng** («*Thứ tự ưu tiên của mỗi quy tắc rất quan trọng!*»): các quy tắc được đánh giá từ trên xuống dưới, quy tắc đầu tiên khớp sẽ kích hoạt. Gợi ý: «*Kéo để thay đổi thứ tự*». Các nút điều khiển thứ tự: **Đầu tiên**, **Cuối cùng**, **Lên trên**, **Xuống dưới**.

Mỗi quy tắc có type: "field". Các nút: **Tạo quy tắc**, **Chỉnh sửa quy tắc**. Gợi ý cho các trường dạng danh sách: «*Các mục cách nhau bằng dấu phẩy*».

Trên trang «Routing», các nút «**Nhập quy tắc**» và «**Xuất quy tắc**» được tập hợp trong menu thả xuống «**thêm**» (more) – giống như trên trang «Outbounds». Nút «**Xuất quy tắc**» không tải xuống tệp ngay lập tức mà mở cửa sổ modal với bản xem trước JSON và các nút «**Sao chép**» và «**Tải xuống**»: nội dung có thể xem trước khi lưu. Xuất outbounds trên trang «Outbounds» hoạt động tương tự.

Route Tester (bộ kiểm tra định tuyến)

Trên tab Routing có sub-tab **Route Tester** – nó hỏi Xray đang chạy xem outbound nào sẽ xử lý một kết nối cụ thể, mà không gửi lưu lượng thực. Chỉ định tên miền hoặc IP, cổng, mạng (TCP/UDP) và nếu cần, inbound và giao thức bị chặn (http / tls / quic / bittorrent), sau đó nhấn **Test Route**. Kết quả được lấy trực tiếp từ engine định tuyến đang chạy.

Phản hồi hiển thị outbound được chọn, và khi sử dụng bộ cân bằng tải – còn có tag của bộ cân bằng tải. Nếu không có quy tắc nào khớp, bộ kiểm tra sẽ thông báo rằng lưu lượng đi vào outbound mặc định (đầu tiên trong danh sách `outbounds`). Điều này rất tiện để kiểm tra thứ tự quy tắc trước khi tin vào chúng.

Bật và tắt từng quy tắc riêng lẻ

Một quy tắc định tuyến có thể được **tắt** tạm thời bằng công tắc, không cần xóa. Trong bảng quy tắc có cột **«Bật»** với công tắc (Switch), và trong biểu mẫu quy tắc có trường **«Bật»** – cũng là công tắc. Quy tắc bị tắt sẽ không được đưa vào cấu hình Xray cuối cùng, nhưng được lưu trong mẫu và có thể bật lại bất kỳ lúc nào.

Quy tắc thông kê dịch vụ (`inboundTag: ["api"] → outboundTag: "api"`) không thể tắt – công tắc của nó bị khóa để không làm hỏng tính năng đếm lưu lượng của bảng điều khiển (xem [11.10](#)).

Các trường biểu mẫu quy tắc

Trường biểu mẫu	Nhân	Trường JSON	Mô tả
Nguồn	Nguồn	<code>source</code>	Địa chỉ IP/mạng con nguồn. Danh sách phân cách bằng dấu phẩy.
Cổng nguồn	Cổng nguồn	<code>sourcePort</code>	Cổng nguồn.
Đích	Đích	<code>domain + ip + port</code>	Tên miền, IP và cổng đích. Tên miền hỗ trợ tiền tố <code>domain: , full: , regexp: , keyword: ,</code> cũng như <code>geosite:* ; IP – geoip:*</code> và CIDR.
Mạng	–	<code>network</code>	<code>tcp , udp</code> hoặc <code>tcp,udp</code> .
Giao thức	–	<code>protocol</code>	<code>http , tls , bittorrent</code> (xác định qua sniffing).
Người dùng	Người dùng	<code>user</code>	Lọc theo e-mail/định danh người dùng.
Thuộc tính / Giá trị	Thuộc tính / Giá trị	<code>attrs</code>	Thuộc tính tiêu đề HTTP để khớp.
VLESS route	VLESS route	–	Định tuyến theo trường route của VLESS.
Tags inbound	Tags inbound	<code>inboundTag</code>	Một hoặc nhiều tag inbound áp dụng quy tắc (bao gồm cả <code>api</code> dựng sẵn và tag DNS từ cài đặt DNS). Trong danh sách inbound hiển thị là «tag (remark)» nếu inbound có ghi chú riêng, nếu không – chỉ tag; trong quy tắc đã lưu vẫn chỉ lưu tag.
Tag outbound	Tag outbound / Kết nối đi	<code>outboundTag</code>	Nơi chuyển lưu lượng khớp đến.
Tag bộ cân bằng tải	Tag bộ cân bằng tải / Bộ cân bằng tải	<code>balancerTag</code>	Gợi ý: « <i>Chuyển lưu lượng qua một trong các bộ cân bằng tải đã cấu hình</i> ».

Loại trừ lẫn nhau giữa `outboundTag` và `balancerTag` : «*Không thể sử dụng đồng thời `balancerTag` và `outboundTag`. Khi sử dụng đồng thời, chỉ `outboundTag` hoạt động.*» Trong một quy tắc, chỉ đặt tag `outbound` hoặc tag bộ cân bằng tải.

Các quy tắc tích hợp sẵn trong mẫu tham chiếu

Trong `config.json` tiêu chuẩn, phần `routing` chứa ba quy tắc (theo thứ tự này):

- `inboundTag: ["api"]` → `outboundTag: "api"` – quy tắc dịch vụ cho gRPC-API thống kê bảng điều khiển.
- `ip: ["geoip:private"]` → `outboundTag: "blocked"` – chặn các dải địa chỉ riêng tư.
- `protocol: ["bittorrent"]` → `outboundTag: "blocked"` – chặn BitTorrent.

Quy tắc `api` → `api` luôn được tự động đẩy lên vị trí 0 khi lưu (xem 11.10), để yêu cầu thống kê không bị quy tắc catch-all phía trên «nuốt» mất.

Ví dụ quy tắc. Gửi toàn bộ lưu lượng đến các trang web Nga và mạng riêng tư trực tiếp (bỏ qua proxy), phần còn lại – đến bộ cân bằng tải. Thứ tự quan trọng: quy tắc «chuyển thẳng» phải đặt trên quy tắc catch-all.

Trong `routing.rules` :

```
{
  "type": "field",
  "domain": ["geosite:category-ru", "domain:example.ru"],
  "ip": ["geoip:ru", "geoip:private"],
  "outboundTag": "direct"
}
```

Để các quy tắc IP (`geoip:ru`) hoạt động với các yêu cầu tên miền, thường cần đặt

`routing.domainStrategy: "IPIfNonMatch"` ở cấp cao nhất của định tuyến (xem 11.2).

Các nhóm định tuyến được cấu hình sẵn (Kết nối cơ bản)

Ở chế độ «Kết nối cơ bản», bảng điều khiển giúp xây dựng các quy tắc điển hình từ các danh sách có sẵn:

Nhóm	Trường	Gợi ý
Chặn theo giao thức/ trang web	–	«Cấu hình để khách hàng không có quyền truy cập vào các giao thức cụ thể»
Chặn theo quốc gia	Địa chỉ IP bị chặn, Tên miền bị chặn	«Các thông số này sẽ chặn lưu lượng tùy thuộc vào quốc gia đích.»
Kết nối trực tiếp	IP trực tiếp, Tên miền trực tiếp	«Kết nối trực tiếp có nghĩa là lưu lượng cụ thể sẽ không được chuyển hướng qua máy chủ khác.»
Quy tắc IPv4	–	«Các thông số này sẽ cho phép khách hàng định tuyến đến các tên miền đích chỉ qua IPv4»

Nhóm	Trường	Gợi ý
Quy tắc WARP	—	«Các tùy chọn này sẽ định tuyến lưu lượng tùy thuộc vào đích cụ thể qua WARP.»
Định tuyến NordVPN	—	«Các tùy chọn này sẽ định tuyến lưu lượng tùy thuộc vào đích cụ thể qua NordVPN.»

MTProto-inbound: định tuyến lưu lượng Telegram qua Xray

MTProto-inbound có công tắc **«Route through Xray»** (tắt theo mặc định) và tùy chọn chọn **Outbound**. Khi bật, bảng điều khiển thêm vào cấu hình Xray cầu SOCKS loopback với tag của chính inbound đó, và mtg chuyển hướng lưu lượng Telegram qua đó. Sau đó lưu lượng Telegram đi được bộ định tuyến kiểm soát: có thể khớp với quy tắc thông thường trong tab Routing theo tag inbound hoặc buộc chuyển đến outbound hoặc bộ cân bằng tải đã chọn qua trường **Outbound**. Để **Outbound** trống nếu muốn các quy tắc định tuyến quyết định.

11.4. Outbounds (kết nối đi)

Danh sách outbounds. Các nút: **Tạo kết nối đi**, **Sửa kết nối đi**. Gợi ý: «Thay đổi mẫu cấu hình để xác định các kết nối đi cho máy chủ này».

Trong mẫu tham chiếu có hai outbound bắt buộc:

- `protocol: "freedom", tag: "direct"` — truy cập trực tiếp vào Internet (với `domainStrategy: "AsIs"` và `finalRules: [{action: "allow"}]`);
- `protocol: "blackhole", tag: "blocked"` — «lỗ đen» cho lưu lượng bị chặn.

Các trường biểu mẫu outbound chung

Trường	Nhãn	Mô tả
Tag	Tag (gợi ý: «Tag duy nhất»)	Định danh duy nhất của outbound. Placeholder: «tag-duy-nhat». Xác thực: «Tag là bắt buộc», «Tag đã được sử dụng bởi kết nối đi khác».
Giao thức	—	Loại kết nối đi (xem bên dưới).
Địa chỉ / Cổng	Địa chỉ / Cổng	Đích kết nối. Địa chỉ và cổng là bắt buộc.
Gửi qua	Gửi qua	Địa chỉ IP cục bộ của giao diện đi (<code>sendThrough</code>). Placeholder: «IP cục bộ».
Dialer proxy (chuỗi)	—	Gợi ý: «Kết nối outbound này qua outbound khác (theo tag) để xây dựng chuỗi proxy. Để trống để kết nối trực tiếp.» Placeholder: «Chọn outbound để tạo chuỗi». Được triển khai qua <code>streamSettings.sockopt.dialerProxy</code> .

Các giao thức outbound được hỗ trợ

Các giao thức được hỗ trợ bởi biểu mẫu:

- **freedom** – truy cập trực tiếp. Các trường `settings.domainStrategy` , `finalRules` (xem bên dưới), Happy Eyeballs. Không thể kiểm tra («*Outbound has no testable endpoint*»).
- **blackhole** – loại bỏ lưu lượng. Trường **Loại phản hồi**. Không kiểm tra được.
- **socks** , **http** – danh sách `settings.servers[]` với `address / port` ; trường **Mật khẩu xác thực**.
- **vmess** – `settings.vnext[]` (`address / port`).
- **vless** – `settings.address / settings.port` .
- **trojan** , **shadowsocks** – `settings.servers[]` .
- **wireguard** – `settings.peers[]` với `endpoint` , cộng với các khóa (xem 11.7).
- **hysteria** – `settings.address / settings.port` (UDP-transport).

Đối với outbound loại **loopback**, có khối **Sniffing** với các thông số giống như inbound: bật, **destOverride**, **Metadata Only**, **Route Only** và danh sách **tên miền bị loại trừ**.

Trong mask **UDP** (FinalMask) cho **Hysteria2** có các chế độ bổ sung. Mask **Salamander** có bộ chọn **Mode** với các giá trị **Salamander** và **Gecko**: chế độ Gecko thêm phần đệm ngẫu nhiên cho gói tin với các trường **Min/Max** kích thước (`packetSize` , phạm vi 1–2048, mặc định 512–1200) – điều này bảo vệ chống fingerprinting theo độ dài gói tin. Mask **Realm** (UDP hole-punching) có thêm khối **TLS Config** tùy chọn với các trường **Server Name** (SNI), **ALPN** (`h3 / h2 / http/1.1`), **Fingerprint** (uTLS) và công tắc **Allow Insecure**.

Ví dụ: chuỗi qua SOCKS phía trên. Outbound `upstream` kết nối đến proxy SOCKS5 bên ngoài, còn `chained` gửi lưu lượng của mình qua đó (`dialerProxy`), tạo thành chuỗi. Trong `outbounds` :

```
[
  {
    "tag": "upstream",
    "protocol": "socks",
    "settings": {
      "servers": [{ "address": "203.0.113.10", "port": 1080 }]
    }
  },
  {
    "tag": "chained",
    "protocol": "freedom",
    "streamSettings": {
      "sockopt": { "dialerProxy": "upstream" }
    }
  }
]
```

Bây giờ quy tắc định tuyến với `outboundTag: "chained"` sẽ đưa lưu lượng ra Internet qua `upstream` .

Nhập outbound từ share-link

Outbound có thể được nhập từ share-link (`vless://` , `vmess://` , v.v.). Khi nhập, cài đặt multiplexer **xmux** (XHTTP) được truyền trong khối `extra=` của link cũng được lưu: sau khi nhập, các giá trị của chúng được điền vào biểu mẫu con **XMUX** của outbound được tạo.

Các trường Mux (ghép kênh)

Đồng thời tối đa, Kết nối tối đa, Tái sử dụng tối đa, Yêu cầu tối đa, Giây tái sử dụng tối đa, Chu kỳ keep alive. Các thông số này cấu hình hành vi mux/XUDP của kết nối đi.

Sockopts (cài đặt socket)

Nhóm **Sockopts**: Khoảng thời gian keep alive, Mark (fwmark), Giao diện, Chỉ IPv6, Chấp nhận proxy protocol, Proxy protocol, TCP user timeout (ms), TCP keep-alive idle (s). Dialer-proxy của chuỗi cũng được đặt ở đây.

Freedom finalRules (ghi đè chặn IP riêng tư)

Đối với freedom-outbound có nhóm **Quy tắc cuối**:

Trường	Nhân	Mô tả
<code>overrideXrayPrivateIp</code>	Ghi đè chặn IP riêng tư mặc định trong Xray	Gỡ bỏ lệnh cấm tích hợp trong Xray đối với kết nối đi đến IP riêng tư.
<code>action</code>	Hành động	<code>allow</code> (như trong mẫu tham chiếu: <code>finalRules: [{action: "allow"}]</code>), <code>redirect</code> (Redirect) hoặc các loại khác.
<code>blockDelay</code>	Độ trễ chặn (ms)	Độ trễ trước khi loại bỏ kết nối.
<code>redirect / fragment</code>	Redirect / Fragment	Hành động chuyển hướng và phân mảnh lưu lượng.

Mask fragment: Lengths và Delays theo từng mảnh

Trong mask **fragment** (loại fragment trong FinalMask, cho TCP), các trường đơn Length và Delay được thay bằng danh sách **Lengths** và **Delays**: cho mỗi đoạn có thể đặt phạm vi độ dài riêng (ví dụ `100-200`) và độ trễ tính bằng mili giây (ví dụ `10-20` hoặc `0`). Các hàng trong danh sách có thể thêm và xóa; các giá trị đơn lẻ đã lưu trước đây được chuyển thành mảng một phần tử tự động.

Các trường biểu mẫu khác

- **UDP over TCP** và **Phiên bản UoT** — dành cho các giao thức kiểu shadowsocks.
- **Không có tiêu đề gRPC, Kích thước chunk Uplink** — các thông số transport gRPC.
- Các trường TLS/uTLS: **Xác minh tên peer, Pinned SHA256, Short ID, Vision testpre**, placeholder «tên máy chủ».

Kiểm tra kết nối đi

Các nút: **Kiểm tra, Kiểm tra tất cả**. Trạng thái: **Đang kiểm tra kết nối..., Kiểm tra thành công, Kiểm tra thất bại, Không thể kiểm tra kết nối đi**. Kết quả: **Kết quả kiểm tra**, độ trễ tính bằng mili giây.

Hai chế độ (gợi ý: «TCP: thăm dò dial-only nhanh. HTTP: yêu cầu đầy đủ qua xray.»):

- **TCP** (mode=tcp) – dial đơn giản đến host:port , thực hiện song song trên tất cả các endpoint, ~timeout 5 giây. Chỉ kiểm tra khả năng tiếp cận TCP, không xác thực giao thức proxy. Đối với freedom / blackhole /tag blocked sẽ trả về «Outbound has no testable endpoint».
- **HTTP** (mode=http hoặc trống) – khởi động một phiên bản Xray tạm thời, thực hiện yêu cầu HTTP thực (probe URL = outboundTestUrl phía máy chủ), đo độ trễ thực. Chế độ có thẩm quyền nhưng tốn kém: được tuần tự hóa bởi khóa toàn cục («Another outbound test is already running, please wait»). Timeout một lần thử – 10 giây, cửa sổ chờ kết quả – 15 giây (tăng lên để các outbound lành mạnh trên các kênh chậm hoặc tunnel không bị đánh dấu là «Failed»). Khi thất bại, nguyên nhân thực (lỗi DNS, connection refused, hết deadline, lỗi TLS, v.v.) được ghi vào nhật ký bảng điều khiển/Xray, nơi mà các thông báo về timeout chung chỉ đến.

Các giao thức UDP (wireguard , hysteria) và các transport UDP (kcp , quic , hysteria) **luôn** được kiểm tra ở chế độ HTTP, ngay cả khi yêu cầu TCP – dial UDP thuần không phân biệt được endpoint «sống» với «chết». Đối với wireguard trong cấu hình thử nghiệm, noKernelTun: true được bắt buộc đặt.

Kiểm tra hàng loạt và phân tích theo giai đoạn

Kiểm tra và **Kiểm tra tất cả** ở chế độ HTTP khởi động một phiên bản Xray tạm thời chung cho cả nhóm outbound, tạo SOCKS-inbound loopback với quy tắc cho mỗi cái và gửi song song yêu cầu HTTP thực qua đó; **Kiểm tra tất cả** kiểm tra outbound theo lô. **Kiểm tra tất cả** cũng kiểm tra các outbound nhận được từ subscriptions (bảng «từ subscriptions», chỉ đọc) – các hàng của chúng cũng được tô màu theo kết quả kiểm tra. Trong khi đó, các outbound freedom («direct») và dns không được kiểm tra ở bất kỳ chế độ nào (đây không phải proxy): nút kiểm tra ở chúng không có sẵn, **Kiểm tra tất cả** bỏ qua chúng, và biện pháp bảo vệ phía máy chủ cấm kiểm tra HTTP của chúng ngay cả khi gọi API trực tiếp. Ngoài thành công/lỗi, popup kết quả hiển thị HTTP status phản hồi và phân tích thời gian theo giai đoạn: **Proxy connect** (kết nối đến proxy), **TLS via outbound** (TLS qua outbound) và **First byte** (thời gian đến byte đầu tiên) – điều này giúp hiểu ở bước nào xảy ra độ trễ hoặc sự cố.

Thông kê lưu lượng outbounds

Bảng điều khiển lưu bộ đếm lưu lượng theo tag (up / down / total). Nút đặt lại sẽ đặt lại bộ đếm cho tag cụ thể hoặc cho tất cả (tag = "-alltags-"). Các trường **Thông tin tài khoản** và **Trạng thái kết nối đi** hiển thị bản tóm tắt.

11.5. Bộ cân bằng tải (Balancers)

Danh sách routing.balancers . Các nút: **Tạo bộ cân bằng tải**, **Chỉnh sửa bộ cân bằng tải**.

Trong tab Balancers có các cột trạng thái trực tiếp: **Live Target** hiển thị mục tiêu hiện tại đang hoạt động của bộ cân bằng tải trong Xray đang chạy, còn **Override** cho phép ghi đè thủ công việc chọn mục tiêu (giá trị **Auto (strategy)** trả về lựa chọn theo chiến lược). Trạng thái được cập nhật bằng nút riêng. Nếu bộ cân bằng tải chưa hoạt động trong Xray đang chạy, bảng điều khiển sẽ đề nghị lưu thay đổi hoặc khởi động Xray trước.

Trường	Nhãn	Mô tả
Tag	Tag (gợi ý: « <i>Tag duy nhất</i> »)	Định danh duy nhất. Placeholder: « <i>tag bộ cân bằng tải duy nhất</i> ». Xác thực: « <i>Tag là bắt buộc</i> », « <i>Tag đã được sử dụng bởi bộ cân bằng tải khác</i> ».
Bộ chọn	Bộ chọn	Danh sách các tag outbound (theo chuỗi con) để bộ cân bằng tải lựa chọn đầu ra. Phải chọn ít nhất một: « <i>Chọn ít nhất một kết nối đi</i> ».
Fallback	Fallback	Tag outbound dự phòng nếu không có bộ chọn nào phù hợp.
Chiến lược	Chiến lược	Thuật toán lựa chọn (xem bên dưới).

Chiến lược và thông số quan sát

Chiến lược (`strategy.type`) xác định cách bộ cân bằng tải chọn outbound từ các bộ chọn. Các giá trị Xray-core: `random` (ngẫu nhiên), `roundRobin` (vòng tròn), `leastPing` (độ trễ tối thiểu theo kết quả observatory), `leastLoad` (tải tối thiểu). Đối với `leastLoad / leastPing`, các thông số từ `strategy.settings` được sử dụng:

Trường	Nhãn	Mô tả
<code>expected</code>	Dự kiến	Placeholder: « <i>số node tối ưu</i> » – số mục tiêu đang hoạt động mong muốn.
<code>maxRtt</code>	RTT tối đa	Giới hạn trên của RTT chấp nhận được khi lọc ứng viên.
<code>tolerance</code>	Dung sai	Dung sai (tolerance) khi so sánh độ trễ/tải.
<code>baselines</code>	Baselines	Các giá trị ngưỡng độ trễ để nhóm các node.
<code>costs</code>	Costs	Hệ số trọng số (cost) cho các tag riêng lẻ.

Ví dụ về chiến lược. Khối `strategy` nằm bên trong bộ cân bằng tải (trong JSON – cạnh `tag` và `selector`):

```
"strategy": { "type": "random" }      // lựa chọn ngẫu nhiên từ các bộ chọn
"strategy": { "type": "roundRobin" }  // theo vòng tròn, lần lượt
"strategy": { "type": "leastPing" }   // độ trễ tối thiểu (cần observer)
```

Đối với `leastLoad`, các thông số được đặt trong `settings`:

```
"strategy": {
  "type": "leastLoad",
  "settings": {
    "expected": 2,
    "maxRTT": "1s",
    "tolerance": 0.05,
    "baselines": ["500ms", "1s", "2s"],
    "costs": [
      { "regexp": false, "match": "proxy-premium", "value": 0.1 },
      { "regexp": true, "match": "^proxy-cheap-.+$", "value": 5 }
    ]
  }
}
```

Cách hoạt động (ví dụ). Giả sử observer đo được độ trễ cho các đầu ra: `A = 250 ms` , `B = 280 ms` , `C = 700 ms` , `D = 1500 ms` . Với các cài đặt trên, việc lựa chọn diễn ra như sau:

1. **maxRTT: "1s"** – các đầu ra có độ trễ trên 1 giây bị loại: `D (1500 ms)` bị loại. Còn lại `A` , `B` , `C` .
2. **baselines + expected** – các đầu ra được nhóm theo ngưỡng độ trễ, và ngưỡng **nhỏ nhất** chứa ít nhất `expected` đầu ra được chọn. Ngưỡng `500ms` đã chứa `A` và `B` – đó là 2 (= `expected`) , vì vậy nhóm `{ A , B }` được chọn. `C (700 ms)` không vào lựa chọn khi còn đủ đầu ra nhanh (đây là «dự phòng nóng»).
3. **tolerance: 0.05** – trong nhóm được chọn, các đầu ra có độ trễ chênh lệch không quá 5% được coi là tương đương và tải được chia đều. `A (250)` và `B (280)` chênh lệch ~12% (> 5%), vì vậy khi các điều kiện khác bằng nhau, ưu tiên thuộc về `A` nhanh hơn; nếu chênh lệch trong vòng 5% – lưu lượng sẽ đi qua cả `A` và `B` .
4. **costs** – trước khi so sánh sẽ điều chỉnh «chi phí» của từng đầu ra: `value` nhỏ hơn làm đầu ra hấp dẫn hơn, `value` lớn hơn – ngược lại. Trong ví dụ, `proxy-premium` được `0.1` (trở nên «rẻ hơn» và được ưu tiên chọn hơn), còn tất cả `proxy-cheap-*` (theo regex, `regexp: true`) – `5` (trở nên «đắt hơn» và được sử dụng cuối cùng). Điều này cho phép ưu tiên mềm các đầu ra mà không loại bỏ cứng chúng.

Kết quả: lưu lượng chủ yếu đi qua `A` (khi độ trễ gần nhau – chia đều với `B`) , `C` là dự phòng, `D` bị loại cho đến khi RTT giảm xuống dưới `maxRTT` .

Observer: `observatory` và `burst0bservatory` (đo lường cho `leastPing` / `leastLoad`)

Chiến lược `leastPing` và `leastLoad` không tự đo gì – chúng cần dữ liệu về độ trễ và khả năng sẵn sàng của từng outbound. Dữ liệu này được thu thập bởi **observer** (observatory): nó định kỳ «ping» từng outbound đang được theo dõi và ghi lại thời gian phản hồi và khả năng sẵn sàng. Dữ liệu tương tự được hiển thị trong tab «**Observatory**» (trạng thái **Hoạt động** / **Không khả dụng**, «**Hoạt động cuối**», «**Lần thử cuối**»).

Không có biểu mẫu riêng cho observer trong bảng điều khiển – khối được thêm **thủ công** trong trình soạn thảo cấu hình Xray, ở cấp cao nhất của config (cạnh `routing` và `outbounds`), sau đó cần **khởi động lại Xray**.

Có hai tùy chọn:

- **observatory** – đơn giản: `subjectSelector` + `probeURL` + `probeInterval` .
- **burst0bservatory** – mở rộng, với cấu hình ping chi tiết qua `pingConfig` ; tiện cho nhiều đầu ra.

Ví dụ khối `burst0bservatory` :

```
{
  "subjectSelector": ["WS-SE", "WS-FR", "WS-PL"],
  "pingConfig": {
    "destination": "https://www.google.com/generate_204",
    "interval": "1m",
    "connectivity": "http://connectivitycheck.platform.hicloud.com/generate_204",
    "timeout": "5s",
    "sampling": 2
  }
}
```

Ý nghĩa các trường:

Trường	Chức năng
<code>subjectSelector</code>	Danh sách tiền tố tag outbound để quan sát. Xray lấy tất cả outbound có tag bắt đầu bằng các chuỗi đã chỉ định. Trong ví dụ, các đầu ra <code>WS-SE...</code> , <code>WS-FR...</code> , <code>WS-PL...</code> được quan sát. Các tag này phải khớp với những gì được chọn trong Bộ chọn của bộ cân bằng tải.
<code>pingConfig.destination</code>	URL được yêu cầu qua từng outbound để đo độ trễ. Lấy trang «nhẹ» với phản hồi <code>204</code> không có nội dung – ví dụ <code>https://www.google.com/generate_204</code> . Thời gian đến phản hồi chính là độ trễ được đo.
<code>pingConfig.interval</code>	Tần suất ping từng outbound. Chuỗi thời gian: <code>"1m"</code> – mỗi phút một lần, cũng <code>"30s"</code> , <code>"5m"</code> , v.v. Thường xuyên hơn – dữ liệu mới hơn, nhưng nhiều lưu lượng nền hơn.
<code>pingConfig.connectivity</code>	(tùy chọn) URL kiểm tra kết nối cơ bản của chính máy chủ. Nếu không thể truy cập – nghĩa là vấn đề nằm ở mạng máy chủ, và observer không đánh dấu outbound là không khả dụng (bảo vệ chống báo động giả khi có sự cố cục bộ). Thường cũng là endpoint với phản hồi <code>204</code> .
<code>pingConfig.timeout</code>	Thời gian chờ phản hồi một lần ping trước khi coi lần thử thất bại (ví dụ <code>"5s"</code>).
<code>pingConfig.sampling</code>	Số lần đo cuối cùng cần lưu và lấy trung bình cho mỗi outbound. <code>2</code> – tính đến hai lần ping cuối cùng (làm phẳng các đột biến ngẫu nhiên).

Cách kết nối mọi thứ:

1. Trong trình soạn thảo Xray, thêm khối `burst0bservatory` với `subjectSelector` cần thiết.
2. Tạo bộ cân bằng tải: **Chiến lược** = `leastPing`, trong **Bộ chọn** chỉ định các tag outbound tương tự (`WS-SE`, `WS-FR`, `WS-PL`).
3. Chuyển lưu lượng đến nó bằng quy tắc định tuyến (trường **Tag bộ cân bằng tải**, xem 11.3).
4. Khởi động lại Xray. Trong tab «**Observatory**» sẽ xuất hiện trạng thái đầu ra, và bộ cân bằng tải sẽ bắt đầu chọn đầu ra nhanh nhất trong số các đầu ra đang hoạt động.

Trong một quy tắc không thể đồng thời đặt `balancerTag` và `outboundTag` – chỉ `outboundTag` hoạt động.

11.6. DNS

Phần `dns`. Bật: **Bật DNS** (gợi ý: «*Bật máy chủ DNS tích hợp*»).

Các thông số DNS chung

Trường	Nhân	JSON	Mô tả / gợi ý
<code>tag</code>	Tên tag DNS	<code>dns.tag</code>	«Tag này sẽ có sẵn như tag inbound trong các quy tắc định tuyến.» Cho phép định tuyến các yêu cầu DNS qua <code>inboundTag</code> .

Trường	Nhãn	JSON	Mô tả / gợi ý
clientIp	IP khách hàng	dns.clientIp	«Được sử dụng để thông báo cho máy chủ về vị trí IP được chỉ định trong các yêu cầu DNS» (EDNS Client Subnet).
strategy	Chiến lược truy vấn	dns.queryStrategy	«Chiến lược phân giải tên miền chung». Các giá trị: UseIP , UseIPv4 , UseIPv6 .
disableCache	Tắt bộ nhớ đệm	dns.disableCache	«Tắt bộ nhớ đệm DNS».
disableFallback	Tắt DNS dự phòng	dns.disableFallback	«Tắt các truy vấn DNS dự phòng».
disableFallbackIfMatch	Tắt DNS dự phòng khi khớp	dns.disableFallbackIfMatch	«Tắt các truy vấn DNS dự phòng khi danh sách tên miền của máy chủ DNS khớp».
enableParallelQuery	Bật truy vấn song song	—	«Bật truy vấn DNS song song đến nhiều máy chủ để phân giải nhanh hơn».
useSystemHosts	Sử dụng Hosts hệ thống	dns.useSystemHosts	«Sử dụng tệp hosts từ hệ thống đã cài đặt».

Ví dụ khối dns . Các truy vấn đến tên miền Google được phân giải qua máy chủ DoH Cloudflare, mọi thứ còn lại – qua 1.1.1.1 ; đối với truy vấn Google, chỉ các IP không riêng tư được chờ đợi. Ở cấp cao nhất của config:

```
"dns": {
  "tag": "dns-inbound",
  "queryStrategy": "UseIPv4",
  "servers": [
    {
      "address": "https://cloudflare-dns.com/dns-query",
      "domains": ["geosite:google"],
      "expectIPs": ["geoip:!private"]
    },
    "1.1.1.1"
  ]
}
```

Chuỗi máy chủ ("1.1.1.1") không có trường – đây là máy chủ mặc định cho tất cả các tên miền khác. Tag dns-inbound sau đó có thể được sử dụng như inboundTag trong các quy tắc định tuyến để chuyển chính các yêu cầu DNS qua outbound cần thiết.

Bộ nhớ đệm bản ghi hết hạn

Trường	Nhãn	Mô tả
<code>serveStale</code>	Sử dụng hết hạn	«Trả về kết quả hết hạn từ bộ nhớ đệm trong khi cập nhật trong nền».
<code>serveExpiredTTL</code>	TTL hết hạn	«Thời gian tồn tại (giây) của bản ghi bộ nhớ đệm hết hạn; 0 = vô thời hạn».

Các máy chủ DNS (danh sách `dns.servers`)

Các nút: **Tạo DNS**, **Chỉnh sửa DNS**, **Xóa tất cả** (xác nhận: «Tất cả máy chủ DNS sẽ bị xóa khỏi danh sách. Hành động này không thể hoàn tác.»). Mẫu: **Sử dụng mẫu**, cửa sổ **Mẫu DNS**, bao gồm cả preset **Gia đình**.

Khi nhấn **Chỉnh sửa DNS** trên một bản ghi máy chủ DNS (cũng như bản ghi Fake DNS), cửa sổ chỉnh sửa sẽ điền các giá trị đã lưu của máy chủ, không phải giá trị mặc định.

Các trường máy chủ DNS:

Trường	Nhãn	Mô tả
<code>address</code>	—	Địa chỉ DNS (IP, DoH-URL, <code>localhost</code> , <code>fakedns</code> , v.v.).
<code>domains</code>	Tên miền	Danh sách tên miền sử dụng máy chủ này.
<code>expectIPs</code>	IP dự kiến	Chỉ chấp nhận phản hồi nếu IP nằm trong danh sách.
<code>unexpectIPs</code>	IP không mong muốn	Loại bỏ phản hồi với các IP được chỉ định.
<code>skipFallback</code>	Bỏ qua Fallback	Không sử dụng máy chủ này làm fallback.
<code>finalQuery</code>	Truy vấn cuối	Đánh dấu máy chủ là truy vấn cuối cùng trong chuỗi.
<code>timeoutMs</code>	Timeout (ms)	Timeout truy vấn đến máy chủ.

Hosts (bản ghi tĩnh)

Nhóm **Hosts** (`dns.hosts`). Nút **Thêm Host**; trạng thái trống **Chưa có Host nào**. Các trường: tên miền (placeholder: «Tên miền (ví dụ domain:example.com)») và các giá trị (placeholder: «IP hoặc tên miền – nhập và nhấn Enter»).

Nhật ký DNS

Xem 11.9: cờ **Nhật ký DNS** (`dnsLog`) trong phần ghi nhật ký.

11.7. Fake DNS

Phần `fakedns` . Các nút: **Tạo Fake DNS**, **Chỉnh sửa Fake DNS**.

Trường	Nhãn	Mô tả
<code>ipPool</code>	Mạng con pool IP	Phạm vi CIDR để cấp phát IP giả (ví dụ <code>198.18.0.0/15</code>).

Trường	Nhãn	Mô tả
poolSize	Kích thước pool	Số địa chỉ cần giữ trong pool vòng tròn.

Fake DNS được sử dụng kết hợp với sniffing trên inbound: nhân cấp cho khách hàng một IP giả, ghi nhớ ánh xạ tên miền IP và khôi phục tên miền khi định tuyến. Để Fake DNS hoạt động, máy chủ DNS với địa chỉ `fakedns` phải được thêm vào danh sách máy chủ DNS.

Ví dụ: kết hợp Fake DNS + máy chủ DNS. Trước tiên đặt pool địa chỉ giả, sau đó thêm máy chủ DNS `fakedns` để các truy vấn tên miền nhận IP từ pool này:

```
"fakedns": [
  { "ipPool": "198.18.0.0/15", "poolSize": 65535 }
],
"dns": {
  "servers": [
    { "address": "fakedns", "domains": ["geosite:geolocation-!cn"] },
    "1.1.1.1"
  ]
}
```

Ngoài ra, cần bật sniffing trên inbound với `destOverride: ["fakedns"]`, nếu không nhân sẽ không có nơi nào để lấy tên miền thực để khôi phục.

11.8. WireGuard / WARP / NordVPN

Các trường WireGuard (`wireguard`)

Trường	Nhãn	Mô tả
secretKey	Khóa bí mật	Khóa riêng của giao diện cục bộ.
publicKey	Khóa công khai	Khóa công khai của peer.
psk	Khóa chung	PreShared Key (tùy chọn).
allowedIPs	Địa chỉ IP được phép	Các dải địa chỉ được định tuyến vào tunnel.
endpoint	Điểm cuối	<code>host:port</code> của peer.
domainStrategy	Chiến lược tên miền	Chiến lược phân giải cho WireGuard-outbound.

Cloudflare WARP (`warp`)

Tích hợp sử dụng API `https://api.cloudflareclient.com/v0a4005` (client-version `a-6.30-3596`). Các hành động controller (`/xray/warp/:action`): `config`, `reg`, `license`, `data`, `del`.

Theo từng bước:

1. **Tạo tài khoản WARP** → `reg` : bảng điều khiển tạo/nhận các khóa riêng (`privateKey`) và công khai (`publicKey`), đăng ký thiết bị với Cloudflare và lưu `access_token` , `device_id` , `license_key` , `private_key` (cũng như `client_id`) trong cài đặt `warp` .
2. **Khóa bản quyền WARP / WARP+** → `license` : đặt khóa WARP+ 26 ký tự (placeholder: «*Khóa WARP+ 26 ký tự*»). Khi có lỗi: «*Không thể đặt giấy phép WARP.*» Nếu chưa có config: «*Lấy WARP config trước.*»
3. **Thông tin tài khoản: Tên thiết bị, Model thiết bị, Thiết bị được bật, Loại tài khoản, Vai trò, Dữ liệu WARP+, Hạn mức, Mức sử dụng.**
4. **Thêm kết nối đi** – tạo WireGuard-outbound với các khóa và endpoint Cloudflare đã nhận được.
5. **Xóa tài khoản** → `del` : xóa dữ liệu WARP đã lưu.

NordVPN (`nord` / `nordvpn`)

Tích hợp sử dụng NordLynx (= WireGuard). Các hành động controller (`/xray/nord/:action`): `countries` , `servers` , `reg` , `setKey` , `data` , `del` .

Theo từng bước:

1. **Token truy cập** → `reg` : bảng điều khiển yêu cầu thông tin xác thực NordLynx từ `api.nordvpn.com` và trích xuất `nordlynx_private_key` . Lưu `private_key` và `token` trong cài đặt `nord` . Thay thế – `setKey` : nhập **Khóa riêng** trực tiếp (không được để trống).
2. **Quốc gia** → `countries` tải danh sách quốc gia; **Thành phố** (hoặc **Tất cả thành phố**).
3. **Máy chủ** → `servers` tải máy chủ của quốc gia đã chọn (`countryId` được xác thực là số – bảo vệ chống injection). Bộ lọc: chỉ hiển thị các máy chủ có **Tải** > 7%. Nếu không có máy chủ: «*Không tìm thấy máy chủ nào cho quốc gia đã chọn*». Nếu máy chủ không có khóa công khai NordLynx: «*Máy chủ được chọn không báo cáo khóa công khai NordLynx.*»
4. Tạo/cập nhật kết nối đi: toast «*Đã thêm outbound NordVPN*» / «*Đã cập nhật outbound NordVPN*».

Ưu tiên IPv4 và userspace TUN

Các WireGuard-outbound được tạo bởi wizard WARP và NordVPN sử dụng

`domainStrategy: "ForceIPv4v6"` (ưu tiên IPv4 với fallback sang IPv6 trên các host chỉ có v6) thay vì `ForceIP` – điều này loại bỏ tình trạng «treo» handshake trên các host có IPv6 cấu hình không đầy đủ khi chọn bản ghi AAAA của endpoint Cloudflare. Ngoài ra, `userspace TUN` (`noKernelTun: true`) được bật thay vì `kernel TUN`: loại sau cần quyền và định tuyến fwmark và bị lỗi thâm lặng trên nhiều VPS, trong khi kiểm tra kết nối tích hợp của bảng điều khiển luôn kiểm tra qua `userspace TUN` – giờ đây lưu lượng thực và kiểm tra đi cùng một đường. Thay đổi chỉ áp dụng cho các outbound được thêm mới hoặc đặt lại; các mẫu đã lưu giữ nguyên cài đặt của chúng.

11.9. Reverse-proxy và TUN

Reverse (reverse-proxy)

Phần `reverse` của cấu hình Xray. Trong biểu mẫu outbound có công tắc loại **Reverse-proxy**. Các nút: **Tạo reverse-proxy**, **Chỉnh sửa reverse-proxy**.

Trường	Nhân	Mô tả
Loại	Loại	Bridge hoặc Portal – hai vai trò của reverse-proxy Xray.
Tên miền	Tên miền	Tên miền nhân dịch vụ cho cặp bridge[?]portal.
Tag / Kết nối	Tag / Kết nối	Các tag để liên kết bridge và portal.
Reverse Tag	Tag reverse-proxy	Gợi ý: «Tag kết nối đi cho reverse-proxy VLESS đơn giản. Để trống để tắt.» Placeholder: «tag outbound (trống = tắt)». Triển khai VLESS reverse đơn giản hóa.

Trong biểu mẫu outbound cũng có các trường luồng ngược: **Sniffing ngược**, **Workers**, **Dự phòng**, **Khoảng thời gian tải tối thiểu (ms)**, **Kích thước tải tối đa (byte)**.

TUN (tun)

Trường	Nhân	Mô tả	Mặc định
name	—	«Tên giao diện TUN.»	xray0
mtu	—	«Đơn vị truyền tối đa. Kích thước tối đa của các gói dữ liệu.»	1500
userLevel	Cấp người dùng	«Tất cả các kết nối được thiết lập qua luồng đến này sẽ sử dụng cấp người dùng này.»	0

11.10. Nhật ký và thống kê (Stats, metrics)

Nhật ký (log)

Gợi ý: «Nhật ký có thể làm chậm máy chủ. Chỉ bật những loại nhật ký bạn cần khi cần thiết!» Phần **log** của mẫu tham chiếu: `access: "none" , error: "" , loglevel: "warning" , dnsLog: false , maskAddress: ""`.

Trường	Nhân	JSON	Mô tả	Mặc định
logLevel	Cấp độ nhật ký	loglevel	«Cấp độ nhật ký cho nhật ký lỗi...» Các giá trị: <code>debug</code> , <code>info</code> , <code>warning</code> , <code>error</code> , <code>none</code> .	warning
accessLog	Nhật ký truy cập	access	«Đường dẫn đến tệp nhật ký truy cập. Giá trị đặc biệt «none» tắt nhật ký truy cập.»	none
errorLog	Nhật ký lỗi	error	«Đường dẫn đến tệp nhật ký lỗi. Giá trị đặc biệt «none» tắt nhật ký lỗi.»	"" (mặc định)
dnsLog	Nhật ký DNS	dnsLog	«Bật nhật ký truy vấn DNS»	false
maskAddress	Che giấu địa chỉ	maskAddress	«Khi kích hoạt, địa chỉ IP thực sẽ được thay thế bằng địa chỉ che giấu trong nhật ký.»	"" (tắt)

Thông kê (stats / policy)

Nhóm **Thông kê**. Bật các bộ đếm trong `policy.system` và `policy.levels`. Trong mẫu tham chiếu: `statsInboundUplink: true, statsInboundDownlink: true, statsOutboundUplink: false, statsOutboundDownlink: false`; cho cấp 0 – `statsUserUplink: true, statsUserDownlink: true`.

Trường	Nhân	Mô tả	Mặc định
<code>statsInboundUplink</code>	Thông kê uplink đến	«Bật thu thập thống kê cho lưu lượng đi của tất cả proxy đến.»	true
<code>statsInboundDownlink</code>	Thông kê downlink đến	«Bật thu thập thống kê cho lưu lượng đến của tất cả proxy đến.»	true
<code>statsOutboundUplink</code>	Thông kê uplink đi	«Bật thu thập thống kê cho lưu lượng đi của tất cả proxy đi.»	false
<code>statsOutboundDownlink</code>	Thông kê downlink đi	«Bật thu thập thống kê cho lưu lượng đến của tất cả proxy đi.»	false

Thông kê theo khách hàng và inbounds (uplink/downlink) – là nền tảng hiển thị lưu lượng trên dashboard và ở khách hàng; không nên tắt. Thông kê outbound mặc định bị tắt và chỉ cần nếu bạn theo dõi lưu lượng theo tag kết nối đi.

Metrics

Trong mẫu tham chiếu có phần `metrics` (`listen: "127.0.0.1:11111"` , `tag: "metrics_out"`) và API `metrics_out` tương ứng. Bảng điều khiển sử dụng listener này để thu thập metrics và ảnh chụp observatory: nó phân tích `metrics.listen` từ mẫu, truy vấn `/debug/vars` và tổng hợp lịch sử độ trễ theo tag. Nếu bạn thay đổi địa chỉ/cổng `metrics.listen` , bảng điều khiển sẽ truy cập địa chỉ mới; xóa phần `metrics` sẽ tắt thu thập biểu đồ observatory.

Kiểm tra outbound ở chế độ HTTP khởi động **một phiên bản Xray tạm thời riêng** với listener `metrics` riêng trên cổng ngẫu nhiên – đây không phải listener tương tự trong cấu hình chính.

11.11. Lưu, khởi động lại và các biến đổi tự động

Các nút

Nút	Hành động
Lưu	POST <code>/xray/update</code> : xác thực và lưu mẫu + <code>outboundTestUrl</code> .
Khởi động lại Xray	Tải lại dịch vụ với cấu hình đã lưu. Xác nhận: « <i>Khởi động lại xray?</i> » / « <i>Tải lại dịch vụ xray với cấu hình đã lưu.</i> »

Toast: thành công – «Xray đã được khởi động lại thành công», «Xray đã dừng thành công»; lỗi – «Đã xảy ra lỗi khi khởi động lại Xray.», «Đã xảy ra lỗi khi dừng Xray.» Cửa sổ **Đầu ra khởi động lại Xray** hiển thị đầu ra chẩn đoán của nhân.

Áp dụng nóng thay đổi (không cần khởi động lại hoàn toàn)

Các thay đổi đối với inbounds, outbounds và quy tắc định tuyến được áp dụng «trực tiếp»: khi nhấn **Lưu**, bảng điều khiển tính toán sự khác biệt giữa cấu hình cũ và mới, và chỉ áp dụng các phần đã thay đổi qua gRPC-API Xray (HandlerService/RoutingService), không khởi động lại tiến trình. Khởi động lại hoàn toàn chỉ được thực hiện tự động khi các phần không có API tải lại nóng thay đổi (log , dns , policy , observatory , v.v.). Do đó trên trang Xray không cần nút «Khởi động lại» riêng – **Lưu** tự áp dụng thay đổi. Khởi động lại nhân khi cần thiết vẫn được thực hiện tự động (xem thêm tự động tải lại khi cập nhật subscriptions và xoay vòng WARP).

Khôi phục mẫu mặc định

Endpoint GET /xray/getDefaultJsonConfig trả về mẫu tham chiếu (config.json , tích hợp vào binary). Có thể dùng để đặt lại cấu hình về mặc định nhà máy.

Các biến đổi tự động khi lưu

Khi lưu cài đặt Xray, bảng điều khiển thực hiện (theo thứ tự này):

1. **Gỡ bỏ wrapper** – gỡ bỏ các wrapper dạng { "xraySetting": <config>, "inboundTags": ..., "outboundTestUrl": ... } , nếu chúng vô tình nằm trong giá trị (nếu không, các lớp sẽ tích lũy với mỗi lần lưu). Gỡ tới 8 lớp.
2. **Kiểm tra cấu hình** – JSON được phân tích thành cấu trúc cấu hình Xray; khi có lỗi – từ chối với «xray template config invalid».
3. **Đảm bảo quy tắc thống kê** – quy tắc inboundTag: ["api"] → outboundTag: "api" bắt buộc được đẩy lên vị trí 0 trong routing.rules (hoặc được thêm nếu thiếu). Điều này đảm bảo rằng yêu cầu thống kê gRPC của bảng điều khiển không bị quy tắc catch-all phía trên chặn lại (nếu không, khách hàng có thể hiển thị offline với lưu lượng bằng không trong khi proxy đang hoạt động).

Do mục 3, đừng cố xóa hoặc di chuyển quy tắc api → api – bảng điều khiển sẽ trả nó về chỗ cũ ở lần lưu tiếp theo. Đây là cơ sở hạ tầng thống kê dịch vụ, không phải route người dùng.

11.12. Outbound từ subscription (với tự động cập nhật)

Kể từ phiên bản 3.3.0, bảng điều khiển có thể nhập outbound trực tiếp từ URL subscription – cùng định dạng mà các nhà cung cấp VPN cấp cho các ứng dụng khách. Subscription được đọc lại định kỳ ở nền, vì vậy tập hợp outbound trên máy chủ được giữ cập nhật mà không cần chỉnh sửa mẫu cấu hình thủ công.

Trong giao diện tiếng Anh, phần được gọi là «**Outbound Subscriptions**», mô tả: «Nhập outbound từ URL subscription từ xa (vmess/vless/trojan/ss/...). Các tag không thay đổi để sử dụng trong bộ cân bằng tải và quy tắc định tuyến. Cập nhật được thực hiện tự động.» Phần nằm trên trang Xray, phía trên bảng cấu hình outbound .

Cách hoạt động

Subscription được lưu riêng khỏi mẫu cấu hình Xray. Mẫu **không bao giờ bị ghi đè**: các `outbound` nhận được từ subscription được thêm vào cấu hình cuối cùng khi tạo config Xray.

Thêm subscription

Trong biểu mẫu «Thêm subscription» có các trường sau:

Trường	Khóa	Mặc định	Mục đích
URL subscription	<code>url</code>	– (bắt buộc)	Địa chỉ subscription. Placeholder: «https://... (danh sách liên kết trong base64)». Chỉ chấp nhận HTTP(S); địa chỉ được kiểm tra an toàn.
Ghi chú	<code>remark</code>	trống	Nhân tùy ý (placeholder «ví dụ: node HK»).
Tiền tố tag	<code>tagPrefix</code>	<code>subN-</code>	Tiền tố mà các tag của các <code>outbound</code> được nhập bắt đầu. Nếu để trống, bảng điều khiển sẽ tự chọn số nhỏ nhất còn trống dạng <code>sub1-</code> , <code>sub2-</code> , v.v.
Khoảng thời gian cập nhật	<code>updateInterval</code>	600 giây (10 phút)	Tần suất đọc lại subscription. Trong UI được đặt bằng giờ/phút.
Đã bật	<code>enabled</code>	có (<code>true</code>)	Chỉ các subscription đã bật mới được đưa vào config và tự động cập nhật.
Cho phép địa chỉ riêng tư	<code>allowPrivate</code>	không (<code>false</code>)	Cho phép URL trên localhost, LAN và IP riêng tư. Mặc định tắt để bảo vệ chống SSRF – chỉ bật cho nguồn cục bộ đáng tin cậy.
Trước outbound thủ công	<code>prepend</code>	không (<code>false</code>)	Nếu bật, các <code>outbound</code> của subscription này được đặt trước các <code>outbound</code> thủ công trong mẫu, và một trong số chúng có thể trở thành <code>outbound</code> mặc định. Nếu không, chúng được thêm sau .

Nút **«Xem trước»** (`POST /outbound-subs/parse`) cho phép tải và phân tích URL trước khi lưu và xem các `outbound` và tag nào sẽ nhận được; không có gì được ghi vào cơ sở dữ liệu. Nếu không nhận ra gì từ URL, hiển thị «Không tìm thấy outbound nào từ URL này.»

Thứ tự của nhiều subscription trong danh sách `outbound` chung được đặt theo ưu tiên (`priority`) và thay đổi bằng mũi tên lên/xuống (`POST /outbound-subs/:id/move`).

Các định dạng subscription được chấp nhận

Nội dung phản hồi từ URL được xử lý như sau:

- Nội dung trước tiên được thử dưới dạng **base64** (biến thể chuẩn và URL-safe, với tự động thêm padding và loại bỏ khoảng trắng/xuống dòng). Nếu là base64 – được giải mã; nếu không – lấy nguyên như vậy.
- Sau đó nội dung được chia thành các dòng. Mỗi dòng không rỗng không bắt đầu bằng `#` được phân tích như một liên kết. Các dòng không nhận ra (comment, giao thức không hỗ trợ) bị bỏ qua trong im lặng.
- Các schema liên kết được hỗ trợ: `vmess://`, `vless://`, `trojan://`, `ss://` (Shadowsocks), `hysteria2://` / `hy2://`, `wireguard://` / `wg://`.

Tức là phù hợp với subscription thông thường dạng «danh sách liên kết được mã hóa base64», như ở hầu hết các nhà cung cấp.

Các tag ổn định

Mỗi liên kết được tính một «danh tính» bền vững (URI cốt lõi không có fragment-ghi chú; cho vmess – JSON nội bộ không có trường `ps`). Ánh xạ «danh tính → tag» được lưu, và ở lần cập nhật tiếp theo, cùng một máy chủ nhận cùng tag, ngay cả khi ghi chú hoặc thông số phụ đã thay đổi. Điều này được làm đặc biệt để bộ cân bằng tải và quy tắc định tuyến tiếp tục hoạt động sau khi cập nhật:

- Tag chính xác trong bộ cân bằng tải/quy tắc sẽ tiếp tục trở đến cùng một máy chủ.
- Bộ chọn tiền tố/wildcard (ví dụ: `hk-*`) sẽ tự động bắt các máy chủ mới mà subscription trả về sau này – đây là cách được khuyến nghị để «đăng ký nhóm».
- Nếu máy chủ biến mất khỏi subscription, tag của nó đơn giản là biến mất khỏi mảng `outbound` cuối cùng; nếu bộ cân bằng tải có `fallbackTag`, Xray sẽ sử dụng nó.
- Nếu nhà cung cấp đã thay đổi UUID/host/thông tin xác thực của máy chủ, danh tính thay đổi – điều này được coi là `outbound` mới với tag mới.

Trong một lần tải xuống, các tag được loại bỏ trùng lặp bằng hậu tố `-N`. Các tag từ subscription giữ nguyên ký tự không phải ASCII (ví dụ: tiếng Cyrillic) và vẫn có thể đọc được: các chữ cái và số Unicode được giữ nguyên trong slug, còn dấu câu được thay bằng dấu gạch ngang – các tag từ tên tiếng Cyrillic không còn bị rút gọn thành chỉ các con số.

Cách tự động cập nhật hoạt động

- Tác vụ nền cập nhật subscription được chạy theo lịch **mỗi 5 phút**.
- Ở mỗi lần chạy, nó duyệt qua tất cả các subscription đã bật và chỉ cập nhật những cái đã hết khoảng thời gian của chúng: subscription được cập nhật nếu chưa bao giờ được cập nhật hoặc nếu đã trôi qua ít nhất `updateInterval` kể từ lần cập nhật cuối. Như vậy tác vụ kiểm tra subscription thường xuyên, nhưng mỗi subscription cụ thể được đọc lại không thường xuyên hơn `updateInterval` của nó (mặc định 10 phút). Điều này được phản ánh bằng gợi ý tương ứng trong UI.
- Cập nhật: URL được kiểm tra an toàn lại như URL công khai (địa chỉ riêng tư bị chặn nếu subscription không có `allowPrivate`), yêu cầu đi qua proxy-client của bảng điều khiển với header `User-Agent: 3x-ui-outbound-sub/1.0`. Chuỗi chuyển hướng được giới hạn 10 bước, và mỗi bước cũng được kiểm tra tính riêng tư (bảo vệ chống SSRF). HTTP 200 được mong đợi; nếu không – ghi nhận lỗi.
- Sau khi phân tích thành công, kết quả được lưu, thời gian cập nhật cuối được đặt, lỗi được xóa. Khi có lỗi, văn bản của nó hiển thị trong UI như «Lỗi cuối», còn các `outbound` đã nhận trước đó vẫn có hiệu lực.
- Nếu ít nhất một subscription thực sự được cập nhật, tác vụ đánh dấu Xray cần khởi động lại và gửi thông báo vô hiệu hóa UI để giao diện cập nhật các `outbound` mới. Khởi động lại thực tế Xray diễn ra ở chu kỳ 30 giây gần nhất của quản lý viên.

Cập nhật thủ công một subscription – nút **«Cập nhật ngay»** (`POST /outbound-subs/:id/refresh`); nó cũng đánh dấu Xray cần khởi động lại. Thêm, thay đổi, xóa subscription cũng kích hoạt cờ khởi động lại Xray (khi xóa, các `outbound` của nó sẽ biến khỏi config ở lần tải lại tiếp theo). UI gợi ý: «Sau khi thêm hoặc cập nhật, hãy khởi động lại Xray (hoặc đợi lần tự động tải lại tiếp theo) để các outbound trở nên hoạt động.»

Cách đưa vào cấu hình Xray

Ở mỗi lần tạo cấu hình Xray, các `outbound` subscription đang hoạt động được chia thành hai nhóm – `prepend` (cờ «Trước outbound thủ công») và các cái còn lại – và được ghép với mẫu: `[prepend subscription] + [outbound từ mẫu] + [subscription còn lại]`. Bên trong mỗi nhóm, subscription được sắp xếp theo ưu tiên. Các `outbound` thủ công từ mẫu không bị ảnh hưởng; nếu mảng `outbound` của mẫu vì lý do nào đó không phân tích được, các `outbound` subscription không được trộn vào (để không mất các `outbound` thủ công).

Các `outbound` được nhập cũng được hiển thị trong bảng `outbound` chính trong một khối riêng «**Từ outbound subscriptions (chỉ đọc)**» – không thể chỉnh sửa chúng ở đó, quản lý chỉ qua phần «Outbound Subscriptions».

11.13. Xoay vòng IP trong WARP

Trong 3X-UI có thể thiết lập WARP-outbound – kết nối WireGuard đi đến Cloudflare WARP (tag `warp` trong config Xray). Bảng điều khiển tự đăng ký tài khoản thiết bị trên máy chủ Cloudflare, nhận khóa WireGuard và địa chỉ, và đưa chúng vào outbound với tag `warp`. Qua outbound này, lưu lượng ra Internet dưới địa chỉ IP của Cloudflare WARP. Tính năng mới trong phiên bản 3.3.0 – khả năng thay đổi IP đi này thủ công hoặc theo lịch, mà không cần tạo lại tài khoản WARP thủ công.

Quản lý nằm trong phần **Xray** trong thẻ WARP (sau khi nhấn «Tạo tài khoản WARP» và nhận config; trước đó các hành động không khả dụng – bảng điều khiển sẽ gợi ý «Lấy WARP config trước»).

Điều gì xảy ra khi thay đổi IP

Nút «**Thay đổi IP**» bắt đầu thay đổi IP. Logic:

1. Một cặp khóa WireGuard mới được tạo.
2. Với khóa mới, thiết bị WARP được đăng ký lại trên máy chủ Cloudflare (mới `device_id`, `access_token`, địa chỉ và dữ liệu peer).
3. Dữ liệu mới được ghi vào WARP-outbound của config Xray: cập nhật `secretKey`, `address` (v4 /32 và v6 /128), `reserved` (từ `client_id`), cũng như `publicKey` và `endpoint` ở peer.
4. Nếu trước đó đã đặt khóa bản quyền WARP+ (độ dài ít nhất 26 ký tự), nó sẽ tự động được cài đặt lại trên tài khoản mới. Khi thất bại, đây chỉ là cảnh báo trong nhật ký – thay đổi IP không bị hủy bỏ.
5. Sau khi thay đổi thành công, Xray được đánh dấu là cần khởi động lại để outbound mới có hiệu lực.

Khi thành công, giao diện hiển thị «Địa chỉ IP WARP đã được thay đổi thành công!».

Xoay vòng tự động theo lịch

Trong thẻ WARP có công tắc «**Tự động cập nhật địa chỉ IP**» và trường «**Khoảng thời gian (ngày)**». Gợi ý: «0 – tắt. Tự động thay đổi địa chỉ IP.»

Thông số	Giá trị
Cài đặt trong DB	<code>warpUpdateInterval</code> (số nguyên, ≥ 0)

Thông số	Giá trị
Giá trị mặc định	0 (tự động xoay vòng tắt)
Đơn vị đo	ngày
0	tắt tự động thay đổi
> 0	thay đổi IP mỗi N ngày

Ghi khoảng thời gian lưu `warpUpdateInterval`, và nếu giá trị lớn hơn 0, đặt lại «thời gian cập nhật cuối» về thời điểm hiện tại – nếu không bộ lập lịch sẽ thay đổi IP ngay ở lần tick gần nhất.

Lịch được thực thi bởi tác vụ nền chạy mỗi giờ một lần – tức là bảng điều khiển một giờ một lần kiểm tra xem đã đến lúc xoay vòng chưa. Thuật toán kiểm tra:

- nếu khoảng thời gian ≤ 0 – không làm gì;
- nếu «thời gian cập nhật cuối» bằng 0 (ví dụ: khoảng thời gian được đặt bằng cách chỉnh sửa DB trực tiếp) – đây là lần chạy đầu tiên: tác vụ chỉ ghi lại mốc thời gian cơ sở và KHÔNG thay đổi IP ngay lập tức;
- nếu đã trôi qua ít nhất khoảng thời gian $\times 24 \times 3600$ giây kể từ lần cập nhật cuối – thực hiện cùng thay đổi IP, cập nhật mốc thời gian và lên lịch khởi động lại Xray.

Chi tiết quan trọng: thay đổi thủ công bằng nút «Thay đổi IP» cũng đặt lại mốc thời gian cập nhật cuối. Do đó sau khi xoay vòng thủ công, đếm ngược khoảng thời gian tự động bắt đầu lại và việc thay đổi theo lịch sẽ không xảy ra ngay sau đó.

«Qua proxy bảng điều khiển»

Đã thay đổi trong 3.3.1. Cài đặt riêng biệt «Proxy mạng bảng điều khiển» (`panelProxy`) đã bị xóa. Lưu lượng đi của chính bảng điều khiển (bao gồm cả yêu cầu đến WARP API) hiện được định tuyến qua **outbound cho lưu lượng bảng điều khiển** đã chọn – Xray-outbound hoặc bộ cân bằng tải (xem phần 13). Mô tả bên dưới áp dụng cho các phiên bản trước 3.3.1.

Tất cả các yêu cầu đến Cloudflare WARP API (đăng ký, nhận config, đặt bản quyền, thay đổi IP) đều không đi trực tiếp mà qua HTTP-client của bảng điều khiển với timeout 15 giây. Client này tôn trọng cài đặt **«Proxy mạng bảng điều khiển»** (`panelProxy`) từ cài đặt bảng điều khiển.

Từ mô tả cài đặt: proxy định tuyến các yêu cầu đi của chính bảng điều khiển (cập nhật geo-database, kiểm tra phiên bản Xray/bảng điều khiển, Telegram, và bây giờ cả các lệnh gọi đến WARP) – để vượt qua bộ lọc phía máy chủ. Chấp nhận các địa chỉ dạng `socks5://` hoặc `http(s)://`, ví dụ inbound SOCKS cục bộ của chính Xray. Nếu trường trống hoặc proxy được đặt không đúng – sử dụng kết nối trực tiếp (hành vi không bị phá vỡ).

Lợi ích cho WARP: nếu máy chủ không thể trực tiếp tiếp cận `api.cloudflareclient.com`, việc đăng ký và xoay vòng trước đây sẽ thất bại. Bây giờ, bằng cách chỉ định proxy hoạt động trong `panelProxy` (bao gồm cả inbound Xray riêng), bạn có thể đảm bảo khả năng truy cập WARP API và hoạt động của cả nút thủ công lẫn xoay vòng theo lịch.

Khi nào điều này hữu ích

- Thay đổi thường xuyên IP đi cho outbound qua WARP – giảm nguy cơ bị chặn và theo dõi theo một địa chỉ.
 - «Làm mới» IP thủ công nếu địa chỉ Cloudflare hiện tại nằm trong danh sách đen hoặc hoạt động chậm.
 - Các máy chủ không có quyền truy cập trực tiếp đến Cloudflare WARP API: định tuyến yêu cầu qua `panelProxy` làm cho việc đăng ký và xoay vòng hoạt động được.
-

12. Các nút (đa bảng điều khiển, master/slave)

Mục **Các nút** biến cài đặt 3X-UI thông thường thành **bảng điều khiển trung tâm (master)**, có khả năng giám sát và quản lý từ xa các bảng điều khiển 3X-UI khác (bảng con). Mỗi nút là một cài đặt 3X-UI riêng biệt trên máy chủ của nó; master kết nối với nút thông qua HTTP API riêng của nút đó, truy vấn trạng thái và đồng bộ các inbound và client được gán cho nút đó. Đây chính là tính năng **đa bảng điều khiển**: thay vì đăng nhập vào từng bảng riêng lẻ, bạn có thể xem tất cả các máy chủ trong một danh sách và quản lý chúng tập trung.

Nguyên tắc quan trọng: **nút không phải là agent mà là một bảng điều khiển 3X-UI đầy đủ chức năng**. Master không "cài đặt" gì lên nó — chỉ kết nối tới API của nút bằng token. Xóa một nút khỏi danh sách chỉ dừng quá trình giám sát; bảng điều khiển từ xa không bị ảnh hưởng (gợi ý: «This will stop monitoring the node. The remote panel will not be affected»).

12.1. Tổng quan ở đầu danh sách

Phía trên bảng danh sách nút hiển thị các bộ đếm tổng hợp:

Trường	Mô tả
Tổng số nút	Tổng số nút trong danh sách.
Trực tuyến	Số nút có trạng thái online .
Ngoại tuyến	Số nút có trạng thái offline .
Độ trễ trung bình	Độ trễ (ping) trung bình đến các nút, tính bằng mili giây.

12.2. Thêm và chỉnh sửa nút

Các nút **Thêm nút** và **Chỉnh sửa nút** mở biểu mẫu với các trường của nút.

Các trường bắt buộc (gợi ý: «Name, address, port and API token are required») là **Tên**, **Địa chỉ**, **Cổng** và **API Token**.

Khi nhấn «Lưu» (cả khi thêm mới lẫn khi chỉnh sửa), bảng điều khiển **trước tiên kiểm tra khả năng tiếp cận của nút** với thời gian chờ 6 giây. Nếu nút không phản hồi, bản ghi sẽ không được lưu và hiển thị lỗi. Tức là không thể thêm một nút rõ ràng không thể tiếp cận được.

Các trường trong biểu mẫu

Trường	Mặc định	Giá trị hợp lệ	Mô tả
Tên	— (bắt buộc)	chuỗi không rỗng, duy nhất	Tên nội bộ của nút. Cột tên có ràng buộc duy nhất — không thể tạo hai nút có cùng tên. Placeholder gợi ý: <code>napr.de-frankfurt-1</code> . Khi lưu, khoảng trắng đầu và cuối sẽ bị cắt bỏ.
Ghi chú	rỗng	bất kỳ chuỗi nào	Ghi chú/mô tả tùy chọn về nút. Không ảnh hưởng đến hoạt động.

Trường	Mặc định	Giá trị hợp lệ	Mô tả
Giao thức	<code>https</code>	<code>http / https</code>	Giao thức kết nối đến bảng điều khiển từ xa. Nếu để trống hoặc giá trị không hợp lệ, quá trình chuẩn hóa sẽ đặt thành <code>https</code> . Nếu nút phản hồi qua HTTP thông thường nhưng giao thức được đặt là <code>https</code> , bảng điều khiển sẽ hiển thị gợi ý rõ ràng: «the server speaks HTTP, not HTTPS; set the node scheme to http».
Địa chỉ	– (bắt buộc)	tên máy chủ hoặc IP	Địa chỉ của bảng điều khiển từ xa. Placeholder: <code>panel.example.com</code> hoặc <code>1.2.3.4</code> . Địa chỉ được chuẩn hóa; theo mặc định, các địa chỉ riêng tư/cục bộ bị cấm để bảo vệ khỏi SSRF – xem «Cho phép địa chỉ riêng tư».
Cổng	– (bắt buộc)	số nguyên 1–65535	Cổng web của bảng điều khiển nút từ xa. Các giá trị ngoài phạm vi sẽ bị từ chối («node port must be 1-65535»).
Đường dẫn cơ sở	<code>/</code>	chuỗi đường dẫn	Đường dẫn cơ sở (web base path) của bảng điều khiển từ xa nếu được cấu hình. Được chuẩn hóa: đảm bảo bắt đầu và kết thúc bằng <code>/</code> (giá trị rỗng → <code>/</code>). Bảng điều khiển nối thêm <code>panel/api/server/status</code> vào đường dẫn này khi truy vấn.
API Token	– (bắt buộc)	token của bảng điều khiển từ xa	Bearer token để truy cập API của nút. Được truyền trong header <code>Authorization: Bearer <token></code> . Placeholder: «Token từ trang Cài đặt của bảng điều khiển từ xa». Gợi ý: «The remote panel displays its API token in Settings → API Token». Nghĩa là token phải được tạo trên chính nút đó (Cài đặt → API Token), sau đó dán vào đây.
Đã bật	<code>true</code>	có/không	Bật giám sát và đồng bộ hóa nút. Các nút bị tắt không được truy vấn bởi các tác vụ nền (heartbeat và traffic-sync bỏ qua chúng) và không tham gia vào cập nhật bảng điều khiển hàng loạt.
Cho phép địa chỉ riêng tư	<code>false</code>	có/không	Tắt bảo vệ SSRF và cho phép kết nối đến nút qua địa chỉ riêng tư/cục bộ. Gợi ý: «Enable only for nodes in a private network or VPN». Chỉ bật khi nút thực sự nằm trong mạng riêng tư hoặc có thể truy cập qua VPN.

Lấy và tái tạo token ở phía nút

Token được lấy từ bảng điều khiển từ xa trong mục **Cài đặt** → **API Token**. Tại đây cũng có thể tái cấp phát token: nút **Tạo lại token** với cảnh báo: «Regenerating will invalidate the current token. Any central panel using it will lose access until updated. Continue?». Sau khi tái tạo, token cũ trong bảng điều khiển master sẽ ngừng hoạt động – cần cập nhật trong biểu mẫu nút.

Kết nối đi (Connection outbound)

Trường **Connection outbound** (Kết nối đi, `outboundTag`) xác định cách lưu lượng truy cập từ master đến API của nút này rời khỏi máy chủ. Nếu chọn tag Xray-outbound, các yêu cầu của bảng điều khiển đến nút sẽ không đi trực tiếp mà qua outbound được chỉ định; bảng điều khiển tự thêm một bridge-inbound trên loopback vào cấu hình đang chạy và áp dụng thay đổi ngay lập tức mà không cần khởi động lại. Gợi ý: «Route this node's panel API traffic through the selected Xray outbound. A loopback bridge inbound is added to the running config automatically and applied live. Leave empty for a direct connection».

Bộ chọn hoạt động như bộ chọn outbound trong bảng điều khiển: các tag được nhóm thành **Outbounds** (outbound thông thường) và **Balancers** (bộ cân bằng tải), các outbound blackhole bị ẩn khỏi danh sách. Giá trị rỗng (placeholder «Direct connection») = kết nối trực tiếp đến nút.

Nhập inbound (chọn inbound để đồng bộ hóa)

Trong biểu mẫu nút có cài đặt **Nhập inbound** (inboundSyncMode) với hai chế độ: **Tất cả inbound** (all , mặc định) và **Đã chọn** (selected). Theo mặc định, master đồng bộ đến nút tất cả các inbound có nút đó được chọn; các nút hiện có tiếp tục hoạt động ở chế độ «Tất cả inbound».

Ở chế độ **Đã chọn**, bên dưới trường xuất hiện bộ chọn đa giá trị cho tag inbound. Nhân **Tải inbound** – master sẽ sử dụng các thông số kết nối đã nhập (chưa lưu) để yêu cầu danh sách inbound của nút (endpoint `POST /panel/api/nodes/inbounds`) và hiển thị các tag của chúng; đánh dấu các tag cần thiết. Bảng điều khiển sẽ chỉ đồng bộ và triển khai lên nút các tag đã đánh dấu, còn các inbound khác tồn tại trực tiếp trên nút sẽ không bị ảnh hưởng – master không xóa và không quản lý chúng.

Ví dụ: yêu cầu danh sách inbound của nút để nhập có chọn lọc. Thân yêu cầu chứa các thông số kết nối chưa lưu; phản hồi trả về các tag của inbound có sẵn trên nút:

```
POST /panel/api/nodes/inbounds
Content-Type: application/json

{ "name": "de-fra-1", "scheme": "https", "address": "node1.example.com",
  "port": 2053, "basePath": "/", "apiToken": "abcdef..." }
```

12.3. Xác minh TLS (cho nút https)

Nhóm trường xác định cách master kiểm tra chứng chỉ HTTPS của nút. Các cài đặt này **chỉ áp dụng cho giao thức https**; đối với nút `http`, chúng bị bỏ qua.

Xác minh TLS – danh sách thả xuống, gợi ý: «How the panel verifies the node's HTTPS certificate. Pin or Skip – for self-signed certificates (https nodes only)».

Chế độ	Giá trị	Mặc định	Mô tả
Xác minh (CA tiêu chuẩn)	verify	có (mặc định)	Xác minh chuỗi chứng chỉ thông thường bằng CA đáng tin cậy. Phù hợp với các nút có chứng chỉ công khai/Let's Encrypt. Cũng được sử dụng cho tất cả nút <code>http</code> .
Ghim chứng chỉ (SHA-256)	pin	–	Chuỗi CA không được xác minh, nhưng SHA-256 của chứng chỉ lá của nút được so sánh với dấu vân tay đã lưu (so sánh constant-time). Bảo toàn bảo vệ chống MITM cho chứng chỉ tự ký . Yêu cầu điền trường dấu vân tay.
Bỏ qua xác minh	skip	–	Xác minh chứng chỉ bị tắt hoàn toàn. Cảnh báo: «Skipping verification removes protection against man-in-the-middle attacks – the API token may be intercepted. Pinning the certificate is better».

Ngoài ba chế độ trên, trong phiên bản 3.4.0 có thêm chế độ thứ tư – **Mutual TLS (client certificate)** (mtls), cũng chỉ khả dụng với giao thức `https`.

Chế độ	Giá trị	Mặc định	Mô tả
Mutual TLS (chứng chỉ client)	<code>mtls</code>	–	Ngoài việc xác minh chứng chỉ của nút, master còn xác thực bản thân với nút bằng chứng chỉ client do CA của chính nút phát hành. Đối với nút ở chế độ này, API token trở thành tùy chọn – nút nhận diện master qua chứng chỉ. Khi chọn chế độ này, gợi ý hiển thị: «This node authenticates the panel with a client certificate. Copy this panel's CA from the Node mTLS section onto the node, set its Trusted parent CA, then restart it».

Để bật TLS lẫn nhau cho nút: ở phía nút, đặt chế độ **Mutual TLS**, sao chép CA của bảng điều khiển quản lý từ mục **Node mTLS** (xem bên dưới), thiết lập CA đó trên nút là **CA gốc đáng tin cậy** và khởi động lại nút.

Nếu chọn bất kỳ giá trị nào khác ngoài `skip`, `pin` hoặc `mtls`, quá trình chuẩn hóa sẽ bắt buộc đặt thành `verify`.

Ghim chứng chỉ

Khi chọn **Ghim chứng chỉ**, các mục sau xuất hiện:

- **SHA-256 của chứng chỉ đã ghim** – trường nhập liệu. Chấp nhận dấu vân tay ở định dạng **base64** (định dạng `pinnedPeerCertSha256` từ Xray) hoặc **hex** có dấu hai chấm hoặc không (kiểu `openssl -fingerprint`). Gợi ý: «SHA-256 of the node's certificate in base64 or hex. Click "Get" to fetch it from the node now». Placeholder: «SHA-256 in base64 or hex». Khi chọn `pin`, dấu vân tay trống hoặc không hợp lệ sẽ gây ra lỗi xác thực khi lưu.

Ví dụ: cùng một dấu vân tay ở hai định dạng. Trường chấp nhận bất kỳ định dạng nào – cả hai đều chỉ cùng một chứng chỉ:

```
# base64 (định dạng pinnedPeerCertSha256 từ Xray)
607TNg3l2k0pq8R1sT2uV3wX4yZ5a6B7c8D9e0F1g2=

# hex với dấu hai chấm (kiểu openssl x509 -fingerprint -sha256)
E8:E2:D3:60:DE:5D:9A:4D:29:AB:CF:11:B2:7C:34:...
```

Nếu dấu vân tay chưa biết, hãy nhấn **Lấy** – master sẽ tự lấy từ nút qua HTTPS và điền vào trường.

- Nút **Lấy** – kết nối đến nút qua HTTPS mà không xác minh chứng chỉ và đọc SHA-256 của chứng chỉ là hiện tại (endpoint `POST /certFingerprint`), sau đó điền vào trường. Sau khi thành công – «Current node certificate retrieved»; nếu thất bại – «Failed to retrieve certificate». Chỉ khả dụng cho nút https.

Node mTLS (xác thực TLS lẫn nhau giữa các bảng điều khiển)

Trên trang **Các nút** có mục riêng **Node mTLS** – cài đặt xác thực TLS lẫn nhau, bổ sung yếu tố thứ hai (chứng chỉ client) ngoài API token cho các cuộc gọi «bảng điều khiển → nút». TLS lẫn nhau là tùy chọn; nếu các trường của mục để trống, các nút hoạt động theo sơ đồ cũ – **chỉ với API token** (gợi ý: «Mutual TLS adds a

client-certificate factor on top of the API token for node-to-node calls. It is opt-in: leave it empty to keep token-only auth»). Mục có hai thao tác:

- **Sao chép CA của bảng điều khiển này** (`POST /panel/api/nodes/mtls/ca`) – sao chép chứng chỉ gốc (CA) của bảng điều khiển này vào clipboard. CA này cần được chuyển đến các nút được quản lý để chúng tin tưởng chứng chỉ client của bảng điều khiển; sau đó trên chính các nút đó phải đặt chế độ xác minh TLS **Mutual TLS** (gợi ý: «Hand this CA to the nodes this panel manages, then set their TLS verification to Mutual TLS»). Sau khi sao chép – «CA certificate copied to clipboard».
- **CA gốc đáng tin cậy** (`Trusted parent CA` , `POST /panel/api/nodes/mtls/trustCA`) – trường được sử dụng khi chính bảng điều khiển này đóng vai trò nút cho bảng điều khiển quản lý cấp trên. Dán CA của bảng điều khiển quản lý vào đây để yêu cầu chứng chỉ client từ bảng đó, rồi nhấn **Save trust CA**. Thay đổi này yêu cầu **khởi động lại bảng điều khiển** (gợi ý: «When this panel is itself a node, paste the managing panel's CA here to require its client certificate. Restart the panel to apply»).

12.4. Thông tin hiển thị cho từng nút

Các cột trong bảng và trường trong thẻ nút (trạng thái quan sát được, được cập nhật sau mỗi lần truy vấn heartbeat):

Trường	Mô tả
Trạng thái	<code>online</code> / <code>offline</code> / <code>unknown</code> – xem bên dưới.
CPU	Mức tải bộ xử lý của máy chủ từ xa theo phần trăm.
Bộ nhớ	Mức sử dụng RAM theo phần trăm (tính theo <code>current/total*100</code>).
Thời gian hoạt động	Thời gian hoạt động liên tục của máy chủ (tính bằng giây).
Độ trễ	Thời gian phản hồi của nút trong lần truy vấn gần nhất (ms).
Ping cuối	Thời điểm heartbeat thành công gần nhất (unix-giây; <code>0</code> = «chưa bao giờ»; giá trị gần đây hiển thị là «vừa xong»).
Phiên bản Xray	Phiên bản Xray-core đang chạy trên nút.
Phiên bản bảng điều khiển	Phiên bản 3X-UI trên nút – được so sánh với phiên bản hiện tại để hiển thị chỉ báo cập nhật.
(inbound)	Số inbound được đặt vật lý trên nút này.
(client)	Số client trên các inbound của nút.
(trực tuyến)	Số client của nút hiện đang trực tuyến.
(đã hết)	Số client của nút đã hết hạn hoặc vượt giới hạn lưu lượng . Client bị tắt thủ công không được tính vào bộ đếm này.
(tốc độ)	Tốc độ truyền dữ liệu hiện tại (trực tiếp) trên các inbound được đặt trên nút.

Bộ đếm inbound/client/trực tuyến được gắn với nút theo GUID ổn định (`panelGuid`) của nó, không phải theo id cục bộ – để client trên nút con được tính đúng cho nút con đó, không phải cho nút trung gian mà nó được đồng bộ qua.

Đối với các inbound được đặt trên nút, trang hiển thị client trực tuyến, các bộ đếm và **tốc độ truyền dữ liệu hiện tại**. Việc gắn kết theo GUID ổn định cũng phân biệt chính xác các nút «nhân bản» có cùng `panelGuid`.

Trạng thái nút

Trạng thái	Khi nào được thiết lập
online	Nút đã phản hồi <code>success=true</code> cho truy vấn <code>panel/api/server/status</code> .
offline	Nút không phản hồi, trả về lỗi HTTP, <code>success=false</code> hoặc phản hồi không nhận dạng được.
unknown	Giá trị ban đầu, khi nút chưa được truy vấn lần nào.

Khi truy vấn thất bại, văn bản lỗi được lưu lại và hiển thị dưới dạng thông báo rõ ràng, giúp chẩn đoán nguyên nhân «offline».

12.5. Thao tác với nút

- **Kiểm tra kết nối** (`POST /test`) – trong biểu mẫu nút, kiểm tra kết nối theo các thông số đã nhập (chưa lưu) với thời gian chờ 6 giây. Kết quả: «Kết nối tốt ({ms} ms)» hoặc «Không thể kết nối». Tiện lợi để gỡ lỗi địa chỉ/cổng/token/TLS trước khi lưu.
- **Kiểm tra ngay** (nút «Kiểm tra ngay», `POST /probe/:id`) – truy vấn đột xuất nút đã lưu; ngay lập tức cập nhật trạng thái và số liệu (CPU/bộ nhớ/thời gian hoạt động/độ trễ/phiên bản) và ghi lại heartbeat. Nếu thất bại – «Kiểm tra thất bại».

Ví dụ: kiểm tra và truy vấn nút qua API của master. «Kiểm tra kết nối» thử nghiệm các thông số chưa lưu từ biểu mẫu:

```
POST /panel/api/nodes/test
Content-Type: application/json

{ "scheme": "https", "address": "de-frankfurt-1.example.com", "port": 2053,
  "basePath": "/", "apiToken": "eyJhbGciOi...", "tlsMode": "verify" }
```

Truy vấn đột xuất nút đã lưu với id 7:

```
POST /panel/api/nodes/probe/7
```

- **Cập nhật bảng điều khiển** (`POST /updatePanel` với thân `{ids: [...]}`) – khởi chạy trình tự cập nhật tích hợp trên nút: nút tải xuống bản phát hành 3X-UI mới nhất và khởi động lại với phiên bản đó. Nút **Cập nhật đã chọn ({count})** thực hiện thao tác này cho nhiều nút được đánh dấu cùng lúc. Bên cạnh mỗi nút hiển thị chỉ báo: **Có bản cập nhật** hoặc **Đã cập nhật**, dựa trên việc so sánh phiên bản bảng điều khiển của nút với phiên bản mới nhất.

Ví dụ: cập nhật nhiều nút bằng một yêu cầu. Thân yêu cầu chứa id của các nút được đánh dấu; chỉ các nút đã bật và `online` được cập nhật, các nút còn lại được trả về là đã bỏ qua.

```
POST /panel/api/nodes/updatePanel
Content-Type: application/json

{ "ids": [3, 7, 12] }
```

Phản hồi dạng «Đã khởi động cập nhật trên 2 nút, 1 thất bại»: ví dụ nút 12 có thể offline và do đó bị bỏ qua.

- Xác nhận: «Cập nhật {count} nút lên phiên bản mới nhất? Mỗi nút được chọn sẽ tải bản phát hành mới nhất và khởi động lại. Chỉ các nút đã bật và đang trực tuyến mới được cập nhật».
- **Chỉ các nút đã bật và có trạng thái online mới được cập nhật.** Nút bị tắt trong kết quả được đánh dấu «node is disabled», offline – «node is offline». Kết quả: «Đã khởi động cập nhật trên {ok} nút, {failed} thất bại». Nếu không có nút phù hợp nào được chọn – «Vui lòng chọn ít nhất một nút đã bật và đang trực tuyến».
- **Set Cert from Panel** (phụ trợ, GET /webCert/:id) – khi tạo inbound trên nút, cho phép điền đường dẫn đến chứng chỉ web-TLS của chính nút đó (không phải bảng điều khiển trung tâm), để các tệp tồn tại đúng trên nút. Yêu cầu nút đã bật và có thể truy cập.
- **Xóa nút** (POST /del/:id) – xác nhận: «Xóa nút "{name}"? Thao tác này sẽ dừng giám sát nút. Bảng điều khiển từ xa sẽ không bị ảnh hưởng». Xóa bản ghi nút và thống kê lưu lượng tích lũy của nó; bảng điều khiển từ xa tiếp tục hoạt động bình thường. **Chỉ có thể xóa nút sau khi tất cả inbound đã được gỡ khỏi nút đó.** Nếu vẫn còn ít nhất một inbound được gắn với nút (qua node_id), bảng điều khiển sẽ từ chối xóa với lỗi dạng «cannot delete node: N inbound(s) still attached to it; detach or delete them first» – hãy gỡ hoặc xóa các inbound đó trước, sau đó mới xóa nút. Điều này ngăn chặn các inbound «mồ côi» có tham chiếu treo đến nút đã xóa.

12.6. Lịch sử số liệu

Nút/biểu đồ lịch sử gọi đến GET /history/:id/:metric/:bucket. Các số liệu khả dụng: **cpu** và **mem** – chúng được tích lũy sau mỗi lần heartbeat thành công. Kích thước khoảng tổng hợp (bucket, tính bằng giây) bị giới hạn bởi danh sách trắng:

Ví dụ: yêu cầu lịch sử. Biểu đồ tải CPU của nút 7 với tổng hợp theo khoảng 60 giây (trả về tối đa 60 điểm):

```
GET /panel/api/nodes/history/7/cpu/60
```

Đối với bộ nhớ và chế độ «thời gian thực» (2 giây) – tương ứng .../7/mem/60 và .../7/cpu/2. Các giá trị ngoài danh sách trắng bị từ chối («invalid metric» / «invalid bucket»).

Bucket (giây)	Mục đích
2	Chế độ thời gian thực
30	Khoảng 30 giây
60	Khoảng 1 phút
120	Khoảng 2 phút
180	Khoảng 3 phút
300	Khoảng 5 phút

Trả về tối đa 60 điểm. Số liệu hoặc bucket không hợp lệ bị từ chối («invalid metric» / «invalid bucket»).

12.7. Cách đồng bộ hóa inbound và client

Inbound «thuộc về» nút thông qua trường `node_id` (trong trình chỉnh sửa inbound, có thể chọn nút):

Ví dụ: token trong biểu mẫu nút. Token được lấy từ bảng điều khiển con (Cài đặt → API Token) và dán vào trường **API Token** của master. Trong mỗi lần truy vấn, master gửi token trong header:

```
GET https://panel.example.com:2053/panel/api/server/status
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.abc123...
```

Nếu bảng điều khiển con có **đường dẫn cơ sở** (web base path), ví dụ `/secret/`, master tự thêm nó vào trước `panel/api/server/status` → `https://panel.example.com:2053/secret/panel/api/server/status`.

1. **Triển khai cấu hình (reconcile).** Bất kỳ thay đổi nào đối với inbound/client được gắn với nút đều đánh dấu nút là «bắt». Tác vụ nền cho mỗi nút đã bắt ở **trạng thái online** khi có thay đổi sẽ triển khai các inbound của nút (theo `node_id`) lên nút đó, sau đó xóa cờ «bắt». Nút bị tắt, offline hoặc «bắt» được coi là «đang chờ» – quá trình triển khai lên nút đó bị hoãn đến khi kết nối được khôi phục.

2. **Thu thập lưu lượng.** Cùng tác vụ đó yêu cầu ảnh chụp lưu lượng từ nút và hợp nhất vào thống kê cục bộ. Dựa trên lưu lượng đã hợp nhất, hệ thống kiểm tra việc vượt giới hạn/hết hạn và tắt client khi cần; bộ đếm «đã hết» theo nút phản ánh chính xác điều này. Nếu nút không thể truy cập, danh sách client trực tuyến của nó sẽ bị xóa.

Đối với client được gắn với nhiều bảng điều khiển cùng lúc, master trong cùng tác vụ đó còn gửi đến các nút **tổng lưu lượng của client trên tất cả bảng điều khiển** (trong một bảng riêng biệt trên nút, khóa là GUID của master; bị ghi đè sau mỗi lần gửi, do đó việc đặt lại ở phía master cũng được lan truyền). Trên nút, lưu lượng của client hiển thị giá trị lớn hơn trong hai giá trị – cục bộ hoặc được gửi đến – và khi vượt tổng hạn ngạch, client bị tắt **cục bộ trên chính nút đó** (thông qua cơ chế khởi động lại Xray giống như khi tự động tắt, ngắt các kết nối đã thiết lập). Điều này loại bỏ tình huống nút chỉ thấy phần lưu lượng của mình, tính thiếu và tiếp tục phục vụ client đã vượt tổng giới hạn. Khi đặt lại lưu lượng, tự động gia hạn hoặc xóa client, các bộ đếm đã gửi sẽ bị xóa.

3. **Heartbeat.** Một tác vụ nền riêng biệt định kỳ truy vấn tất cả các nút **đã bắt** (với giới hạn song song) thông qua `panel/api/server/status`, cập nhật trạng thái/số liệu/phần bản và, khi có web-client, gửi cây nút đã cập nhật qua WebSocket.

12.8. Chuỗi nút (nút con / nút trung gian)

Cấu trúc liên kết có thể không phẳng: một nút có thể tự là master cho các nút của nó. Các bảng điều khiển cấp dưới như vậy được hiển thị với bạn dưới dạng **Nút con** – đây là **các chiều chỉ đọc** nhận được từ nút trực tiếp.

- Gợi ý: «Read-only: a subordinate node accessible through {parent}. Manage it from {parent}'s own panel». Nghĩa là nút con không thể chỉnh sửa, xóa hoặc cập nhật ở đây – tất cả thao tác với nó được thực hiện từ bảng điều khiển của cha trực tiếp.

- Danh tính của nút con được xác định bởi GUID của nó; nhờ đó, client trực tuyến và inbound được tính đúng cho nút vật lý đang lưu trữ chúng, ngay cả trong chuỗi `Node1 → Node2 → Node3` (master «đi qua» một cấp sâu hơn qua từng nút trực tiếp).
- Nếu nút trực tiếp không thể truy cập, bộ nhớ cache nút con của nó sẽ bị xóa và các nút con biến mất khỏi cây cho đến khi kết nối được khôi phục.

12.9. Các nút: tính năng mới trong 3.3.0

Trong phiên bản 3.3.0, mục **Các nút** nhận được ba cải tiến đáng chú ý: phân bổ lưu lượng và client trực tuyến chính xác trong cấu trúc liên kết đa cấp (multi-hop), đồng bộ hóa client-IP giữa các nút và chỉ báo trạng thái riêng biệt cho trường hợp bảng điều khiển nút đang hoạt động nhưng lỗi Xray trên đó bị sập.

1. Multi-hop: phân bổ lưu lượng chính xác trong chuỗi nút con

Trước đây, các bộ đếm (số inbound, client trực tuyến, đã hết) được tính ở cấp nút «trực tiếp». Nếu bạn có chuỗi `Master → Node1 → Node2 → Node3`, mọi thứ vật lý nằm trên `Node2 / Node3` bị gán nhầm cho `Node1` qua đó nó đến được master. Trong 3.3.0, phân bổ diễn ra theo nguồn thực tế.

Cách hoạt động:

- **Nút con trở nên hiển thị như các hàng riêng biệt.** Mỗi bảng điều khiển công bố danh sách các nút trực tiếp của mình; chỉ các nút có `Guid` đã biết mới được đưa vào – danh tính ổn định cần thiết để gán nút vào một «bước nhảy» phía trên. Master định kỳ (từ tác vụ heartbeat) tải các danh sách này và lưu vào cache, sau đó thêm các nút con «trung gian» vào cạnh các nút trực tiếp.
- **Nút trung gian chỉ đọc.** Trong giao diện người dùng, chúng được đánh dấu là **«Nút con»** với gợi ý: *«Chỉ đọc: nút con có thể truy cập qua {cha}. Quản lý nó từ bảng điều khiển riêng của {cha}.»* Không có nút quản lý cho hàng đó – nút được quản lý từ bảng điều khiển của cha trực tiếp.
- **Phân cấp thông qua GUID.** `ParentGuid` của nút trực tiếp là GUID của chính master; của nút trung gian là GUID của nút cha. Như vậy tạo thành một cây.
- **Nguồn chân lý cho bộ đếm – `origin_node_guid` trên inbound.** Đây là `panelGuid` của nút vật lý đang giữ inbound đó. Nó được gán khi đồng bộ inbound từ nút và **được giữ nguyên qua các bước nhảy tiếp theo**, vì vậy inbound lồng sâu được gán cho nút thực tế, không phải nút trung gian. Theo GUID này, các bộ đếm số inbound, client trực tuyến và client đã hết được tính lại. Logic chọn khóa:

Trạng thái inbound	Được gán cho
<code>origin_node_guid</code> được đặt	GUID này (nút nguồn thực tế)
trống, nhưng <code>node_id</code> được đặt	GUID tổng hợp của nút (bản dựng cũ, chưa báo cáo <code>panelGuid</code>)
trống và <code>node_id</code> trống	GUID riêng của master (inbound trên Xray cục bộ)

Client trực tuyến cũng được nhóm theo GUID, vì vậy mỗi hàng nút chỉ hiển thị những người thực sự được kết nối với nút đó.

Điều người dùng thấy: trong cấu trúc liên kết phẳng (các nút trực tiếp dưới master) không có gì thay đổi – bộ đếm theo GUID và theo `id` trùng khớp. Nhưng ngay khi xuất hiện chuỗi nút, các hàng «Nút con» xuất hiện trong danh sách, và con số inbound/trực tuyến/đã hết của mỗi nút phản ánh đúng tải trọng của chính nó, không phải tổng của tất cả những gì đi qua nó theo cách trung gian.

2. Đồng bộ hóa client-IP từ access.log giữa các nút

Giới hạn theo IP (`limitIp` của client) dựa trên các địa chỉ mà Xray ghi vào access.log. Trước đây mỗi nút chỉ thấy các kết nối đến chính nó, vì vậy giới hạn «không quá N IP trên mỗi client» không hoạt động trong cụm: client có thể kết nối đến các nút khác nhau và vượt giới hạn. Trong 3.3.0, các IP đã quan sát được đồng bộ trên toàn cụm.

Cách hoạt động:

- Trên mỗi nút, tác vụ nền phân tích access.log, trích xuất từ mỗi dòng IP, email client và dấu thời gian, lưu vào bảng cục bộ (một bản ghi cho mỗi email, IP được lưu dưới dạng mảng JSON `{ip, timestamp}`). Địa chỉ cục bộ `127.0.0.1` và `::1` bị loại bỏ.
- Đồng bộ hóa **mỗi 10 giây** thực hiện trao đổi hai chiều cho mỗi nút đã bật trong mạng: tải IP từ nút và hợp nhất vào bảng cục bộ, sau đó gửi cho nút bức tranh tổng hợp của master.
- Quá trình hợp nhất kết hợp các quan sát cũ và mới **mà không đếm trùng** một IP đã thấy trên nhiều nút, và **không hồi sinh** các bản ghi lỗi thời: áp dụng cùng ngưỡng tuổi như tác vụ cục bộ – **30 phút**. Dấu thời gian mới nhất được lưu cho mỗi IP. Các bản ghi từ nút khác nhận id cục bộ mới (không gian id của các nút độc lập); việc chèn đồng thời cùng một email được bảo vệ khỏi trùng lặp.
- Khi tính giới hạn, IP được coi là «còn sống» nếu nó được phát hiện trong lần quét cục bộ hiện tại hoặc có dấu thời gian rất mới từ cơ sở dữ liệu đã đồng bộ (**trong vòng 2 phút**). Chính điều này khiến giới hạn hoạt động trên toàn bộ cụm, ngay cả khi địa chỉ được phát hiện trên nút khác. Khi vượt giới hạn, các IP «còn sống» cũ nhất được gửi đến nhật ký fail2ban và các kết nối bị ngắt bắt buộc (remove/re-add client qua Xray API).

Điều người dùng thấy: giới hạn theo số IP bây giờ áp dụng cho toàn bộ cụm, không phải cho từng nút riêng lẻ; trong bảng điều khiển, theo client có thể thấy các IP được phát hiện trên bất kỳ nút nào (trong cửa sổ 30 phút). Không có nút/cài đặt riêng biệt cho điều này – đồng bộ hóa diễn ra tự động ở chạy nền, miễn là access.log của nút được bật và có thể truy cập (để áp dụng giới hạn cũng cần Fail2Ban trên nút).

3. Chỉ báo trạng thái riêng biệt: bảng điều khiển nút trực tuyến nhưng Xray bị sập

Trước đây trạng thái nút về cơ bản là «trực tuyến / ngoại tuyến». Nếu bảng điều khiển nút phản hồi, nút được coi là trực tuyến – ngay cả khi lõi Xray trên đó không hoạt động và client thực tế không thể kết nối. Trong 3.3.0, tình trạng bảng điều khiển và tình trạng lõi Xray được tách biệt.

Cách hoạt động:

- Khi truy vấn nút, master lấy từ phản hồi của `/panel/api/server/status` từ xa các trường `xray.state` và `xray.errorMessage` và lưu chúng vào nút. Các trường này được điền ngay cả khi ping bảng điều khiển thành công nhưng lõi không khỏe mạnh – chính xác để phân biệt khả năng tiếp cận bảng điều khiển với trạng thái Xray.

- Các giá trị của `xray.state` : "running" (đang chạy), "stop" (đã dừng), "error" (lỗi).
- Các giá trị này được dịch thành trạng thái nút. Ngoài các trạng thái quen thuộc, có thêm các trạng thái mới:

Khóa trạng thái	Khi nào hiển thị
<code>online</code>	bảng điều khiển phản hồi, Xray đang chạy (<code>running</code>)
<code>offline</code>	bảng điều khiển không thể truy cập / ping thất bại
<code>unknown</code>	trạng thái chưa được xác định
<code>xrayError</code>	bảng điều khiển trực tuyến, nhưng lỗi Xray ở trạng thái <code>error</code> (có <code>errorMsg</code>)
<code>xrayStopped</code>	bảng điều khiển trực tuyến, nhưng Xray đã dừng (<code>stop</code>)

- Đối với trạng thái như vậy, giao diện người dùng sử dụng **chỉ báo màu tím riêng biệt** (màu khác với màu xanh «trực tuyến» và màu đỏ «ngoại tuyến»). Màu tím báo hiệu trực tiếp: nút có thể liên lạc được, vấn đề nằm ở chính lỗi Xray, không phải ở mạng hay bảng điều khiển.

Điều người dùng thấy: thay vì «màu xanh» gây hiểu nhầm khi lỗi bị sập, nút được tô sáng bằng **màu tím** với trạng thái **«Lỗi Xray»** hoặc **«Đã dừng»**. Điều này ngay lập tức cho thấy cần sửa Xray trên nút (khởi động lại lỗi, xem `errorMsg`), không phải điều tra khả năng tiếp cận của chính nút. Cùng `xrayState` / `xrayError` cũng được truyền đến các nút con trung gian (xem điểm 1), vì vậy trạng thái lỗi không chính xác có thể thấy được trên toàn bộ chuỗi.

13. Cài đặt bảng điều khiển

Phần «Cài đặt» (tiêu đề trang — **Cài đặt**, tiếng Anh *Panel Settings*) quản lý hành vi của chính bảng điều khiển web 3X-UI: địa chỉ và cổng nào nó lắng nghe, cách bảo mật, cách tương tác với Telegram bot và các dịch vụ bên ngoài, múi giờ nào để thực thi các tác vụ định kỳ. Mỗi thông số được lưu trong bảng `settings` của cơ sở dữ liệu dưới dạng cặp «khóa — giá trị»; nếu giá trị không có trong CSDL, giá trị mặc định sẽ được áp dụng.

Quan trọng — áp dụng thay đổi. Mọi thay đổi trên trang này cần được lưu bằng nút **Lưu** (*Save*), sau đó khởi động lại bảng điều khiển để thay đổi có hiệu lực. Gợi ý nguyên văn: «Lưu thay đổi và khởi động lại bảng điều khiển để áp dụng.» Khi lưu, thông báo «Cài đặt đã được thay đổi» sẽ hiển thị.

13.1. Lưu và khởi động lại bảng điều khiển

Phần tử	Mục đích
Lưu (<i>Save</i>)	Ghi tất cả các trường của biểu mẫu vào CSDL (<code>POST /panel/setting/update</code>). Trước khi ghi, các giá trị được xác thực — địa chỉ, cổng hoặc đường dẫn không hợp lệ sẽ bị từ chối và bảng điều khiển sẽ trả về lỗi.
Khởi động lại bảng điều khiển (<i>Restart Panel</i>)	Khởi động lại máy chủ web của bảng điều khiển (<code>POST /panel/setting/restartPanel</code>). Việc khởi động lại diễn ra sau độ trễ 3 giây. Gợi ý: «Bạn có chắc muốn khởi động lại bảng điều khiển không? Xác nhận và bảng điều khiển sẽ khởi động lại sau 3 giây. Nếu bảng điều khiển không phản hồi, hãy kiểm tra log máy chủ». Khi thành công — «Bảng điều khiển đã được khởi động lại thành công».
Khôi phục cài đặt mặc định (<i>Reset to Default</i>)	Xóa tất cả cài đặt đã lưu trong CSDL, sau đó bảng điều khiển sẽ sử dụng các giá trị mặc định. Thông tin đăng nhập của quản trị viên không bị đặt lại bởi thao tác này.

Việc khởi động lại được thực hiện bằng cách gửi tín hiệu `SIGHUP` đến tiến trình bảng điều khiển (hoặc thông qua hook khởi động lại đã đăng ký). Trên Windows, khởi động lại tự động qua tín hiệu không được hỗ trợ. **Các thay đổi về thông số lắng nghe (IP, cổng, đường dẫn, tên miền, chứng chỉ, múi giờ) chỉ có hiệu lực sau khi khởi động lại bảng điều khiển.**

13.2. Cài đặt chung (tab «Bảng điều khiển» / *General*)

Ngôn ngữ giao diện (*Language*)

Ngôn ngữ của giao diện web bảng điều khiển. Các ngôn ngữ có sẵn: `en-US` (tiếng Anh), `ru-RU` (tiếng Nga), `zh-CN`, `zh-TW`, `fa-IR`, `ar-EG`, `es-ES`, `id-ID`, `ja-JP`, `pt-BR`, `tr-TR`, `uk-UA`, `vi-VN`. Đây là cài đặt hiển thị và không ảnh hưởng đến hoạt động của Xray.

Loại lịch (*Calendar Type*)

- **Khóa:** `datepicker`
- **Giá trị mặc định:** `gregorian` (lịch Gregory).

- **Mục đích:** loại lịch được sử dụng trong bộ chọn ngày (ví dụ: khi đặt ngày hết hạn của khách hàng). Gợi ý: «Các tác vụ định kỳ sẽ được thực thi theo lịch này.» Giá trị thay thế – lịch Ba Tư (jalali), được ưa chuộng trong cộng đồng Iran sử dụng bảng điều khiển.

Kích thước phân trang (*Pagination Size*)

- **Khóa:** `pageSize`
- **Giá trị mặc định:** `25`
- **Giá trị cho phép:** số nguyên từ `0` đến `1000`.
- **Mục đích:** số hàng trên mỗi trang trong các bảng (danh sách kết nối/inbound). Gợi ý: «Xác định kích thước trang cho bảng kết nối. Đặt 0 để tắt» – khi `0`, phân trang bị tắt và tất cả các bản ghi hiển thị trong một danh sách duy nhất.
- **Không cần khởi động lại bảng điều khiển** (cài đặt hiển thị).

Khởi động lại Xray sau khi tự động tắt (*Restart Xray After Auto Disable*)

- **Khóa:** `restartXrayOnClientDisable`
- **Giá trị mặc định:** `true`
- **Mục đích:** khi khách hàng bị tắt tự động (do hết hạn hoặc đạt giới hạn lưu lượng), Xray sẽ được khởi động lại để ngắt các kết nối đã thiết lập của khách hàng đó. Gợi ý: «Khi khách hàng bị tắt tự động do hết hạn hoặc vượt giới hạn lưu lượng, hãy khởi động lại Xray.» Bản thân chức năng không thay đổi – công tắc chỉ nằm ở tab «Bảng điều khiển» (*General*) cùng với các cài đặt chung khác.

Mô hình ghi chú và ký tự phân tách (*Remark Model & Separation Character*)

- **Khóa:** `remarkModel`
- **Giá trị mặc định:** `-ieo`
- **Mục đích:** xác định cách hình thành tên (remark) của cấu hình trong subscription. Chuỗi bao gồm **ký tự đầu tiên** – dấu phân tách, và tiếp theo là **chuỗi chữ cái thứ tự**:
 - `i` – ghi chú inbound (*inbound remark*);
 - `e` – email của khách hàng;
 - `o` – nhân bổ sung (*extra*).

Với giá trị mặc định `-ieo`, dấu phân tách là `-`, và thứ tự các phần: inbound → email → extra (ví dụ: `MyInbound-user@mail-extra`). Các phần trống sẽ bị bỏ qua. Trường «Ví dụ ghi chú» (*Sample Remark*) trong giao diện hiển thị bản xem trước tên được tạo. Việc đưa email vào tên còn phụ thuộc vào thông số «Bao gồm Email trong tên» trong cài đặt subscription (phần về subscription).

Ví dụ: cách giá trị `remarkModel` ảnh hưởng đến tên cấu hình. Giả sử inbound có tên `VLESS-Reality`, email của khách hàng là `alex@vpn`, và nhân bổ sung là `RU`. Khi đó:

Giá trị trường	Tên kết quả (remark)
<code>-ieo</code> (mặc định)	<code>VLESS-Reality-alex@vpn-RU</code>

Giá trị trường	Tên kết quả (remark)
_ie	VLESS-Reality_alex@vpn
-ei	alex@vpn-VLESS-Reality
o (dấu cách làm phân tách, chỉ nhận)	RU

Ký tự đầu tiên của chuỗi luôn là dấu phân tách; các chữ cái còn lại xác định phần nào và theo thứ tự nào sẽ có trong tên.

13.3. Quyền truy cập bảng điều khiển: IP, cổng, đường dẫn, tên miền, chứng chỉ

Nhóm này xác định điểm vào mạng của bảng điều khiển. **Tất cả các thay đổi ở đây chỉ có hiệu lực sau khi khởi động lại bảng điều khiển.**

Trường	Khóa	Giá trị mặc định	Mô tả
Địa chỉ IP để quản lý bảng điều khiển (<i>Listen IP</i>)	webListen	"" (trống)	IP mà bảng điều khiển web lắng nghe. Trống = lắng nghe trên tất cả IP. Gợi ý: «Để trống để cho phép kết nối từ bất kỳ IP nào». Nếu được đặt, phải là địa chỉ IP hợp lệ (nếu không, quá trình lưu sẽ bị từ chối).
Tên miền bảng điều khiển (<i>Listen Domain</i>)	webDomain	"" (trống)	Tên miền của bảng điều khiển để xác minh yêu cầu theo tên miền. Trống = chấp nhận kết nối từ bất kỳ tên miền và IP nào. Gợi ý: «Để trống để cho phép kết nối từ bất kỳ tên miền và IP nào.»
Cổng bảng điều khiển (<i>Listen Port</i>)	webPort	2053	Cổng mà bảng điều khiển hoạt động. Gợi ý: «Cổng mà bảng điều khiển hoạt động». Cho phép 1–65535. Cổng phải trống; bảng điều khiển và dịch vụ subscription không thể đồng thời sử dụng cùng một cặp IP: cổng.
Đường dẫn URI (<i>URI Path</i>)	webBasePath	/	Đường dẫn cơ sở URL của bảng điều khiển (basePath). Gợi ý: «Phải bắt đầu bằng '/' và kết thúc bằng '/'». Khi lưu, bảng điều khiển tự động thêm dấu gạch chéo đầu và cuối nếu thiếu. Các ký tự không được phép trong đường dẫn sẽ bị từ chối.

Chứng chỉ bảng điều khiển (TLS / HTTPS)

Trường	Khóa	Giá trị mặc định	Mô tả
Đường dẫn đến file khóa công khai của chứng chỉ bảng điều khiển (<i>Public Key Path</i>)	webCertFile	""	Đường dẫn đầy đủ đến file chứng chỉ (chuỗi). Gợi ý: «Nhập đường dẫn đầy đủ bắt đầu bằng '/'».
Đường dẫn đến file khóa riêng tư của chứng chỉ bảng điều khiển (<i>Private Key Path</i>)	webKeyFile	""	Đường dẫn đầy đủ đến file khóa riêng tư. Gợi ý: «Nhập đường dẫn đầy đủ bắt đầu bằng '/'».

Nếu **ít nhất một** trong các đường dẫn chứng chỉ/khóa được đặt, bảng điều khiển khi lưu sẽ cố tải cặp «chứng chỉ + khóa»; nếu xảy ra lỗi (file không tồn tại, khóa và chứng chỉ không khớp), quá trình lưu sẽ bị từ

chối. Khi cả hai đường dẫn hợp lệ được đặt, bảng điều khiển chuyển sang HTTPS. Cả hai trường trống = bảng điều khiển hoạt động qua HTTP thông thường.

Cảnh báo bảo mật (*Security warnings*). Bảng điều khiển hiển thị khối «Bảng điều khiển của bạn có thể bị lộ:» với các cảnh báo nếu phát hiện cấu hình không an toàn:

- hoạt động qua HTTP thông thường – «hãy cấu hình TLS cho môi trường production»;
- cổng mặc định 2053 – «hãy đổi sang cổng ngẫu nhiên»;
- đường dẫn cơ sở mặc định / – «hãy đổi sang đường dẫn ngẫu nhiên»;
- đường dẫn subscription mặc định /sub/ và JSON subscription /json/ – «hãy thay đổi». Đây là khuyến nghị, không phải chặn.

13.4. Phiên, proxy bảng điều khiển và proxy tin cậy (tab «Proxy và máy chủ» / *Proxy and Server*)

Thời gian phiên (*Session Duration*)

- **Khóa:** `sessionMaxAge`
- **Giá trị mặc định:** 360 (phút, tức là 6 giờ).
- **Giá trị cho phép:** từ 1 đến 525600 phút (1 năm).
- **Mục đích:** thời gian quản trị viên duy trì trạng thái đăng nhập mà không cần đăng nhập lại. Đơn vị – phút. Gợi ý: «Thời gian phiên trong hệ thống (đơn vị: phút)».

Outbound cho lưu lượng bảng điều khiển (*Panel Traffic Outbound*)

- **Khóa:** `panelOutbound`
- **Giá trị mặc định:** "" (trống = kết nối trực tiếp).
- **Mục đích:** chỉ định Xray-outbound mà qua đó bảng điều khiển gửi **các yêu cầu của chính mình** – kiểm tra phiên bản và tải xuống bảng điều khiển/Xray, kết nối với Telegram, cập nhật file geo thông thường – để vượt qua bộ lọc máy chủ cho GitHub/Telegram. Trường này là **danh sách thả xuống**: nó liệt kê các outbound từ template cấu hình Xray, các outbound từ subscription outbound, cũng như các **bộ cân bằng tải** định tuyến (thành nhóm riêng). Các outbound kiểu `blackhole` bị loại khỏi danh sách – định tuyến tải xuống vào «hố đen» là vô nghĩa. Gợi ý nguyên văn: «Định tuyến các yêu cầu của chính bảng điều khiển – kiểm tra phiên bản và tải xuống bảng điều khiển/Xray, Telegram và cập nhật file geo thông thường – qua outbound Xray này để vượt qua bộ lọc máy chủ GitHub/Telegram. Một loopback inbound cầu nối cục bộ được tự động thêm vào cấu hình đang chạy và áp dụng ngay lập tức. Tính năng tự động cập nhật Geodata tích hợp trong Xray không bị ảnh hưởng; nó có outbound riêng để tải xuống. Để trống để kết nối trực tiếp.»

Cách hoạt động. Khi chọn outbound, bảng điều khiển tự thêm vào cấu hình hoạt động một loopback inbound dịch vụ (SOCKS bridge với tag `panel-egress`) và quy tắc định tuyến chuyển hướng lưu lượng HTTP của chính bảng điều khiển đến outbound đã chọn. Nếu chọn bộ cân bằng tải, `balancerTag` được đưa vào quy tắc và lưu lượng bảng điều khiển được phân phối giữa các thành viên của nó. Bridge và quy tắc được áp dụng **ngay lập tức**, mà không cần khởi động lại toàn bộ bảng điều khiển. Để trống

trường để kết nối trực tiếp. Tính năng tự động cập nhật dữ liệu geo tích hợp trong Xray **không bị ảnh hưởng** bởi cài đặt này – nó có outbound riêng trong định tuyến Xray.

- **Định dạng:** `socks5://` (hoặc `socks5h://`) hoặc `http(s)://`, khi cần với thông tin xác thực dạng `socks5://user:pass@host:port`. Các scheme được hỗ trợ chính xác là: `socks5`, `socks5h`, `http`, `https` – các scheme khác được coi là không hợp lệ và bảng điều khiển sẽ quay lại kết nối trực tiếp. Ví dụ điển hình – SOCKS inbound cục bộ của chính Xray.
- Gợi ý nguyên văn: «Định tuyến các yêu cầu đi ra của chính bảng điều khiển (cập nhật geo, kiểm tra phiên bản Xray/bảng điều khiển, Telegram) qua proxy này để vượt qua bộ lọc máy chủ GitHub/Telegram. Chấp nhận `socks5://` hoặc `http(s)://`, ví dụ SOCKS inbound cục bộ của Xray. Để trống để kết nối trực tiếp.»
- Proxy không hợp lệ không dẫn đến lỗi lưu – bảng điều khiển chỉ đơn giản sử dụng kết nối trực tiếp và ghi cảnh báo vào log.

Ví dụ các giá trị trường. Nếu trên máy chủ đã có SOCKS inbound cục bộ của Xray trên cổng `10808`, hãy định tuyến các yêu cầu của chính bảng điều khiển qua nó:

```
socks5://127.0.0.1:10808
```

Đối với HTTP proxy bên ngoài có xác thực:

```
http://user:pass@proxy.example.com:8080
```

Sau khi lưu và khởi động lại, bảng điều khiển sẽ tải cập nhật cơ sở dữ liệu geo, kiểm tra phiên bản và kết nối Telegram qua proxy đã chỉ định.

CIDR proxy tin cậy (Trusted proxy CIDRs)

- **Khóa:** `trustedProxyCIDRs`
- **Giá trị mặc định:** `127.0.0.1/32, ::1/128` (chỉ máy chủ cục bộ).
- **Định dạng:** danh sách địa chỉ IP hoặc mạng con CIDR được phân tách bằng dấu phẩy (ví dụ `10.0.0.0/8, 192.168.1.5`). Mỗi phần tử được kiểm tra như IP hoặc CIDR – giá trị không hợp lệ sẽ bị từ chối khi lưu.
- **Mục đích:** liệt kê các nguồn được phép đặt các header `X-Forwarded-Host`, `X-Forwarded-Proto` và header IP thực của khách hàng. Gợi ý nguyên văn: «IP/CIDR cách nhau bằng dấu phẩy được phép đặt các header forwarded host, proto và IP khách hàng.» Cần cấu hình nêu bảng điều khiển hoạt động sau reverse proxy (nginx, Caddy, v.v.) để xác định chính xác IP khách hàng và scheme.

Ví dụ: bảng điều khiển sau reverse proxy. Nếu nginx nằm trên cùng máy chủ và proxy các yêu cầu đến bảng điều khiển, hãy giữ tin tưởng chỉ với máy chủ cục bộ (giá trị mặc định):

```
127.0.0.1/32, ::1/128
```

Nếu proxy nằm trên máy chủ riêng trong mạng nội bộ `10.0.0.0/8`, hãy thêm mạng con của nó, nếu không bảng điều khiển sẽ bỏ qua các header mà proxy đã gửi và sẽ thấy IP của proxy thay vì IP thực của khách hàng:

```
127.0.0.1/32,::1/128,10.0.0.0/8
```

Ví dụ về khối nginx tương ứng truyền IP thực và scheme:

```
proxy_set_header X-Real-IP      $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;
proxy_set_header X-Forwarded-Host $host;
```

13.5. Telegram bot (tab «Telegram bot» / *Telegram Bot*)

Bật Telegram bot (*Enable Telegram Bot*)

- **Khóa:** `tgBotEnable`
- **Loại/mặc định:** `boolean`, `false`.
- **Mục đích:** bật hoạt động của Telegram bot. Gợi ý: «Truy cập các tính năng bảng điều khiển qua Telegram bot».

Token Telegram (*Telegram Token*)

- **Khóa:** `tgBotToken`
- **Mặc định:** `""`.
- **Mục đích:** token của bot. Gợi ý: «Cần lấy token từ trình quản lý bot Telegram @botfather».
- **Đặc điểm bảo mật:** token là giá trị bí mật. Nó không được trả về trong phản hồi của bảng điều khiển khi đọc cài đặt (trường bị xóa, chỉ trả về cờ «đã cấu hình/chưa cấu hình»). Nếu để trống trường khi lưu, token đã lưu trước đó **được giữ nguyên** (không bị xóa).

Ngôn ngữ Telegram bot (*Telegram Bot Language*)

- **Khóa:** `tgLang`
- **Mặc định:** `en-US`.
- **Mục đích:** ngôn ngữ của tin nhắn bot (độc lập với ngôn ngữ giao diện web). Danh sách các ngôn ngữ có sẵn trùng với ngôn ngữ của bảng điều khiển.

User ID của quản trị viên bot (*Admin Chat ID*)

- **Khóa:** `tgBotChatId`
- **Mặc định:** `""`.
- **Định dạng:** một hoặc nhiều Telegram User ID dạng số **phân tách bằng dấu phẩy**.
- **Mục đích:** người nhận thông báo và quản trị viên được phép quản lý bảng điều khiển qua bot. Gợi ý: «Một hoặc nhiều User ID của quản trị viên Telegram bot. Để lấy User ID, hãy sử dụng @userinfobot hoặc lệnh '/id' trong bot.»

Tần suất thông báo (*Notification Time*)

- **Khóa:** `tgRunTime`
- **Mặc định:** `@daily` (một lần mỗi ngày).
- **Định dạng:** chuỗi ở định dạng **Crontab** (hỗ trợ cả biểu thức cron tiêu chuẩn và viết tắt như `@daily`, `@hourly`, `@every 1h`). Gợi ý: «Chỉ định khoảng thời gian thông báo ở định dạng Crontab». Kiểm soát các báo cáo định kỳ của bot.

Ví dụ các giá trị trường.

Giá trị	Khi bot gửi báo cáo
<code>@daily</code>	một lần mỗi ngày vào nửa đêm (mặc định)
<code>@hourly</code>	mỗi giờ
<code>@every 6h</code>	mỗi 6 giờ
<code>0 9 * * *</code>	hàng ngày lúc 09:00
<code>30 8 * * 1</code>	mỗi thứ Hai lúc 08:30

Thời gian được tính theo múi giờ từ cài đặt «Múi giờ» (mục 13.6).

SOCKS proxy (*SOCKS Proxy*)

- **Khóa:** `tgBotProxy`
- **Mặc định:** `""`.
- **Mục đích:** SOCKS5 proxy riêng cho kết nối bot với Telegram. Gợi ý: «Nếu bạn cần proxy Socks5 để kết nối với Telegram, hãy cấu hình các thông số của nó theo hướng dẫn.» Áp dụng đặc biệt cho lưu lượng bot (khác với «Proxy mạng chung của bảng điều khiển» từ mục 13.4).

Telegram API Server (*Telegram API Server*)

- **Khóa:** `tgBotAPIServer`
- **Mặc định:** `""` (sử dụng máy chủ tiêu chuẩn `api.telegram.org`).
- **Định dạng:** URL `http(s)://...`; khi lưu sẽ kiểm tra tính hợp lệ của URL – địa chỉ không hợp lệ sẽ bị từ chối. Gợi ý: «Máy chủ API Telegram được sử dụng. Để trống để sử dụng máy chủ mặc định.» Cần thiết cho Telegram Bot API server tự triển khai.

Thông báo bot (nhóm «Thông báo» / *Notifications*)

Trường	Khóa	Mặc định	Mô tả
Sao lưu cơ sở dữ liệu (<i>Database Backup</i>)	<code>tgBotBackup</code>	<code>false</code>	Gửi file sao lưu CSDL kèm báo cáo đến Telegram. Gợi ý: «Gửi thông báo kèm file sao lưu cơ sở dữ liệu».

Trường	Khóa	Mặc định	Mô tả
Thông báo đăng nhập (<i>Login Notification</i>)	tgBotLoginNotify	true	Thông báo khi có lần thử đăng nhập vào bảng điều khiển. Gợi ý: «Hiển thị tên người dùng, địa chỉ IP và thời gian khi có ai đó cố đăng nhập vào bảng điều khiển của bạn.»
Độ trễ thông báo hết hạn phiên (<i>Expiration Date Notification</i>)	expireDiff	0	Số ngày trước khi hết hạn của khách hàng để gửi thông báo. 0 – đã tắt. Cho phép ≥ 0 . Gợi ý: «Nhận thông báo về ngày hết hạn phiên trước khi đạt ngưỡng (đơn vị: ngày)».
Ngưỡng thông báo lưu lượng (<i>Traffic Cap Notification</i>)	trafficDiff	0	Ngưỡng lưu lượng còn lại để thông báo. Gợi ý: «Nhận thông báo về việc cạn kiệt lưu lượng trước khi đạt ngưỡng (đơn vị: GB)». Cho phép 0–100.
Ngưỡng tải CPU (<i>CPU Load Notification</i>)	tgCpu	80	Thông báo cho quản trị viên nếu tải CPU vượt quá ngưỡng (tính bằng %). Cho phép 0–100. Gợi ý: «Thông báo cho quản trị viên trong Telegram nếu tải CPU vượt quá ngưỡng này (đơn vị: %)».

13.6. Ngày và giờ (tab «Ngày và giờ» / *Date and Time*)

Múi giờ (*Time Zone*)

- **Khóa:** timeLocation
- **Giá trị mặc định:** Local (múi giờ hệ thống của máy chủ).
- **Định dạng:** tên vùng từ cơ sở dữ liệu IANA tz (ví dụ: Europe/Moscow, UTC, Asia/Tehran).
- **Mục đích:** múi giờ mà bảng điều khiển thực thi các tác vụ định kỳ (báo cáo bot, đặt lại/kiểm tra lưu lượng, hết hạn). Gợi ý: «Các tác vụ định kỳ được thực thi theo giờ trong múi giờ này».
- **Xác thực:** khi lưu, vùng được kiểm tra – vùng không tồn tại sẽ bị từ chối. Nếu sau đó CSDL chứa giá trị không hợp lệ, bảng điều khiển trong runtime sẽ quay lại Local, và nếu không khả dụng – về UTC.

13.7. Lưu lượng bên ngoài và hành vi Xray (tab «Lưu lượng bên ngoài» / *External Traffic*)

Trường	Khóa	Mặc định	Mô tả
Thông tin lưu lượng bên ngoài (<i>External Traffic Inform</i>)	externalTrafficInformEnable	false	Thông báo cho API bên ngoài khi mỗi lần cập nhật lưu lượng. Gợi ý: «Thông báo cho API bên ngoài khi mỗi lần cập nhật lưu lượng.»
URI thông tin lưu lượng bên ngoài (<i>External Traffic Inform URI</i>)	externalTrafficInformURI	""	URL mà bảng điều khiển gửi cập nhật lưu lượng đến. Kiểm tra tính hợp lệ của URL khi lưu. Gợi ý: «Cập nhật lưu lượng được gửi đến URI này».

Trường	Khóa	Mặc định	Mô tả
Khởi động lại Xray sau khi tự động tắt (<i>Restart Xray After Auto Disable</i>)	<code>restartXrayOnClientDisable</code>	<code>true</code>	Khởi động lại Xray khi khách hàng bị tắt tự động do hết hạn hoặc vượt giới hạn lưu lượng. Gợi ý: «Khi khách hàng bị tắt tự động do hết hạn hoặc vượt giới hạn lưu lượng, hãy khởi động lại Xray.» Công tắc nằm ở tab «Bảng điều khiển» (General) – xem mục 13.2; đây đưa ra để đầy đủ.

13.8. Khác: template cấu hình Xray và URL kiểm tra

Template cấu hình Xray (*xrayTemplateConfig*)

- **Khóa:** `xrayTemplateConfig`
- **Mặc định:** template JSON tích hợp (embedded) được cung cấp cùng với bản build.
- **Mục đích:** template cấu hình JSON cơ sở của Xray-core, trên đó bảng điều khiển xây dựng thêm inbound/outbound. Giá trị này **không được trả về** trong đầu ra thông thường của tất cả cài đặt và được chỉnh sửa trên trang cấu hình Xray riêng biệt, không phải trong danh sách trường cài đặt chung của bảng điều khiển. Template tiêu chuẩn mặc định có thể truy cập qua `GET /panel/setting/getDefaultJsonConfig`.

URL kiểm tra outbound (*xrayOutboundTestUrl*)

- **Khóa:** `xrayOutboundTestUrl`
- **Mặc định:** `https://www.google.com/generate_204`
- **Mục đích:** URL được sử dụng khi kiểm tra khả năng hoạt động của các kết nối outbound. Khi thiết lập, nó được làm sạch như HTTP(S)-URL.

13.9. Tài khoản quản trị viên và API token

Các thông số này nằm ở tab liên kế («Tài khoản» / *Authentication*) và được xem xét chi tiết trong phần về bảo mật; đây là tóm tắt ngắn gọn về các khóa.

- **Thay đổi thông tin đăng nhập** (các trường «Đăng nhập hiện tại», «Mật khẩu hiện tại», «Đăng nhập mới», «Mật khẩu mới») được lưu bằng yêu cầu riêng `POST /panel/setting/updateUser`. Yêu cầu đăng nhập và mật khẩu hiện tại chính xác; đăng nhập và mật khẩu mới không được trống. Thông báo: «Bạn đã thay đổi thông tin đăng nhập quản trị viên thành công.» / «Tên người dùng hoặc mật khẩu không đúng».
- **Xác thực hai yếu tố (2FA)** – các khóa `twoFactorEnable` (mặc định `false`) và bí mật `twoFactorToken`. Token là bí mật: khi 2FA được bật, trường trống khi lưu không xóa token hiện có. Khi **lần đầu** bật 2FA, bảng điều khiển vô hiệu hóa các phiên hiện tại (tăng «epoch đăng nhập»).
- **API token** được quản lý bởi các endpoint riêng (`/panel/setting/apiTokens...`): danh sách, tạo (`apiTokens/create`), xóa, bật/tắt. Bản thân token chỉ được hiển thị **một lần khi tạo** và không được lưu ở dạng đọc được: «Sao chép token này ngay bây giờ. Vì lý do bảo mật, nó không được lưu ở dạng đọc được và sẽ không được hiển thị lại.»

Chi tiết về 2FA, mật khẩu, đồng bộ hóa LDAP và định dạng subscription (JSON/Clash, fragmentation, noises, mux) được trình bày trong các phần riêng biệt tương ứng của hướng dẫn.

13.10. Thay đổi API trong 3.3.0 (quan trọng cho các tích hợp)

Trong phiên bản 3.3.0, cấu trúc đường dẫn API phía máy chủ đã thay đổi. Nếu bạn có các tích hợp bên ngoài (script, bot, bảng điều khiển trung tâm, tác vụ CI) truy cập bảng điều khiển qua HTTP, chúng **cần được sửa**, nếu không chúng sẽ ngừng hoạt động.

⚠ BREAKING CHANGE: các endpoint `/panel/setting/*` và `/panel/xray/*` đã chuyển sang `/panel/api`

Trước đây, quản lý cài đặt bảng điều khiển và cấu hình Xray nằm riêng biệt, dưới các đường dẫn `/panel/setting/*` và `/panel/xray/*`. Bây giờ cả hai tập hợp được đăng ký bên trong nhóm API chung `/panel/api`. Các đường dẫn cũ **đã bị xóa hoàn toàn** – yêu cầu đến chúng sẽ trả về 404.

Lý do thực hiện điều này: toàn bộ nhóm `/panel/api` đi qua kiểm tra quyền truy cập thống nhất, tức là các endpoint này bây giờ chấp nhận cùng header `Authorization: Bearer <token>` như phần còn lại của API. API token là quyền truy cập quản trị viên đầy đủ, và như vậy toàn bộ bề mặt API đã trở nên thống nhất.

Những gì KHÔNG thay đổi: các trang giao diện web (SPA routes) `/panel/settings` và `/panel/xray` vẫn còn nguyên vị trí – chỉ các endpoint API phía máy chủ mới bị ảnh hưởng.

Bảng tương ứng đường dẫn (cũ → mới)

Tiền tố cho tất cả các đường dẫn bên dưới – chỉ thêm `api/` sau `/panel/`.

Đường dẫn cũ (≤ 3.2.x)	Đường dẫn mới (3.3.0)	Phương thức
<code>/panel/setting/all</code>	<code>/panel/api/setting/all</code>	POST
<code>/panel/setting/defaultSettings</code>	<code>/panel/api/setting/defaultSettings</code>	POST
<code>/panel/setting/update</code>	<code>/panel/api/setting/update</code>	POST
<code>/panel/setting/updateUser</code>	<code>/panel/api/setting/updateUser</code>	POST
<code>/panel/setting/restartPanel</code>	<code>/panel/api/setting/restartPanel</code>	POST
<code>/panel/setting/getDefaultJsonConfig</code>	<code>/panel/api/setting/getDefaultJsonConfig</code>	GET
<code>/panel/setting/apiTokens</code>	<code>/panel/api/setting/apiTokens</code>	GET
<code>/panel/setting/apiTokens/create</code>	<code>/panel/api/setting/apiTokens/create</code>	POST
<code>/panel/setting/apiTokens/delete/:id</code>	<code>/panel/api/setting/apiTokens/delete/:id</code>	POST
<code>/panel/setting/apiTokens/setEnabled/:id</code>	<code>/panel/api/setting/apiTokens/setEnabled/:id</code>	POST
<code>/panel/xray/</code>	<code>/panel/api/xray/</code>	POST
<code>/panel/xray/update</code>	<code>/panel/api/xray/update</code>	POST
<code>/panel/xray/getDefaultJsonConfig</code>	<code>/panel/api/xray/getDefaultJsonConfig</code>	GET

Đường dẫn cũ (≤ 3.2.x)	Đường dẫn mới (3.3.0)	Phương thức
/panel/xray/getXrayResult	/panel/api/xray/getXrayResult	GET
/panel/xray/getOutboundsTraffic	/panel/api/xray/getOutboundsTraffic	GET
/panel/xray/resetOutboundsTraffic	/panel/api/xray/resetOutboundsTraffic	POST
/panel/xray/testOutbound	/panel/api/xray/testOutbound	POST
/panel/xray/warp/:action	/panel/api/xray/warp/:action	POST
/panel/xray/nord/:action	/panel/api/xray/nord/:action	POST
/panel/xray/outbound-subs (và /outbound-subs/*)	/panel/api/xray/outbound-subs (và /outbound-subs/*)	GET/POST/DELETE

Bản thân các tên đường dẫn con, phần thân yêu cầu và định dạng phản hồi không thay đổi – chỉ có **tiền tố** thay đổi.

Cách sửa các tích hợp hiện có

1. Tìm trong các script/câu hình của bạn tất cả các đường dẫn `/panel/setting/` và `/panel/xray/`.
2. Thay thế tiền tố: thêm `api/` ngay sau `/panel/` (ví dụ: `/panel/setting/all` → `/panel/api/setting/all`).
3. Không cần sửa phần thân yêu cầu, tham số và định dạng phản hồi – chỉ URL thay đổi.
4. Vì cài đặt và cấu hình Xray bây giờ nằm dưới `/panel/api`, chúng có thể (và nên) được truy cập bằng cùng API token `Authorization: Bearer <token>` như `/panel/api/inbounds/*` và các endpoint khác. Đừng quên về CSRF-middleware được bật cho toàn bộ nhóm `/panel/api`.

Ví dụ: đọc tất cả cài đặt qua API. Trước đây (≤ 3.2.x):

```
curl -sk -X POST "https://panel.example.com:2053/MyPath/panel/setting/all" \
-H "Authorization: Bearer <token>"
```

Bây giờ (3.3.0) – đã thêm `api/` sau `/panel/`:

```
curl -sk -X POST "https://panel.example.com:2053/MyPath/panel/api/setting/all" \
-H "Authorization: Bearer <token>"
```

Tương tự cho khởi động lại bảng điều khiển: `POST /panel/api/setting/restartPanel`. Đường dẫn cũ `/panel/setting/restartPanel` bây giờ sẽ trả về 404.

API có kiểu: schema và tài liệu (Swagger / OpenAPI)

Trong 3.3.0, đặc tả OpenAPI đã trở nên hoàn toàn có kiểu. Trước đây, các phản hồi có kiểu được mô tả bằng đối tượng rỗng `{}`; bây giờ các component và schema (`components.schemas`) được tạo trực tiếp từ các mô hình dữ liệu. Nhờ đó:

- Swagger UI hiển thị các mô hình dữ liệu thực, chứ không phải các placeholder vô nghĩa.

- Các bộ tạo bên ngoài (`openapi-generator` và tương tự) có thể xây dựng client sẵn sàng theo ngôn ngữ cần thiết dựa trên đặc tả.
- Mỗi phản hồi có kiểu được đính kèm `$ref` đến mô hình cụ thể và kèm theo các ví dụ phản hồi.

Nơi xem tài liệu API:

- **Trang Swagger tích hợp.** Trong menu bảng điều khiển – mục «**Tài liệu API**» (SPA route `/panel/api-docs`). Đây liệt kê tương tác tất cả các endpoint với mô tả, phần thân yêu cầu và ví dụ phản hồi.
 - **Đặc tả OpenAPI 3.0 thô** được trả về tại địa chỉ `/panel/api/openapi.json` . URL này có thể đưa trực tiếp vào Postman, Insomnia hoặc `openapi-generator` . Đặc tả được tích hợp vào binary khi build; khi bảng điều khiển chạy dưới `webBasePath` không chuẩn, trường `servers` trong đặc tả tự động được ghi lại theo đường dẫn cơ sở hiện tại, để nút «Try it out» và các bộ tạo bên ngoài nhắm đến tiền tố đúng.
-

14. Telegram Bot

Bảng điều khiển 3X-UI tích hợp sẵn Telegram bot, cho phép nhận thông báo về trạng thái máy chủ và các client, đồng thời quản lý từng client ngay trong ứng dụng nhắn tin. Bot hoạt động theo công nghệ long polling (thăm dò liên tục với Telegram), do đó không cần tên miền ngoài hay cổng mở – chỉ cần có kết nối đầu ra đến máy chủ Telegram.

Bot phân biệt hai loại người dùng:

- **Quản trị viên** – người dùng có Telegram User ID được chỉ định trong cài đặt bot (trường «User ID quản trị viên bot»). Có quyền truy cập đầy đủ: thống kê máy chủ, sao lưu, quản lý client, khởi động lại Xray.
- **Client** – bất kỳ người dùng nào khác có Telegram User ID được liên kết với một client cụ thể của kết nối inbound (trường `tgId` của client). Chỉ thấy thông tin về các subscription của chính mình.

Ví dụ: liên kết client với Telegram. Để người dùng nhận được thông kê về subscription của mình, Telegram User ID dạng số của họ được ghi vào trường `tgId` của client. Trong cài đặt JSON của client, điều này trông như sau:

```
{
  "email": "ivan",
  "id": "6f1e6b1a-0c3d-4f2a-9b7e-1a2b3c4d5e6f",
  "tgId": "123456789",
  "enable": true,
  "limitIp": 2,
  "totalGB": 53687091200,
  "expiryTime": 0
}
```

Sau đó, người dùng có User ID `123456789` có thể gửi cho bot lệnh `/usage ivan` và xem thống kê của mình. Quản trị viên cũng có thể đặt ID tương tự thông qua nút « Đặt người dùng Telegram» trong thẻ client – không cần phải sửa thủ công trong JSON.

14.1. Bật và cấu hình bot

Tất cả các tham số bot được đặt trong bảng điều khiển tại mục **Cài đặt** → **Telegram Bot**. Sau khi thay đổi cài đặt, cần lưu lại và khởi động lại bảng điều khiển – bot được khởi tạo khi máy chủ web khởi động.

Trường (UI)	Khóa cài đặt	Giá trị mặc định	Mô tả
Bật Telegram bot	<code>tgBotEnable</code>	<code>false</code>	Công tắc chính. Chú thích: «Truy cập các chức năng bảng điều khiển qua Telegram bot». Khi tắt, bot không khởi động và các tác vụ thông báo không được lên lịch.
Telegram token	<code>tgBotToken</code>	(trống)	Token của bot. Chú thích: «Cần lấy token từ trình quản lý bot Telegram <code>@botfather</code> ». Không có token hợp lệ, bot sẽ không khởi động.

Trường (UI)	Khóa cài đặt	Giá trị mặc định	Mô tả
SOCKS proxy	<code>tgBotProxy</code>	(trống)	Proxy để kết nối với Telegram. Chú thích: «Nếu bạn cần proxy Socks5 để kết nối với Telegram, hãy cấu hình các tham số theo hướng dẫn».
Telegram API Server	<code>tgBotAPIServer</code>	(trống)	Máy chủ API Telegram thay thế. Chú thích: «Máy chủ API Telegram đang sử dụng. Để trống để dùng máy chủ mặc định».
User ID quản trị viên bot	<code>tgBotChatId</code>	(trống)	Một hoặc nhiều Telegram User ID của quản trị viên, phân cách bằng dấu phẩy. Chú thích: «Để lấy User ID, hãy dùng <code>@userinfobot</code> hoặc lệnh <code>/id</code> trong bot».
Tần suất thông báo cho quản trị viên từ bot	<code>tgRunTime</code>	<code>@daily</code>	Lịch báo cáo định kỳ theo định dạng crontab. Chú thích: «Nhập khoảng thời gian thông báo theo định dạng Crontab».
Sao lưu cơ sở dữ liệu	<code>tgBotBackup</code>	<code>false</code>	Chú thích: «Gửi thông báo kèm file sao lưu cơ sở dữ liệu». Đính kèm bản sao lưu vào báo cáo định kỳ.
Thông báo đăng nhập	<code>tgBotLoginNotify</code>	<code>true</code>	Chú thích: «Hiển thị tên người dùng, địa chỉ IP và thời gian khi ai đó cố gắng đăng nhập vào bảng điều khiển của bạn».
Ngưỡng tải CPU để thông báo	<code>tgCpu</code>	<code>80</code>	Ngưỡng tải CPU tính theo phần trăm (hợp lệ từ 0–100). Chú thích: «Thông báo cho quản trị viên qua Telegram nếu tải CPU vượt quá ngưỡng này (giá trị: %). Khi giá trị là 0, việc kiểm tra CPU bị tắt».
Ngôn ngữ Telegram bot	—	—	Ngôn ngữ mà bot dùng để soạn tất cả thông báo.

Lấy token qua @BotFather

- Mở hội thoại với **@BotFather** trong Telegram.
- Gửi lệnh `/newbot` và làm theo hướng dẫn (tên bot và `username` duy nhất kết thúc bằng `bot`).
- BotFather sẽ cung cấp token dạng `123456789:AA...`. Sao chép vào trường **Telegram token**.

Lấy User ID quản trị viên

User ID là mã định danh số của tài khoản (không phải username). Có hai cách để biết:

- Nhắn tin cho bot **@userinfobot**.
- Khởi động bot đã được cấu hình và gửi lệnh `/id` — bot sẽ trả về ID của bạn.

Nhập số nhận được vào trường **User ID quản trị viên bot**. Để chỉ định nhiều quản trị viên, hãy liệt kê ID của họ cách nhau bằng dấu phẩy (ví dụ: `11111111,22222222`). Mỗi ID được kiểm tra là số nguyên; giá trị không hợp lệ sẽ gây lỗi khi khởi động bot.

Ví dụ: giá trị trường «User ID quản trị viên bot». Một quản trị viên — chỉ là một số:

123456789

Hai quản trị viên cách nhau bằng dấu phẩy (có thể không cần khoảng trắng):

123456789,987654321

Mỗi giá trị phải là số nguyên. Dạng @username hoặc 123 456 (có khoảng trắng bên trong số) sẽ không hoạt động – bot sẽ không khởi động.

Proxy

Hỗ trợ các giao thức socks5://, http:// và https://. Nếu trường proxy để trống, bot sẽ cố dùng proxy chung của bảng điều khiển (nếu đã được đặt và giao thức được hỗ trợ). URL có giao thức không được hỗ trợ hoặc cú pháp không hợp lệ sẽ bị bỏ qua – bot sẽ kết nối trực tiếp. Proxy hữu ích khi truy cập trực tiếp vào API Telegram từ máy chủ bị chặn.

Thông báo qua email (SMTP)

Ngoài Telegram, cùng các sự kiện đó cũng có thể được nhận qua email. Kênh này được cấu hình trong mục **Cài đặt** → **Email** trên tab **SMTP Settings**:

Trường (UI)	Khóa cài đặt	Giá trị mặc định	Mô tả
Enable Email Notifications	smtpEnable	false	Công tắc chính cho thông báo email qua SMTP.
SMTP Host	smtpHost	(trống)	Máy chủ SMTP (ví dụ: smtp.gmail.com).
SMTP Port	smtpPort	587	Cổng máy chủ SMTP.
SMTP Username	smtpUsername	(trống)	Tên người dùng để xác thực SMTP. Cũng được dùng làm địa chỉ người gửi (From).
SMTP Password	smtpPassword	(trống)	Mật khẩu để xác thực SMTP. Được lưu ẩn; nếu mật khẩu đã được đặt, trường hiển thị trạng thái «đã cấu hình» và có thể để trống để giữ nguyên mật khẩu hiện tại.
Recipients	smtpTo	(trống)	Danh sách người nhận phân cách bằng dấu phẩy (ví dụ: admin@example.com, ops@example.com).
Encryption	smtpEncryptionType	starttls	Loại mã hóa kết nối: none (không mã hóa), starttls (STARTTLS) hoặc tls (TLS ngầm định).

Nút **Send Test Email** gửi email thử nghiệm và hiển thị kết quả theo từng bước: **Connection** (kết nối), **Authentication** (xác thực) và **Send** (gửi). Nếu có sự cố, thông tin chẩn đoán chỉ ra bước nào xảy ra lỗi (ví dụ: «Authentication failed – check username and password» hoặc «Server requires STARTTLS – change encryption type»), giúp dễ dàng điều chỉnh tham số.

Trên tab thứ hai (**Notifications**) có thể chọn các sự kiện sẽ được gửi qua email – bằng cùng các nhóm thẻ như đối với Telegram (xem «Hệ thống sự kiện và lựa chọn thông báo» trong mục 14.5).

Máy chủ API Telegram

Theo mặc định, bot kết nối với API Telegram chính thức. Trong trường **Telegram API Server**, có thể chỉ định địa chỉ máy chủ Bot API riêng (telegram-bot-api). URL được kiểm tra độ an toàn; địa chỉ bị chặn hoặc không hợp lệ sẽ bị bỏ qua, và máy chủ mặc định sẽ được sử dụng.

14.2. Menu chính và các nút

Menu được gọi bằng lệnh **/start** . Các nút là bàn phím inline gắn với tin nhắn; bộ nút hiển thị phụ thuộc vào việc bạn là quản trị viên hay client.

Menu quản trị viên

Nút	Hành động
 Báo cáo sử dụng lưu lượng đã sắp xếp	Liệt kê tất cả client được sắp xếp theo lưu lượng, với mức tiêu thụ của từng người; các email « <i>đư thừa</i> » không có dữ liệu được đánh dấu « Không có kết quả».
 Trạng thái máy chủ	Tổng quan về máy chủ (xem mục 14.5). Nút «  Làm mới» cập nhật dữ liệu.
 Đặt lại toàn bộ lưu lượng	Đặt lại bộ đếm lưu lượng của tất cả client. Yêu cầu xác nhận (« <i>Bạn có chắc không?</i> »), sau đó hiển thị «  Thành công» hoặc «  Thất bại» cho từng client, cuối cùng là «  Đặt lại lưu lượng hoàn tất cho tất cả client».
 Sao lưu DB	Gửi file cơ sở dữ liệu và config.json (xem mục 14.6).
 Nhật ký ban	Gửi các file nhật ký các địa chỉ IP bị cấm do vượt giới hạn IP.
 Kết nối inbound	Tổng quan về tất cả inbound: Remark, cổng, lưu lượng, số lượng client, ngày hết hạn.
 Sắp hết hạn	Danh sách các inbound và client sắp hết lưu lượng hoặc hết hạn (xem mục 14.5).
 Lệnh	Hiển thị trợ giúp về các lệnh quản trị viên.
 Trực tuyến	Số lượng và danh sách client đang trực tuyến; nhấn vào email để mở thẻ client. Nút «  Làm mới».
 Tất cả client	Mở danh sách chọn inbound, sau đó hiển thị danh sách client của nó – để xem/quản lý.
 Client mới	Khởi động trình hướng dẫn thêm client (chọn inbound → bản nháp → xác nhận).
 Cài đặt subscription / liên kết riêng / QR code	Chọn inbound và client để lấy liên kết subscription, liên kết riêng hoặc QR code.

Menu client

Client chỉ có bộ nút hạn chế:

Nút	Hành động
 Thông kê client	Hiển thị dữ liệu cho tất cả subscription liên kết với Telegram User ID của client.
 Lệnh	Hiển thị trợ giúp về các lệnh client.

Nút	Hành động
Cài đặt subscription	Chọn client của mình → liên kết subscription.
Liên kết riêng	Chọn client của mình → liên kết riêng.
QR code	Chọn client của mình → QR code.

Nếu người dùng không có client nào với Telegram User ID của họ, bot trả lời: «  Không tìm thấy cấu hình của bạn!  Vui lòng yêu cầu quản trị viên sử dụng Telegram User ID của bạn trong cấu hình.  User ID của bạn: ...». ID này cần được cung cấp cho quản trị viên để nhập vào trường của client.

14.3. Lệnh bot

Bốn lệnh được đăng ký cho bot, hiển thị trong menu «/» của Telegram:

Lệnh	Mô tả (từ menu)	Quyền truy cập	Chức năng
/start	Hiển thị menu chính	tất cả	Chào mừng; quản trị viên còn thấy thêm «  Chào mừng đến với bot quản lý!» và menu chính.
/help	Trợ giúp về bot	tất cả	Hiển thị lời chào chung và đề nghị chọn mục menu.
/status	Kiểm tra trạng thái bot	tất cả	Trả lời «  Bot đang hoạt động bình thường».
/id	Hiển thị Telegram ID của bạn	tất cả	Trả về «  User ID của bạn: ... ». Tiện lợi để lấy User ID của chính mình.

Ngoài các lệnh đã đăng ký, còn có thêm ba lệnh đối số được xử lý (không hiển thị trong menu «/» nhưng hoạt động bình thường):

- **/usage [Email]** – tìm kiếm client theo email.
 - Đối với **quản trị viên** – hiển thị thẻ client đầy đủ (với các nút quản lý).
 - Đối với **client** – chỉ hiển thị subscription của chính họ với email được chỉ định (theo liên kết Telegram User ID). Không có đối số, bot yêu cầu cung cấp email: «  Vui lòng chỉ định email để tìm kiếm».
- **/inbound [tên kết nối]** – chỉ dành cho quản trị viên. Tìm kiếm inbound theo Remark và hiển thị các tham số cùng thống kê của tất cả client. Không có đối số (hoặc đối với client) – «  Lệnh không xác định».
- **/restart** – chỉ dành cho quản trị viên. Khởi động lại Xray Core. Các phản hồi có thể: «  Xray Core đã khởi động lại thành công», «  Xray Core không chạy» (nếu nhân không hoạt động), «  Lỗi khi khởi động lại Xray core. <Lỗi>». Bất kỳ đối số nào sau /restart đều dẫn đến thông báo lệnh không xác định kèm gợi ý /restart.

Trong chat nhóm, lệnh dạng /lệnh@botusername chỉ được xử lý nếu username trùng với tên bot hiện tại.

Trợ giúp quản trị viên (nút «Lệnh»):

 Để khởi động lại Xray Core: `/restart`
 Để tìm kiếm client theo email: `/usage [Email]`
 Để tìm kiếm kết nối inbound (với thông kê client): `/inbound [tên kết nối]`
 Telegram User ID của bạn: `/id`

Trợ giúp client:

 Để xem thông tin subscription của bạn: `/usage [Email]`
 Telegram User ID của bạn: `/id`

14.4. Quản lý client (chỉ quản trị viên)

Sau khi mở thẻ client (qua «Tắt cả client», «Trực tuyến», «Sắp hết hạn» hoặc `/usage`), quản trị viên thấy thông tin về client (email, các inbound liên kết, trạng thái «Đang hoạt động», trạng thái kết nối, ngày hết hạn, mức tiêu thụ lưu lượng) và các nút inline quản lý:

Nút	Chức năng
 Làm mới	Tải lại thẻ client.
 Đặt lại lưu lượng	Đặt lại bộ đếm lưu lượng của client. Yêu cầu xác nhận «  Xác nhận đặt lại lưu lượng? ».
 Giới hạn lưu lượng	Đặt giới hạn lưu lượng. Các giá trị có sẵn: ☹ Không giới hạn (0), 1/5/10/20/30/40/50/60/80/100/150/200 GB hoặc «  Tùy chỉnh » – nhập số bằng bàn phím số tích hợp (nút 0–9, «  » – đặt lại về 0, «  » – xóa chữ số cuối, «  Xác nhận: N »). Giá trị được đặt tính bằng gigabyte.
 Thay đổi ngày hết hạn	Các tùy chọn có sẵn: ☹ Không giới hạn, «  Tùy chỉnh », thêm 7/10/14/20 ngày, 1/3/6/12 tháng. Số dương kéo dài thời hạn (cộng thêm ngày vào ngày hết hạn hiện tại hoặc «bây giờ» nếu đã hết hạn); 0 xóa giới hạn thời hạn.
 Nhật ký IP	Hiển thị các địa chỉ IP đã ghi nhận của client (kèm dấu thời gian nếu có). Từ nhật ký có thể truy cập «  Làm mới » và «  Xóa IP » (với xác nhận «  Xác nhận xóa IP? »).
 Giới hạn IP	Giới hạn số IP đồng thời. Các tùy chọn: ☹ Không giới hạn (0), 1–10 hoặc «  Tùy chỉnh » (bàn phím số).
 Đặt người dùng Telegram	Hiển thị Telegram User ID hiện tại được liên kết với client; cho phép xóa liên kết («  Xóa người dùng Telegram » với xác nhận). Việc liên kết người dùng mới được thực hiện thông qua hệ thống chọn liên lạc Telegram.
 Bật/Tắt	Bật hoặc tắt client. Yêu cầu xác nhận «  Xác nhận bật/tắt người dùng? ».

Tất cả các thao tác thay đổi cấu hình (giới hạn lưu lượng/IP, ngày hết hạn, liên kết/hủy liên kết người dùng Telegram, bật/tắt) sẽ đánh dấu Xray để khởi động lại khi cần thiết, để các thay đổi có hiệu lực. Sau khi thao tác thành công, bot hiển thị xác nhận dạng «  : ... » và hiển thị lại thẻ.

Mọi nhập số trong các trình hướng dẫn được giới hạn ở giá trị < 999999.

14.5. Thông báo và báo cáo

Thông báo được gửi đến tất cả quản trị viên (tất cả User ID trong `tgBotChatId`).

Hệ thống sự kiện và lựa chọn thông báo

Thông báo được xây dựng trên một hệ thống sự kiện thống nhất, với hai kênh phân phối — **Telegram** và **email (SMTP)**. Đối với mỗi kênh, có thể chọn riêng các sự kiện cần thông báo. Trong **Cài đặt** → **Telegram**, điều này được thực hiện trên tab **Notifications**, trong **Cài đặt** → **Email** — trên tab cùng tên.

Các sự kiện được nhóm thành thẻ; mỗi nhóm có công tắc chính kèm bộ đếm sự kiện đã bật (n/tổng cộng) và trạng thái trung gian khi chỉ chọn một phần. Các nhóm có sẵn:

- **Outbound** — «Down» (`outbound.down`) và «Up» (`outbound.up`): outbound bị ngừng và phục hồi.
- **Xray Core** — «Crash» (`xray.crash`): nhân Xray bị sập bất thường.
- **Nodes** — «Down» (`node.down`) và «Up» (`node.up`): nút không còn khả dụng hoặc đã phục hồi.
- **System** — «CPU high (%)» (`cpu.high`) và «Memory high (%)» (`memory.high`): tải CPU và RAM cao. Cả hai sự kiện đều có trường inline ngưỡng tính theo phần trăm bên cạnh.
- **Security** — «Login attempt» (`login.attempt`): cố gắng đăng nhập vào bảng điều khiển.

Bộ sự kiện được bật lưu riêng biệt: cho Telegram — trong `tgEnabledEvents` , cho Email — trong `smtpEnabledEvents` . Theo mặc định, cả hai kênh đều bật «Login attempt» và «CPU high» (giá trị `login.attempt,cpu.high`).

Thông báo đăng nhập vào bảng điều khiển

Được kiểm soát bằng ô chọn **Thông báo đăng nhập** (`tgBotLoginNotify` , mặc định bật). Mỗi lần cố gắng đăng nhập vào bảng điều khiển web, quản trị viên nhận được tin nhắn:

- Khi thành công: «  Đăng nhập vào bảng điều khiển thành công.» + máy chủ, tên người dùng, IP, thời gian.
- Khi thất bại: «  Lỗi đăng nhập vào bảng điều khiển.» + máy chủ, **lý do** (ví dụ: «Lỗi 2FA» khi nhập sai yêu tố thứ hai), tên người dùng, IP, thời gian.

Vượt ngưỡng tải CPU và bộ nhớ

Mỗi phút, bảng điều khiển kiểm tra tải CPU và RAM. Nếu ngưỡng `tgCpu` > 0 và tải CPU trung bình trong một phút vượt qua ngưỡng đó, quản trị viên nhận được: «  Tải CPU là N%, vượt ngưỡng M%». Tương tự, tải RAM được kiểm tra theo ngưỡng `tgMemory` (mặc định 80%) — sự kiện «Memory high (%)».

Cả hai ngưỡng được đặt bằng các trường inline bên cạnh sự kiện «CPU high (%)» và «Memory high (%)» trong nhóm **System** trên tab Notifications (xem «Hệ thống sự kiện và lựa chọn thông báo» bên dưới). Đối với kênh Email, các khóa riêng biệt `smtpCpu` và `smtpMemory` được áp dụng. Khi giá trị ngưỡng là 0, việc kiểm tra tương ứng bị tắt.

Báo cáo định kỳ (theo lịch)

Được lên lịch theo biểu thức cron từ trường **Tần suất thông báo** (`tgRunTime` , mặc định `@daily`). Nếu giá trị trống hoặc không hợp lệ, `@daily` được sử dụng. Báo cáo bao gồm:

Trình tạo lịch

Trường **Tần suất thông báo cho quản trị viên từ bot** được đặt không phải bằng cách nhập chuỗi thủ công mà thông qua trình tạo lịch. Trước tiên, chọn chế độ trong danh sách thả xuống:

- **@every** – **lặp lại theo khoảng thời gian** – xuất hiện trường số và lựa chọn đơn vị (**Giây / Phút / Giờ**); kết quả được tổng hợp thành biểu thức dạng `@every 6h`.
- **@hourly** – **mỗi giờ**, **@daily** – **mỗi ngày lúc 00:00**, **@weekly** – **mỗi tuần**, **@monthly** – **mỗi tháng** – các preset có sẵn, được lưu dưới dạng macro tương ứng (`@hourly`, `@daily`, `@weekly`, `@monthly`).
- **Tùy chỉnh (crontab)** – trường để nhập biểu thức crontab riêng. Bộ lập lịch của bảng điều khiển hoạt động với giây được bật, vì vậy biểu thức tùy chỉnh bao gồm **6 trường**: giây, phút, giờ, ngày tháng, tháng, ngày tuần (ví dụ: `0 30 8 * * *` – mỗi ngày lúc 08:30:00). Khi chuyển sang chế độ này, trường được điền tương đương crontab của lựa chọn hiện tại để có điểm khởi đầu.

Ví dụ: giá trị trường «Tần suất thông báo» (`tgRunTime`). Hỗ trợ cả viết tắt có sẵn và định dạng crontab đầy đủ:

Giá trị	Thời điểm kích hoạt
<code>@daily</code>	một lần mỗi ngày vào nửa đêm (giá trị mặc định)
<code>@hourly</code>	mỗi giờ
<code>@every 6h</code>	mỗi 6 giờ
<code>0 9 * * *</code>	mỗi ngày lúc 09:00
<code>0 9 * * 1</code>	mỗi thứ Hai lúc 09:00
<code>0 */12 * * *</code>	mỗi 12 giờ (lúc 00:00 và 12:00)

Thứ tự trường trong crontab: phút, giờ, ngày tháng, tháng, ngày tuần.

1. Dòng « Báo cáo đã lên lịch: <lich>» và ngày/giờ hiện tại.
2. **Trạng thái máy chủ** (xem bên dưới).
3. Khối «Sắp hết hạn» theo inbound và client.
4. Thông báo cá nhân cho các client có Telegram User ID liên kết – mỗi client không phải quản trị viên nhận danh sách subscription của mình sắp hết lưu lượng/hết hạn (kể cả những cái đã tắt).
5. Nếu bật **Sao lưu cơ sở dữ liệu** (`tgBotBackup`) – bản sao lưu DB cho quản trị viên.

Trạng thái máy chủ chứa: máy chủ, phiên bản 3X-UI và Xray, IPv4/IPv6, thời gian hoạt động (tính bằng ngày), tải trung bình (Load1/2/3), RAM (hiện tại/tổng cộng), số lượng client trực tuyến, số lượng kết nối TCP/UDP, tổng lưu lượng mạng (↑/↓) và trạng thái Xray.

«**Sắp hết hạn**» hiển thị:

- theo inbound: số lượng bị tắt và số lượng «sắp hết», sau đó liệt kê các inbound đó (Remark, cổng, lưu lượng, ngày hết hạn);
- theo client: tương tự, cộng thêm thẻ client và nút với email của họ (nhấn để mở thẻ client).

Ngưỡng «sắp hết» được lấy từ cài đặt chung của bảng điều khiển: dự phòng lưu lượng (tính bằng GB) và dự phòng thời hạn (tính bằng ngày). Một inbound/client được coi là «sắp hết» khi lưu lượng còn lại đến giới hạn nhỏ hơn ngưỡng HOẶC thời gian còn lại đến ngày hết hạn nhỏ hơn ngưỡng.

14.6. Sao lưu và nhật ký

- **Sao lưu DB** (nút « Sao lưu DB» hoặc ô chọn trong báo cáo định kỳ): bot gửi thời gian sao lưu, file cơ sở dữ liệu (`x-ui.db` , hoặc `x-ui.dump` cho PostgreSQL) và file cấu hình Xray `config.json` .
- **Nhật ký ban** (nút « Nhật ký ban»): gửi file nhật ký hiện tại và trước đó về các địa chỉ IP bị cấm do vượt giới hạn IP. Các file trống không được gửi.

14.7. Đặc điểm hoạt động

- **Tin nhắn dài** được chia thành các phần (ngưỡng ~2000 ký tự), bàn phím inline được gắn vào phần cuối cùng.
- **Tính song song**: các lệnh và nhấn nút được xử lý đồng thời (nhóm lên đến 10 trình xử lý đồng thời).
- **Độ tin cậy gửi tin**: khi có lỗi kết nối, tin nhắn được gửi lại với độ trễ tăng dần theo cấp số nhân (1s/2s/4s, tối đa 3 lần thử).
- **Bộ nhớ đệm**: dữ liệu «Trạng thái máy chủ» được lưu vào bộ nhớ đệm để các lần nhấn «Làm mới» thường xuyên không gây tải cho hệ thống.
- **Khởi động lại bot**: khi lưu cài đặt và khởi động lại bảng điều khiển, vòng lặp thăm dò trước đó dừng lại đúng cách và vòng lặp mới được khởi động – chỉ có một phiên bản nhận cập nhật hoạt động tại một thời điểm.

15. Cơ sở dữ liệu địa lý (geoip / geosite và tùy chỉnh)

Cơ sở dữ liệu địa lý là các tệp nhị phân `.dat` mà Xray-core sử dụng để định tuyến và lọc lưu lượng theo quốc gia (dải IP) hoặc theo danh mục tên miền. Panel có khả năng tải và cập nhật cả bộ tệp địa lý tiêu chuẩn lẫn các nguồn tùy chỉnh tùy ý do người dùng chỉ định qua URL. Tất cả tệp được lưu trong thư mục `bin` cạnh tệp nhị phân Xray (đường dẫn mặc định `bin`, có thể ghi đè bằng biến môi trường `XUI_BIN_FOLDER`).

15.1. geoip.dat và geosite.dat là gì

- **geoip.dat** – cơ sở dữ liệu ánh xạ «địa chỉ IP → mã quốc gia/khu vực». Được sử dụng trong các quy tắc định tuyến dưới dạng `geoip:<mã>`, ví dụ `geoip:ru`, `geoip:cn`, cũng như cho các nhân đặc biệt `geoip:private` (mạng riêng tư/cục bộ). Về bản chất đây là câu trả lời cho câu hỏi «IP này thuộc quốc gia nào».
- **geosite.dat** – cơ sở dữ liệu ánh xạ «tên miền → danh mục/danh sách». Được sử dụng dưới dạng `geosite:<danh mục>`, ví dụ `geosite:category-ads-all` (tên miền quảng cáo), `geosite:google`, `geosite:ru`. Về bản chất đây là các danh sách tên miền được nhóm lại.

Các tệp này cần thiết để xây dựng các quy tắc kiểu «toàn bộ lưu lượng đến IP/tên miền của Nga – kết nối trực tiếp, còn lại – qua outbound» và các quy tắc tương tự. Bản thân các quy tắc được thiết lập trong phần định tuyến của Xray; cơ sở dữ liệu địa lý chỉ cung cấp dữ liệu cho chúng. Nếu không có tệp địa lý cập nhật, các quy tắc tham chiếu đến `geoip:` / `geosite:` sẽ không hoạt động hoặc sẽ dựa vào các danh sách lỗi thời.

Ví dụ: quy tắc «tên miền và IP của Nga – kết nối trực tiếp». Quy tắc này trong phần định tuyến chuyển toàn bộ lưu lượng đến tài nguyên của Nga sang outbound có thể `direct`:

```
{
  "type": "field",
  "domain": ["geosite:category-ru"],
  "ip": ["geoip:ru"],
  "outboundTag": "direct"
}
```

15.2. Tệp địa lý tiêu chuẩn và cách cập nhật

Panel chứa danh sách cho phép (allowlist) cố định gồm sáu tệp tiêu chuẩn với các nguồn tải đã được mã hóa cứng. Việc cập nhật được thực hiện qua `POST /panel/api/server/updateGeofile/:fileName` (hoặc không có tên tệp – để cập nhật tất cả cùng lúc).

Ví dụ: cập nhật một tệp và tất cả tệp qua API. Chỉ cập nhật `geoip_RU.dat`:

```
curl -X POST 'https://panel.example.com:2053/panel/api/server/updateGeofile/
geoip_RU.dat' \
-H 'Cookie: 3x-ui=<session-cookie>'
```

Cập nhật tất cả sáu tệp tiêu chuẩn trong một yêu cầu (không chỉ định tên tệp):

```
curl -X POST 'https://panel.example.com:2053/panel/api/server/updateGeofile' \
-H 'Cookie: 3x-ui=<session-cookie>'
```

Phản hồi thành công:

```
{ "success": true, "msg": "Geofile updated successfully", "obj": null }
```

Tên tệp	Nguồn (kho phát hành)
geoup.dat	github.com/Loyalsoldier/v2ray-rules-dat (geoup.dat)
geosite.dat	github.com/Loyalsoldier/v2ray-rules-dat (geosite.dat)
geoup_IR.dat	github.com/chocolate4u/iran-v2ray-rules (geoup.dat)
geosite_IR.dat	github.com/chocolate4u/iran-v2ray-rules (geosite.dat)
geoup_RU.dat	github.com/runetfreedom/russia-v2ray-rules-dat (geoup.dat)
geosite_RU.dat	github.com/runetfreedom/russia-v2ray-rules-dat (geosite.dat)

Đặc điểm cập nhật tệp tiêu chuẩn:

- **Nút cập nhật một tệp.** Trước khi tải xuống sẽ hiển thị xác nhận: «Bạn có thực sự muốn cập nhật tệp địa lý không?» với ghi chú «Thao tác này sẽ cập nhật tệp #filename#.» (tiếng Anh: *Do you really want to update the geofile? This will update the #filename# file.*). Khi thành công sẽ hiển thị thông báo «Tệp địa lý đã được cập nhật thành công» (tiếng Anh: *Geofile updated successfully*).
- **Nút «Update all»** (cập nhật tất cả) tải xuống tất cả sáu tệp. Xác nhận: «Thao tác này sẽ cập nhật tất cả tệp địa lý.» (tiếng Anh: *This will update all geofiles.*).
- **Tải xuống có điều kiện.** Nếu tệp cục bộ đã tồn tại, yêu cầu sẽ gửi kèm header `If-Modified-Since` với thời gian sửa đổi của tệp. Phản hồi `304 Not Modified` từ máy chủ có nghĩa là tệp không thay đổi – tệp sẽ không được tải lại, chỉ cập nhật mốc thời gian của tệp.
- **Bảo mật tên tệp.** Chỉ chấp nhận các tên có trong allowlist; tên được kiểm tra không chứa `..`, dấu phân cách đường dẫn `/` và `\`, đường dẫn tuyệt đối và phải khớp với mẫu `^[a-zA-Z0-9._-]+\..dat$`. Bất kỳ tên nào ngoài danh sách sẽ bị từ chối với lỗi «Invalid geofile name».
- **Khởi động lại Xray.** Sau khi tải tệp địa lý, Xray-core sẽ được khởi động lại để đọc lại các cơ sở dữ liệu đã cập nhật. Nếu không thể khởi động lại, thông báo lỗi sẽ chứa dòng tương ứng.

Cập nhật cơ sở dữ liệu địa lý từ dòng lệnh (x-ui)

Cơ sở dữ liệu địa lý cũng có thể được cập nhật mà không cần panel – thông qua menu tương tác `x-ui` (mục cập nhật tệp địa lý) hoặc bằng lệnh không tương tác `x-ui update-all-geofiles`. Đối với từng tệp trong bộ (geoup/geosite, bao gồm cả bộ IR và RU) sẽ hiển thị trạng thái riêng: «đã cập nhật», «đã là phiên bản mới nhất» hoặc «lỗi tải xuống». Khi tải xuống thất bại sẽ không in thông báo thành công giả. Việc khởi động lại Xray (và do đó ngắt các kết nối đang hoạt động) chỉ xảy ra nếu có ít nhất một tệp thực sự được cập nhật; nếu không có tệp nào thay đổi (tất cả đều trả về `304 Not Modified`), panel và Xray sẽ không được khởi động lại.

15.3. Tự động cập nhật dữ liệu địa lý bằng Xray (Geodata Auto-Update)

Các nguồn `.dat` bổ sung theo URL tùy ý được thêm không phải qua panel mà thông qua phần `geodata` gốc của Xray-core. Phần tương ứng được đặt trong cửa sổ modal cập nhật Xray (Dashboard → cập nhật Xray, `xrayUpdates`) – đây là tab «Geodata Auto-Update» (Tự động cập nhật Geodata). Panel ở đây chỉ chỉnh sửa khóa `geodata` trong mẫu cấu hình Xray; việc tải xuống, kiểm tra và tải lại nóng các tệp do bản thân lõi Xray thực hiện.

Ở phần trên của mục hiển thị gợi ý: «Xray tải xuống các tệp này theo lịch và tải lại nóng mà không cần khởi động lại. URL phải là HTTPS. Tệp phải đã tồn tại trong thư mục bin trước khi Xray có thể cập nhật nó.» (tiếng Anh: *Xray downloads these files on schedule and hot-reloads them without a restart. URLs must be HTTPS. Each file must already exist in the bin folder once before Xray can update it.*).

Các trường của mục

- **Schedule (cron)** (Lịch cron) – chuỗi cron gồm 5 trường; giá trị mặc định `0 4 * * *` (hàng ngày lúc 04:00). Khi lưu sẽ kiểm tra rằng chuỗi chứa đúng 5 trường, nếu không sẽ hiển thị lỗi «Cron phải chứa 5 trường, ví dụ `0 4 * * *`».
- **Download through outbound (optional)** (Tải xuống qua outbound (tùy chọn)) – danh sách thả xuống với các thẻ outbound có sẵn (cộng outbound của subscription), qua đó Xray sẽ tải các tệp; các outbound có giao thức `blackhole` không xuất hiện trong danh sách. Trường có thể để trống – khi đó sẽ sử dụng kết nối trực tiếp. Lựa chọn này độc lập với outbound cho các yêu cầu của chính panel (xem §11): tự động cập nhật geodata có outbound tải xuống riêng của nó.
- **Danh sách tệp** – mỗi dòng xác định một cặp «URL + File name» (Tên tệp). URL phải bắt đầu bằng `https://` (nếu không sẽ hiển thị «Cần có HTTPS URL cho mỗi tệp.»). Tên tệp chỉ định đơn giản, không có đường dẫn và dấu phân cách – chỉ các ký tự `^[A-Za-z0-9._-]+$` (nếu không sẽ hiển thị «Tên tệp phải đơn giản, ví dụ `geosite_custom.dat` (không có đường dẫn).»). Khi nhập URL, panel sẽ cố gắng tự động điền tên tệp từ phần cuối của đường dẫn. Nút «Add file» (Thêm tệp) thêm một dòng, nút thùng rác xóa nó.

Nếu danh sách trống, hiển thị gợi ý: «Chưa có tệp nào được cấu hình. Tham chiếu các tệp trong quy tắc định tuyến dưới dạng `ext:geosite_custom.dat:category`» (tiếng Anh: *No files configured. Reference files in routing rules as ext:geosite_custom.dat:category*).

Lưu

Nút «Save & Restart Xray» (Lưu và khởi động lại Xray) hiển thị xác nhận «Lưu cài đặt geodata?» với ghi chú «Mẫu cấu hình Xray sẽ được cập nhật và Xray sẽ được khởi động lại.» (tiếng Anh: *Save geodata settings? This updates the Xray config template and restarts Xray*). Sau khi lưu, khóa `geodata` được ghi vào mẫu cấu hình (`POST /panel/api/xray/update`) và Xray được khởi động lại (`POST /panel/api/server/restartXrayService`). Nếu danh sách tệp trống, khóa `geodata` sẽ bị xóa khỏi mẫu.

Các đặc điểm quan trọng:

- **Tệp phải đã tồn tại trong bin**. Xray chỉ cập nhật các tệp `.dat` đã có trong thư mục `bin` tại thời điểm khởi động. Do đó, tệp tùy chỉnh mới trước tiên phải được đặt vào `bin` theo cách thủ công (hoặc ít nhất tạo phiên bản trống/lỗi thời ở đó với tên cần thiết), và chỉ sau đó Xray mới bắt đầu duy trì nó theo lịch.

- **Tải lại nóng.** Sau khi tải xuống theo lịch, Xray sẽ đọc lại các cơ sở dữ liệu đã cập nhật mà không cần khởi động lại toàn bộ tiến trình.
- **Khả năng tương thích.** Các tệp địa lý đã tải xuống trước đó (cả tiêu chuẩn lẫn tùy chỉnh) tiếp tục hoạt động trong các quy tắc định tuyến với cú pháp `ext:` mà không có thay đổi.

Nếu danh sách trống, hiển thị gợi ý: «Chưa có nguồn địa lý tùy chỉnh nào – nhấn «Thêm» để tạo» (tiếng Anh: *No custom geo sources yet – click Add to create one*).

Các cột bảng và trường của nguồn

Trường (UI)	JSON	Giá trị mặc định	Mô tả
Type (Loại)	<code>type</code>	– (bắt buộc)	Loại tài nguyên: chỉ <code>geosite</code> hoặc <code>geoip</code> . Xác định tên tệp kết quả.
Alias (Bí danh)	<code>alias</code>	– (bắt buộc)	Định danh ngắn của nguồn. Tên tệp được tạo từ nó và loại.
URL (<i>URL</i>)	<code>url</code>	– (bắt buộc)	Liên kết trực tiếp đến tệp <code>.dat</code> (http / https).
Enabled (Đã bật)	–	–	Trạng thái hoạt động của nguồn trong danh sách.
Last updated (Cập nhật lần cuối)	<code>lastUpdatedAt</code>	<code>0</code>	Thời gian cập nhật thành công gần nhất (Unix time; <code>0</code> – chưa được cập nhật).
Routing (ext:...) (Định tuyến (ext:...))	–	–	Chuỗi sẵn sàng cho quy tắc định tuyến: <code>ext:<tệp.dat>:tag</code> .
Actions (Hành động)	–	–	Các nút «Chỉnh sửa», «Xóa», «Cập nhật ngay».

Ngoài ra, các trường dịch vụ được lưu trong cơ sở dữ liệu: `localPath` (đường dẫn thực tế đến tệp trong thư mục `bin`), `lastModified` (giá trị header `Last-Modified` từ máy chủ, được sử dụng cho tải xuống có điều kiện), `createdAt` và `updatedAt`.

Đặt tên tệp

Tên tệp kết quả được tạo tự động từ loại và bí danh:

- loại `geoip` → `geoip_<alias>.dat`;
- loại `geosite` → `geosite_<alias>.dat`.

Ví dụ, nguồn có loại `geosite` và bí danh `myads` sẽ tạo tệp `geosite_myads.dat`.

Ví dụ: thêm nguồn qua API. Thêm danh sách tên miền quảng cáo tùy chỉnh dưới dạng tài nguyên `geosite` với bí danh `myads`:


```
curl -X POST 'https://panel.example.com:2053/panel/api/server/customGeo/add' \
-H 'Cookie: 3x-ui=<session-cookie>' \
-H 'Content-Type: application/json' \
-d '{
  "type": "geosite",
  "alias": "myads",
  "url": "https://example.com/lists/myads.dat"
}'
```

Panel sẽ tải tệp vào thư mục `bin` dưới tên `geosite_myads.dat`, lưu bản ghi và khởi động lại Xray.

Các nút và hành động

- **Add** (Thêm) – mở biểu mẫu «Add custom geo» (Thêm nguồn tùy chỉnh). Nút lưu – «Save» (Lưu). API: `POST /add`.
- **Edit** (Chỉnh sửa) – biểu mẫu «Edit custom geo» (Chỉnh sửa nguồn tùy chỉnh). API: `POST /update/:id`. Khi thay đổi loại hoặc bí danh, tệp cũ sẽ bị xóa và tệp mới sẽ được tải xuống lại.
- **Delete** (Xóa) – xác nhận «Xóa nguồn tùy chỉnh này không?» (tiếng Anh: *Delete this custom geo source?*). Xóa bản ghi khỏi cơ sở dữ liệu và bản thân tệp `.dat`. API: `POST /delete/:id`. Khi thành công: «Tệp địa lý tùy chỉnh «<tên>» đã được xóa».
- **Update now** (Cập nhật ngay) – tải lại nguồn cụ thể và cập nhật mốc thời gian. API: `POST /download/:id`. Khi thành công: «Geofile «<tên>» đã được cập nhật».
- **Cập nhật tất cả** – cập nhật tất cả nguồn tùy chỉnh cùng lúc. API: `POST /update-all`. Khi hoàn toàn thành công: «Tất cả nguồn địa lý tùy chỉnh đã được cập nhật» (tiếng Anh: *All custom geo sources updated*). Nếu ít nhất một nguồn không cập nhật được, thao tác được coi là thành công một phần với thông báo «Không thể cập nhật một hoặc nhiều nguồn địa lý tùy chỉnh» (tiếng Anh: *One or more custom geo sources failed to update*), và phản hồi liệt kê các nguồn thành công và thất bại.

Sau bất kỳ hành động nào trong số này (thêm, chỉnh sửa, xóa, cập nhật, cập nhật tất cả khi có thành công) Xray-core sẽ được khởi động lại.

Từng bước: thêm nguồn

1. Nhấn «Add» (Thêm).
2. Trong trường «Type» (Loại) chọn `geosite` hoặc `geoip`.
3. Trong trường «Alias» (Bí danh) nhập định danh (chỉ chữ thường Latin, chữ số, `-` và `_`; gợi ý placeholder: `a-z 0-9 _ -`).
4. Trong trường «URL» chỉ định liên kết trực tiếp đến tệp `.dat` (phải bắt đầu bằng `http://` hoặc `https://`).
5. Nhấn «Save» (Lưu). Panel sẽ ngay lập tức tải tệp vào thư mục `bin`, lưu bản ghi và khởi động lại Xray.

15.4. Xác thực và giới hạn

Khi tạo và chỉnh sửa nguồn sẽ thực hiện các kiểm tra nghiêm ngặt. Thông báo lỗi:

Điều kiện	Thông báo (RU)	Thông báo (EN)
Loại không phải <code>geosite</code> / <code>geoip</code>	Тип должен быть geosite или geoip	<i>Type must be geosite or geoip</i>
Bí danh trống	Укажите псевдоним	<i>Alias is required</i>
Ký tự không hợp lệ trong bí danh (không khớp <code>^[a-z0-9_-]+\$</code>)	Псевдоним содержит недопустимые символы	<i>Alias must match allowed characters</i>
Bí danh bị dành riêng	Этот псевдоним зарезервирован	<i>This alias is reserved</i>
URL trống	Укажите URL	<i>URL is required</i>
URL không phân tích được	Некорректный URL	<i>URL is invalid</i>
Scheme không phải <code>http/https</code>	URL должен использовать http или https	<i>URL must use http or https</i>
Host trống/không hợp lệ hoặc bị chặn bởi bảo vệ SSRF	Некорректный хост URL	<i>URL host is invalid</i>
Trùng lặp «loại + bí danh»	Такой псевдоним уже используется для этого типа	<i>This alias is already used for this type</i>
Không tìm thấy nguồn	Источник не найден	<i>Custom geo source not found</i>
Lỗi tải xuống	Ошибка загрузки	<i>Download failed</i>

Gợi ý trong biểu mẫu (xác thực phía client): «Bí danh: chỉ a-z, chữ số, - và _» (*Alias may only contain lowercase letters, digits, - and _*) và «URL phải bắt đầu bằng `http://` hoặc `https://`» (*URL must start with http:// or https://*).

Các giới hạn kỹ thuật bổ sung:

- **Bí danh bị dành riêng.** Không thể sử dụng các bí danh xung đột với tệp tiêu chuẩn. Bị dành riêng (so sánh không phân biệt chữ hoa/thường, dấu gạch ngang được coi tương đương với dấu gạch dưới): `geoip`, `geosite`, `geoip_ir`, `geosite_ir`, `geoip_ru`, `geosite_ru`. Ví dụ, `geosite-ru` sẽ bị từ chối như `geosite_ru`.
- **Bảo vệ SSRF.** Host URL được phân giải thành IP, và nếu nó trở đến địa chỉ riêng tư/nội bộ, việc tải xuống sẽ bị chặn (người dùng thấy «Host URL không hợp lệ»). Điều này ngăn việc sử dụng panel để truy cập các dịch vụ nội bộ.
- **Bảo vệ path traversal.** Đường dẫn cuối cùng của tệp phải nằm trong thư mục `bin` (với việc giải quyết symlink); mọi nỗ lực thoát ra ngoài đều bị từ chối.
- **Kích thước tệp tối thiểu.** Tệp tải xuống chỉ được coi là hợp lệ nếu nó không nhỏ hơn 64 byte; tệp quá nhỏ sẽ bị từ chối với lỗi tải xuống.
- **Proxy và tải xuống có điều kiện.** Nếu cài đặt panel có cấu hình proxy, việc tải xuống sẽ đi qua đó; trong các trường hợp khác sử dụng kết nối trực tiếp với transport an toàn SSRF. Cũng như với tệp tiêu chuẩn, sử dụng `If-Modified-Since / 304 Not Modified` (tệp không thay đổi sẽ không được tải lại). Timeout tải xuống – 10 phút, kiểm tra khả năng truy cập URL (HEAD, nếu cần – GET một phần) – 12 giây.

15.5. Kiểm tra tự động khi khởi động panel

Khi khởi động, panel duyệt qua tất cả nguồn tùy chỉnh và kiểm tra sự tồn tại và tính toàn vẹn của tệp cục bộ cho từng nguồn (tệp không tồn tại, là thư mục hoặc nhỏ hơn 64 byte). Nếu tệp bị thiếu hoặc bị hỏng, sẽ thực hiện kiểm tra nguồn và thử tải xuống lại. Điều này đảm bảo rằng sau khi cài đặt lại hoặc mất thư mục `bin`, các tệp địa lý tùy chỉnh sẽ được khôi phục tự động.

15.6. Sử dụng cơ sở dữ liệu địa lý trong quy tắc định tuyến

Trong các quy tắc định tuyến Xray, cơ sở dữ liệu địa lý được sử dụng trong các trường như `domain / ip` qua các tiền tố:

- **geoip:** cho cơ sở dữ liệu IP — `geoip:<mã>`. Ví dụ: `geoip:ru`, `geoip:cn`, `geoip:private`. Lấy từ `geoip.dat` (hoặc `geoip_RU.dat` v.v., nếu quy tắc trỏ đến tệp cụ thể).
- **geosite:** cho cơ sở dữ liệu tên miền — `geosite:<danh mục>`. Ví dụ: `geosite:category-ads-all`, `geosite:google`, `geosite:ru`. Lấy từ `geosite.dat`.

Ví dụ: chặn quảng cáo qua geosite. Quy tắc gửi tất cả tên miền quảng cáo vào «lỗ đen» (giả sử có outbound với thẻ `blocked` và giao thức `blackhole`):

```
{
  "type": "field",
  "domain": ["geosite:category-ads-all"],
  "outboundTag": "blocked"
}
```

Đối với các tệp **tùy chỉnh** sử dụng cú pháp tệp ngoài `ext:`. Gợi ý trong UI: «Trong quy tắc định tuyến hãy sử dụng giá trị như `ext:tệp.dat:tag` (thay thế tag).» (tiếng Anh: *In routing rules use the value column as ext:file.dat:tag (replace tag)*). Định dạng:

```
ext:<tên_tệp.dat>:<tag>
```

trong đó `<tên_tệp.dat>` — là `geoip_<alias>.dat` hoặc `geosite_<alias>.dat`, còn `<tag>` — danh sách/danh mục cụ thể bên trong tệp. Panel trong cột «Routing (ext:...» (Định tuyến (ext:...)) hiển thị mẫu sẵn sàng dạng `ext:geosite_myads.dat:tag` — chỉ cần thay `tag` bằng thẻ cần thiết. Tên tệp như vậy được chỉ định trong mục «Geodata Auto-Update» (xem §15.3) trong trường «File name» (Tên tệp) — ví dụ `geosite_custom.dat`; tham chiếu đến nó trong quy tắc dưới dạng `ext:geosite_custom.dat:category`.

Ví dụ: quy tắc dựa trên tệp tùy chỉnh. Nếu đã thêm nguồn loại `geosite` với bí danh `myads`, và danh sách bên trong tệp `.dat` được đánh dấu bằng thẻ `ads`, quy tắc định tuyến trông như sau:

```
{
  "type": "field",
  "domain": ["ext:geosite_myads.dat:ads"],
  "outboundTag": "blocked"
}
```

Đối với nguồn IP (loại `geoip` , bí danh `mycorp` , thẻ `office`) trường sẽ là `"ip":`
`["ext:geoip_mycorp.dat:office"]`.

16. Vận hành: sao lưu, nhật ký, cập nhật, CLI

Phần này đề cập đến việc bảo trì hàng ngày của bảng điều khiển: tạo và khôi phục bản sao lưu cơ sở dữ liệu, xem nhật ký (log) của bảng điều khiển và Xray, khởi động lại và dừng dịch vụ, cập nhật Xray và bản thân bảng điều khiển, các tác vụ định kỳ (cron) và gỡ cài đặt bảng điều khiển. Một số thao tác được thực hiện từ giao diện web (các tab trên trang «Dashboard» và «Cài đặt bảng điều khiển»), một số – từ menu console `x-ui` trên máy chủ.

16.1. Sao lưu và khôi phục cơ sở dữ liệu

Tất cả dữ liệu của bảng điều khiển (inbound, client, nhóm, node, cài đặt) được lưu trong một cơ sở dữ liệu duy nhất. Quản lý sao lưu có thể truy cập từ trang «Dashboard» trong tab «Sao lưu», tiêu đề khối – «Sao lưu và khôi phục».

Bảng điều khiển hỗ trợ hai engine CSDL và hành vi sao lưu phụ thuộc vào đó:

- **SQLite** (tùy chọn mặc định) – dữ liệu được lưu trong file `x-ui.db`.
- **PostgreSQL** – nếu bảng điều khiển được cấu hình dùng PostgreSQL, trong khối sẽ hiển thị gợi ý:

«Bảng điều khiển này đang chạy trên PostgreSQL. «Sao lưu» tải xuống bản lưu trữ `pg_dump` (.dump), còn «Khôi phục» tải nó lên thông qua `pg_restore`. Công cụ client PostgreSQL (`pg_dump` và `pg_restore`) phải được cài đặt trên máy chủ.»

Xuất (tạo bản sao)

Nút «**Xuất cơ sở dữ liệu**» (eng. `Back Up`) tải file bản sao lưu về thiết bị của bạn.

Engine CSDL	Tên file	Điều gì xảy ra trên máy chủ
SQLite	<code>x-ui.db</code>	Trước tiên thực hiện checkpoint WAL để file chứa các bản ghi mới nhất, sau đó file được đọc toàn bộ và gửi để tải xuống
PostgreSQL	<code>x-ui.dump</code>	Chạy <code>pg_dump</code> , lưu trữ được gửi để tải xuống

Gợi ý trong giao diện:

- **SQLite**: «Nhân để tải file .db chứa bản sao lưu cơ sở dữ liệu hiện tại của bạn về thiết bị.»
- **PostgreSQL**: «Nhân để tải bản dump PostgreSQL (.dump) của cơ sở dữ liệu hiện tại về thiết bị.»

Về mặt kỹ thuật, xuất là yêu cầu `GET /panel/api/server/getDb`. Tên tệp đính kèm được máy chủ tạo ra (`Content-Disposition`) tùy theo engine.

Tên file sao lưu được tạo theo địa chỉ máy chủ, không phải cố định là `x-ui.db` / `x-ui.dump`. Khi tải xuống từ trình duyệt, nó lấy từ địa chỉ bảng điều khiển trong thanh địa chỉ (host của yêu cầu), nếu không – từ web domain đã cấu hình, còn nếu không có – từ IP công cộng của máy chủ (IPv4 trước, rồi IPv6), với dự phòng là

`x-ui`. Điều này giúp dễ dàng phân biệt các bản sao lưu từ các máy chủ khác nhau. Phần mở rộng vẫn là `.db` cho SQLite và `.dump` cho PostgreSQL; các bản sao lưu qua Telegram được đặt tên theo cùng domain/IP.

Ví dụ: tải bản sao lưu qua API. Có thể lấy cùng bản xuất bằng yêu cầu từ console – ví dụ, cho script sao lưu tự động. Cần phiên đã xác thực (cookie đăng nhập):

```
# 1) Đăng nhập và lưu cookie phiên
curl -s -c cookies.txt \
  -d 'username=admin&password=admin' \
  https://panel.example.com:2053/panel/login

# 2) Tải file cơ sở dữ liệu (tên do máy chủ đặt: x-ui.db hoặc x-ui.dump)
curl -s -b cookies.txt -OJ \
  https://panel.example.com:2053/panel/api/server/getDb
```

Nếu bảng điều khiển mở theo đường dẫn cơ sở (Web Base Path), cần thêm nó vào URL: `...:2053/<base_path>/panel/api/server/getDb`.

Nhập (khôi phục)

Nút «**Nhập cơ sở dữ liệu**» (eng. `Restore`) mở hộp thoại chọn file và tải nó lên máy chủ để khôi phục (`POST /panel/api/server/importDB`, trường form `db`).

Gợi ý trong giao diện:

- SQLite: «Nhấn để chọn và tải file `.db` từ thiết bị của bạn để khôi phục cơ sở dữ liệu từ bản sao lưu.»
- PostgreSQL: «Nhấn để chọn và tải file `.dump` để khôi phục cơ sở dữ liệu PostgreSQL. Điều này sẽ thay thế tất cả dữ liệu hiện tại.»

Quá trình nhập cho SQLite (quan trọng phải hiểu rằng nó có tính nguyên tử và có rollback):

1. File đã tải lên được kiểm tra định dạng – phải là cơ sở dữ liệu SQLite hợp lệ; nếu không sẽ trả về lỗi «Invalid db file format».
2. File được lưu vào `x-ui.db.temp` tạm thời và trải qua kiểm tra toàn vẹn.
3. **Xray bị dừng** trước khi thay thế CSDL.
4. CSDL hiện tại được đổi tên thành `x-ui.db.backup` dự phòng (fallback).
5. File tạm thời được di chuyển vào vị trí CSDL làm việc, thực hiện khởi tạo và di chuyển schema, sau đó di chuyển inbound.
6. **Nếu bất kỳ bước nào xảy ra lỗi** – thực hiện rollback: khôi phục CSDL cũ từ `x-ui.db.backup`, và Xray khởi động lại trên dữ liệu cũ.
7. Khi thành công, file dự phòng được xóa và **Xray tự động khởi động lại** trên dữ liệu đã khôi phục.

Thông báo giao diện theo kết quả:

Kết quả	Nội dung
Thành công	«Cơ sở dữ liệu đã được nhập thành công»
Lỗi nhập	«Đã xảy ra lỗi khi nhập cơ sở dữ liệu»

Kết quả	Nội dung
Lỗi đọc file	«Đã xảy ra lỗi khi đọc cơ sở dữ liệu»

Khôi phục sẽ thay thế hoàn toàn dữ liệu hiện tại. Do Xray tạm thời bị dừng trong quá trình, các kết nối hiện có của client sẽ bị ngắt trong thời gian nhập.

File di chuyển giữa các engine (SQLite ⇌ PostgreSQL)

Ngoài sao lưu thông thường, còn có chức năng **«Tải file di chuyển»** (Download Migration, yêu cầu GET / panel/api/server/getMigration). Nó tạo ra file có thể chuyển đổi để chuyển sang engine CSDL khác:

Engine hiện tại	Nội dung tải xuống	Tên file	Mục đích
SQLite	Dump SQL có thể chuyển đổi (văn bản)	x-ui.dump	Nạp dữ liệu của bạn vào PostgreSQL
PostgreSQL	Cơ sở dữ liệu SQLite được tạo từ dữ liệu PostgreSQL	x-ui.db	Chuyển bảng điều khiển trở về SQLite

Gợi ý:

- Trên SQLite: «Nhấn để tải xuất .dump có thể chuyển đổi (văn bản SQL) của cơ sở dữ liệu SQLite.»
- Trên PostgreSQL: «Nhấn để tải cơ sở dữ liệu SQLite (.db) được tạo từ dữ liệu PostgreSQL của bạn và sẵn sàng để chạy bảng điều khiển trên SQLite.»

Chuyển đổi .db ⇌ .dump cho SQLite cũng có thể thực hiện từ CLI bằng lệnh `x-ui migrateDB [file]` (xem phần 16.7).

Sao lưu qua Telegram bot

Nếu Telegram bot đã được cấu hình (xem phần về thông báo), nó có thể gửi bản sao lưu trực tiếp vào chat của quản trị viên. Sao lưu qua Telegram bao gồm **hai file**: bản thân cơ sở dữ liệu (x-ui.db, hoặc x-ui.dump trên PostgreSQL) và cấu hình Xray config.json. Trước tin nhắn có dòng « Thời gian sao lưu: ...».

Có hai cách nhận sao lưu trong Telegram:

1. **Theo yêu cầu.** Nút « Sao lưu CSDL» trong menu bot – bot ngay lập tức gửi file vào chat hiện tại.
2. **Tự động cùng với báo cáo.** Trong cài đặt bot có công tắc **«Sao lưu cơ sở dữ liệu»** (Database Backup) với mô tả «Gửi thông báo kèm file bản sao lưu cơ sở dữ liệu». Khi được bật, mỗi lần gửi báo cáo định kỳ, bot sẽ gửi bản sao lưu cho tất cả quản trị viên sau báo cáo. Chu kỳ gửi báo cáo được đặt theo lịch cron của bot (xem phần 16.6). Giữa các file và giữa các quản trị viên, bot tạm dừng để không vượt quá giới hạn của Telegram.

Sao lưu qua bot chỉ được gửi khi bot đang chạy; trên PostgreSQL nó cũng yêu cầu có `pg_dump` trên máy chủ.

16.2. Xem nhật ký

Bảng điều khiển có hai công cụ xem nhật ký độc lập, cả hai đều mở từ tab «**Nhật ký**» trên «Dashboard». Mỗi cửa sổ có thể làm mới (biểu tượng «làm mới» trong tiêu đề) và tải nội dung hiển thị vào file `x-ui.log` (nút với biểu tượng tải xuống).

Nhật ký bảng điều khiển (ứng dụng / syslog)

Cửa sổ nhật ký bảng điều khiển (`POST /panel/api/server/logs/{count}`). Các điều khiển:

Phần tử	Giá trị mặc định	Mô tả
Số dòng	20	Danh sách thả xuống: 10 / 20 / 50 / 100 / 500
Cấp độ	Info	Cấp độ tối thiểu: Debug / Info / Notice / Warning / Error
SysLog (hộp kiểm)	tắt	Nguồn lấy nhật ký: từ bộ đệm ứng dụng hoặc từ nhật ký hệ thống

Hành vi phụ thuộc vào hộp kiểm **SysLog**:

- **Tắt (mặc định):** nhật ký được lấy từ bộ đệm vòng nội bộ của bảng điều khiển, được lọc theo cấp độ đã chọn. Các bản ghi hiển thị với cấp độ (DEBUG / INFO / NOTICE / WARNING / ERROR) và nguồn: `X-UI`: – thông báo của chính bảng điều khiển, `XRAY`: – thông báo chuyển tiếp từ Xray.
- **Bật:** bảng điều khiển thực thi trên máy chủ
`journalctl -u x-ui --no-pager -n <count> -p <level>`, tức là hiển thị nhật ký hệ thống của dịch vụ `x-ui`. Số dòng được phép – từ 1 đến 10000; cấp độ chấp nhận các giá trị syslog (`emerg/0`, `alert/1`, `crit/2`, `err/3`, `warning/4`, `notice/5`, `info/6`, `debug/7`). Trên Windows chế độ SysLog không được hỗ trợ – sẽ hiển thị cảnh báo rằng cần bỏ chọn hộp kiểm và sử dụng nhật ký ứng dụng. Nếu `systemd` /dịch vụ không khả dụng, sẽ xuất hiện thông báo lỗi khởi động `journalctl`.

Ví dụ: cùng nhật ký từ console máy chủ. Khi bảng điều khiển không khả dụng (ví dụ, không khởi động được), nhật ký hệ thống có thể đọc trực tiếp – đây chính là lệnh mà bảng điều khiển thực thi ở chế độ SysLog:

```
# 100 dòng cuối cấp warning trở lên
journalctl -u x-ui --no-pager -n 100 -p warning

# theo dõi nhật ký theo thời gian thực
journalctl -u x-ui -f
```

Cấp độ trong cửa sổ này lọc **đầu ra**. Cấp độ tối thiểu nào được ghi vào console/syslog được xác định bởi cấp độ ghi nhật ký của bảng điều khiển (biên môi trường, mặc định là `Info`; vào file bảng điều khiển luôn ghi ở cấp `DEBUG`).

Nhật ký Xray (nhật ký truy cập)

Cửa sổ riêng biệt cho access-log của Xray (`POST /panel/api/server/xraylogs/{count}`). Nó phân tích các dòng nhật ký truy cập Xray và hiển thị chúng dưới dạng bảng: **Date, From, To, Inbound, Outbound, Email**.

Phần tử	Giá trị mặc định	Mô tả
Số dòng	20	10 / 20 / 50 / 100 / 500
Bộ lọc	trống	Tìm kiếm văn bản theo chuỗi con (áp dụng khi nhấn Enter)
Direct (hộp kiểm)	bật	Hiển thị kết nối trực tiếp (lưu lượng qua freedom-outbound)
Blocked (hộp kiểm)	bật	Hiển thị kết nối bị chặn (lưu lượng vào blackhole-outbound)
Proxy (hộp kiểm)	bật	Hiển thị lưu lượng được proxy

Loại sự kiện được xác định tự động theo thẻ kết nối đầu ra trong dòng nhật ký: khớp với thẻ freedom → «DIRECT» (xanh lá), blackhole → «BLOCKED» (đỏ), tất cả còn lại → «PROXY» (xanh dương). Các dòng `api` → `api` và dòng trống bị bỏ qua.

Để cửa sổ này hiển thị bản ghi, Xray phải bật **nhật ký truy cập** với đường dẫn đến file (không phải `none`) – xem bên dưới. Nếu access-log bị tắt hoặc file không khả dụng, cửa sổ sẽ trống («No Record...»).

16.3. Cấp độ và cấu hình ghi nhật ký Xray

Các tham số ghi nhật ký của chính Xray được đặt trên trang «**Cấu hình Xray**» trong khối «**Nhật ký**» (Log) với cảnh báo:

«Nhật ký có thể làm chậm máy chủ. Chỉ bật các loại nhật ký bạn cần khi cần thiết!»

Trường	Dịch	Giá trị mặc định	Mô tả
Cấp độ nhật ký (<code>logLevel</code>)	Log Level	warning	Cấp độ chi tiết của nhật ký lỗi Xray. Các giá trị được phép: <code>debug</code> , <code>info</code> , <code>notice</code> , <code>warning</code> , <code>error</code> . Gợi ý: «Cấp độ nhật ký cho nhật ký lỗi, chỉ định thông tin cần ghi lại.»
Nhật ký truy cập (<code>accessLog</code>)	Access Log	none	Đường dẫn đến file nhật ký truy cập. Giá trị đặc biệt <code>none</code> tắt nhật ký truy cập. Gợi ý: «Đường dẫn đến file nhật ký truy cập. Giá trị đặc biệt «none» tắt nhật ký truy cập.»
Nhật ký lỗi (<code>errorLog</code>)	Error Log	trống (đường dẫn mặc định)	Đường dẫn đến file nhật ký lỗi; <code>none</code> tắt chúng. Gợi ý: «Đường dẫn đến file nhật ký lỗi. Giá trị đặc biệt «none» tắt nhật ký lỗi.»
Nhật ký DNS (<code>dnsLog</code>)	DNS Log	false (tắt)	Bật ghi nhật ký yêu cầu DNS. Gợi ý: «Bật nhật ký yêu cầu DNS».
Ẩn địa chỉ (<code>maskAddress</code>)	Mask Address	trống (tắt)	Khi được kích hoạt, địa chỉ IP thực sẽ tự động được thay thế bằng địa chỉ ẩn danh trong nhật ký. Gợi ý: «Khi được kích hoạt, địa chỉ IP thực được thay thế bằng địa chỉ ẩn danh trong nhật ký.»

Vì mặc định **«Nhật ký truy cập» = none**, cửa sổ «Nhật ký Xray» (phần 16.2) ban đầu trống. Để nó hoạt động, hãy đặt đường dẫn đến access-log ở đây và khởi động lại Xray.

Lưu ý: access-log trống chỉ ảnh hưởng đến cửa sổ này. Danh sách client online trên «Dashboard» và giới hạn số lượng IP trong form client **không phụ thuộc** vào access-log – bảng điều khiển xác định client online và đếm địa chỉ IP của họ thông qua online-stats API của nhân Xray (thống kê kết nối). Trên các phiên bản nhân cũ không có API này, bảng điều khiển tự động quay về cách cũ (đọc access-log), và khi đó đường dẫn đến access-log ở đây vẫn cần thiết cho giới hạn IP.

Giới hạn số lượng IP và fail2ban. Chính giới hạn số lượng IP của client (trường «IP Limit» trong form client và khi thêm hàng loạt) chỉ được áp dụng trên máy chủ nếu đã cài đặt **fail2ban** – chính nó chặn các địa chỉ vượt quá giới hạn. Bảng điều khiển kiểm tra sự có mặt của fail2ban (GET /panel/api/server/fail2banStatus); nếu không có, trường «IP Limit» sẽ không khả dụng với gợi ý giải thích (trên Windows – thông báo riêng), và các giới hạn đã đặt trước đó trên các máy chủ như vậy sẽ tự động bị đặt về 0 vì chúng không có tác dụng dù sao. Khóa của fail2ban áp dụng cho cả TCP và UDP. Trên các máy chủ thông thường, fail2ban hiện được cài đặt và cấu hình tự động khi cài đặt và cập nhật bảng điều khiển (xem phần 16.5).

Ví dụ: khối Log để cửa sổ «Nhật ký Xray» bắt đầu hiển thị bản ghi. Trong cấu hình JSON của Xray trông như thế này:

```
{
  "log": {
    "loglevel": "warning",
    "access": "./access.log",
    "error": "",
    "dnsLog": false,
    "maskAddress": ""
  }
}
```

Điều quan trọng – thay "access": "none" bằng đường dẫn đến file (ví dụ, "./access.log"). Sau khi lưu, hãy khởi động lại Xray, và bảng trong cửa sổ «Nhật ký Xray» sẽ được điền các dòng.

16.4. Quản lý Xray: dừng và khởi động lại

Trạng thái của Xray được quản lý từ thẻ Xray trên «Dashboard». Trạng thái hiện tại được hiển thị bằng một trong các giá trị: **Đang chạy** (Running), **Đã dừng** (Stopped), **Không xác định** (Unknown), **Lỗi** (Error). Khi có lỗi, có sẵn tooltip «Lỗi khi khởi động Xray».

Nút	Dịch	Endpoint	Hành động
Dừng	Stop	POST /panel/api/server/stopXrayService	Dừng tiến trình Xray. Khi thành công – thông báo cảnh báo «Xray service has been stopped».

Nút	Dịch	Endpoint	Hành động
Khởi động lại	Restart	POST /panel/api/server/restartXrayService	Khởi động lại (hoặc khởi động) Xray với cấu hình hiện tại. Khi thành công – thông báo «Xray service has been restarted successfully».

Sau bất kỳ thao tác nào, bảng điều khiển phát trạng thái mới qua WebSocket, vì vậy trạng thái trên «Dashboard» được cập nhật mà không cần tải lại trang. Nếu thao tác kết thúc với lỗi, trạng thái Xray trở thành «Lỗi», và văn bản lỗi xuất hiện trong thông báo.

Ngoài khởi động lại thủ công, bảng điều khiển tự kiểm tra xem có cần khởi động lại Xray không (tác vụ nền mỗi 30 giây) và xem tiến trình có bị crash không (kiểm tra mỗi giây) – xem phần 16.6.

16.5. Khởi động lại và cập nhật bảng điều khiển

Khởi động lại bảng điều khiển

Trên trang «Cài đặt bảng điều khiển» có hành động «**Khởi động lại bảng điều khiển**» (Restart Panel , POST /panel/api/setting/restartPanel). Sau khi xác nhận, bảng điều khiển khởi động lại **sau 3 giây**.

Thông báo:

- Xác nhận: «Bạn có chắc chắn muốn khởi động lại bảng điều khiển không? Xác nhận và khởi động lại sẽ xảy ra sau 3 giây. Nếu bảng điều khiển không khả dụng, hãy kiểm tra nhật ký máy chủ.»
- Thành công: «Bảng điều khiển đã được khởi động lại thành công».

Về mặt kỹ thuật trên Linux, khởi động lại được thực hiện bằng cách gửi tín hiệu `SIGHUP` cho tiến trình bảng điều khiển (hoặc thông qua hook đã đăng ký). Trên Windows, việc gửi `SIGHUP` không được hỗ trợ.

Tự cập nhật bảng điều khiển (Update Panel)

Trên «Dashboard» có chức năng «**Cập nhật bảng điều khiển**» (Update Panel) – cập nhật 3X-UI lên bản phát hành mới nhất trực tiếp từ giao diện web.

Trước khi cập nhật, bảng điều khiển so sánh phiên bản (GET /panel/api/server/getPanelUpdateInfo), yêu cầu bản phát hành 3x-ui mới nhất từ GitHub:

Trường	Dịch
Phiên bản bảng điều khiển hiện tại	Current panel version
Phiên bản bảng điều khiển mới nhất	Latest panel version
Bảng điều khiển đã cập nhật / «Đã cập nhật»	Panel is up to date / Up to date – hiển thị nếu không có phiên bản mới

Khởi chạy cập nhật – POST /panel/api/server/updatePanel . Hộp thoại xác nhận:

- «Bạn có thực sự muốn cập nhật bảng điều khiển không?»

- «Điều này sẽ cập nhật 3X-UI lên phiên bản #version# và khởi động lại dịch vụ bảng điều khiển.»

Sau khi khởi chạy – thông báo popup «Đã bắt đầu cập nhật bảng điều khiển» (`Panel update started`); nếu kiểm tra phiên bản thất bại – «Kiểm tra cập nhật bảng điều khiển thất bại» (`Panel update check failed`).

Điều gì xảy ra trên máy chủ: tự cập nhật chỉ được hỗ trợ **trên Linux** (trên các hệ điều hành khác sẽ trả về lỗi «panel web update is supported only on Linux installations»). Bảng điều khiển tải script chính thức `update.sh` từ GitHub (`raw.githubusercontent.com/MHSanaei/3x-ui/main/update.sh`) và chạy nó trong một tiến trình riêng biệt: ưu tiên qua `systemd-run` trong unit riêng (`x-ui-web-update-<timestamp>`), còn khi không có `systemd` – như một tiến trình tách biệt độc lập. Sau khi hoàn thành, script cập nhật các thành phần và khởi động lại dịch vụ bảng điều khiển. Để chạy cần có `bash` .

Nếu trong quá trình cập nhật script tạo ra đường dẫn cơ sở mới ngẫu nhiên của web-panel (Web Base Path), dịch vụ `x-ui` sẽ khởi động lại tự động để đường dẫn mới hoạt động ngay lập tức. (Nếu không khởi động lại, máy chủ sẽ tiếp tục phục vụ đường dẫn cũ, trong khi giao diện hiển thị đường dẫn mới, và địa chỉ mới sẽ không khả dụng cho đến khi khởi động lại thủ công.)

Trên các node, bảng điều khiển cùng 3x-ui này được cập nhật tập trung qua `POST /panel/api/nodes/updatePanel` – xem phần về các node.

Cài đặt tự động fail2ban

Để giới hạn số lượng IP của client (phần 16.3) hoạt động ngay từ đầu, khi cài đặt và cập nhật bảng điều khiển trên máy chủ thông thường, `fail2ban` hiện được cài đặt và cấu hình tự động (trước đây điều này chỉ xảy ra trong Docker image). Hành vi được kiểm soát bởi biến môi trường `XUI_ENABLE_FAIL2BAN` : cấu hình được thực hiện nếu biến không được đặt hoặc bằng `true` . Có thể chạy thủ công bằng lệnh `x-ui setup-fail2ban` . Lỗi cấu hình fail2ban không làm gián đoạn quá trình cài đặt hoặc cập nhật bảng điều khiển.

Cài đặt và cập nhật trên máy chủ chỉ có IPv6

Các script `install.sh` và `update.sh` hiện hoạt động chính xác trên các máy chủ chỉ có IPv6: tải xuống bản phát hành, script `x-ui.sh` và file dịch vụ không còn bắt buộc dùng IPv4 (`curl -4`), mà sử dụng giao thức khả dụng. Do đó, bảng điều khiển có thể được cài đặt và cập nhật trên máy chủ không có địa chỉ IPv4.

Ghi đề cổng bảng điều khiển bằng biến `XUI_PORT`

Cổng lắng nghe của web-panel có thể được ghi đề bằng biến môi trường `XUI_PORT` – nó chỉ có hiệu lực trong thời gian chạy của tiến trình hiện tại và **không thay đổi** giá trị `webPort` đã lưu trong cơ sở dữ liệu. Các giá trị được phép từ `1` đến `65535` ; giá trị trống, không hợp lệ hoặc ngoài phạm vi bị bỏ qua (sử dụng `webPort`) với cảnh báo trong nhật ký. Điều này tiện lợi khi triển khai, chủ yếu trong Docker: khi sử dụng mạng bridge, cổng container được publish phải khớp với `XUI_PORT` – ví dụ, `XUI_PORT=8080` và `ports: "8080:8080"` .

Cập nhật và chuyển đổi phiên bản Xray-core

Ngay trên «Dashboard» có thể quản lý phiên bản Xray-core riêng biệt với bảng điều khiển.

- **Cập nhật Xray** (Xray Updates) / **Chọn phiên bản** (Version) – danh sách thả xuống các phiên bản có sẵn. Gợi ý: «Chọn phiên bản bạn cần» và cảnh báo «Quan trọng: các phiên bản cũ có thể không hỗ trợ các cài đặt hiện tại».
- Cài đặt/thay đổi phiên bản – `POST /panel/api/server/installXray/{version}` . Hộp thoại: «Chuyển đổi phiên bản Xray» / «Bạn có thực sự muốn thay đổi phiên bản Xray không?». Khi thành công – «Xray đã được cập nhật thành công».

Ví dụ: thay đổi phiên bản Xray-core qua yêu cầu API. Phiên bản được chỉ định bằng thẻ phát hành từ XTLS/Xray-core (với tiền tố `v`). Ví dụ, chuyển sang `v1.8.24` :

```
curl -s -b cookies.txt -X POST \
https://panel.example.com:2053/panel/api/server/installXray/v1.8.24
```

(`cookies.txt` – file với cookie từ ví dụ trong phần 16.1.) Sau khi cài đặt, Xray sẽ tự động khởi động lại với phiên bản đã chọn.

Trên máy chủ khi thay đổi phiên bản, Xray trước tiên bị dừng, bản lưu trữ phiên bản cần thiết được tải xuống từ GitHub (XTLS/Xray-core), binary được giải nén và thay thế, sau đó Xray khởi động lại với kiểm tra kích thước tệp lưu trữ/binary.

16.6. Các tác vụ định kỳ (cron)

Bảng điều khiển đăng ký một số tác vụ nền khi khởi động. Lịch của chúng được cố định (không thể cấu hình trong UI, ngoại trừ lịch báo cáo Telegram và đồng bộ LDAP). Dưới đây là các tác vụ liên quan đến vận hành.

Tác vụ	Lịch	Mục đích
Kiểm tra hoạt động Xray	mỗi 1 giây	Kiểm soát rằng tiến trình Xray đang chạy
Kiểm tra cần khởi động lại Xray	mỗi 30 giây	Khởi động lại nếu cấu hình được đánh dấu là đã thay đổi
Thu thập lưu lượng Xray	mỗi 5 giây (bắt đầu sau 5 giây kể từ khi khởi động)	Ghi lại lưu lượng inbound/client
Kiểm tra IP client	mỗi 10 giây	Kiểm soát giới hạn IP theo nhật ký
Heartbeat và đồng bộ lưu lượng node	mỗi 5 giây	Trao đổi với các node
Xóa nhật ký	hàng ngày (@daily)	Xóa nhật ký giới hạn IP và access-log liên tục, xoay vòng nhật ký hiện tại thành <code>*.prev.log</code>
Đặt lại lưu lượng theo chu kỳ	@hourly , @daily , @weekly , @monthly	Đặt lại bộ đếm lưu lượng của các inbound (và client của chúng) có cài đặt chu kỳ tự động đặt lại tương ứng

Tác vụ	Lịch	Mục đích
Báo cáo Telegram	được đặt trong cài đặt bot (mặc định @daily)	Gửi báo cáo cho quản trị viên; khi bật tùy chọn – kèm theo bản sao lưu CSDL đính kèm (phần 16.1)
Đặt lại bộ lưu trữ hash Telegram	mỗi 2 phút	Chỉ khi bot được bật
Kiểm soát tải CPU cho Telegram	mỗi 10 giây	Chỉ nếu ngưỡng CPU > 0 được đặt

Bổ sung:

- **Đặt lại lưu lượng định kỳ** chỉ kích hoạt cho các inbound có chọn chế độ tự động đặt lại tương ứng (hàng giờ/hàng ngày/hàng tuần/hàng tháng). Tác vụ đặt lại lưu lượng của chính inbound và tất cả client của nó.
- **Kiểm tra hết hạn và cạn kiệt.** Vô hiệu hóa client khi hết hạn và khi hết giới hạn lưu lượng được thực hiện trong khuôn khổ ghi lại lưu lượng: các client có `expiry_time` đã hết hoặc dung lượng đã cạn được đánh dấu và vô hiệu hóa, nếu cần sẽ tính kỳ hạn tiếp theo (cho giới hạn chu kỳ và chế độ «đếm từ lần sử dụng đầu tiên»). Trên «Dashboard» và trong danh sách, điều này phản ánh qua trạng thái «Hết hạn»/«Đã cạn»/«Sắp hết».
- **Sao lưu tự động trong Telegram** – là hiệu ứng phụ của tác vụ báo cáo, không có lịch cron riêng chỉ cho sao lưu. Do đó, tần suất tự động sao lưu bằng tần suất báo cáo của bot.

16.7. Menu console và CLI (`x-ui`)

Trên máy chủ, bảng điều khiển được quản lý bằng lệnh `x-ui` . Không có đối số, mở menu tương tác «3X-UI Panel Management Script»; với đối số, thực thi lệnh con cụ thể. Các mục menu liên quan đến vận hành:

Số trong menu	Mục	Hành động
1	Install	Cài đặt bảng điều khiển (tải xuống và chạy <code>install.sh</code>)
2	Update	Cập nhật tất cả thành phần x-ui lên phiên bản mới nhất không mất dữ liệu; sau đó – tự động khởi động lại
3	Update Menu	Chỉ cập nhật script menu <code>x-ui</code>
4	Legacy Version	Cài đặt phiên bản cũ được chỉ định của bảng điều khiển theo số đã nhập (ví dụ: <code>2.4.0</code>)
5	Uninstall	Gỡ cài đặt hoàn toàn bảng điều khiển và Xray (xem 16.8)
6	Reset Username & Password	Đặt lại tên đăng nhập/mật khẩu quản trị viên
7	Reset Web Base Path	Đặt lại đường dẫn cơ sở của web-panel
8	Reset Settings	Đặt lại cài đặt về giá trị mặc định
9	Change Port	Thay đổi cổng bảng điều khiển
10	View Current Settings	Xem cài đặt hiện tại

Số trong menu	Mục	Hành động
11–13	Start / Stop / Restart	Khởi động, dừng, khởi động lại dịch vụ bảng điều khiển
14	Restart Xray	Chỉ khởi động lại Xray
15	Check Status	Trạng thái hiện tại của dịch vụ
16	Logs Management	Xem và xóa nhật ký (xem bên dưới)
17–18	Enable / Disable Autostart	Bật/tắt tự động khởi động dịch vụ khi hệ điều hành khởi động
25	Update Geo Files	Cập nhật các file địa lý (GeoIP/GeoSite)
27	PostgreSQL Management	Quản lý PostgreSQL

Quản lý nhật ký trong CLI (mục 16)

Trong submenu «Logs Management»:

- **Debug Log** – xem luồng nhật ký dịch vụ: `journalctl -u x-ui -e --no-pager -f -p debug` (trên Alpine – `grep` theo `/var/log/messages`).
- **Clear All logs** – xóa nhật ký hệ thống: `journalctl --rotate + journalctl --vacuum-time=1s`, sau đó dịch vụ khởi động lại. (Không khả dụng trên Alpine.)

Các lệnh con trực tiếp của `x-ui`

Tất cả các lệnh con có sẵn:

Lệnh	Mô tả
<code>x-ui</code>	Mở menu quản trị
<code>x-ui start</code>	Khởi động bảng điều khiển
<code>x-ui stop</code>	Dừng bảng điều khiển
<code>x-ui restart</code>	Khởi động lại bảng điều khiển
<code>x-ui restart-xray</code>	Khởi động lại Xray
<code>x-ui status</code>	Trạng thái hiện tại
<code>x-ui settings</code>	Hiển thị cài đặt hiện tại
<code>x-ui enable</code>	Bật tự động khởi động khi hệ điều hành khởi động
<code>x-ui disable</code>	Tắt tự động khởi động
<code>x-ui log</code>	Xem nhật ký
<code>x-ui banlog</code>	Xem nhật ký chặn của Fail2ban
<code>x-ui setup-fail2ban</code>	Cài đặt và cấu hình fail2ban cho giới hạn IP (xem 16.5)

Lệnh	Mô tả
<code>x-ui update</code>	Cập nhật bảng điều khiển
<code>x-ui update-all-geofiles</code>	Cập nhật tất cả file địa lý (với khởi động lại tiếp theo)
<code>x-ui migrateDB [file]</code>	Chuyển đổi cơ sở dữ liệu <code>.db</code> ⇌ <code>.dump</code> (SQLite)
<code>x-ui legacy</code>	Cài đặt phiên bản cũ
<code>x-ui install</code>	Cài đặt bảng điều khiển
<code>x-ui uninstall</code>	Gỡ cài đặt bảng điều khiển

Lệnh `x-ui update` tải xuống và chạy `update.sh` chính thức (giống như cập nhật web từ phần 16.5), yêu cầu xác nhận: «This function will update all x-ui components to the latest version, and the data will not be lost.» Sau khi hoàn thành, bảng điều khiển tự động khởi động lại.

Cờ `-webCert` / `-webCertKey` trong lệnh con `setting`. Đường dẫn đến chứng chỉ và khóa riêng tư của web-panel có thể được đặt trực tiếp trong lệnh con `x-ui setting -webCert <đường dẫn> -webCertKey <đường dẫn>` – chỉ định bất kỳ cờ nào trong số này sẽ lưu đường dẫn tương ứng (cũng như lệnh con `cert` riêng biệt), và bảng điều khiển sẽ ngay lập tức chuyển sang HTTPS.

Lấy API token qua CLI

Lệnh lấy API token qua CLI (mục menu/lệnh `x-ui`) không hiển thị token đã được cấp trước đó. API token chỉ được lưu ở dạng hash, vì vậy không thể lấy token hiện có ở dạng văn bản thuần. Nếu token đã được cấu hình, lệnh thông báo số lượng của chúng, khuyên nên quản lý token trong bảng điều khiển (**Settings** → **API Tokens**, xem phần về API token) và ngay lập tức tạo **token dự phòng mới** với tên dạng `cli-fallback-<timestamp>` và hiển thị nó, để CLI vẫn hữu ích mà không cần vào giao diện.

16.8. Gỡ cài đặt bảng điều khiển

Gỡ cài đặt được thực hiện từ CLI – mục menu **5 (Uninstall)** hoặc lệnh `x-ui uninstall`. Trước khi gỡ cài đặt, cần xác nhận (mặc định «không»): «Are you sure you want to uninstall the panel? xray will also be uninstalled!».

Sau khi xác nhận, script:

- Dừng dịch vụ và tắt tự động khởi động (`systemctl stop/disable x-ui`, hoặc trên Alpine – `rc-service / rc-update`), xóa file unit dịch vụ và tải lại cấu hình systemd.
- Xóa thư mục dữ liệu và ứng dụng (`/etc/x-ui/`, thư mục cài đặt) và file env dịch vụ (`/etc/default/x-ui`, `/etc/conf.d/x-ui` hoặc `/etc/sysconfig/x-ui` – tùy theo phân phối).
- Xóa chính script `x-ui` và hiển thị thông báo «Uninstalled Successfully.», cùng với lệnh để cài đặt lại.

Gỡ cài đặt là không thể đảo ngược: cùng với bảng điều khiển, Xray và tất cả dữ liệu (bao gồm cơ sở dữ liệu) sẽ bị xóa. Nếu dữ liệu có thể cần thiết, hãy xuất cơ sở dữ liệu trước (phần 16.1).

16.9. Lệnh `x-ui migrateDB`

Bắt đầu từ phiên bản 3.3.0, script quản lý `x-ui.sh` có thêm lệnh `con migrateDB` — một wrapper xung quanh binary `x-ui` tích hợp (`x-ui migrate-db`) để chuyển đổi cơ sở dữ liệu bảng điều khiển SQLite giữa hai định dạng: nhị phân `.db` và dump văn bản có thể chuyển đổi `.dump` (văn bản SQL thông thường).

Lệnh làm gì

Lệnh hoạt động theo hai hướng, và hướng được xác định **tự động** theo file đầu vào:

Hướng	Tên gọi	Điều gì xảy ra
<code>.db → .dump</code>	dump (xuất)	cơ sở dữ liệu SQLite nhị phân được xuất thành file SQL văn bản
<code>.dump → .db</code>	restore (khôi phục)	từ file SQL văn bản, cơ sở dữ liệu SQLite nhị phân được tạo lại

Bên trong, script gọi binary của bảng điều khiển:

- xuất: `x-ui migrate-db --src <đầu vào> --dump <đầu ra>`
- khôi phục: `x-ui migrate-db --restore <đầu vào> --out <đầu ra>`

Cú pháp gọi

```
x-ui migrateDB [file.db|file.dump] [output]
```

- **[file.db|file.dump]** — file đầu vào (đối số thứ nhất). Nếu không chỉ định, sẽ lấy cơ sở dữ liệu bảng điều khiển đã cài đặt mặc định: `/etc/x-ui/x-ui.db`.
- **[output]** — đường dẫn đến file đầu ra (đối số thứ hai). Không bắt buộc: khi vắng mặt, tên được chọn tự động cạnh file đầu vào (xem bên dưới).

Ví dụ:

```
x-ui migrateDB                               # xuất /etc/x-ui/x-ui.db -> /etc/x-ui/x-
ui.dump
x-ui migrateDB /etc/x-ui/x-ui.db backup.dump
x-ui migrateDB backup.dump restored.db       # tạo .db từ dump
```

Cách xác định hướng

Script nhìn vào phần mở rộng của file đầu vào:

- `*.db`, `*.sqlite`, `*.sqlite3` → chế độ **dump** (xuất thành văn bản);
- `*.dump`, `*.sql` → chế độ **restore** (tạo cơ sở dữ liệu).

Nếu phần mở rộng không được nhận dạng, script đọc 16 byte đầu tiên của file: chữ ký SQLite format 3 có nghĩa là cơ sở dữ liệu nhị phân (chế độ dump), ngược lại file được coi là dump (chế độ restore).

Tên file đầu ra, nếu đối số thứ hai không được đặt:

- khi xuất – cùng tên với file đầu vào, với phần mở rộng `.dump` ;
- khi khôi phục – cùng tên với phần mở rộng `.db` .

Kiểm tra bảo vệ và hành vi

- **Sự có mặt của binary.** Nếu binary `x-ui` không tìm thấy hoặc không thể thực thi – hiển thị lỗi «x-ui binary not found ... Is the panel installed?».
- **Hỗ trợ chức năng trong bản dựng.** Script kiểm tra rằng binary hỗ trợ `migrate-db --dump/--restore` (qua `x-ui migrate-db -h`). Nếu không – đề xuất trước tiên cập nhật bảng điều khiển bằng lệnh `x-ui update` .
- **Sự tồn tại của file đầu vào.** Khi file đầu vào không có, in ra lỗi và dòng với cú pháp gọi.
- **Ghi đè đầu ra.** Nếu file đầu ra đã tồn tại, yêu cầu xác nhận (mặc định «không»); không có xác nhận, thao tác bị hủy. Khi khôi phục, file đầu ra cũ được xóa trước.
- **Bảo vệ cơ sở dữ liệu «đang sống».** Khi khôi phục vào cơ sở dữ liệu mặc định `/etc/x-ui/x-ui.db` , khi bảng điều khiển đang chạy, thao tác bị từ chối với yêu cầu trước tiên dừng bảng điều khiển (`x-ui stop`) hoặc chọn đường dẫn đầu ra khác. Điều này ngăn chặn việc ghi đè cơ sở dữ liệu đang hoạt động của dịch vụ đang chạy.
- Khi tạo cơ sở dữ liệu thất bại, file đầu ra chưa ghi hoàn chỉnh sẽ bị xóa.

Tại sao cần điều này

- **Sao lưu.** Văn bản `.dump` – có thể đọc được bởi người, thuận tiện để lưu trữ trong hệ thống kiểm soát phiên bản và để xem nội dung cơ sở dữ liệu theo kiểu diff.
- **Di chuyển.** Dump có thể chuyển đổi giữa các máy và bền vững trước các khác biệt phiên bản định dạng file SQLite – trên máy chủ mới, `.db` làm việc được tạo từ nó.
- **Chẩn đoán.** Từ `.dump` có thể xem cấu trúc và dữ liệu của bảng điều khiển bằng mắt mà không cần có công cụ SQLite trong tay.

Chế độ tương tác

Ngoài việc gọi trực tiếp, chuyển đổi có thể thực hiện từ menu tương tác. Trong submenu PostgreSQL (`x-ui` → phần làm việc với PostgreSQL) có mục **9. Convert SQLite .db <-> .dump** : nó hỏi đường dẫn đến file đầu vào (mặc định `/etc/x-ui/x-ui.db`) và đến file đầu ra (có thể để trống để tự đặt tên), còn hướng, như trong chế độ CLI, được xác định tự động.

Tài liệu được chuẩn bị dựa trên mã nguồn 3X-UI. Nếu bất kỳ mục nào trong giao diện ở phiên bản của bạn khác – ưu tiên cho hành vi của bảng điều khiển và các gợi ý trong chính UI.